

狼叔

更了不起的
Node.js

卷1

狼叔◎著



中国工信出版集团



电子工业出版社
PUBLISHING HOUSE OF ELECTRONICS INDUSTRY
<http://www.phei.com.cn>

更了不起的 Node.js

狼叔◎著

卷1



電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

内 容 简 介

Node.js 开发简单，性能极好，一经发布便成了明星级项目。随着大前端领域的蓬勃发展，跨平台开发、API 构建、Web 应用开发等场景愈加常见，Node.js 也成为大前端开发的必备“神器”。

本书以 Node.js 为主，讲解了 Node.js 的基础知识、开发调试方法、源码原理和应用场景，旨在向读者展示如何通过最新的 Node.js 和 npm 编写出更具前端特色、更具工程化优势的代码。本书还讲解了 Node.js 中最核心、最复杂的异步流程控制，展望了未来异步流程的发展方向，非常适合大前端领域及后端领域的测试、运维及软件开发从业者阅读、学习。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。
版权所有，侵权必究。

图书在版编目（CIP）数据

狼书. 卷 1, 更了不起的 Node.js / 狼叔著. —北京：电子工业出版社，2019.7
ISBN 978-7-121-35907-1

I. ①狼… II. ①狼… III. ①JAVA 语言—程序设计 IV. ①TP312.8

中国版本图书馆 CIP 数据核字（2019）第 011590 号

责任编辑：张春雨

印 刷：

装 订：

出版发行：电子工业出版社

北京市海淀区万寿路 173 信箱 邮编：100036

开 本：787×980 1/16 印张：19.5 字数：435 千字

版 次：2019 年 7 月第 1 版

印 次：2019 年 7 月第 1 次印刷

定 价：79.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888，88258888。

质量投诉请发邮件至 zltz@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式：010-51260888-819，faq@phei.com.cn。

推荐序 1

提起国内的 Node.js 布道师，我脑海中出现的第一个名字就是狼叔（i5ting）。

狼叔从 2015 年开始活跃于 CNode 社区，至今累计发表文章 200 余篇，主题丰富多样——Node.js 底层原理、npm 目录结构改进、前后端分离实践、全栈工程师之路等。这几年来，狼叔同时运营着自己的微信公众号“Node 全栈”，每日笔耕不辍，源源不断地将最新鲜、最“硬核”的资讯分享给国内的开发者们。不得不说，他的这种乐于分享的精神，实属难得。

我与狼叔也是在 2015 年相识的。2015 年是 Node.js 的普及度呈爆发式增长的一年，但那一年的 Node.js 还远远谈不上被广泛使用。当时我在阿里巴巴数据平台任职，那时我们所做的部分项目的 JavaScript 压缩工具和测试覆盖率工具还是用 Java 实现的，这在现在看来可以说是非常匪夷所思的，JavaScript 工作流中的工具竟然还有用 Java 而不是用 Node.js 实现的！

时过境迁，转眼到了 2019 年，JavaScript 在大前端领域遍地开花，Node.js 也已经被广泛应用于 Web 开发的方方面面，成了 Web 开发流程中不可或缺的部分。大家不再怀疑 Node.js 能不能用，而是要开始思考该如何用 Node.js 实现我们想要的功能。

一门技术的好与坏，不仅仅在于技术本身具有什么优势。布道如果做得不好，酒香也怕巷子深。技术的进步与受众群体的反馈是相互促进的。Node.js 在国内逐渐生根发芽的这几年，狼叔无疑是推广该技术的中坚力量。

经过多年的积累和沉淀，狼叔带着他的新书《狼书（卷 1）：更了不起的 Node.js》与大家见面了。这本书内容循序渐进，概念清晰明了，技术描述有点有面，是一本理论架构完整且实战案例典型的好书！相信各位读者一定能够从中获益！

最后，衷心祝愿 Node.js 发展越来越好，也祝愿狼叔的布道事业蒸蒸日上！

CNode 社区管理员，alsotang

推荐序 2

狼叔邀请我为《狼书（卷 1）：更了不起的 Node.js》写推荐的时候，我的内心是忐忑的。因为我对 Node.js 并不熟悉，不是这方面的专家。但对于狼叔我是了解的，同为技术社区推动者和文字爱好者，我深知在国内要写一本严肃的技术图书是一件多么吃力不讨好的事情。正因如此，狼叔的这本书就更值得推荐给准备学习和正在学习 Node.js 的工程师们。

十年前，Node.js 刚刚诞生，那时我就接触到了它。后来，Node.js 的迭代和进步之快完全超出了我的预期，它变得越来越好用，逐渐成为全栈工程师的首选。这样的结果离不开强大、活跃的 Node.js 社区和无私的 Node.js 贡献者们的付出，而狼叔就是国内 Node.js 贡献者的代表。

有了 Node.js，前端工程师也可以编写后端程序，并且成为手机应用的跨平台开发主力。客户端、前端和服务端已呈现出大统一趋势。在我自己熟悉的 Web 服务器领域，可以说 Nginx 内置的 njs 就是冲着替代 OpenResty 这一目标迅速发展的。

在这种技术趋势下，学习 JavaScript 和 Node.js 无疑是一个性价比很高的选择，这样一来，我们便可以打通从手机应用、网站应用到服务端接口的整条链路。而学习一门技术最好的方式，就是选择一本好书。

一本好书对作者的要求很高——技术功底扎实只是基础，更要有丰富的项目经验、深厚的文字功底和洞察读者心理的能力。平日里像“诗人”一样的狼叔绝对是为数不多的具备上述能力的“牛人”，所以我相信他写的书也一定是一本好书。希望大家能通过这本好书提升自己的技术水平。

学习从来不是一件容易的事，但却是一件快乐的事情，共勉。

温铭

OpenResty 软件基金会主席、OpenResty Inc. 合伙人

推荐序 3

1995 年, Brendan Eich 花了 10 天时间开发出了一门脚本语言, 用来弥补 Java Applets 的不足, 随后 Marc Andreessen 将这门语言命名为 Mocha。Mocha 的最初定位是, 服务于测试脚本编写人员、业余编程爱好者、设计师。

1995 年 5 月, Mocha 被集成到了 Netscape 浏览器中, 不久后被更名为 LiveScript。同年年底, Netscape 公司和 Sun 公司达成协议并获得了 Java 商标的使用权, 于是 LiveScript 正式更名为 JavaScript。

有人觉得, 正是因为更名为 JavaScript 才使这门语言成了浏览器执行的唯一语言。但时至今日, JavaScript 已经不仅仅局限于实现网页特效了, 而真正发展成了一门全功能的编程语言。

2009 年, Joyent 公司的一名软件工程师 Ryan Dahl 开发了 Node.js, 这是一个基于 Chrome V8 引擎的 JavaScript 运行时环境。Node.js 使得 JavaScript 拥有了操作文件系统、I/O、网络, 甚至数据库的能力。虽然 Node.js 不是第一个将 JavaScript 带离浏览器的软件, 但无疑是最成功的一个。

如今 Node.js 社区已经成了最活跃的编程社区之一, 其 npm 的包数量也已经超越了 Java 的 Maven、Ruby 的 Gem、PHP 的 Composer。

狼叔是国内最早一批的 Node.js 使用者, 也是 Node.js 社区最活跃的布道者之一。几年前狼叔来天津创业, 我有缘与他结识。在那之前我就已经拜读过狼叔的文章和课程, 而当时狼叔就与我谈起要写一本关于 Node.js 的书。说来也巧, Node.js 于 2009 年发布, 而《金刚狼》系列电影也于 2009 年开始上映。《金刚狼》系列电影一共 3 部, 而狼叔的《狼书》系列图书也有 3 卷, 希望《狼书》系列能如《金刚狼》一样受到欢迎。

目前 Node.js 十分火热, 但很大一部分使用者是前端开发人员。和 Java、Python、Ruby 等后端语言对比, 尤其在书籍出版方面, Node.js 还需持续深入, 而《狼书》的面世正好弥补了这

一方面的不足——第 1 卷比较系统全面地介绍了 Node.js，第 2 卷和第 3 卷则会更加深入地介绍 Node.js 的高级应用。如果你想深入学习 Node.js 的核心原理并掌握使用 Node.js 开发大型系统的要诀，那么这套书绝对值得你精读。

迷渡 (justjavac)

2019 年 5 月于天津

推荐序 4

俗话说，十年磨一剑，慢工出细活。狼叔撰写的《狼书》系列书籍很好地诠释了这两句话。

众所周知，狼叔是 Node.js 布道者、“Node 全栈”公众号的作者，他活跃于 CNode 社区，组织了不少线下 Node.js 沙龙，同时常作为讲师在各种技术交流会上进行分享，为 Node.js 在国内的推广做出了很大的贡献。我觉得这是一种情怀，也是一种责任。当你爱上一件事，你就会全情投入。

Node.js 的出现在很大程度上满足了前端工程师想要探索更广阔的编程世界的愿望，为前端工程师提供了更好的了解后端工作的机会，对于前后端协同而言具有巨大价值。十年时间，Node.js 几经波折，但这并不妨碍它快速发展，如今它已经成为最流行的技术之一。

近些年，不少大型互联网公司都开始基于 Node.js 构建应用。我和狼叔在去哪儿网相识，平时和他对话或闲聊，最后总能聊到 Node.js 上，我能深切地感受到他对 Node.js 的热爱。那时候的狼叔正在努力为去哪儿网建设更完善的 Node.js 基础设施，他的努力为去哪儿网注入了新鲜活力，加快了 Node.js 在机票购买业务中的落地。

《狼书》系列图书正是狼叔 Node.js 情怀的最终寄托。本书由浅及深，由粗至细，几经雕琢，很好地承载了狼叔对 Node.js 的热爱，将 Node.js 的发展历史、Node.js 的特征和应用场景、Node.js 基本语法、高阶异步流程控制和展望等热门话题娓娓道来，就像一杯老酒，越品越有味道。我相信每一位拿到此书的读者都会有不同的收获，无论你是初入前端领域的“小白”，还是深耕多年的“老手”。

杜瑶

美团研究员

去哪儿网前高级技术总监

推荐序 5

自 2009 年 Node.js 诞生以来，它一直在快速发展，不断扩大自身的能力范围。

基于 Chrome V8 执行引擎的 Node.js 在保证其性能和稳定性的同时，也收获了许多由强大的技术社区提供的优秀 npm 包，因此基于单线程和异步流程控制的 Node.js 开发在效率上得到了保障与提升。

得益于这些优势，Node.js 可以被广泛应用于诸多场景——从数据库到 API，从 Web 应用框架到 SSR 服务，从命令行到前端工程化，甚至在操作系统开发和桌面应用设计中都能占有一席之地。这些足以说明 Node.js 的“了不起”与空前繁荣。

近几年来，Node.js 在国内发展迅猛，无论是大型企业的中台服务，还是中小型企业的全栈式研发模式，几乎都将 Node.js 作为首选技术。事实证明，它并没有让大家失望。

如今 Node.js 的稳定版本为 v10.x，新特性和新功能不断加入，版本也快速迭代。我们可以看到，有非常多的 Node.js 开发工程师专门从事这项技术的研究，也有很多企业在招聘时将 Node.js 作为应聘者的必会技能进行考查。这些都足以证明，Node.js 正在被进一步发扬光大。

本书的作者狼叔，一直活跃在 CNode 技术社区。作为一名 Node.js 布道者，他一直深耕在 Node.js 领域，不断在各个平台与大家分享他的技术见解。这本《狼叔(卷 1): 更了不起的 Node.js》是狼叔多年技术心血的结晶，它很好地向读者介绍了 Node.js 的发展历程、基本特性、编程方法、应用场景和核心模块。这本书是对狼叔多年实践经验的总结。无论你是想入门 Node.js 还是想进行企业级深度实践，都可以参考这本书。只要你热衷于 Node.js，这本书便值得你阅读！

河伯

腾讯技术总监

腾讯 IVWEB 团队负责人

推荐语

Node.js 是为数不多的中国程序员不是跟从者而是开创者的技术领域。中国程序员在 Node.js 的布道方面贡献了很多，从推广 Node.js 社区到组织各种会议，当然也包括出版书籍。对所有优秀的程序员来说，写书都是一件辛苦的事，所以愿意在这方面投入精力的程序员基本上都是有情怀的。狼叔花了三年多时间写成了这本书，其中既包含 Node.js 基础知识，也包含宝贵的工程实践，为所有从业者提供了参考，期待狼叔能够一直写下去。

极客时间《重学前端》专栏作者，程邵非（winter）

两年前曾和狼叔聊起过一个颇为枯燥的技术问题，当时他把那个问题解释得非常精彩，让我印象颇深。所以得知狼叔在写书时，我充满了期待。一方面，我相信狼叔一定能把严肃的技术问题讲得通俗易懂；另一方面，要想将 Node.js 生态讲得透彻，狼叔是优秀人选。

ioredis 作者、《Redis 入门指南》作者、石墨文档技术总监，李子骅（luin）

这不是一本简单的 Node.js 入门书，而是一本纵观 Node.js 发展历史、带你领略 Node.js 底层风采，并且能对你的 Node.js 知识体系进行查漏补缺的书。在如今各式各样的 Node.js 书籍中，这样的好书真的非常难得。

《Node.js：来一打 C++ 扩展》作者，死月

狼叔是国内比较知名的 Node.js 技术布道者，为 Node.js 在中国的发展做出了巨大的贡献。本书中既有对 Node.js 知识点的详细介绍，也有对狼叔多年宝贵经验的深度总结，非常值得大家阅读、学习，建议各位持卷品读。

ThinkJS 框架作者，李成银

本书从多个使用场景深度探究了 Node.js 的能力，讲解了 Node.js 的体系、工具链和方法论。在 Node.js 发展迅猛、各种新生框架如雨后春笋般涌现之时，我们十分需要这样一本书。书中凝聚了狼叔在 Node.js 领域深耕多年的总结。通读全书后，相信读者一定能体会到 Node.js 更了不起之处。

TypeScript 布道者、Midway 框架作者，陈仲寅（张挺）

传说中的《狼叔（卷1）：更了不起的 Node.js》终于出版了。这本书凝聚了狼叔多年以来的技术心血，也填补了目前市面上没有一套大而全的“Node.js 红宝书”的缺憾，值得每一位 Node.js 开发者阅读，也让我更加期待后两卷的面世。

《Node.js 调试指南》作者，赵坤（nswbmw）

这本《狼书（卷1）：更了不起的 Node.js》涵盖 Node.js 方方面面的内容，更详细解读了 Node.js 的应用场景。读者可以通过这本书了解 Node.js 的“前世今生”，将 Node.js 的精髓融会贯通。无论是初学者还是全栈工程师，都可以从这本书中获益。在这本书中，狼叔将带你进入更宽广的 Node.js 世界，照亮你的 Node.js 学习道路！

GMTC（全球大前端技术大会）主编，孟夕

这本书是狼叔花了三年多时间打造的，书中融入了他丰富的开发经验和实践技巧，可以指导你深入研究 Node.js，探索其中的奥秘，助你成为 JavaScript 全栈工程师。无论你是刚开启前端之旅的“小白”，还是有经验的高级工程师，都能从本书中获得经验和启发。

《前端架构：从入门到微前端》作者，黄峰达（Phodal）

《狼书（卷1）：更了不起的 Node.js》真的来了，让我们通过这本书跟狼叔一起“进化”吧！不管是新人，还是资深程序员，都能从本书中获得构建完善的 Node.js 知识体系的能力，让自己真正“刚”起来！

Trek.js 作者，fundon

狼叔亲历了 Node.js 在国内的兴起、发展和成熟，并将他眼中的 Node.js 的核心知识完整融入本书。本书深入浅出地介绍了什么是 Node.js、如何学习 Node.js，以及 Node.js 的核心原理和独有特性，非常适合各个阶段的前后端工程师阅读、学习，从而构建出更了不起的 Node.js 应用。

新浪移动前端技术专家、Daruk 框架作者，付强（小燊）

作为同时在两地推动 NodeParty 线下聚会的同仁和网友，我时常被狼叔对社区投入的热情所感染。狼叔的技术能力和技术视野是毋庸置疑的。现在看到狼叔在教育领域又有进步，我不禁感慨，希望大家不负狼叔多年的付出，从书中吸取精华内容，快速成长，成为社区建设的中坚力量，一起推动 Node.js 的未来发展！

NodeParty 开源基金会发起人、大搜车无线团队负责人，芋头

2015 年 10 月，我便知道狼叔在筹备一本关于 Node.js 的书，不禁满心期待。虽然等待了三年多，但看到这本书面世，依然惊喜。本书汇集了许多 Node.js 发展历程中的精彩故事，还涵盖了很多 Node.js 的核心技术观点。相信对于读者而言定是一场知识盛宴！奔跑吧，更了不起的 Node.js！

宋小菜前端负责人，Scott

Node.js 是我最喜欢的技术之一，因为它给 JavaScript 带来了无限可能。本书着重讲解了 Node.js 的应用场景、核心模块、运行原理等内容，能够带领你了解更全面、更了不起的 Node.js。如果你想提升自己的 JavaScript 编程能力，就从这本书开始吧！

iView 作者，梁灏

狼叔是国内知名的 Node.js 技术布道者和推广先驱，他将 Node.js 技术的精华提取出来并完全融入本书。这本书深入浅出，不仅解释了 Node.js 的技术细节，更教会你学习的方法，同时结合作者多年的实践心得和宝贵经验，可以让读者少走很多弯路，是一本真正的开发者之书。

思否开发者社区 CTO，祁宁

我曾与狼叔探讨过关于“业界对 Node.js 存在争议”的问题，当时狼叔展现出的那种破除前后端开发分工隔阂的大局观，以及以业务需求为导向去解决实际问题的思维方式，让我十分佩服。这本书是狼叔的心血结晶，相信大家都能从中获得技术提升，扩展自己的视野和格局。

天津谷歌开发者社区核心组织者，朱峰

序

本书从 2015 年 10 月开始写作。

在那之前，我还在天津创业，顶着 CTO 的头衔干着各种最基础的编码工作。由于公司在天津的位置很偏僻，所以公司招人成了一个大问题。更要命的是，创始人没有工资可拿，现在想想只能说是情怀在支撑我吧。

公司招人不便，那就只能想办法把人才从北上广拉到天津，于是就动了扩大技术影响力的心思——我开始在 CNode 社区上发帖，后面慢慢尝试做“Node 全栈”公众号，效果还不错。我还记得 CNode 社区管理员、知名 Node.js 开发者 alsotang 曾评论过我的一篇文章，说我是 Node.js 布道者。当时我臭美了很久，之后便自然而然地走上了布道之路。

2015 年，我结婚了，财权上交，发觉生活窘迫，又不好意思向老婆要钱，于是便开始在网上教授 VSCode，之后我又和极客邦旗下的 StuQ 合作课程，获得收入的同时又可以进一步扩大技术影响力。而技术影响力扩大的体现就是，我被出版社的编辑发现了。由于早有布道的心思，自然希望能够出一本书，于是便开始了写书之旅。

可是写书从来都不是一件容易的事。阅历浅，写不来；无恒心，写不来。从 2015 年 10 月到 2019 年 2 月，历时三年多，很多朋友催书，以至于我经常在演讲中“自黑”：“我的书从 Node.js v4 写到 Node.js 8，然而还没有写完。”出版社约稿时，Node.js 才刚刚发布 4.0 版本，而 2019 年年初，Node.js 已经发布了 11.10 版本。本书几经修改，最终确定以 Node.js 的 8.0 版本为核心版本。虽然后面 Node.js 更新的版本里又有新功能，但整体来看 Node.js 的 API 设计得非常好，几乎都是向后兼容的，所以即使是 11.10 版本，和 8.0 版本的差别也不大，而且在本书的编辑过程中我又进行了一定的更新，因此不会影响读者阅读和学习。

在这三年多的时间里，Node.js 稳定高效地发布了多个版本，国内外的 Node.js 使用率也渐渐达到了一个前所未有的高度。感谢前端领域的爆发式增长，这极大扩展了 Node.js 的应用场景，而且新语法、新特性的使用也开始成为大前端开发团队中的标配。

人生之美好就是在苦难之后能够获得成果。写书的过程是痛苦的，但也让我对于“成就别人，才能成就自己”这句话有了更深刻的认识。最开始写书是为了布道，希望更多人能从中受益，没想到最先受益的是自己。通过长时间的积累，我完善了自己的知识体系，受益匪浅。通过与 CNode 社区、出版社的编辑以及 Node.js 爱好者们之间的交流，我有了更好的学习机会。通过写书、演讲、组织社区活动，我有了更丰富的人生经历。

以前见到图书的前言中总有致谢话语，还以为只是出版“套路”。然而今时今日，历经三年多的写作，我确实要感谢很多人。

感谢我的家人，写书会牺牲很多陪伴家人的时间，感谢他们的理解和支持。最难过的是周一到周五，只能看老婆通过微信发来的宝宝的视频，一遍一遍地看，一遍一遍地想哭。

感谢所有推荐本书以及为本书进行技术审校的专家们，若没有他们的帮助，这本书恐怕无法以最佳状态与各位读者见面。他们的宝贵建议使得本书的内容不至于空洞，也让我受益良多。

感谢博文视点的张春雨编辑和孙奇俏编辑，他们一次次地叮嘱我、鼓励我，面对面指导我如何规范写作，这种耐心和包容是极其难得的。这本书在审校初期，有 6 位出版社的编辑都参与其中。那时我是崩溃的——感觉自己数学不好，常常上面说 3 项下面列 4 项；语文也不好，连基本的语句都表达不清，很符合那句玩笑话：“你的语文是体育老师教的吧”。我能够想象编辑们在修改书稿之时是多么的“痛苦”，因此再次感谢各位编辑，感谢他们的辛苦付出，因为他们，本书才能够顺利出版。

回想这三年多的写作过程，其实几次都想放弃，想将 Node.js 系统地讲明白，真的不是一件容易的事。可是话都说出去了，不想让一直以来支持我的读者失望，更不能自己“打脸”，所以，这本书最终还是跟大家见面了。感谢各位粉丝在各个技术群里“花式”催书，感谢他们对我的鞭策。

再次感谢所有的小伙伴们。

所有未见面的读者，但愿狼叔的“碎碎念”，能够带你们打开 Node.js 世界的大门，领略大前端领域璀璨的星光。

狼叔

2019 年 4 月于北京

前言

Node.js 诞生于 2009 年，是由 Joyent 公司的员工 Ryan Dahl 开发完成的，之后 Joyent 公司一直扮演着 Node.js 孵化者的角色。由于诸多原因，Ryan 于 2012 年离开了 Node.js 社区，随后在 2015 年，由于 Node.js 的贡献者们在 ES6 新特性集成问题上产生意见分歧，因此分裂出 io.js。

io.js 的分裂最终促成了 2015 年 Node.js 基金会的成立，同年 Node.js v4.0 顺利发布。Node.js 基金会的创始成员包括 Google、Joyent、IBM、PayPal、Microsoft、Fidelity 和 Linux 基金会，这些创始成员将共同掌管过去由 Joyent 一家企业掌控的 Node.js 开源项目。此后，Node.js 基金会发展得非常好，稳定地发布了 5.x、6.x、7.x、8.x、9.x、10.x、11.x 等多个版本，截止到本书完稿之时，最新版本已经是 v11.14，最新的长期支持（LTS）版本是 v10.15。

Node.js 不是一门语言也不是一个框架，它是基于 Chrome V8 引擎的 JavaScript 运行时环境，同时结合 libuv 扩展了 JavaScript 功能，使得 JavaScript 能够支持浏览器 DOM 操作，同时具有只有后端语言才有的 I/O、文件读写与操作数据库等能力，是目前使用最简单的全栈式环境。

本书内容

从整体上来说，本书以 Node.js 为主，首先介绍了 Node.js 的发展历史，然后简要概括了 Node.js 的特点和使用场景，之后讲解了 Node.js 实现过程中的新增内容（如语法、模块、单进程等）的基本用法。读者入门 Node.js 之后，可以继续从本书中了解 Node.js 的执行原理，深入解读源码。最后，本书还讲解了 Node.js 中最核心、最复杂的异步流程控制，对未来异步流程的发展方向进行了展望。

本书共分 7 章，每章的内容简介如下。

第 1 章 Node.js 初识

本章介绍了 Node.js 的一些基础知识，包括什么是 Node.js、Node.js 和 JavaScript 的关系、

Node.js 的特点和应用场景等。

第 2 章 Node.js 安装与入门

本章介绍了 Node.js 安装与使用的基本方法, 包括 3m (即 nvm、nrm、npm) 安装法、Node.js 基础示例, 以及编辑器和调试等内容。

第 3 章 更了不起的 Node.js

本章更加详细地介绍了 Node.js 的各类应用场景, 对 Node.js 的核心作用进行了概括与总结, 还对如何成为全栈工程师提供了宝贵建议。

第 4 章 更好的 Node.js

本章介绍了 Node.js 的各种写法, 包括单线程与集群, 以及最新提出的各种最佳实践, 包括 ES 语法、多模块管理器 Lerna、npm 的替代品 Yarn 等。

第 5 章 Node.js 是如何执行的

本章介绍了 Node.js 的源码构建和调试过程, 阐述了 Node.js 是如何执行的, 还介绍了 API 的调用过程, 以及事件循环机制。

第 6 章 模块与核心

本章介绍了 Node.js 中的 CommonJS 规范、SDK 模块与核心技术, 还对未来的 ES6 模块功能进行了预测与展望。

第 7 章 异步写法与流程控制

本章介绍了异步流程控制的演进过程、Node.js 的核心异步写法, 以及更好的异步流程控制机制, 如 Thunk、Promise、async 函数等。

本书中的各章在内容上基本是相互独立的, 因此各位读者可以挑选自己感兴趣的章节阅读。这本书是《狼书》系列的第 1 卷, 还有第 2 卷和第 3 卷稍后会和各位读者见面, 内容涉及 Web 应用和性能优化等, 搭配阅读, 效果更好。

目标读者

本书的目标读者有以下三类。

- 正在学习 JavaScript 开发，对 JavaScript 语言有基本的了解和熟悉度，且希望能够了解 JavaScript 发展情况的人。
- 正从事 JavaScript 开发相关工作，熟悉 JavaScript 的基本开发要领，在日常工作中经常接触 Node.js，想要深入了解 Web 应用、BFF、API 代理等内容，以进一步提升自我的 Web 工程师（此处不区分前端与后端）。
- 具有极客精神，想要深入研究 JavaScript 语言及 Node.js 的全栈工程师。

同时，本书也适合正使用其他编程语言（如 Go、PHP、Python、Ruby、Java 等）进行 Web 开发的工程师阅读、学习。

阅读准备

要想运行本书中的示例，需要安装以下系统及软件。

- 操作系统：推荐 Linux，以及 macOS X 10.9 或以上版本，使用 Windows 操作系统可能会报错。
- 浏览器：Google Chrome、Safari、Firefox、Internet Explorer 11、Windows Edge。
- 运行环境：以 Node.js 8.6 版本为主。

联系作者

由衷地感谢你购买此书，希望你会喜欢它，也希望它能够为你带来你希望获得的知识。虽然我已经非常细心地检查了书中的所有内容，但仍有可能存在疏漏。若你在阅读过程中发现错误，在此我先表示歉意。同时欢迎你对本书的内容和相关源代码发表意见和评论。你可以通过我的私人邮箱 i5ting@126.com 与我取得联系，我会一一解答你的疑惑。

我的更多联系方式如下，大家可通过任意方式与我联系。

- 个人主页：<http://i5ting.com>
- GitHub：<https://github.com/i5ting>
- Twitter：<https://twitter.com/i5ting>

最后送给各位读者一句话，也是狼叔常说的——少抱怨，多思考，未来更美好！

读者服务

轻松注册成为博文视点社区用户 (www.broadview.com.cn), 扫码直达本书页面。

- **提交勘误:** 您对书中内容的修改意见可在 [提交勘误](#) 处提交, 若被采纳, 将获赠博文视点社区积分 (在您购买电子书时, 积分可用来抵扣相应金额)。
- **交流互动:** 在页面下方 [读者评论](#) 处留下您的疑问或观点, 与我们和其他读者一同学习交流。

页面入口: <http://www.broadview.com.cn/35907>



目录

第 1 章 Node.js 初识.....	1
1.1 引子.....	1
1.2 JavaScript	7
1.3 什么是 Node.js.....	9
1.3.1 Node.js 概述.....	9
1.3.2 Node.js 的特点.....	12
1.3.3 Node.js 的应用场景.....	16
1.4 本章小结.....	18
第 2 章 Node.js 安装与入门.....	19
2.1 安装 Node.js.....	19
2.1.1 3m 安装法	19
2.1.2 nvm.....	20
2.1.3 npm.....	26
2.1.4 nrm.....	32
2.1.5 从源码进行编译	35
2.1.6 状态理论	35
2.2 Hello Node.js!	36
2.2.1 Hello World.....	36
2.2.2 Hello CommonJS.....	37
2.2.3 Hello HTTP	38
2.3 编辑器与调试.....	41
2.3.1 IDE/编辑器.....	41
2.3.2 VSCode.....	42

2.3.3 调试	45
2.4 本章小结	52
第 3 章 更了不起的 Node.js	53
3.1 架构升级	53
3.1.1 从 LAMP 到 MEAN	54
3.1.2 前后端分离	55
3.1.3 页面即服务	58
3.1.4 场景决定选型	59
3.2 贯穿开发全过程	60
3.2.1 静态 API	60
3.2.2 现代 Web 开发	63
3.2.3 后端开发	68
3.3 更多乐趣	78
3.3.1 更多应用场景	78
3.3.2 C/C++ 扩展	79
3.3.3 团队优化	80
3.3.4 全栈之路	81
3.4 本章小结	85
第 4 章 更好的 Node.js	86
4.1 选择	86
4.1.1 语法可难可易	86
4.1.2 开发大型软件	90
4.1.3 特定场景下的快速开发	91
4.2 单线程会“死”吗	92
4.2.1 uncaughtException	93
4.2.2 异常捕获	94
4.2.3 forever	95
4.2.4 小集群: 单台服务器上多个实例	95
4.2.5 大集群: 多台机器	96
4.3 为 Node.js 正名	98
4.3.1 版本帝?	98

4.3.2	已无性能优势？	99
4.3.3	异步和回调地狱	100
4.3.4	技术栈演进	101
4.4	更好的实践	102
4.4.1	ES.next	102
4.4.2	类型系统	110
4.4.3	更好的 npm 替代品——Yarn	111
4.4.4	多模块管理器 Lerna	113
4.5	本章小结	114
第 5 章	Node.js 是如何执行的	115
5.1	准备	115
5.1.1	编辑器	116
5.1.2	编译	117
5.1.3	调试	118
5.2	编译步骤	120
5.2.1	configure	120
5.2.2	make	130
5.2.3	make install	132
5.3	从入口开始	135
5.3.1	核心流程	137
5.3.2	构造 process 对象	139
5.3.3	LoadEnvironment	147
5.3.4	bootstrap_node.js	148
5.3.5	EventLoop 启动方法	160
5.4	API 调用过程	162
5.4.1	相关的引用	163
5.4.2	FSReqWrap	163
5.4.3	核心 open 方法	164
5.4.4	src/node_file.cc	164
5.5	事件循环机制	167
5.5.1	概览	167
5.5.2	生命周期	169

5.5.3	microtask 和 macrotask	170
5.5.4	process.nextTick(callback)	173
5.6	本章小结	175
第 6 章	模块与核心	176
6.1	CommonJS 规范	176
6.1.1	简介	176
6.1.2	核心技术	181
6.2	Node.js 模块	189
6.2.1	从源码分析实现原理	189
6.2.2	从 Node.js 代码执行开始	191
6.2.3	深入理解模块	195
6.2.4	全局对象	205
6.2.5	Node.js 模块详解	215
6.3	未来展望: ES 模块	220
6.3.1	ES 模块入门	221
6.3.2	模块导入	222
6.3.3	模块导出	222
6.3.4	ES 模块示例	223
6.3.5	兼容性更好的@std/esm	224
6.4	本章小结	224
第 7 章	异步写法与流程控制	225
7.1	异步调用	226
7.1.1	异步与同步	226
7.1.2	浏览器中的异步	227
7.1.3	Node.js 异步原理	227
7.1.4	API 和示例	229
7.1.5	代码优化	231
7.2	Node.js 自带的异步写法	236
7.2.1	错误优先的回调方式	236
7.2.2	EventEmitter	240
7.2.3	该选择哪种风格的写法	247

7.3 更好的异步流程控制	248
7.3.1 回调地狱	248
7.3.2 Thunk	252
7.3.3 Promise	254
7.3.4 Generator	276
7.3.5 async 函数	282
7.4 本章小结	287

第 1 章

Node.js 初识

Node.js 从 2009 年诞生开始，到现在已有 10 岁，在这 10 年里，它的成长和成熟是大家有目共睹的。它因后端简化并发编程而被关注，因作为前端辅助开发工具而流行，因异步流程控制和回调地狱而被人诟病，因 npm 批量安装模块而被人敬仰。无论之前你是否接触过 Node.js，它都值得你去了解，下面就跟我一起来认识一下 Node.js 吧！

1.1 引子

我有一位好朋友有 10 余年的 Java 开发经验，对 jQuery 和 Angular 掌握得都还不错，其公司架构升级，需要将开发流程打通，当中重要的一环就是“API 文档化”，他采用的技术选型是 Swagger UI，一个 API 在线文档生成和测试利器。他遇到一个问题，当时的 Swagger UI 版本是 3.1，是基于他不熟悉的 Babel、React、Webpack 技术栈开发的。面对前端组件化，以及更新频繁且十分复杂的技术栈，他不知道该如何下手，于是向我提出了一个极小的需求，希望通过修改 UI 里的头部信息来了解当前最“酷”的前端开发过程。

如果对 Node.js 项目熟悉，这个问题是很容易解决的。首先打开项目根目录下的 package.json，它是 Node.js 模块的核心配置文件；查看 dependencies 可以知道当前项目在产品环境下使用了哪些 Node.js 模块，便于我们对当前项目使用的技术栈有一定了解。下面的代码是 Swagger UI 中的核心模块，该模块将 Less 用作 CSS 预处理器，将 React 用作前端组件化框架。

```
"dependencies": {  
  ...  
  "less": "2.7.1",  
  ...  
  "react": "^15.4.0",  
  ...  
},
```

查看 `devDependencies` 可以知道当前项目在开发环境下需要依赖的模块。为了使用最新的 ECMAScript 语法（后文中简称为 ES6），需要使用 Babel 编译工具，同时使用 ESLint 来检查代码质量，使用 Standard 来统一编码风格，使用 Webpack 作为打包器。最后要说的是，`autoprefixer` 模块为了兼容所有浏览器，需要通过 CSS 属性为不同的浏览器加上对应的前缀。在写代码的时候可以不加，但在最后编译的时候会由 `autoprefixer` 自动加上，这是提高前端开发效率的常见手段。

```
"devDependencies": {
  "autoprefixer": "7.1.1",
  "babel-core": "^6.23.1",
  ...
  "eslint": "^4.1.1",
  ...
  "standard": "^10.0.2",
  "webpack": "^2.6.1",
  ...
},
```

那么，为什么要区分开发环境和产品环境呢？让我们在前端的发展过程中求解吧！图 1-1 展示了前端技术的发展过程。自 2014 年 Backbone 框架中引入了 MVC 开始，前端领域中逐渐出现了各种概念，例如 MVVM、Router、IoC、Virtual Dom 等，再加上 Angular、React、Vue 等大量框架井喷式地出现，导致前端对于传统的 HTML、CSS、JavaScript 写法产生了“反叛”情绪，期望找到更好、更高效的开发方式。在这种情况下，Node.js 无疑是前端开发的最佳搭档，它能编写大量辅助前端开发的模块，上面所列举的都是常用的模块。

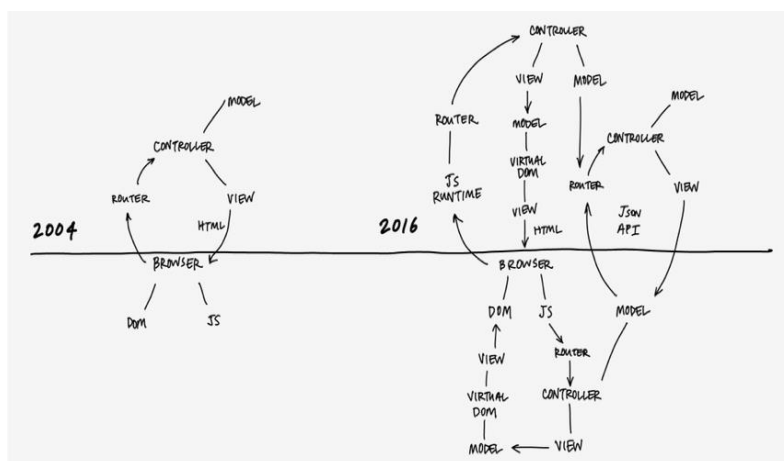


图 1-1

图片来源：<https://twitter.com/sstephenson/status/730039913052176384>

现在的前端开发方式早已不是将代码直接放在浏览器里运行了，而是采用更好的写法。经过转译处理，最终输出可在浏览器中运行的代码，如图 1-2 所示。比如将 ES6 语法降级为 ES5，将 Less 编译为 CSS，将 React 组件通过 React 引擎渲染为 HTML 等，这些还是你熟悉的前端开发方式吗？

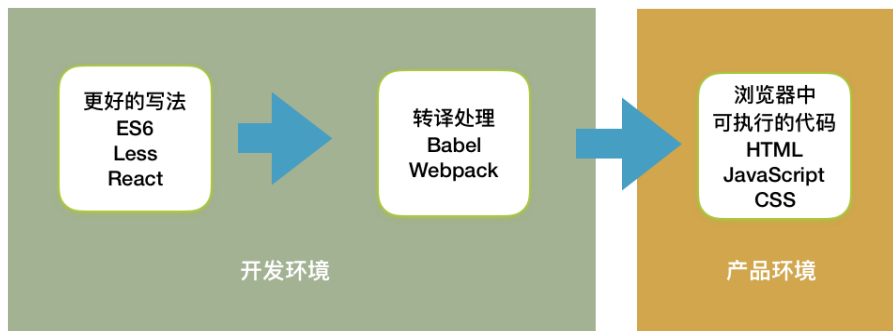


图 1-2

下面让我们来看一下如何进行开发。一般开发以终端命令行 CLI 的方式来实现，最常用的就是使用 npm scripts 来实现，具体定义如下。

```
"scripts": {
  "build": "npm run build-core && npm run build-bundle && npm run build-standalone",
  "build-bundle": "webpack --config webpack-dist-bundle.config.js --colors",
  "build-core": "webpack --config webpack-dist.config.js --colors",
  "build-standalone": "webpack --config webpack-dist-standalone.config.js --colors",
  "predev": "npm install",
  "dev": "npm-run-all --parallel hot-server open-localhost",
  "watch": "webpack --config webpack-watch.config.js --watch --progress",
  "open-localhost": "node -e 'require(\"open\")\"(\"http://localhost:3200\")'",
  "hot-server": "webpack-dev-server --host 0.0.0.0 --config webpack-hot-dev-server.config.js --inline --hot --progress --content-base dev-helpers/",
  ...
},
```

其中核心的两个命令如下。

- `npm run build`: 打包，生成最终可在浏览器中运行的代码。
- `npm run dev`: 利用 Node.js 编写的模块辅助开发命令，用于本地开发，不会产生最终文件。

这里我们以本地开发为例。在终端执行 `npm run dev` 命令后会在默认的浏览器中打开如图 1-3 所示的页面，我们的需求是将头部的 `swagger` 改为 `koa`。

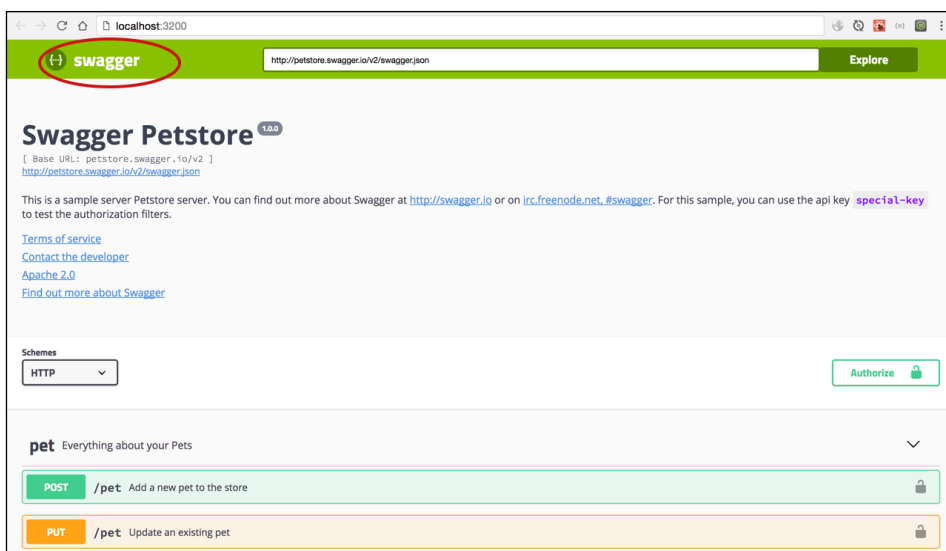


图 1-3

首先要找到入口，当然这并不容易，需要通过启动脚本一个一个进行推导。

`npm run dev` 是通过 `npm-run-all` 来并行执行 `hot-server` 和 `open-localhost` 的。

`npm-run-all` 是一个用于并行或串行执行 `npm scripts` 的命令行辅助工具，它本身也作为依赖模块被安装到了开发环境里。它执行的 `open-localhost` 很有意思，是通过 `node -e 'require("open")( "http://localhost:3200" )'` 实现的，很少有这样的用法，但确实简单直接。

`hot-server` 是通过 `webpack-dev-server` 实现的开发环境服务，其核心配置是 `webpack-hot-dev-server.config.js`，另外还有一个非常重要的配置项 `--content-base dev-helpers/`，表示指定静态服务器的托管目录，即 `http://localhost:3200/` 页面对应的就是 `dev-helpers/index.html`。

```
<div id="swagger-ui"></div>

<script src="./swagger-ui-bundle.js"> </script>
<script src="./swagger-ui-standalone-preset.js"> </script>
<script>
  window.onload = function() {
    window["SwaggerUIBundle"] = window["swagger-ui-bundle"]
    window["SwaggerUIStandalonePreset"] = window["swagger-ui-standalone-preset"]
    // 创建一个系统
    const ui = SwaggerUIBundle({
      url: "http://petstore.swagger.io/v2/swagger.json",
      dom_id: '#swagger-ui',
```

```
    presets: [
      SwaggerUIBundle.presets.apis,
      SwaggerUIStandalonePreset
    ],
    plugins: [
      SwaggerUIBundle.plugins.DownloadUrl
    ],
    layout: "StandaloneLayout"
  })
  ...
}
```

</script>

以上代码中的要点如下。

- 核心是 SwaggerUIBundle。
- SwaggerUIBundle 的布局是 StandaloneLayout。
- SwaggerUIStandalonePreset 只是 SwaggerUIBundle 的预置扩展。

webpack-hot-dev-server.config.js 文件是 Webpack 配置，是我们需要掌握的最流行的 Node.js 模块。在 Webpack 配置文件里，entry 是入口，具体的配置如下。

```
module.exports = require("./make-webpack-config") (rules, {
  ...
  entry: {
    "swagger-ui-bundle": [
      "./src/polyfills",
      "./src/core/index.js"
    ],
    "swagger-ui-standalone-preset": [
      "./src/style/main.scss",
      "./src/polyfills",
      "./src/standalone/index.js",
    ]
  },
  ...
})
```

在 dev-helpers/index.html 中引用的 swagger-ui-bundle.js 文件和 wagger-ui-standalone-preset.js 文件就是这样来的，由入口配置多个文件，最终打包成一个可供浏览器使用的文件。该文件在开发阶段由 webpack-dev-server 提供，在产品阶段通过 Webpack 打包输出到 dist 目录中，进而提供给页面使用。

这里推断核心模块 SwaggerUIBundle 的入口为 ./src/core/index.js，原因很简单，模块配置的

入口只有两个文件，利用常识判断，便知道 polyfills 是用于实现浏览器并不支持的原生 API 的代码，所谓的补丁肯定不会是入口。

其实完整阅读 src 目录下的代码才发现，最后代码存在于 src/standalone/layout.jsx 文件的 StandaloneLayout 布局中的 Topbar 组件内。

```
export default class StandaloneLayout extends React.Component {
  ...
  render() {
    ...
    const Topbar = getComponent("Topbar", true)
    ...

    return (

      <Container className='swagger-ui'>
        { Topbar ? <Topbar /> : null }
        ...
      </Container>
    )
  }
}
```

Topbar 组件位于 src/plugins/topbar 目录下，我们要更改的是显示的内容，所以直接研究模板 topbar.jsx，将其中的 `<span>swagger</span>` 改为 `<span>koa</span>` 即可实现需求，代码如下。

```
export default class Topbar extends React.Component {
  ...
  render() {
    ...
    return (
      <div className="topbar">
        <div className="wrapper">
          <div className="topbar-wrapper">
            <Link href="#" title="Swagger UX">
              <img height="30" width="30" src={ Logo } alt="Swagger UI"/>
              <span>swagger</span>
            </Link>
            <form className="download-url-wrapper" onSubmit={ formOnSubmit }>
              {control}
            </form>
          </div>
        </div>
      </div>
    )
  }
}
```

```
}  
}
```

大家可以想一想，对于这样一个修改字符串的简单需求，为什么我们要占用如此大的篇幅来讲呢？在整个修改过程中，我们看到了前端开发颠覆性的变化，无论是开发方式，还是构建过程、组件写法，这种典型的多技术栈组合的开发方式难度是相当大的。从另外一个角度来看，开发过程中会大量应用 Node.js 模块，如果对 Node.js 开发足够熟悉，学习前端开发时基本上只需要付出学习“纯前端”技术的时间成本，如果不熟悉 Node.js，那么学习前端开发还要同时学习 Node.js 开发。如果换一门开发语言，比如使用 Go 语言呢？那么你要学的便是 3 门技能：Node.js 开发、前端技术和 Go 语言编程。前端技术尚且学不过来，你真的有精力学习其他知识吗？

当然，Node.js 不只是前端辅助开发工具，它还可以实现更多功能，下面就跟我一起开启更了不起的 Node.js 之旅吧！

1.2 JavaScript

基于万维网之父 Tim Berners-Lee 提出的最小功效原则——选择最合适的解决方案而不是最强的，语言的功效越弱，对于储存在该语言中的数据，我们能做的事情就越多，Jeff Atwood 在 2007 年提出了下面这条著名的 Atwood 定律。

任何能够用 JavaScript 实现的应用系统，最终都必将用 JavaScript 实现。

语言的语法结构越简单，进行数据提取和分析就越容易，用来开发互联网应用也越理想。如果对象创建和线程管理都不是必要的机制，那么作为一种基于原型而不是面向对象的语言，JavaScript 就是完美的：它虽然没有类的概念，但所有内容都是对象，无须创建过程就能使用，而且它是单线程的。同时，JavaScript 也面向后端，即 Node.js，明显简化了并发编程，可以轻松访问各种数据库，构建 Web 应用、API 与微服务，让 Web 开发者实现使用 JavaScript 开发前后端的全栈理想。

JavaScript 诞生于 1995 年，其第一版仅 10 天就被编写完成，起初它的主要目的是处理以前由服务器端负责处理的一些表单验证。第一版 JavaScript 的名称是 LiveScript，为了赶在发布日期前完成 LiveScript 的开发，当年 Netscape 公司与 Sun 公司还特意成立了一个开发联盟。Netscape 公司为了搭上“媒体热炒 Java”的顺风车，临时将 LiveScript 更名为 JavaScript，所以从本质上来说，JavaScript 和 Java 并没有什么关系。

图 1-4 展示了 JavaScript 语言的演进过程（本图作者为 justjavac）。

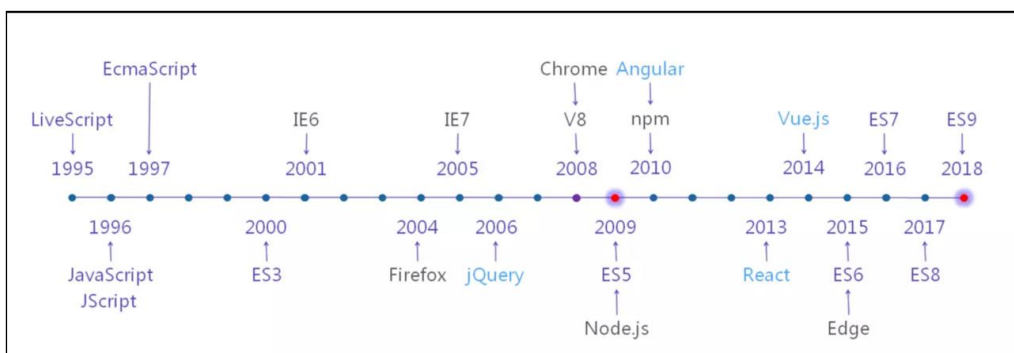


图 1-4

JavaScript 1.0 获得了巨大的成功，1997 年，以 JavaScript 1.1 为蓝本的建议被提交给了欧洲计算机制造商协会（European Computer Manufactures Association，ECMA），该协会指定 39 号技术委员会（TC39）负责将其标准化，TC39 由关注脚本语言发展的各大公司的程序员组成，他们的工作是定义 ECMAScript 语言的标准。第二年，国标标准化组织和国际电工委员会（ISO/IEC）也采用了 ECMAScript 作为标准，即 ISO/IEC-16262。之后的几年又陆续发布了几个 ECMAScript 版本，但影响都不大，直至 2009 年 ECMAScript 5（简称 ES5）语言标准发布，该标准才逐渐被大家熟知。

同样在 2009 年，Node.js 横空出世，开启了 JavaScript 用于后端领域编程的新时代！2015 年，ECMAScript 6（简称 ES6）语言标准发布，Node.js 也从 4.x 版本开始支持 ES6 特性，目前 Node.js 6.x 以上的版本几乎可以支持 ES6 中 90% 以上的特性。之后的每年，TC39 都会发布一个新版本的 ECMAScript 语言标准，但标准发布与实现还要相隔很长时间，大家如果有精力就提前学习一下，掌握技术趋势。

下面展示一些对 JavaScript 有正面影响的其他统计数据，可以说，JavaScript 实现了前后端语法统一，让全栈开发变得更容易。

- 在 GitHub 上，JavaScript 开源项目的数量最多，几乎是 Java 开源项目数量的 1.5 倍。
- 在“StackOverflow 2017 年开发者调查报告”中，Node.js 被评为最受欢迎的开发平台。
- JavaScript 是 StackOverflow 中最流行的编程语言。

1.3 什么是 Node.js

1.3.1 Node.js 概述

Node.js 官方网站中的描述如下。

Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient. Node.js' package ecosystem, npm, is the largest ecosystem of open source libraries in the world.

在这段介绍中，需要解读的要点如下。

- Node.js 不是 JavaScript 应用，不是编程语言（JavaScript 是编程语言），不是像 Rails（基于 Ruby）、Laravel（基于 PHP）或 Django（基于 Python）一样的框架，也不是像 Nginx 一样的 Web 服务器。Node.js 是 JavaScript 的运行时环境。
- Node.js 构建在 Chrome V8 这个著名的 JavaScript 引擎之上，Chrome V8 引擎是通过 C 或 C++编写的，相当于要将 JavaScript 转换为底层的 C 或 C++代码后再执行，通过 JavaScript 这层包装，大大降低了偏底层的编程语言的学习成本。
- Node.js 是轻量且高效的，每个函数都是同步的，而 I/O 操作是异步的。所有由 JavaScript 编写的函数的 I/O 操作最终都将由 libuv（由 C/C++ 编写）事件循环处理库来处理，隐藏了非阻塞 I/O 的具体细节，简化了并发编程模型，可以轻松编写高性能的 Web 应用。
- Node.js 使用 npm 作为包管理器，目前 npm 是开源库里最大的包管理生态，功能强大，截至 2019 年 3 月，模块数量已经超过了 90 万。

大多数人认为，Node.js 只能用于编写网站后台或者前端工具，这其实是不全面的，Node.js 的目标是让并发编程更简单，主要应用在以网络编程为主的 I/O 密集型应用中。它是开源的，跨平台的，并且是高效的（尤其是在 I/O 处理方面），IBM、Microsoft、Yahoo、SAP、PayPal、WalMart 以及 GoDaddy 都是 Node.js 的用户。

图 1-5 源自 Node.js 之父 Ryan Dahl 的演讲稿，展示了 Node.js 早期的架构模式，但今天依然不过时。Node.js 是基于 Chrome V8 引擎构建的，由事件循环（Event Loop）分发 I/O 任务，最终工作线程（Work Thread）会将任务放到线程池（Thread Pool）中执行，而事件循环只要等

待执行结果就可以了。

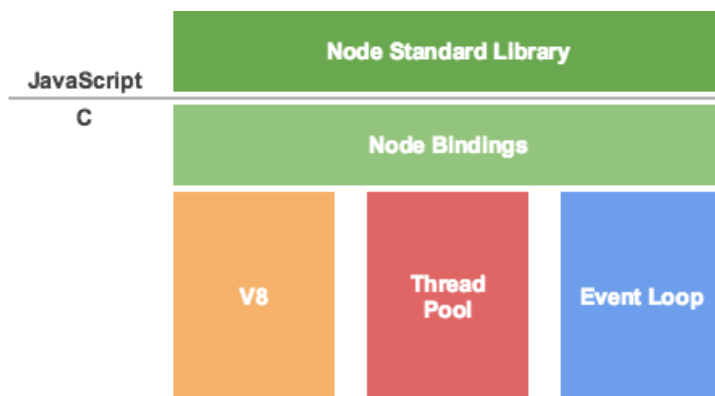


图 1-5

下面带领大家梳理一下上面介绍的内容。

- **Chrome V8 引擎：**Google 发布的开源 JavaScript 引擎，采用 C/C++ 编写，在 Google 的 Chrome 浏览器中被使用。Chrome V8 引擎可以独立运行，也可以嵌入 C/C++ 应用程序中被执行。
- Node.js 内置 Chrome V8 引擎，所以使用的是 JavaScript 语法。
- JavaScript 语言的一大特点就是单线程，即同一时间只能做一件事。单线程就意味着所有的任务都需要排队，前一个任务结束才会执行后一个任务。如果前一个任务耗时很长，后一个任务就不得不一直等待。
- 一般情况下，排队的时候 CPU 是闲着的。其实 CPU 完全可以不管 I/O 设备而直接挂起处于等待中的任务，先运行排在后面的任务。
- 将等待中的 I/O 任务放到事件循环中，事件循环由 libuv 提供。
- 事件循环负责将文件 I/O 任务放到线程池中，线程池由 libuv 提供。网络 I/O 任务不通过线程池完成。
- 只要有 CPU 资源，就应尽力执行。

我们再换一个维度来看一下 Node.js 的原理，如图 1-6 所示。

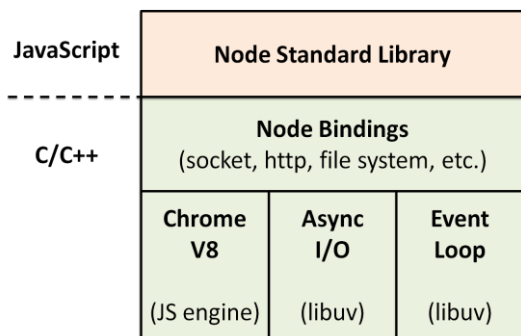


图 1-6

在图 1-6 中，Chrome V8 引擎解释并执行 JavaScript 代码（这就是浏览器能执行 JavaScript 的原因），libuv 由事件循环和线程池组成，负责所有 I/O 任务的分发与执行。

在解决并发问题方面，异步是最好的方案，我们可以类比排队和叫号机制来理解。在排队的时候，除了等待我们什么都做不了。但如果有叫号机制，我们就可以先取号码，等轮到自己的时候，系统会发出通知，这中间，我们可以做任何想做的事。

Node.js 其实就是帮我们构建了类似的机制。写代码的过程实际上就是取号的过程，由事件循环来接受处理，而真正执行操作的是具体的线程池中的 I/O 任务。之所以说 Node.js 是单线程的，是因为它在接受任务的时候是单线程的，无须切换进程/线程，非常高效，但它在执行具体任务的时候是多线程的。

Node.js 公开宣称的目标是“提供一种简单的构建可伸缩网络程序的方法”，毫无疑问，它确实做到了。这种做法将并发编程模型简化了，事件循环和具体线程池等细节被 Node.js 封装了，进而将异步调用 API 写法暴露给开发者。这种方式也是福祸相依的，一方面简化了并发编程，能让更多人轻而易举地写出高性能的程序，另一方面在写法上埋下了祸根。

Node.js Bindings 层做的事就是将 Chrome V8 引擎暴露的 C/C++ 接口转换成 JavaScript API，并且结合这些 API 编写 Node.js 标准库，所有这些 API 被统称为 Node.js SDK，后面关于模块的章节中会更详细地讨论。

Microsoft 在 2016 年宣布在 MIT 许可协议下开放 Chakra 引擎，并以 ChakraCore 为名在 GitHub 上开放了源代码，ChakraCore 是一个完整的 JavaScript 虚拟机，拥有着和 Chakra 几乎相同的功能与特性。Microsoft 向 Node.js 主分支提交了代码合并请求，让 Node.js 用上了 ChakraCore 引擎。实际上 Microsoft 是通过创建名为 Chakra Shim 的库赋予了 ChakraCore 处理 Google Chrome

V8 引擎指令的能力，其原理如图 1-7 所示。

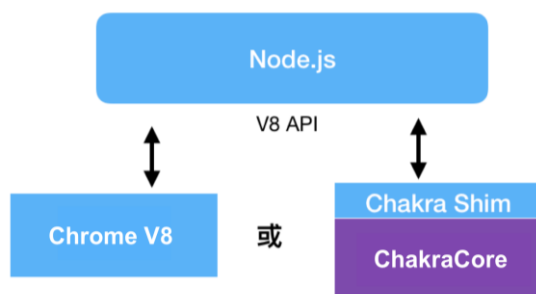


图 1-7

目前，Node.js 同时支持 ChakraCore 和 Chrome V8 这两种 JavaScript 引擎，二者在性能和特点上各有千秋，ChakraCore 给人感觉更“潮”一些，曾是第一个支持 `async` 函数的引擎，但目前 Node.js 还是以 Chrome V8 引擎为主，ChakraCore 版本需要单独安装，大家了解即可。

1.3.2 Node.js 的特点

上面的内容主要概括了 Node.js 的定位，总结成一句话——Node.js 是可扩展的适合用于构建高性能 Web 应用的最简单的解决方案。

这句话中有 4 个要点：适合构建 Web 应用、高性能、简单、可扩展，这 4 点也是 Node.js 的特点，下面具体介绍。

👉 适合构建 Web 应用

1. 构建网站

此处是指构建传统网站，不进行前后端分离，将视图渲染和数据库访问都放在同一个项目中。用 Node.js 做入门开发，和传统的 Java、PHP 开发没有什么区别。构建成功的应用是典型的单体式应用，CNode 社区就是基于 Node.js 编写的网站。

2. 构建 API

从 2010 年移动端开发变得十分火爆起，大前端领域便开始变得复杂，不再是单纯的 PC 端 Web 开发，也包含了 iOS、Android、HTML5 等多种客户端开发，这就导致了一个问题：为这些客户端提供可以复用的 API 接口变得更加困难。基于此，API 接口开发被提升到了一个前所

未有的重要程度。

移动端 API 与前端的 Ajax 调用、API 调用一样，一般以返回 JSON 或 XML 结构数据为主。API 的写法很多，最为推荐的写法是对返回的数据加壳，如下所示。

```
// 包装示例
{
  "status": {
    "code": 10000,
    "message": 'Success'
  },
  "response": {
    ...results...
  }
}
```

至于是选用 GitHub V3 式的纯 RESTful API 还是微博 API 式的自定义约定，按团队喜好即可。

对于用 Node.js 实现 API 接口开发，在技术选型上，无论是选择通用型的 Express、Koa 框架，还是选择专门为 API 开发编写的 Restify、Hapi 框架，都是极其简单的。一些大型工程的 API 相对复杂，因此在后端的 API 接口开发上封装一层专门供前端使用的 API Proxy 是非常有必要的。后端 API 接口开发用于数据库访问，将返回的数据进行包裹，以 HTTP 形式返回；API Proxy 针对前端使用的 API 进行优化，使前端开发更人性化。这是一个相当不错的架构升级，后面的章节会详细讲解。

3. 构建 RPC 服务

在微服务架构十分流行的今天，服务化已经是必经之路，其中使用最多的就是 RPC (Remote Procedure Call, 远程过程调用) 协议服务，常见的做法是将数据库访问返回的数据，以 TCP 形式传输给调用方。相对于 HTTP，RPC 服务采用的是 TCP，在协议和传输上有明显优势。著名的 RPC 服务有 Java 版本的 Dubbo、Google 出品的跨语言 RPC 库 gRPC 等。

前面讲过，Node.js 是非常适合用于网络应用开发的，其中 Socket 编程就是一种典型的网络应用开发场景，也就是说，Node.js 一样适合用于 Socket 编程，使用 Node.js 开发 RPC 服务是非常合适的。当然，Node.js 实现的 RPC 服务有很多种，例如纯粹用 Node.js 实现的 DNode、微服务工具集 Seneca、跨语言的 gRPC 客户端，可以看出 Node.js 在微服务架构下的应用场景也是非常多的。

- RPC 服务：数据库访问，将返回的数据进行包裹，以 TCP 形式传输给调用方。

- RPC 组装服务: 组装多个 RPC 服务, 完成某项业务。
- 将 RPC 服务包装成 HTTP API, 这也是 Node.js 擅长的。

4. 前后端分离

自 2014 年以来, 前端开发的受关注度爆增, React、Vue、Angular 框架三足鼎力, 每个月还会冒出新框架, 可谓风光一时无二。但是这也导致了一个问题: 对于大型项目而言, 如果不进行前后端分离, 很难满足当下的需求。

前后端分离的应用场景大致有如下 4 个。

- 前端页面静态化 (Page Static)
- 前端页面服务化 (PAAS, Page as Service)
- 服务端渲染 (SSR, Server Side Render)
- 渐进式 Web 应用 (PWA, Progressive Web App)

5. 适用于 Serverless

Node.js 横空出世之后马上受到各个云计算厂商的青睐, Joyent 本身也是云计算厂商, 说 Node.js 是为云计算而生的, 也未尝不可。

前端领域演进速度很快, Node.js 的应用场景也越来越多, 暴露出来的问题自然也非常多, 涉及运维、API 快速开发、服务端渲染等方面, 这些问题其实都可以通过 Serverless 架构来解决。开发者不需要关心运维、流量处理和容器编排, 通过一个函数 (内置 RPC、缓存、配置等 API) 就可以完成所有开发。可以简单理解为, Serverless 是云计算的延伸, Node.js 在这种应用层开发上有着得天独厚的优势。

有了 Serverless, 前端无须关心运维实现, 写一个函数就可以搞定 API、服务端渲染等, 大幅优化了开发方式, 可以想象得到, 依靠“纯”前端方法实现所有业务也是可行的: 无须关心运维 (Serverless 平台自带运维功能, 一般集成 Kubernetes 这种容器编排管理系统), 无须关心流量 (Serverless 平台自带服务网格功能, 比如 Istio, 根据流量可以快速实例化容器), 无须关心高并发 (API 层有缓存)。目前, Google 开源的 Knative 正在逐渐走向成熟, 是一个不错的技术选型。

👉 高性能

Node.js 的高性能具体是指以下几点。

- 执行速度快：Node.js 是构建在 Chrome V8 引擎之上的，执行速度可能是动态语言运行时环境里最快的。
- 天生异步：事件驱动和非阻塞 I/O 特性决定了 Node.js 必须采用异步机制，每个 I/O 任务都是异步的，因此集成到 libuv 的事件循环里才能让开发者代码对并发操作无感知。
- 适用于 I/O 密集的网络应用开发：网络应用开发（包括 Web 应用开发）的瓶颈在于 I/O 处理，而这恰恰是 Node.js 的强项。对于 CPU 密集型应用而言，能够使用其他语言开发最好使用其他语言，如果必须使用 Node.js，可以通过 C/C++ 扩展机制来实现。因为采用 Node.js 就认为所有功能都要使用 Node.js 实现，这是错误的，合理的采用其他技术栈，利用其优势部分，除了能够加快开发迭代的速度，对系统稳定性也是非常有帮助的。

狼叔常说：Node.js 苦了自己却让更多人能够轻松实现并发编程，并以最小的成本获得更高的性能。

👉 简单

总体来讲，Node.js 使用起来非常简单，具体如下。

- 语法简单：JavaScript 语法简单易学，对新手非常友好，对于有前端开发经验或有其他方面 JavaScript 编程经验的开发者而言，更加简单。
- 并发编程简单：有了事件驱动和非阻塞 I/O 机制，Node.js 可以使用非常少的资源处理非常多的连接和任务，处理低延时请求，完美应对实时及 I/O 密集型应用等高并发场景。
- 部署运维简单：无须使用额外的服务器软件，不像 Java 需要用到 Tomcat 之类的 JavaEE 容器，在分布式集群中，负载均衡、多核系统方面都有完善的配套设施，现有的各种自动化运维工具（如 Ansible、SaltStack 等）都可以直接使用。
- 开发简单：目前 Node.js 内置模块和 npm 上的模块都遵守“小而美”的设计哲学，相对比较简单，对开发、迭代、上线有明显帮助，不像 Java 里的三大框架包含那么多内容。当然，现在 Spring Boot 也开始设计 Java 版本的小而美框架了。

👉 可扩展

Node.js 还具备可扩展的特性, 具体如下。

- 可以使用 npm 上的大量模块。
- 可以通过编写 C/C++ 扩展实现 CPU 密集型任务。
- 可以轻松搭配 Java、Rust 等语言使用。
- 架构互补: 在架构上以业务边界来进行服务拆分, 外加各种“组合拳”, 可以让合适的轮子出现在合适的位置上, 比如 Java 在基础平台建设及大数据等领域有非常深厚的基础, 那么直接使用即可。

1.3.3 Node.js 的应用场景

Node.js in Action 一书中说: “Node.js 所针对的应用程序有一个专门的简称——DIRT, 表示数据密集型实时 (Data-Intensive Real-Time) 程序。因为 Node.js 自身在 I/O 上是非常轻量级的, 善于将数据从一个管道混排或代理到另一个管道中, 这在处理大量请求时会持有很多开放的连接, 并且只占用一小部分内存, 可以保证响应能力, 与浏览器一样。”

上面的话不假, 但在今天来看, DIRT 的设定范围还是偏小。随着大前端的发展, Node.js 早已不再是只与 I/O 处理相关的框架, 现在的 Node.js 已经有了更多的应用场景!

具体来说, Node.js 的应用场景主要分为以下 4 大类, 如图 1-8 所示。

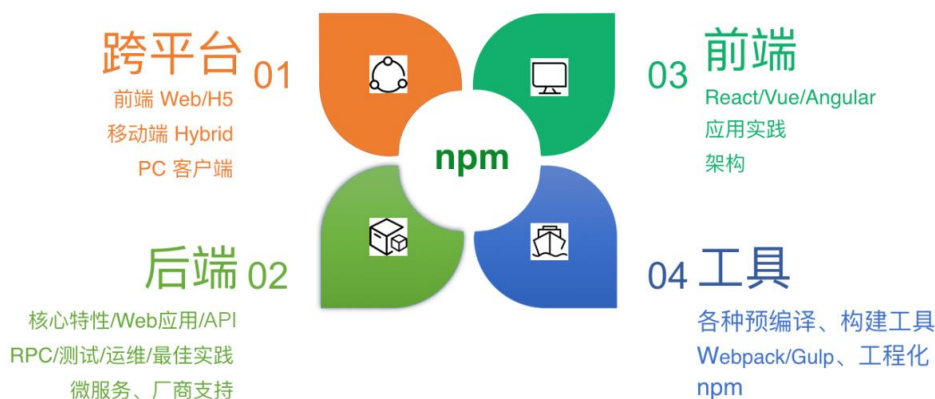


图 1-8

- 跨平台开发：几乎覆盖你能想到的面向用户的所有平台，传统的 PC Web 端、移动端、HTML5、React Native、Weex，以及硬件 Ruff.io 等
- 后端开发：面向网站、API、RPC 服务等。
- 前端开发：三大框架 React、Vue、Angular 辅助开发及工程化演进过程（使用 Gulp、Webpack 构建 Web 开发工具）。
- 工具开发：主要是开发 npm 上的各种工具模块，包括各种前端预编译工具、构建工具 Gulp、脚手架、命令行工具等。

表 1-1 列出了 Node.js 的具体应用场景。

表 1-1

应用场景	描 述	相关模块
网站	类似于 <code>cnodejs.org</code> 这样的传统网站	Express、Koa
API	同时提供给移动端、PC 端、HTML5 等前端使用的 HTTP API 接口	Restify、Hapi
API 代理	为前端提供，主要对后端 API 接口进行再处理，以便更好适应前端开发	Express、Koa
IM 即时聊天	实时应用，很多是基于 WebSocket 协议的	Socket.io、SockJS
反向代理	提供类似于 Nginx 反向代理功能，但对前端更友好	AnyProxy、node-http-proxy、hiproxy
前端构建工具	辅助前端开发，尤其是在预编译、构建相关的工具上，能够极大提高前端开发效率	Grunt、Gulp、Bower、Webpack、FIS3、Ykit
命令行工具	使用命令行是非常“酷”的方式，前端开发自定义了很多相关工具，无论是 shell 命令、Node.js 脚本，还是各种脚手架，几乎每个公司和小组都有自己的命令行工具集	Cordova、Shell.js
操作系统	能实现，但估计不会有很多人使用	NodeOS
跨平台打包工具	使用 Web 开发技术开发 PC 客户端是目前最流行的方式，会有更多前端开发工具采用这种方式	PC 端的 Electron、NW.js，比如钉钉 PC 客户端、微信小程序 IDE、微信客户端；移动端的 Cordova，即以前的 PhoneGap；还有更加有名的一站式开发框架 Ionic Framework
P2P	用于区块链开发和 BT 客户端	WebTorrent、IPFS
编辑器	Atom 和 VSCode 都是基于 Electron 模块的	Electron
物联网与硬件	Ruff.io 和很多硬件都支持 Node.js SDK	Ruff、Rokid、Node-RED

可以说,目前大家能够看到的、用到的软件中都有 Node.js 的身影,当下最流行的软件写法也大都基于 Node.js,比如 PC 客户端 `luin/redis` 采用 Electron 打包的方式,写法采用 `React+Redux` 的方式。我自己一直在实践的“Node 全栈”也是基于这种趋势而形成的。未来,Node.js 的应用场景会更加广泛,更多内容可参见 `sindresorhus/awesome-nodejs`。

1.4 本章小结

通过本章的学习,相信你已经对 Node.js 有了一定的了解,尤其是它的 4 个特点:适合构建 Web 应用、高性能、简单、可扩展。另外,关于 Node.js 的 4 类使用场景的讲解,相信也能让你耳目一新。

恭喜你,你已经进入了 Node.js 的世界!

第 2 章

Node.js 安装与入门

看了 Node.js 的特点和应用场景，你是否也有些跃跃欲试了呢？

要想使用 Node.js 进行开发，首先需要安装 Node.js 运行环境，由于 JavaScript 是脚本语言，只需要解释器就可以执行，所以当你编写完 Node.js 代码之后，可以直接通过 `node` 命令来执行编写的脚本，并且立刻看到结果！这是非常简单的，但只有这些是不够的，你还需要了解编辑器和调试技巧。本章会展示两个例子，分别是最常见的 Hello World 和 Node.js 最擅长的 Web 应用实例，希望能够让你快速掌握 Node.js 的使用技巧。

2.1 安装 Node.js

Node.js 支持 macOS、Linux 以及 Windows 等多个主流操作系统，但在 Windows 平台下问题较多，其中很多莫名其妙的问题不容易解决，所以一般推荐使用 macOS 或 Linux（比如 Ubuntu 桌面版）系统。在实际生产环境中，推荐使用 Linux 服务器，常用的是 CentOS 或 Ubuntu，选用对应的 64 位的 LTS（长期支持）版本即可。

这里以 Linux 系统下的安装为标准，先介绍 3m 安装法，之后讲解如何从源码开始进行编译，以便大家可以深入理解。

2.1.1 3m 安装法

各个平台都有相关的包管理工具，比如 Ubuntu 下面有 `apt-get`，CentOS 下面有 `yum`，macOS 下面有 `homebrew`，这些都是安装软件时非常方便的利器。在本书的写作过程中，Node.js 经历了多次版本升级，但对于这个名副其实的“版本帝”而言，上述工具显然是不合适的。首先 Node.js

的版本更新非常快, 开发机器上可能需要同时存在几个 Node.js 的大版本, 每个 Node.js 内置的 npm 又有版本的差异, 而且国内网络访问 npmjs.org 镜像的速度非常慢, 鉴于以上种种原因, 这里为大家介绍一种最佳实践——3m 安装法, 具体如下。

- nvm (node version manager): 用于开发阶段, 解决多版本共存、切换、测试等问题。
- npm (node package manager): 解决 Node.js 模块安装问题, 其本身也是一个 Node.js 模块, 每次安装都会内置某个版本的 npm。
- nrm (node registry manager): 解决 npm 镜像访问慢的问题, 提供测速、切换下载源 (registry) 功能。

2.1.2 nvm

nvm 是一个开源的 Node.js 版本管理器, 通过简单的 shell 脚本来管理和切换多个 Node.js 版本。提供 nvm 类似功能的还有 TJ Holowaychuk 写的 n 以及新发布的跨平台 nvs, 它们的功能大同小异, 相比之下 nvm 稍强大一些。值得注意的是, n 目前不支持 Windows 系统, 而 nvm 中有 nvmw, 可以支持 Windows 系统, 另外 nvs 默认支持 Windows 系统, 也是一个不错的选择。

关于 nvm 的官方介绍如下。

Node Version Manager - Simple bash script to manage multiple active node.js versions.

nvm 除了安装方便, 可以随意切换需要安装的 Node.js 版本, 还可以免除安装权限, 通过 nvm 命令安装的 Node.js 位于用户目录下, 而非系统目录下。在 npm 安装全局模块的时候, 可以避免除操作系统超级用户授权问题。

👉 安装 nvm

在安装 Node.js 之前, 需要先安装 nvm, 然后通过 nvm 命令去安装多个版本的 Node.js。nvm 本身是 shell 脚本, 直接下载安装即可。首先, 在终端中执行如下命令。

```
$ curl -o- https://raw.githubusercontent.com/creationix/nvm/v0.29.0/install.sh | bash
```

显示结果如下。

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
100	7731	100	7731	0	0	37998	0

```
=> Downloading nvm from git to '/home/ubuntu/.nvm'
=> Cloning into '/home/ubuntu/.nvm'...
remote: Counting objects: 4715, done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4715 (delta 0), reused 0 (delta 0), pack-reused 4712
Receiving objects: 100% (4715/4715), 1.24 MiB | 265.00 KiB/s, done.
Resolving deltas: 100% (2801/2801), done.
Checking connectivity... done.
* (HEAD detached at v0.29.0)
  master

=> Appending source string to /home/ubuntu/.bashrc
=> Close and reopen your terminal to start using nvm
```

意思是通过 `curl` 命令下载 `install.sh` 脚本并执行，执行完成后，把 `nvm` 命令的执行路径放到 `~/.bashrc` 文件下，我们可以通过 `cat` 命令来查看。

```
$ cat ~/.bashrc | grep nvm
export NVM_DIR="/home/ubuntu/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh" # This loads nvm
```

确认存在以上配置后，通过执行 `source` 命令，使系统环境变量生效。

```
$ source ~/.bashrc
```

至此，我们就完成了 `nvm` 的安装。通过在终端执行 `nvm --version` 可以查看版本号，如果能够打印出版版本号就证明 `nvm` 已经安装成功了，如下所示。

```
$ nvm --version
0.25.1
```

如果系统使用的是 `zsh`，则需要手动将环境变量放到 `~/.zshrc` 里，命令如下。

```
$ vi ~/.zshrc
```

然后，追加内容到 `~/.zshrc` 里，命令如下。

```
export NVM_DIR="/home/ubuntu/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && . "$NVM_DIR/nvm.sh" # This loads nvm
```

最后，通过执行 `source` 命令使环境变量生效，其他和 `bash` 里的操作相同。

```
$ source ~/.zshrc
```

👉 查看可安装版本

在安装 `Node.js` 之前，我们需要了解通过 `nvm` 可以安装哪些版本的 `Node.js`，通过 `nvm ls-remote` 命令列出当前 `nvm` 可以安装的版本，显示如下。

```
$ nvm ls-remote
v0.1.14
...
v0.10.45
v0.11.0
...
v0.11.16
v0.12.0
...
v0.12.14
iojs-v1.0.0
...
iojs-v3.3.1
v4.0.0
v5.0.0
...
v6.11.2 (Latest LTS: Boron)
-
v7.10.1
-
v8.2.1
```

读者不需要一一了解以上版本,只需要了解 LTS 版本和 Current 版本这两个重要概念即可,具体如下。

- LTS 版本指的是长期支持 (Long-Term Support) 版本,有官方支持,推荐给绝大多数用户使用,一般在生产环境中使用,对于 Bug 和安全问题的修复相当及时。
- Current 版本指的是当前正在开发的尝鲜版本。它通常较新,具有大的功能变动,比如支持 async 函数、支持 ES6 模块,但不完全稳定,需要再经过一段时间的测试、开发、修复 Bug 才可能变为 LTS 版本,一般用于开发者学习,基本不会用在线上生产环节中。

以 Node.js 官方网站为例,当前的 LTS 版本是 v10.15.3,而 Current 版本是 v11.13.0,如图 2-1 所示。

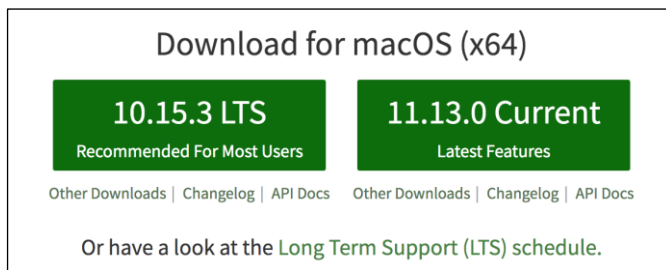


图 2-1

“八卦”一下：Node.js 的可用版本是非常多的，一般奇数版本（比如 v5.x、v7.x、v9.x、v11.x）都是尝试性的，其中包含最新的特性，而偶数版本（如 v4.x、v6.x、v8.x 和 v10.x）都是 LTS 版本，非常稳定，适合在线上生产环境中使用！

👉 安装 Node.js

目前 Node.js v10.15.3 是 LTS 版本，无论是开发还是用于其他场景，都推荐使用此版本。下面带领大家一起来安装此版本的 Node.js。

```
$ nvm install 10.15.3
Downloading https://nodejs.org/dist/v10.15.3/node-v10.15.3-linux-x64.tar.xz...
##### 100.0%
Computing checksum with shasum -a 256
Checksums matched!
Now using node v10.15.3 (npm v6.4.1)
```

第一步：确定安装位置。

通过 nvm 安装的 Node.js 位于用户目录下，而非系统目录下。在 npm 安装全局模块的时候，可以避免操作系统超级用户授权问题。

```
$ which node
/home/i5ting/.nvm/versions/node/v10.15.3/bin/node
```

第二步：指定默认版本。

如果想要默认使用 Node.js v6.11.2 来编译代码，你需要手动指定一个 default 别名，具体操作如下。

```
$ nvm alias default node
default -> node (-> v10.15.3)
```

此时，输入 node -v，以后在终端的任何地方使用的默认版本都会是你设置的版本。

```
$ node -v
v10.15.3
```

这里需要说明一下，当安装完 Node.js 之后，node 命令会存在于环境变量里，你可以在终端的任何位置使用。node 命令是用于解释并执行 Node.js 代码的，由于 JavaScript 是脚本语言，所以 node 命令实际上是 Node.js 代码的解释器。

第三步：切换版本。

Node.js 对 ES6 新特性的支持是有版本差别的, v4.8.4 支持 57% 的 ES6 特性, 而 v6.11.2 支持 99% 的 ES6 特性。比如 `let`、`const` 等在 v4 中的非严格模式下是不被支持的, 这时候我们就需要使用更高的版本了。如果要想使用 `async` 函数, 需要安装 v7.6 以上的版本。如果要想支持 ES6 模块, 需要安装 8.5 以上的版本。

这里以安装 Node.js 8.x 版本为例, 演示如何通过 `nvm` 进行 Node.js 版本切换, 命令如下。

```
$ nvm install 8
Downloading https://nodejs.org/dist/v8.2.1/node-v8.2.1-linux-x64.tar.xz...
##### 100.0%
Now using node v8.2.1 (npm v5.3.0)
```

切换到 8.x 版本, 注意它内置的 `npm` 是 v5 的。

```
$ nvm use 8
Now using node v8.2.1 (npm v5.3.0)
$ node -v
v8.2.1
```

此时, 你的 Node.js 代码就可以在 8.x 版本下执行了, 可以使用任何 8.x 版本支持的特性。像上面这样使用 `nvm use` 命令实现的切换版本, 只在当前终端会话内有效, 是一次性切换, 要和设置默认别名区分开。

第四步: 列出本机版本。

如何知道本机通过 `nvm` 安装过哪些 Node.js 版本呢? 方法如下。

```
$ nvm ls
   v0.10.48
   v6.11.4
   v8.7.0
   v8.9.4
   v8.15.0
   v10.12.0
->  v10.15.3
default -> node (-> v10.15.3)
node -> stable (-> v10.15.3) (default)
stable -> 10.15 (-> v10.15.3) (default)
iojs -> N/A (default)
lts/* -> lts/dubnium (-> v10.15.3)
lts/argon -> v4.9.1 (-> N/A)
lts/boron -> v6.17.0 (-> N/A)
lts/carbon -> v8.15.1 (-> N/A)
lts/dubnium -> v10.15.3
```

对以上代码说明如下。

- 本机上一共有 3 个 Node.js 版本，一个是系统默认的（system），另外两个是通过 nvm 命令安装的。
- v10.15.3 表示当前版本是 v10.x。
- default -> 10.15.3 (-> v10.15.3) 表示默认版本。
- iojs -> iojs- (-> system) (default) 对于不同机器而言可能不一样，如果之前通过 apt-get 或 homebrew 安装过 Node.js，此处会有显示。

第五步：重新安装全局模块。

若经常切换版本，最痛苦的莫过于全局模块需要重新安装，比如全局安装了 gulp-cli 模块之后又重新安装了一个 Node.js 版本，那么此时的 gulp 命令是无法使用的，唯一能做的就是重新安装全局模块。针对这种情况，nvm 提供了一个很贴心的一键安装全局模块的 nvm reinstall-packages 命令，具体如下。

```
$ nvm reinstall-packages 6
VERSION=''
Reinstalling global packages from v6.11.2...
/Users/sang/.nvm/versions/node/v8.2.1/bin/gulp -> /Users/sang/.nvm/versions/node/
v8.2.1/lib/node_modules/gulp-cli/bin/gulp.js
+ gulp-cli@1.4.0
added 137 packages in 14.08s
Linking global packages from v6.11.2...
```

👉 指定远端下载地址

nvm 的默认远端下载地址是 <https://nodejs.org/dist>，大部分开发场景用到的 Node.js 都可以从这个地址下载。如果想安装自定义的 Node.js 版本，可以指定 nvm 的远端下载地址，对于想要体验 node-chakracore 和其他测试特性的尝鲜版本的用户来说是必要的。

```
$ NVM_NODEJS_ORG_MIRROR=https://nodejs.org/download/chakracore-nightly
```

查看远端可用版本，命令如下。

```
$ nvm ls-remote
v7.0.0-nightly201701136413bab8a5
v8.0.0-nightly2017060948524b8340
v8.1.0-nightly20170613d03aa0a20d
v9.0.0-nightly2017061300a6407bc8
...
```

获取到版本信息后, 直接安装即可。

```
$ nvm install v8.1.0-nightly20170613d03aa0a20d
VERSION_PATH=''
##### 100.0%
Computing checksum with shasum -a 256
Checksums matched!
Now using node v8.1.0-nightly20170613d03aa0a20d (npm v5.0.3)
```

对于一些正在测试的尝鲜版本, 可以在下面的地址下载。注意, 非 LTS 版本尽量不要用在生产环境中。

```
$ NVM_NODEJS_ORG_MIRROR=https://nodejs.org/download/test
```

本节一共介绍了 6 个 nvm 中的常用命令, 总结如下。

- 安装: `nvm install`。
- 设置系统默认的 Node.js 版本: `nvm alias default`。
- 切换版本: `nvm use`。
- 列出当前的本地版本: `nvm ls`。
- 列出远端可安装版本: `nvm ls-remote`。
- 一键安装全局模块: `nvm reinstall-packages`。

可以说这 6 个命令在开发过程中已经足够用了, 其实 nvm 还有很多高级特性, 比如, 在项目根目录下创建 `.nvmrc` 文件来指定特定的 Node.js 版本就可以自动切换到对应版本。类似的特性很多, 此处就不一一列举了, 请各位读者自行学习体会吧。

2.1.3 npm

npm, 通常称为 Node.js 包管理器, 它的主要功能是管理 Node.js 包, 包括安装、卸载、更新、查看、搜索、发布等。npm 最开始只是 Node.js 的包管理器, 但随着大前端技术 React、Webpack、Vue、Browerfy、Ionicframework 等的发展, 它的定位变成了广义的包管理器, 可以实现 JavaScript、React、移动开发、Angular、浏览器、jQuery、Cordova、Bower、Gulp、Gunt、Browserify、Docpad、Nodebots、Tessel 等包或模块管理, 是目前开源世界里最大、生态最健全的包管理器, 截止到 2019 年 3 月, 包的总数已经超过 94.6 万, 月下载量超过 47.8 亿, 如图 2-2 所示。

Popular libraries	Discover packages	By the numbers
lodash	IoT	Packages
request	mobile	946,452
chalk	front end	
react	backend	Downloads - Last Week
express	robotics	11,259,862,949
commander	css	
moment	testing	Downloads - Last Month
debug	cli	47,826,168,701
async	documentation	
bluebird	math	
prop-types	coverage	
react-dom	frameworks	

图 2-2

npm 是随 Node.js 一起安装的包管理工具，能解决 Node.js 在模块管理上的很多问题，常见的应用场景有以下几种。

- 从 npm 镜像服务器下载第三方模块。
- 从 npm 镜像服务器下载并安装命令行程序到本地。
- 自己发布模块到 npm 镜像服务器供他人使用。

安装 Node.js 时，npm 也一并安装好了，可以通过在终端里输入 `npm -v` 命令来测试是否成功安装，出现版本提示则表示安装成功，具体如下。

```
$ npm -v
5.3.0
```

👉 安装 npm

- 不同版本的 Node.js 内置不同版本的 npm，每个版本的 npm 在功能上都有一定的差别。
- npm v2: 典型的树形依赖，层次深，依赖重叠，安装时间长，但相对更清晰。
 - npm v3: 扁平化依赖，将依赖移到了顶层，同一模块多版本依赖时需采用 npm v2 模式，由于 Node.js 项目模块依赖非常多，因此在查找依赖时特别麻烦，虽没有 npm v2 用起来体验好，但下载速度确实快了不少。
 - npm v5: 自动记录依赖树，下载使用强校验，重写缓存系统等功能得到了升级和改造。代表性特征是新增了 `package-lock.json` 文件，在操作依赖时默认生成，用于记录和锁定依赖树的信息，与 Yarn 类似。

如果你安装的是旧版本的 npm，由于 npm 本身也是一个 Node.js 模块，因此可以很容易地通过 npm 命令来升级版本，命令如下。

```
$ [sudo] npm install -g npm
/home/ubuntu/.npm/versions/node/v6.11.2/bin/npm -> /home/ubuntu/.npm/versions/node/v6.11.2/lib/node_modules/npm/bin/npm-cli.js
npm@3.9.5 /home/ubuntu/.npm/versions/node/v6.11.2/lib/node_modules/npm
```

sudo 是可选的，如果 Node.js 是通过 nvm 安装的，可以不使用 sudo 临时授权。当然，也可以安装指定的某个版本以获得更多的功能支持，比如 npm v2.9 之后的版本支持了私有模块。

```
$ node -v
v4.0.0
$ npm -v
2.14.2
$ [sudo] npm install -g npm@2.9
```

在遇到使用问题的时候，一定要先查看版本，适当升级或降级是十分必要的。

👉 使用 npm 安装模块

npm 的包安装是最核心的功能，分为本地安装（local）、全局安装（global）两种，从键入的命令行来看，两者的差别只是有没有-g 而已，帮助文档如下。

```
$ npm -h install

npm install (with no args, in package dir)
npm install [<@scope>/]<pkg>
npm install [<@scope>/]<pkg>@<tag>
npm install [<@scope>/]<pkg>@<version>
npm install [<@scope>/]<pkg>@<version range>
npm install <folder>
npm install <tarball file>
npm install <tarball url>
npm install <git:// url>
npm install <github username>/<github project>

aliases: i, isntall, add
common options: [--save-prod|--save-dev|--save-optional] [--save-exact] [--no-save]
```

常用选项及说明见表 2-1。

表 2-1

命 令	简 写	说 明
无选项	无	将模块安装到本地 node_modules 目录下，但不保存到 package.json 里

续表

命 令	简 写	说 明
--save-prod	-P	将模块安装到本地 node_modules 目录下，同时保存到 package.json 里的 dependencies，这在安装模块时是必须安装的
--save-dev	-D	将模块安装到本地 node_modules 目录下，同时保存到 package.json 里的 devDependencies，仅供开发时使用，一般测试模块等辅助开发工具都会这样安装
--global	-g	安装的模块为全局模块，如果是命令行模块，会直接链接到环境变量里

关于本地安装，具体说明如下。

- 将安装包放在 node_modules(运行 npm 命令时所在的目录)下,如果没有 node_modules 目录，则会在当前执行 npm 命令的目录下生成。
- 可以通过 require()来引入本地安装的包。

在以下实例中，我们使用 npm 命令安装常用的 Node.js 调试模块 debug，命令如下。

```
$ npm install debug
```

安装好之后，debug 包存在于工程目录下的 node_modules 目录中，因此在代码中只执行 require('debug')即可，无须指定第三方包路径。

```
const debug = require('debug')('YOUR_DEBUG_PREFIX');
```

关于全局安装，具体说明如下。

- 如果不是使用 nvm 安装的，安装包将放在/usr/local 下，安装全局模块需要超级用户授权。如果是使用 nvm 安装的，则需要将安装包放到用户目录的 nvm 版本对应的 bin 目录~/.nvm/versions/node/v6.0.0/bin/下，一般用户对于用户目录下的所有文件拥有完整权限，所以不需要增加 sudo 授权，可以直接在命令行里安装全局模块。
- 不能通过 require()来引入本地安装的包。

采用 nvm 安装 Node.js 之后，基本不会再遇到全局安装时的权限问题。安装包存在于用户目录的 nvm 版本对应的 bin 目录下，不用加 sudo 授权。对于初学者而言，经常会遇到 Linux 或 macOS 系统的权限问题。

为避免引用模块缺失，保证依赖模块都出现在 package.json 里，使用 npm i --save 是一个好习惯。

👉 区分 npm 版本

不同版本的 npm 差异非常大, 需要区分, 这样可以避免很多问题。比如有的模块在 npm v2.x 下是正常的, 而在 npm v3.x 下就莫名其妙地无法使用了, 所以这个问题值得大家花点时间来理解。

首先, 看一下 npm v2.x 的结构。为了演示方便, 我们统一在 npm v2.x 和 npm v3.x 里都安装一个名为 debug 的用于根据环境变量来决定是否打印调试日志的 Node.js 模块, 代码如下。

```
$ cd book-source/nodejs/npm/2.x
// 初始化 package.json, 一直按回车键即可
$ npm init
$ nvm use 4
Now using node v6.11.2 (npm v2.15.5)
$ npm -v
2.15.5
$ npm i -S debug
npm WARN package.json 2.x@1.0.0 No description
npm WARN package.json 2.x@1.0.0 No repository field.
npm WARN package.json 2.x@1.0.0 No README data
debug@2.2.0 node_modules/debug
└─ ms@0.7.1
```

查看一下目录结构。

```
$ tree .
.
├── node_modules
│   └── debug
│       ├── History.md
│       ├── Makefile
│       ├── Readme.md
│       ├── bower.json
│       ├── browser.js
│       ├── component.json
│       ├── debug.js
│       ├── node.js
│       ├── node_modules
│       │   └── ms
│       │       ├── History.md
│       │       ├── LICENSE
│       │       ├── README.md
│       │       ├── index.js
│       │       └── package.json
│       └── package.json
└── package.json
```

这里使用的 npm v2.x 的依赖是嵌套的，比如当模块依赖 debug 模块，而 debug 模块又依赖 ms 模块时，就会导致结构变成下面这样。tree 命令的 -d 选项是只显示目录的意思。

```
$ tree . -d
.
├── node_modules
│   └── debug
│       └── node_modules
│           └── ms
4 directories
```

很明显，它们的依赖关系很清晰，但如果不同模块依赖了相同模块呢？这种情况下，模块间彼此互相依赖，各种版本的依赖交织在一起，就会变得非常混乱。也因为这样，npm 被人诟病。为了解决这个问题，ied 这样的 npm 替代方案便出现了。

实际上，npm 也是做了改进的，npm v3.x 中有了明显改善，代码如下。

```
$ cd book-source/nodejs/npm/3.x
// 初始化 package.json，一直按回车键即可
$ npm init
$ nvm use 6
Now using node v6.0.0 (npm v3.8.6)
$ npm i -S debug
2.x@1.0.0 /Users/sang/workspace/17koa/book-source/nodejs/npm/2.x
├── debug@2.2.0
└── ms@0.7.1
```

查看结构，并进行对比。

```
$ tree .
.
├── node_modules
│   ├── debug
│   │   ├── History.md
│   │   ├── Makefile
│   │   ├── Readme.md
│   │   ├── bower.json
│   │   ├── browser.js
│   │   ├── component.json
│   │   ├── debug.js
│   │   ├── node.js
│   │   └── package.json
│   └── ms
│       ├── History.md
│       ├── LICENSE
│       └── README.md
```



```

├── index.js
├── package.json
└── package.json

```

3 directories, 15 files

同样是 debug 模块, 不过在 npm v3.x 下面, debug 和 ms 是同级的。

```

$ tree . -d
.
├── node_modules
│   ├── debug
│   └── ms

```

3 directories

现在, npm v3 模块的目录结构已经扁平化了, 但又保持了树和依赖版本共存的可能性, Node.js v5.0 之后便升级到了 npm v3, 也就是说, npm v3 已经普遍可用了。更多关于 npm v3 变更的内容见官方文档。图 2-3 展示了 npm v2 与 npm v3 的差异。

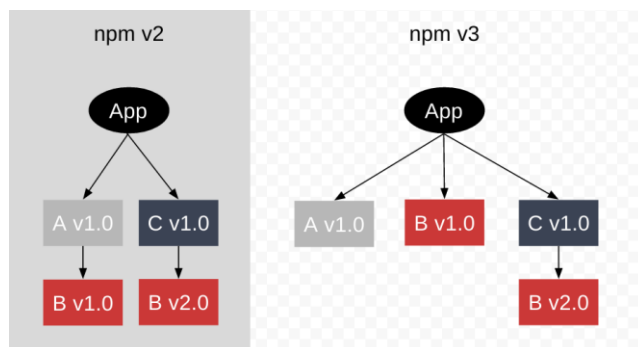


图 2-3

- npm v2.x 的依赖都在当前模块下面, 所以目录层级会比较多。
- npm v3.x 的结构是扁平式的, 当依赖多个版本的时候, 才会和 npm v2.x 采用同样的方式。

2.1.4 nrm

Node.js 和其他语言一样都提供了模块管理工具, 默认将模块托管在 npmjs.org 下, 这是官方的下载源, 当然其他组织或个人也可以自建下载源, 这些非官方的镜像会定期和 npm 官方源进行同步, 一般 10 分钟左右进行一次, 略有延时。

npm 官方源是在国外托管的, 所以对于国内用户来说, 访问起来速度会慢一些, 而且 Node.js

本身又因遵守小而美哲学而导致多模块依赖，有时会慢得让人头疼，所以就近选择一些速度更快的源是刚需，其实 `npm` 中是提供了这样的配置的。

但当同时有几个源需要切换时，如果还用这种方式，相信整个人都会崩溃，那么这种情况下该怎么办呢？

`nrm` 就是专门用于解决这个问题的，它可以帮助我们简单、快速地在不同的 `npm` 之间进行切换，它默认内置了很多常用的源，包括 `npm`、`cnpm`、`taobao`、`nj`、`rednpm`、`npmMirror` 等，当然也可以通过 `nrm add` 命令来维护自己的源。可以说，`nrm` 是工程复杂到一定程度的必然产物，也是最佳实践。

👉 安装 `nrm`

`nrm` 本身是 Node.js 模块，所以它是通过 `npm` 来安装的，它是一个命令行模块，需要使用 `--global` 参数进行全局安装，命令如下。

```
$ [sudo] npm install --global nrm
```

也可以使用 `nrm -h` 命令来查看 `nrm` 的帮助信息。

👉 测速

为了确定哪个源下载速度更快，我们需要对这些源进行网络测速，通过测速结果来决定具体使用哪个源，测速时使用 `nrm test` 命令，具体如下。

```
$ nrm test
* npm ---- 1623ms
cnpm --- 561ms
taobao - 2184ms
nj ----- 2382ms
rednpm - Fetch Error
npmMirror 3999ms
```

从测速结果上可以看出，`cnpm` 是最快的（*代表当前正在使用的源）。虽然每次测速结果都有浮动，但整体来说，国内 `cnpm` 和 `taobao` 的源速度还是比较快的，尤其是在阿里云上部署时，大家按需取用即可。

👉 查看源

一般我们需要了解自己当前使用的是哪个源, 确定它是否为下载速度最快的源。另外, Node.js 模块只能发布在官方源, 如果使用其他源, 是无法通过 `npm publish` 命令成功发布的。

若要查看源的状态, 使用 `nrm ls` 命令即可。

```
$ nrm ls

* npm ---- https://registry.npmjs.org/
cnpm --- http://r.cnpmjs.org/
taobao - http://registry.npm.taobao.org/
nj ----- https://registry.nodejitsu.com/
rednpm - http://registry.mirror.cqupt.edu.cn
npmMirror https://skimdb.npmjs.com/registry
```

针对以上显示, 有几点说明, 具体如下。

- *标识说明它是当前使用的源。
- npm 是源的名字, 可以在 nrm 里切换。
- 当前版本的 nrm 提供了 6 个源。

👉 切换源

记住源里的具体 URL 地址可能有点难, 但换成名字再记忆就简单多了。nrm 就是通过名字来切换的。使用 `nrm use <registry name>` 命令可以快速切换源。

```
$ nrm use cnpm

Registry has been set to: http://r.cnpmjs.org/
```

此时, 查看一下 nrm 源的情况, 就会知道当前采用的是 cnpm 源。

```
$ nrm ls

npm ---- https://registry.npmjs.org/
* cnpm --- http://r.cnpmjs.org/
taobao - http://registry.npm.taobao.org/
nj ----- https://registry.nodejitsu.com/
rednpm - http://registry.mirror.cqupt.edu.cn
npmMirror https://skimdb.npmjs.com/registry
```

👉 增加源

为了保证开发效率，企业在内网部署一套私有 npm 源是非常必要的，理由如下。

- 内网安装，安装速度快。
- 私有模块，仅供企业内部使用，更加安全。
- 适合多团队开发，前后端都可以使用私有源来进行管理。

通过 nrm 添加自定义 npm 源的命令如下，推荐使用 cnpm 部署私有源镜像服务器，cnpm 本身是 private npm for company 的缩写，就是为企业应用而生的意思。

```
$ nrm add yourcompany http://registry.npm.yourcompany.com/
```

2.1.5 从源码进行编译

Node.js 源码是典型的 C/C++ 代码，使用 make 作为构建工具，所以和其他 C/C++ 项目从源码进行编译并没有什么区别，只是要注意准备好依赖。

安装依赖，命令如下。

```
$ sudo apt-get install g++ curl libssl-dev apache2-utils git-core build-essential
```

下载源码并编译。

```
$ git clone https://github.com/nodejs/node.git
$ cd node
$ ./configure
$ make
$ sudo make install
```

node 命令在终端里可用，即表示已从源码编译成功，后面的章节里会有更详细的说明。

2.1.6 状态理论

本节没有采用 apt-get 或 homebrew 这样的直接安装方式，而是通过 nvm 进行安装。这是因为，在 Node.js 快速发展的今天，需要随时切换版本，无论出于线上环境应尽量保守的原因，还是想在学习尝试超前的想法，3m 安装法都是比较好的选择。

上一节介绍了如何从源码进行编译，主要采用 C/C++ 项目的通用编译法，这部分内容要尽

量掌握，后面在 npm 中开发 C/C++ 扩展时会用到。

本节讲的 3m 安装法是非常复杂的，它涉及 3 个维度，首先 Node.js 有 x 个版本，npm 有 y 个版本，npm 源可能有 z 个，那么这个排列组合的数量就相当大了。对于开发人员来说，要掌握这些也是一件难事，但这些又是必须要区分的，只有足够清楚地区分状态，才能更好地完成工作。

同一时间只做一件事是幸福的，一个脑袋同时运转多进程其实是不可能的，最多只能做到快速切换。不同场景下还能认清自己角色的人是很了不起的，比如工作就工作，学习就学习，陪老婆逛街就要有逛街的样子，如果无法很好地切换状态，陪老婆逛街时还在想代码，下场一定好不到哪里去！很多时候，编程世界和现实世界是一样的，时刻知道自己所处的状态，知道在当前状态下该做什么事，才能拥有更多的快乐！

2.2 Hello Node.js!

本节将以 Hello world 为例，引出浏览器与 Node.js 的差别，进而给出 Node.js 版本中最常见的 Web Server 代码实现。这也是所有书中的默认做法。

2.2.1 Hello World

创建 helloworld.js，代码如下。

```
"use strict";  
  
console.log('Hello World');
```

执行该示例。

```
$ node chapter01/helloworld.js  
Hello World
```

这是最简单的例子，目的是证明 Node.js 支持 JavaScript 语法。Node.js 只是一个 JavaScript 的运行环境，是构建在 Chrome V8 引擎之上的，也就是说，凡是 Chrome V8 支持的 JavaScript 语法，它都支持，唯一不同的就是浏览器内置的 BOM、DOM 等对象是没有的，如图 2-4 所示。如果你懂 JavaScript 语法，就可以非常简单地上手了！

在上面的示例代码中，node 命令只是解释器，用于解释并执行文件中的 Node.js 代码，console.log 是在控制台打印出日志的函数。node 命令和 console.log 函数的差别在于，在浏览器

执行命令的时候，需要调出 Chrome DevTools 里的审查（inspector）的控制台才能看到，而在 Node.js 里是直接在终端输出的。另外"use strict";语句是开启严格模式的规范写法，建议开启。

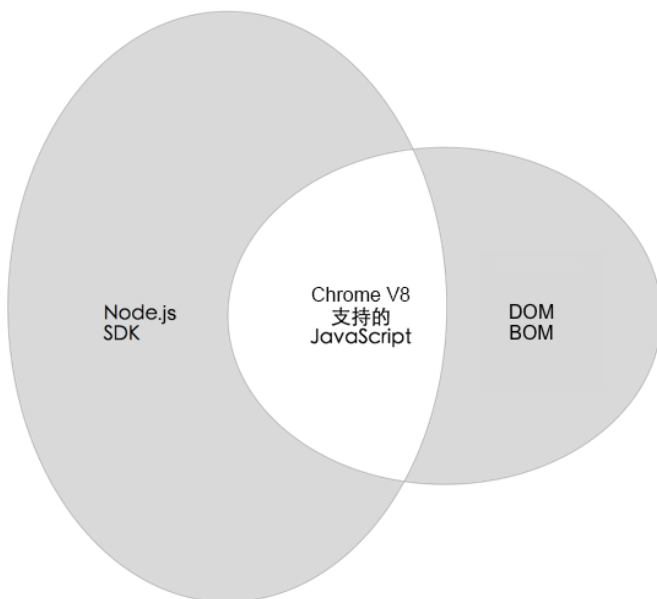


图 2-4

2.2.2 Hello CommonJS

Node.js 是基于 CommonJS 规范的实现，即每个文件都是一个模块，每个模块内代码的写法都必须遵守 CommonJS 规范，多文件调用的核心基于模块对外暴露接口和互相引用，所以 CommonJS 对于学习 Node.js 来说是必须会的知识。下面让我们一起来看看 Node.js 里的 CommonJS 的写法。

本例中会涉及如下两个文件，用来演示多个文件及其引用关系。

- hello_commonjs.js
- hello_commonjs_test.js

👉 创建模块

这里使用 `module.exports` 对外暴露一个匿名函数，在里面打印“Hello CommonJS!”，代码

如下。

```
module.exports = function () {  
  console.log('Hello CommonJS!');  
}
```

这应该是最简单的 Node.js 模块定义，下面看一下如何在 `hello_commonjs_test.js` 里引用该模块。

👉 引用模块

这里通过 `require` 关键字来引用 `hello_commonjs` 模块，里面的 `./hello_commonjs` 表示当前目录下的 `hello_commonjs` 模块。在 `hello_commonjs` 模块里使用 `module.exports` 导出了一个匿名函数，所以才可以直接调用。

```
const hello = require('./hello_commonjs');  
  
hello()
```

以上代码中的核心要点如下。

- 使用 `module.exports` 定义模块。
- 通过 `require` 关键字引用模块。

其实这两点就是 CommonJS 规范的核心内容，后面的章节中会详细介绍。在终端键入如下命令，会打印出我们想要的结果，具体如下。

```
$ node hello_commonjs_test.js  
Hello CommonJS!
```

2.2.3 Hello HTTP

之前我们已经通过 `nvm` 安装了 Node.js，是时候来写一个高级的 Hello World 代码了。一般我们用 Node.js 做的最多是 Web 应用开发，那就来实现一个最简单的示例吧。创建 `hello_node.js` 文件，键入下面的代码。

```
const http = require('http');  
  
http.createServer(function (req, res) {  
  res.writeHead(200, {'Content-Type': 'text/plain'});  
  res.end('Hello Node.js\n');  
}).listen(3000, "127.0.0.1");
```

```
console.log('Server running at http://127.0.0.1:3000/');
```

关于上面的代码，说明如下。

- 引用了 Node.js SDK 内置的名为 `http` 的模块。
- 通过 `http.createServer` 创建了一个 HTTP 服务。
- 通过 `listen` 方法指定端口和 IP 地址，启动服务。
- `res` 是 HTTP 里的 `response`（响应）的别名，通过 `res` 控制对浏览器的操作，设置返回状态码是 200，设置 HTTP 头字段里的 `Content-Type` 为 `text/plain` 类型，最后向浏览器返回 `Hello Node.js\n` 文本，由浏览器解析并渲染。

通过短短几行代码就可以创建一个 HTTP 服务，是不是非常简单？代码执行也非常简单，具体如下。

```
$ node hello_node.js
Server running at http://127.0.0.1:3000/
```

通过解释器执行 `hello_node.js` 文件里的 Node.js 代码，这是典型的 Node.js 执行过程，脚本不需要编译，整体来说还是比较简单的。访问浏览器，结果如图 2-5 所示。



图 2-5

我们把代码稍稍改动一下，创建 `hello_node_es6.js` 文件，修改后如下。

```
// "use strict";
const http = require('http');

http.createServer((req, res) => {
  let status = 200;
  res.writeHead(status, {'Content-Type': 'text/plain'});
  res.end('Hello Node.js\n');
}).listen(3000, "127.0.0.1");

console.log('Server running at http://127.0.0.1:3000/');
```

关于上面的代码，说明如下。

- `var` 变成了 `const`。
- `http.createServer` 里的匿名函数变成了 ES6 里的箭头函数。
- 为了演示方便, 用 `let` 声明了 `status` 变量。

实现的效果和原理都是一模一样的, 只是语法上有些许不同而已。首先, 我们在 Node v4.x 下执行, 看一下效果。

```
$ nvm use 4
Now using node v6.11.2 (npm v2.15.5)
$ node hello_node_es6.js
/Users/sang/workspace/17koa/book-source/nodejs/hello_node_es6.js:4
  let status = 200;
  ^^^

SyntaxError: Block-scoped declarations (let, const, function, class)
not yet supported outside strict mode
    at exports.runInThisContext (vm.js:53:16)
    at Module.compile (module.js:413:25)
    at Object.Module._extensions..js (module.js:452:10)
    at Module.load (module.js:355:32)
    at Function.Module.load (module.js:310:12)
    at Function.Module.runMain (module.js:475:10)
    at startup (node.js:117:18)
    at node.js:951:3
```

这里报的是语法错误, 块作用域声明 (`let`、`const`、`function`、`class`) 在严格模式以外是不支持的。也就是说, 在 Node.js v4.x 里, 如果不使用严格模式, 是无法使用 `let`、`const`、`function`、`class` 这些关键字的。不是 Node.js v4.x 不支持它们, 而是在严格模式以外不支持。

如果在文件第一行增加 `"use strict";`, 就可以正常执行了, 上面的代码将这行用注释形式屏蔽掉了, 就是为了演示。在 Node.js v6.x 下即使使用非严格模式, 这些语法也是可以正常被识别的, 下面我们切换到 Node.js v6.x 下来执行。

```
$ nvm use 6
Now using node v6.2.1 (npm v3.9.3)
$ node hello_node_es6.js
Server running at http://127.0.0.1:3000/
```

以上我们分别演示了使用 `nvm` 在 Node.js v4.x 和 v6.x 间进行版本切换, 以及在两个版本下测试代码, 希望大家可以深刻理解 `nvm` 的优势, 对 Node.js 和 ES6 有一个简单的认识。

本节共讲了 3 个例子, 分别如下。

- **Hello World:** 说明 Node.js 是 JavaScript 的运行环境, 绝大部分代码都是可以通过 `node` 命令解释执行的。
- **Hello CommonJS:** 让大家看到了 Node.js 的模块写法, 这是最常见的写法, 后面会单独介绍 CommonJS 规范。
- **Hello HTTP:** 这是我们在项目里经常用到的最简单的模块, 主要是为了展示 `http` 模块的用法, 让大家能够见证用 Node.js 编写 HTTP 服务是多么简单的事。如果深入学习, 对于理解浏览器渲染原理与性能优化也是有极大好处的。

通过这 3 个例子, 大家应该可以了解 Node.js 的大致写法了, 后面我们会继续深入讲解!

2.3 编辑器与调试

编程有三等境界: 第三等境界是打日志, 这是最简单、便捷的方式, 略显低级, 一般新手或资深程序员偷懒时会用; 第二等境界是断点调试, 在前端、Java、PHP、iOS 开发时非常常用, 通过断点调试可以很直观地跟踪代码执行逻辑、调用栈、变量等, 是非常实用的技巧; 第一等境界是测试驱动开发, 在写代码之前先写测试。与第二等的断点调试刚好相反, 大部分人不是很习惯第一等这种方式, 但在国外开发者或者敏捷爱好者看来, 这是最高效的开发方式, 测试在保证代码质量、重构等方面非常有帮助, 是现代编程开发必不可少的一部分。

Node.js 对这三等境界都支持。

- **打日志:** 可用 `console.log`、`debug` 模块或 Node.js SDK 内置的 `util.log` 等方式。
- **断点调试:** 可用 Node debugger 或 VSCode 编辑器。
- **测试驱动开发:** TDD 和 BDD 测试框架非常多, 比如 Mocha、AVA、Jest、Cucumber 等。

本节将以断点调试为主, 借助编辑器或 IDE (集成开发环境) 的断点调试功能, 帮助各位读者快速上手。

2.3.1 IDE/编辑器

Node.js 的 IDE 和编辑器有很多种, 对比如下, 见表 2-2。

表 2-2

名 称	是否收费	是否支持断点调试	功 能
Webstorm	收费	支持	是 IDE，在代码提示、重构等方面功能非常强大，支持的语言、框架、模板也非常多，支持断点调试，好处是十分智能，然而缺点也是太过智能
Sublime/TextMate	收费	不支持	非常好用的编辑器，TextMate 主要针对 macOS 用户，Sublime 是跨平台的，相信很多前端开发人员都很熟悉
Vim/Emace	免费	不支持	命令行下的编辑器，非常强大，难度也稍大，但功能更为“酷炫”，而且对于服务器部署开发来说是值得一学的
VSCode/Atom	免费	支持	Atom 发布比较早，功能强大，缺点是稍有卡顿。VSCode 是微软发布的，速度快，对于 Node.js 调试、重构、代码提示等方面的支持都非常好

本书推荐使用 VSCode 编辑器！

2.3.2 VSCode

VSCode（Visual Studio Code）是一个运行于 macOS、Windows 和 Linux 系统之上的，用于编写现代 Web 和云应用的跨平台源代码编辑器。它功能强大，便于调试，加上它本身也是基于 Node.js 的 Electron 模块构建的，因此强烈推荐大家使用。VSCode 的下载界面如图 2-6 所示。

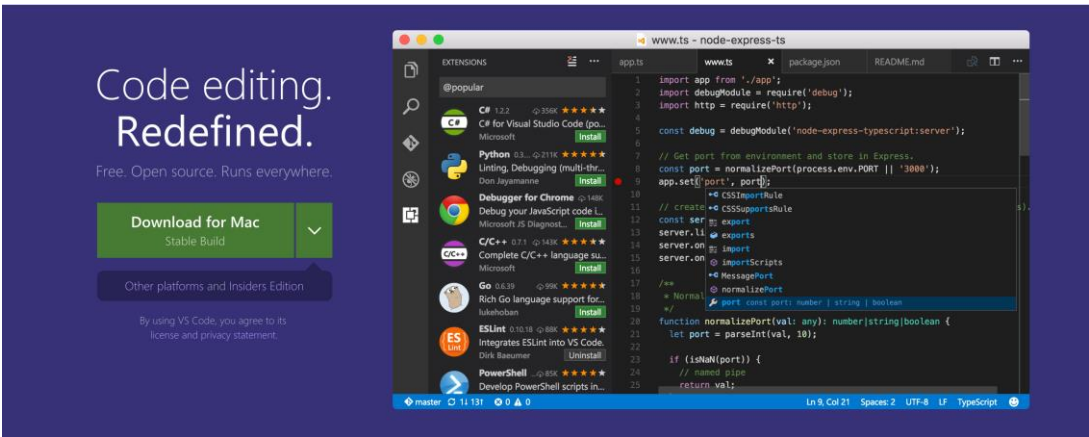


图 2-6

VSCode 具有如下特点。

- 比较新潮，跨平台支持 macOS、Windows 和 Linux，和 TextMate、Sublime、Notepad++、UltraEdit 等比较，功能上毫不逊色。

- 免费、开源，功能足够多。
- 调试功能强大，尤其对于 Node.js、TypeScript、Go 而言效果显著，提供了自定义 Debugger Adapter 和 VSCode Debug Protocol，从而具有自己的调试器。
- 支持语法自动补全及智能提示。
- 支持构建和调试，对 Gulp 等的集成非常好，使用起来非常简单。
- 内置 HTML 开发神器 Emmet (ZenCoding)，对 CSS 及其相关编译型语言 Less 和 Sass 都有很好的支持。
- 速度快、调试效率非常高。
- 支持插件系统、多主题配色方案，在 v0.9.1 之后的版本中使用体验更好。
- 继承了其他编辑器中的高效操作方式和快捷键。

VSCode 扩展可以让第三方支持更多特性，主要包含以下 4 个方面。

- 编程语言：C++、C#、Go、Python。
- 工具：ESLint、JSHint、PowerShell、Visual Studio Team Services。
- 调试：Chrome、PHP XDebug。
- 快捷键映射：Vim、Sublime Text、IntelliJ、Emacs、Atom、Visual Studio、Eclipse。

VSCode 编辑器特别适合用作开发编辑器。当然，最好还是用于写 Node.js 和 JavaScript 代码，这也是接下来要着重介绍的功能。

👉 安装 code 命令

VSCode 编辑器内置了一个 code 命令，可以在命令行下打开文件，对于习惯终端命令的人来说是非常重要的。结合各种命令行工具一起使用时能够实现高效开发，而且看起来特别“酷”！

默认情况下 code 命令是没有安装的，为了设置，需要先打开 Command Palette (⌘P)，并且键入 Shell Command: Install 'code' command in PATH，如图 2-7 所示。

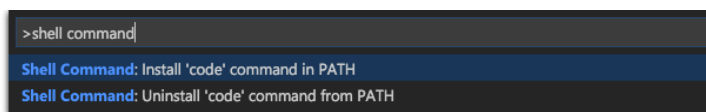


图 2-7

然后重启终端，这样就可以使用 `code` 命令了。在任意目录里键入 `code` 就可以轻松打开该目录下的所有文件。要想删除 `code` 命令，操作同上。

🔧 安装 vscode-icons 插件

VSCode 编辑器支持插件机制，拥有大量各种各样的插件，如果说必须要安装一个插件的话，我推荐 `vscode-icons`，它会根据文件的后缀名来显示不同的图标，让你更直观地知道当前编辑的文件是什么类型的，这对大项目开发来说，好处是极其明显的。

在安装之前，需要先打开 Command Palette (⌘P)，键入如下命令。

```
> ext install vscode-icons
```

如图 2-8 所示，各种文件类型是不是看起来特别直观？

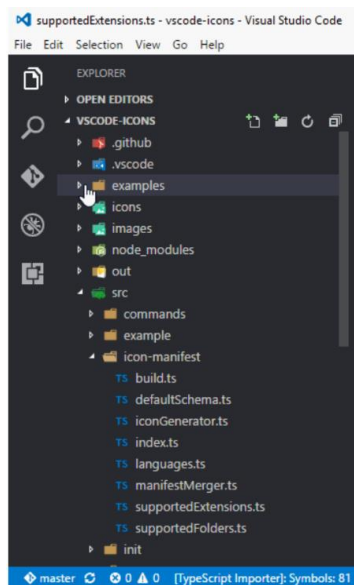


图 2-8

👉 快捷键配置

使用快捷键是提高开发效率的重要手段之一。VSCode 编辑器定制快捷键非常容易，经典做法是将 VSCode 编辑器左侧的 5 个主菜单分别配置为“cmd+数字”的快捷键，方法如下。

```
// 常用快捷键配置
[
  { "key": "cmd+1", "command": "workbench.view.explorer" },
  { "key": "cmd+2", "command": "workbench.view.search" },
  { "key": "cmd+3", "command": "workbench.view.scm" },
  { "key": "cmd+4", "command": "workbench.view.debug" },
  { "key": "cmd+5", "command": "workbench.view.extensions" }
]
```

这和 XCode 开发快捷键一样，非常方便实用。另外，使用 cmd + p 组合键可以快速打开文件，也是必须要掌握的快捷方式。

2.3.3 调试

Node.js 调试具有版本差异，推荐使用最新版本的 VSCode 编辑器，最好是 VSCode v1.1.0 之后的版本。

最早的调试协议是 Node debug，即 Node.js SDK 内置的 v8:debug，通过 debug_agent.js 代理可提供调试功能。一直以来，对于 Node.js 开发而言并没有很好的调试工具，早期调试 Node.js 是非常痛苦的，后来有了 node-inspector 模块，打通了 Chrome DevTools 和 Node debug 协议，使得大家可以在 Chrome Devtools 里调试 Node.js 代码，如图 2-9 所示。

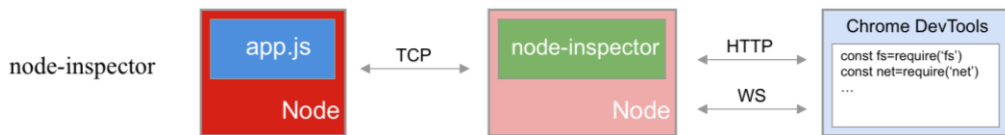


图 2-9

node-inspector 的功能足够强大，也保留了 Chrome 的调试习惯，唯一的问题是，当文件过多的时候，它的启动会尤其慢，甚至让人“抓狂”，所以 2015 年下半年起，我就切换到 VSCode 编辑器上进行开发了。

2016 年 5 月，Google 工程师 Ali Ijaz Sheikh 提交了一个加入 v8inspector 支持的 PR，同年 5 月的 DevTools Google I/O talk 上也提到了此功能。也就是说，v8inspector 可以让 DevTools 直接

连接 Node.js 的 Debugger 进行调试。于是, Node.js v6.3 之后的版本就不再维护 node-inspector 了。

而 VSCode 编辑器完美支持 Node debug 和 Node inspect 两种调试协议, 下面就让我们来体验一下 VSCode 编辑器强大的调试功能吧。

👉 单个文件调试

通过 code 命令打开文件。

```
$ code chapter01/hello_node.js
```

在第 4 行设置断点, 然后按下 F5 快捷键, 这样便会启动 HTTP Server, 注意看调试控制台对应的 Tab 项, 如图 2-10 所示。

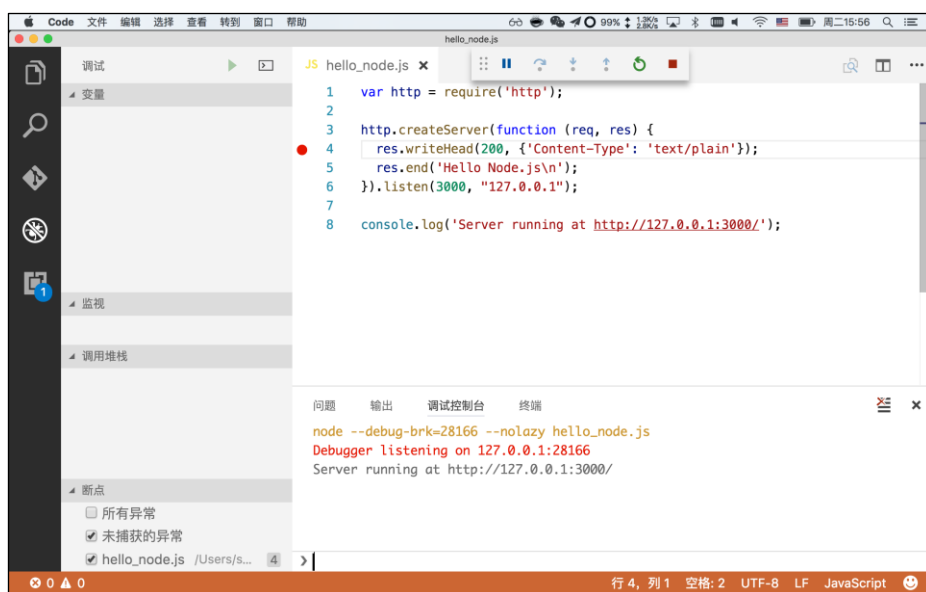


图 2-10

然后通过 curl 命令发送一个简单的 GET 请求, 如图 2-11 所示。当然在浏览器里输入 URL 访问也可以, 效果一样。

接下来, 我们就可以享受真正的断点调试了。

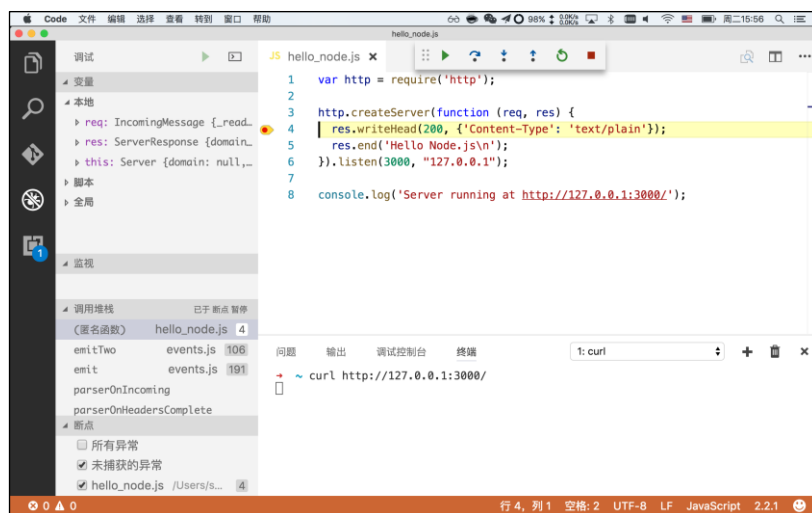


图 2-11

固定文件调试

首先通过 `code` 命令打开 `chapter01` 目录。

```
$ code chapter01
```

打开调试面板，默认是没有配置的，点击“添加配置”项，如图 2-12 所示。

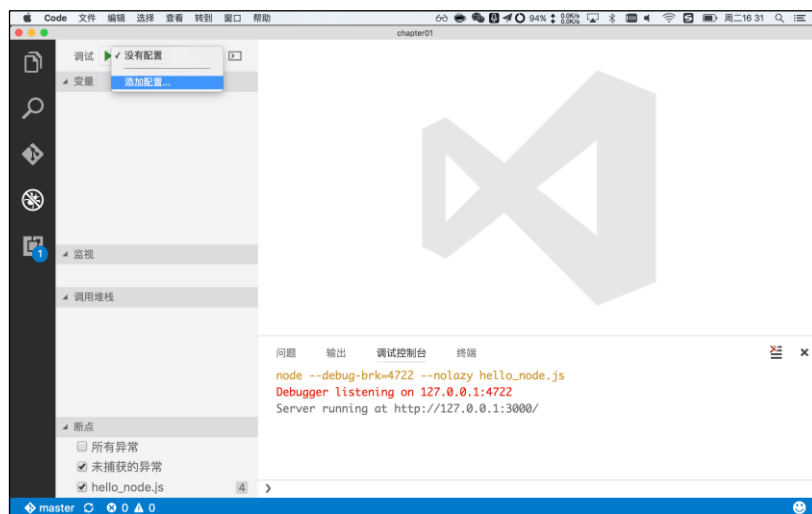


图 2-12

点击后会打开 launch.json 中的 configurations，显示很多选项，如图 2-13 所示。

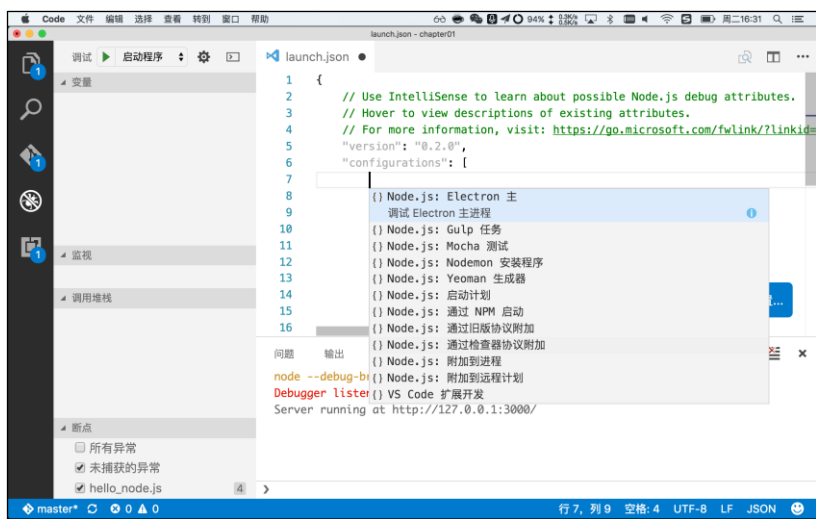


图 2-13

这里我们选择“Node.js: 启动计划”，此时 `luanch.json` 配置如下。

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Program",
      "program": "${workspaceRoot}/app.js"
    },
    {
      "type": "node",
      "request": "launch",
      "name": "启动程序",
      "program": "${file}"
    },
    {
      "type": "node",
      "request": "attach",
      "name": "附加到进程",
      "address": "localhost",
      "port": 5858
    }
  ]
}
```

注意这里面"request": "launch"有两个，上面的指定了 program 为固定值，下面的指定了 program 为\${file}，即 VSCode 支持两种启动方式。

- 当前无任何选中的情况下，按下 F5 键会默认启动固定文件调试，即第一种。
- 当前编辑器激活窗口里打开了某个文件，按下 F5 键会启动当前文件的调试。

上面介绍的是最常用的方法，另外还有几种，如表 2-3 如下。

表 2-3

编 号	名 称	描 述
1	Launch Program	直接执行
2	Launch via NPM	通过 npm 来启动
3	Attach to Port	附加端口，根据端口来查找进程
4	Attach to Process	附加到进程，即跨进程调试
5	Nodemon Setup	代码变动自动重启的 Node monitor，参见 https://github.com/remy/nodemon
6	Mocha Tests	简单，强大的测试框架，参见 https://github.com/mochajs/mocha
7	Yeoman generator	使用最多的脚手架生成器，参见 https://github.com/yeoman/yeoman
8	Gulp task	最流行的流式构建系统，参见 https://github.com/gulpjs/gulp

👉 跨进程调试

在上面的配置项中，request 有两个选项，对应着两种调试模块，具体如下。

- launch: 启动程序，相当于直接在编辑器里启动 Node.js，即本地调试。
- attach: 附加到进程，在编辑器外通过 node --debug 命令启动，然后附加到 debug 进程中，即远程调试。

还是以上面的 hello_node.js 为例，键入以下命令。

```
$ node --inspect hello_node.js
Debugger listening on 127.0.0.1:5858
Server running at http://127.0.0.1:3000/
```

这样就启动了一个可以用于远程调试的进程，注意看 Debugger 监听的端口是 5858，不同场景下也有可能不一样。

在 launch.json 里删除两个包含"request": "launch"的配置项，如图 2-14 所示。

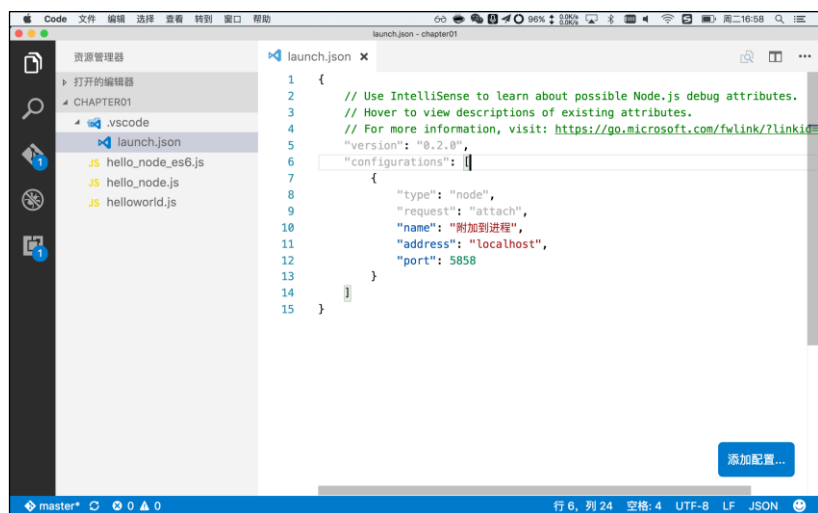


图 2-14

增加断点，按下 F5 快捷键，界面如图 2-15 所示。

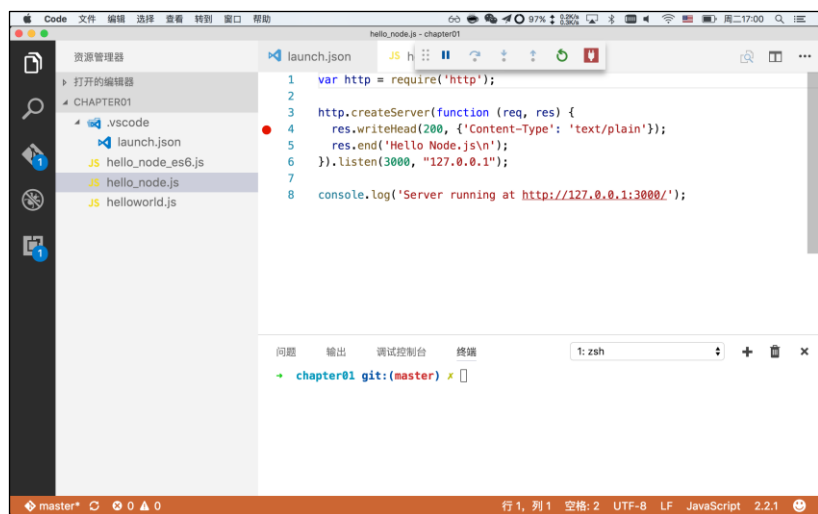


图 2-15

如图 2-16 所示，此时启动 Node.js 调试服务，如果看到调试按钮最右边有一个像电插销一样的按钮，就表示已经和远程 Node.js 线程连接成功了，剩下的就和本地调试一样了。

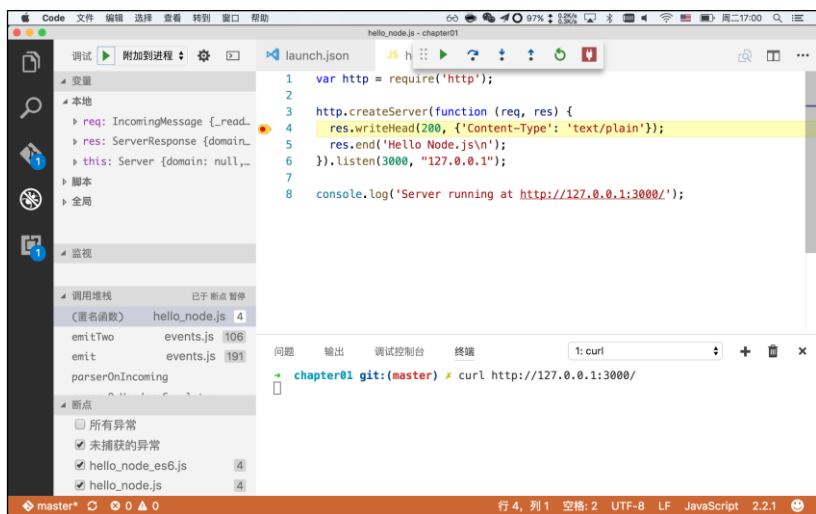


图 2-16

远程调试

上面讲的 3 种调试都是本地调试，在某些情况下可能需要对远程的服务器进行调试，这时只能使用远程调试方式。远程调试的配置和跨进程调试类似，只是要在 `launch.json` 里增加 `address` 等对远程机器的配置信息，具体如下。

```
{
  "type": "node",
  "request": "attach",
  "name": "Attach to Remote",
  "address": "TCP/IP address of process to be debugged",
  "port": 5858,
  "localRoot": "${workspaceRoot}",
  "remoteRoot": "Absolute path to the remote directory containing the program"
},
```

- `address` 是 IP 地址或域名。
- `port` 是远程端口。
- `localRoot` 是本地的代码根目录。
- `remoteRoot` 是远程代码的相对目录。

VSCode 编辑器目前发展也是非常快的，更多相关文档可参见 <https://code.visualstudio.com>。

[com/docs/editor/debugging](https://code.visualstudio.com/docs/editor/debugging)。

2.4 本章小结

通过本章的学习,你应该已经通过 3m 安装法安装好了 Node.js 和 npm 环境。除了能够运行 node、npm 和 nrm 这 3 个命令,你应该也学会了如何执行简单的脚本,以及如何通过 VSCode 编辑器进行 Node.js 程序调试。

恭喜你,你已经入门了,可以去编写更多的 Node.js 代码了!

第 3 章

更了不起的 Node.js

一般来说，某一种语言、框架、设计模式的产生都有其特定的适用领域，但也有很多是例外的，比如 Java 早期想在其 GUI（图形化界面）里使用 MVC 模式，结果没能推广成功，反而在 Java Web 编程领域取得了突破，以致于现在谈到 Web 编程都默认是指 MVC 模式。

Node.js 其实也是这样的例子，本来是为了解决后端并发编程而设计的，不想却成了大前端领域的基石，大前端的火爆难免影响大家对 Node.js 的认知，因此这里有必要为 Node.js 正名。

关于大家对 Node.js 的认知偏差问题，死月曾说过：“早年我刚学 Node.js 的时候，也曾在 Node.js IRC 上发出过类似的疑问，有个人是这么回我的——Don't use a car as a boat. If you want a boat, then use a boat.意思是，它是什么就该是什么，它能干什么就该干什么！”

“了不起”是一个不能随便说的词，对于 Node.js 来说，在简化并发编程方面，用“了不起”来形容并不过分。Node.js 在 2009 年横空出世时，确实是独一无二的。但在今天，已经 10 岁的 Node.js 有了更多、更广泛的应用场景，它的意义已经远远大于设计时的初衷了，我觉得用“更了不起”来形容并不过分。下面就让我们一起来看一下更了不起的 Node.js 吧！

3.1 架构升级

曾经在写 iOS 应用的时候，产品经理提出要实现一个很棒的功能，结果被后端开发人员拒绝了，原因是接口没办法实现，SQL 语句太复杂。产品经理很郁闷，其实作为开发人员的我们也很郁闷。要想解决这件事，只能把相关人员都叫到一起，了解数据库表结构，最后由我们给出 SQL。从此以后，后端开发人员不会说“这个需求做不了”了，最多会说“我考虑一下”。

个人能力越强，能做成的事也越多，有了 Node.js，能做的会更多！也正是因为有了 Node.js，

很多已有的模式需要重新思考，在架构上，我们有了更多更好的选择。

3.1.1 从 LAMP 到 MEAN

Web 开发技术经过了两次大的架构演变，即从 LAMP 到 MEAN，下面分别介绍。

LAMP 是早期表现非常突出的开源 Web 技术集合之一，使用 Linux 操作系统，将 Apache 作为 Web 服务器，使用 MySQL 数据库，并将 PHP（或者 Python、Perl）作为生成基于 HTML 页面的编程语言。随着互联网的发展，这些技术被整合在一起，成为当时的最佳实践。自此以后，Web 技术栈进入大爆发时代，开发者可以快速且轻松地创建一个新的网站。

MEAN 出现在 2014 年左右，是在 Web 社区中赢得大量关注的一种新兴堆栈，涉及 MongoDB、Express、Angular 和 Node.js，如图 3-1 所示。MEAN 代表着一种完全现代的 Web 开发方法：将 JavaScript 作为唯一语言，贯穿应用程序开发的所有环节，从客户端到服务器端，再到数据库持久层，这也是最新潮的全栈 JavaScript 架构。

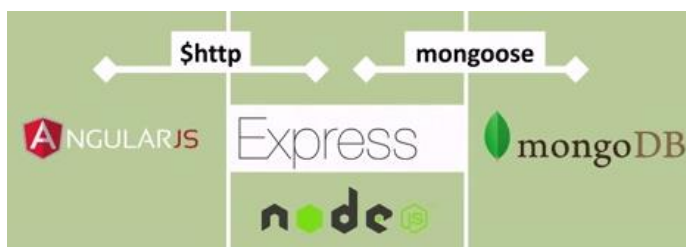


图 3-1

图片来源: <https://www.youtube.com/watch?v=TGPhVOqgQwg>

MEAN 与传统 LAMP 一样，是一种全套开发工具的简称，下面分别说明一下每个字母对应的含义。

- M: MongoDB，一个使用类似 JSON 风格进行存储的数据库，非常适合 JavaScript 语言调用。
- E: Express，一个 Web 应用框架，提供组件和模块，帮助建立网站应用。
- A: Angular，Google 开发的一个前端 MVVM 框架，注意此处说的 Angular 是 1.x 版本的。
- N: Node.js，一个并发、异步、事件驱动的 JavaScript 服务器后端开发平台。

从我个人的角度来看，MEAN 的含义如下。

- MySQL 用 MongoDB 替换，原因是 MongoDB 是 NoSQL 世界里最像关系型数据库的，在开发和性能方面都有优势，尤其对于一些缺少专业 DBA 运维的中小型公司而言。
- Express 作为 Node.js 的示范项目，非常精简，是最成熟稳定的 Web 开发框架。
- Angular 代表了一个时代，双向绑定、指令等都曾让无数人热血沸腾，很多人都说，用过 Angular 之后就再也不想用 jQuery 了。
- Node.js 提供了完全的生态和工具链，你想要的它基本都能实现，感谢 npm 强大的生态，再有就是，在 2015 年以前，Node.js 的性能明显优于 PHP。

技术选型是一件比较严肃的事，可能会决定公司未来几年甚至更长时间的发展方向，在 2015 年，我们选择 MEAN 技术栈的原因如下。

- 成熟、稳定、简单，能够解决实际问题。
- 看好 Node.js 的前景，把握趋势。
- 在架构层面屏蔽可能的风险，不孤注一掷，也不一叶障目，合理使用其他语言，只要求每个功能都以服务形式出现，至于它是用什么语言编写的，并不重要。
- 招聘性价比相对较高，技术栈新，容易吸引人才。

其中最重要的是第一点，在创业初期，出现问题能够解决是最最重要的事。

时至今日，在微服务架构盛行的今天，MEAN 已经因为技术栈过于复杂而被大家慢慢淡忘，更多的是采用阿里巴巴开源的 Egg.js。现在不再流行一整套的技术栈，服务化才是流行的解决方案！本章后面会介绍与服务化相关的部分：RPC 服务、服务组装，以及页面即服务。

3.1.2 前后端分离

绝大部分开发都是从单体式架构（Monolithic Architecture）开始的，简单、集成度高、上手容易。但随着互联网的发展，多人协作势必会导致专业化分工，前后端分离也伴随而来。典型的企业应用大多采用 3 层架构模式。

- 表现层：处理 HTTP 请求，直接返回 HTML 渲染，或者返回 API 结果。对于一个复杂的应用系统，表现层通常是代码中比较重要的部分。

- 业务逻辑层：完成具体的业务逻辑，是应用的核心组成部分。
- 数据访问层：访问基础数据，例如数据库、缓存和消息队列等。

Java 后端的分层非常清晰，具体如下。

- 先定义模型层 (Model)，数据库操作一般采用 ORM 库来简化操作，模型会和数据库里的表进行关联映射。
- DAO (Data Access Object)，就是我们常说的增删改查，主要对单个模型进行操作。
- Service 层就是业务逻辑层，通常组合多个 DAO 对象进行某项业务处理。
- Controller 里组装了多个 Service 对象，可实现具体的功能。

这些分层看起来是非常合理的，所以大家通常也习惯性地认为只有后端才有业务逻辑，可是在重度 API 化的今天，后端的业务逻辑已经被削弱了，更多的业务逻辑被移到了前端：组装 API、RPC 服务、提供配置、静态 API 接口等，除了不直接操作数据库，几乎所有的业务逻辑都由前端承担。这样做有两个好处：一是使得应用分两部分开发、部署和扩展，各自独立，提高开发和测试效率，因为逻辑更多位于前端，能够更好地应对需求变化；二是简化后端开发，让后端开发人员将更多的精力放在后端业务处理上。

现在前后端的大致划分如图 3-2 所示。



图 3-2

按照传统分类方式，表现层即前端，而业务逻辑层和数据访问层都属于后端。目前，最常见的模式是前端+API+后端服务，出于负载均衡、请求转发、反向代理、静态资源托管、防跨

域等原因，往往需要搭配 Nginx 一起使用。

方式 1：请求—> Nginx—> 前端

一般来说，Nginx 是由运维团队（OPS）负责统一管理的，如果想要变更，就需要走变更申请流程，跨部门合作显得特别麻烦。但同时，前端又需要处理一些下线、重定向、兼容多版本等逻辑，解决历史遗留问题，如果每次变更都走变更流程，那开发效率是不可忍受的。既然 Nginx 必不可少，而前端又需要一定的可操作空间，那么是否可以基于这两点找到解决方案呢？最简单最常用的办法，就是在原有方式不变的情况下，在前端引入 Node.js。

方式 2：请求—> Nginx—> Node.js（前端）

这里的 Node.js 可以实现以下功能。

- 提供请求转发、反向代理等功能。
- 提供 Web 服务，所有后端语言（Java、PHP 等）能做到的，Node.js 都能做到。
- 圈定范围，便于维护，只要修改功能包含在当前 Node.js 项目范围内即可维护。
- 充当 HTTP 客户端，访问后端接口，但一般不直接访问数据库。

下面给出一张对比图，以便大家更好地理解，如图 3-3 所示。

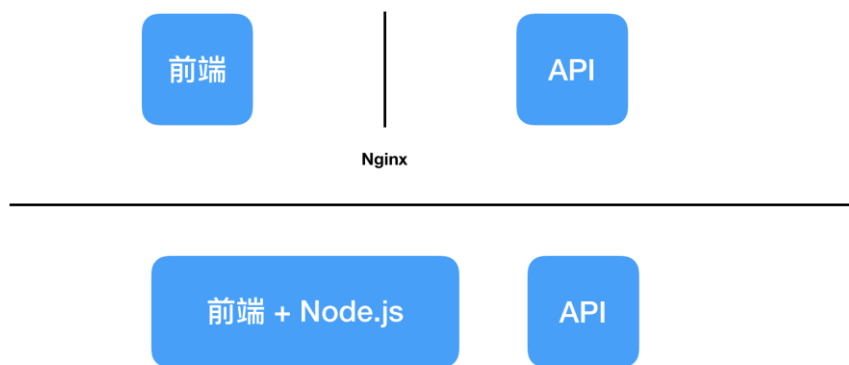


图 3-3

两种方式的差别如下。

- 方式 1：过度依赖 Nginx，在部署运维和团队配合方面（尤其是跨部门）比较麻烦。
- 方式 2：使用 Node.js 来替代 Nginx 的 HTTP 功能，让前端有更多的灵活性。

本书推荐第二种方式！

有了 Node.js 还需不需要 Nginx？

绝大部分场景下是需要的。Nginx 在实现负载均衡、反向代理、缓存、限流限速方面还是比较合适的，业务代理不用操心这些事情。但 Nginx 需要繁杂的配置文件，而且不能动态控制与实时生效，而 OpenResty 只编写少量的 Lua 代码就能完美解决这些问题，但从学习成本上看比 Node.js 要高一些，尤其是对前端开发人员而言。

3.1.3 页面即服务

架构演变是“分分合合”的，对前端架构来说也是一样的。从单体式应用的“大而全”，变为微服务的“一个页面就是一个服务”，完全是由业务需求决定的。

本书将前端的这种“一个页面就是一个服务”的方式定义为“页面即服务 (Page as Service)”，下面来看一下它和单体式应用的区别，如表 3-1 所示。

表 3-1

	单体式应用	页面即服务
开发难度	越大越复杂，大到一定程度会不可维护	单个页面不会特别复杂，更容易把控
可用性	容易雪崩，影响极大	单个页面崩溃对整体来说影响极小
运维	容易	麻烦（必须配合 DevOps）

在上表中可以看到，除了运维比较麻烦，页面即服务几乎都是优点。不过在当下 Docker 和 DevOps 如此成熟的情况下，运维已经变得非常轻松了，所以页面即服务是必然趋势。

比如 Uber 的架构图里拆分出了 3 个服务，那么每个服务的前端都可以是独立应用。如果该服务的前端非常复杂，可以将其再进行拆分，如图 3-4 所示。

如果将 p1、p2、p3 这 3 个服务放到一起，假定它非常复杂，开发难度非常高，那么是不是开发成本也会更高呢？

如果将其拆分成独立的页面即服务呢？每个服务的复杂度降低了，也就是说开发难度也会降低很多，那么是不是开发成本就会低很多，招来的新人甚至实习生都能够进行维护呢？拆分成页面即服务是降低工程复杂度和成本的良方，简单易用。其中唯一增加的成本是拆分后的运维成本，如果是自动化运维的，其实是非常容易的。

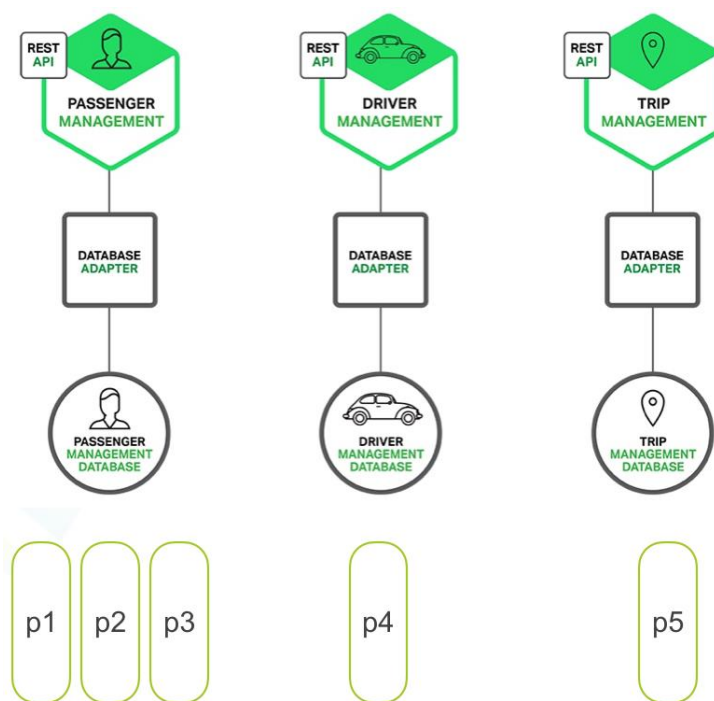


图 3-4

3.1.4 场景决定选型

孙子兵法有云：“势者，因利而制权也。”

Node.js 的应用场景几乎涵盖了开发过程中的所有环节，但并不是每种场景都是 Node.js 擅长的。不要抱着一个语言不变的心态，这个时代已经不接受一根筋的做法了，所以我们一定要区分场景来合理使用，一般应用可能涉及的场景如下。

- 大前端
- API 接口
- RPC 服务（OLTP）
- OLAP
- 数据挖掘

○ AI

上面提到的场景并不都是 Node.js 擅长的场景，下面介绍一下 Node.js 擅长的场景。

- 大前端：都遵循 JavaScript 语法规则，使用 Node.js 做前后端分离是非常合适的。
- API 接口：典型的 Web 应用，绝大部分是 I/O 密集型应用，是 Node.js 最擅长的。
- RPC 服务：主要是对 OLTP（联机事务处理过程）数据库进行操作，Node.js 可以支持，但如果是复杂计算，使用 Go 或 Java 也是极好的。

一定要避免因单一技术栈而导致性能瓶颈问题。有很多时候不是 Node.js 不行，而是没有用对！换句话说，合作共赢才是最好的选择，技术选型不要过多关注技术本身，而是要关注它适合用来做什么，在对应的场景下才能够实现业务价值。

3.2 贯穿开发全过程

第 1 章中讲过，Node.js 的应用场景主要分为 4 大类：跨平台开发、后端开发、前端开发和工具开发。Node.js 天生就是为了解决后端并发编程而生的，却阴差阳错变成了辅助前端开发的工具，在前端和后端开发领域都有非常不错的表现。从软件工程的角度看，如果能够通过 Node.js 提高开发效率，是不是更好呢？

Node.js 可以贯穿开发全过程，优化各个环节，明显提高开发效率，下面给出一些场景，希望能够对读者有一定的启发。

3.2.1 静态 API

书本上的软件工程在互联网高速发展的今天已经不那么适用了，尤其是移动开发火热起来之后，所有的企业都崇尚敏捷开发，快鱼吃慢鱼，甚至觉得两周发布一个迭代版本都很慢，组件化和热更新就是这样产生的。综上所述，我们势必要对传统的软件工程模式重新思考，提高开发和产品迭代的效率是重中之重。

如图 3-5 所示，先反思一下，开发为什么不那么高效？

按照传统的软件工程模式，产品经理提出需求文档（PRD），给出原型图（UE），交给设计师做出高保真的 UI 图，回归到 PRD 中，和前后端开发人员确认后即可排期。整个流程是一环扣着一环的，在没有完整的 PRD 之前，开发人员什么都做不了，后端不给 API 结构，大前端

(App、H5、PC) 的开发人员只能切切图。而后端的 API 设计需要 UI/UE，必须知道功能要做成什么样才能建模，进而给出 API 接口，此过程依赖 UI/UE。设计师也挺“烦”的，产品给出的需求需要论证、思考、排版、设计、配色等，反复和产品经理沟通是最痛苦的环节，所以设计和需求方也是相互依赖的。

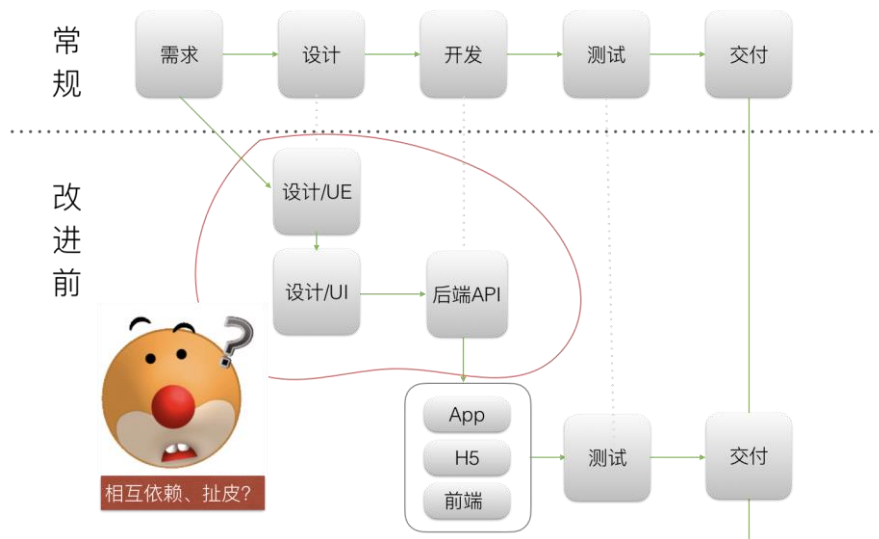


图 3-5

如此倒推，大前端开发者们依赖后端 API 接口，设计依赖 UI/UE，UI/UE 又依赖产品经理和设计师的沟通，效率不高的原因主要在于流程和沟通过程太烦琐，这一点在大公司中体现得尤其明显，上游不给出具体内容，下游就不动手，极易造成相互扯皮的情况。

如果想提高开发效率，该怎么办呢？能否让开发提前接入呢？让开发敏捷起来，在 UE 阶段就让大前端开发者们动手做，是不是能够提高效率呢？如果 4 端（App 端、H5 端、PC 端、后端）都并行开发是不是更好呢？为了能够让 4 端并行开发，最好的方式就是在开发流程中加一层静态 API（也可以叫 Mock API），给出模拟的接口，在测试前预留出和正式接口联调的时间。

流程改进后，在需求讨论阶段，产品经理、设计师、后端 API 开发人员同时接入，在给出 UE 原型图之后，后端 API 开发人员给出静态 API（在设计 API 的时候不需要高保真的 UI 图就可以实现），有了静态 API 就可以让 4 端同时开发了。

- 大前端可以同时开始界面和 API 请求的编写。

此时访问网址 `http://localhost:3000/posts/1`，即我们刚才仿造的静态 API 接口，返回数据如下。

```
{ "id": 1, "title": "json-server", "author": "typicode" }
```

3.2.2 现代 Web 开发

前端领域发展很快，在 2004 年之前，大概只要会“网页三剑客”（DreamWeaver、Fireworks、Flash，一套强大的网页编辑工具，最初是由 MacroMedia 公司开发出来的）就很厉害了。那时候前端还比较“纯洁”，在进入以 Ajax 为代表的异步刷新改进用户体验的 Web 2.0 时代后，大量的库开始涌现，比如 Prototype、jQuery、Mootools、YUI、Extjs、KISSY 等，它们都还只是对浏览器兼容性和工具类函数的封装。可是自从 Backbone、Angular v1.x 相继出现后，前端领域就开始热闹起来了，出现了 MVC、MVVM、IoC（控制反转，Java 著名框架 Spring 里的概念）、前端路由（类似于 Express、Koa 等框架的路由）、Virtual DOM（虚拟 DOM，通过 DOM diff 算法减少对 DOM 的操作）、JSON API（接口规范）、JS 方言（CoffeeScript、Babel、TypeScript）等，各种框架如雨后春笋般冒出来，以前可能半年甚至更长时间才出现一个框架，现在几周就有新的框架诞生，前端领域进入了空前的爆发阶段。

现在前端开发复杂到了一定程度，对比之前的开发方式，可以称为“现代 Web 开发”。这里对前端发展的 4 个阶段进行一下总结，具体如下。

- 基础：HTML、CSS、JavaScript。
- 曾经流行：jQuery、jQuery UI、Extjs、Backbone、Angular。
- 当前流行：React、Vue。

目前整个大前端领域还处于发展期，没有形成固定模式，所以趋势上看是上升的，复杂、混乱、多样性的结果使现在的前端开发比较被看好。

很多人会问，学习前端技术应该采用渐进式方式，还是直接就切入 React、Vue 等核心框架呢？实际上，这两种方式都存在，如果有经济压力就直接学习 React、Vue 等框架，但学会之后，一定要把其他阶段补上。如果没有经济压力，又有时间和耐心的话，循序渐进地学习是最好的。编程没有捷径，无论哪种，都需要脚踏实地多多练习。

当代码写多了，你就会发现规律，具体如下。

- HTML 写烦了，便开始引入模板引擎。

- CSS 写烦了，便开始引入 CSS 预处理器 Sass、Less、Stylus、PostCSS 等。
- JavaScript 写烦了，便开始引入更友好的语言，比如熟悉 Ruby 的会使用 CoffeeScript，熟悉 Java 和 C#的会使用 TypeScript。

于是，前端开发变得复杂，并进入现代 Web 开发时代，如图 3-7 所示。



图 3-7

对于 Node.js 来说，所有前端框架都是视图层的展现技术而已，可以非常方便地和各种框架集成，并按照业务需求进行更好的实现。另外，所有的前端框架都采用 Node.js 和 npm 作为辅助开发工具，使用了大量 Node.js 模块。但前端框架使用的模块大同小异，如果熟练掌握了 Node.js 和 npm，对于学习前端技术来说，要学的只是纯前端的部分，其他技术的复用价值非常高。

和 Node.js 相关的前端开发模块实在是太多了，这里简单列举一些，如表 3-2 所示。

表 3-2

编 号	分 类	举 例
1	压缩	UglifyJS、JSMIn、CSSO
2	依赖管理	npm、Bower
3	模块系统	CommonJS、AMD、ES6 Modules

续表

编 号	分 类	举 例
4	模块加载器	Require.js、jspm、Sea.js、System.js
5	模块打包器	Browserify、Webpack
6	CSS 预处理	PostCSS、Less、Sass、Scss、Stylus
7	构建工具	Grunt、Gulp
8	模板引擎	Jade、Handlebars、Nunjucks
9	JavaScript 友好语言	CoffeeScript、Babel、TypeScript
10	跨平台打包工具	Electron、NW.js、Cordova
10	生成器	Yeoman、Slush、vue-cli、create-react-app
11	其他	图片压缩 imagemin、资源处理 DataURL 等

👉 预处理器

前端预处理可分为模板引擎、CSS 预处理器、JavaScript 友好语言 3 种。这些都离不开 Node.js 的支持，对于前端工程师来说，使用 Node.js 来实现这些是最方便不过的，如表 3-3 所示。

表 3-3

名 称	实 现	描 述
模板引擎	Mustache、EJS、Jade 等	在 JavaScript 的世界里，模板引擎有上百种之多，通过模板和数据编译成 HTML，可以完成更多操作
CSS 预处理器	Less、Sass、Scss、Rework、PostCSS	自定义语法规则，编译成 CSS
JavaScript 友好语言	CoffeeScript、TypeScript	自定义语法规则、编译成 JavaScript

👉 跨平台

跨平台指的是跨 H5 端、PC 端、移动端、物联网，对比如下，见表 3-4。

表 3-4

平 台	实 现	描 述
H5 端	纯移动前端	在手机浏览器上运营的 HTML5
PC 端	NW.js 和 Electron	Atom 和 VSCode 编辑器最为著名，像钉钉 PC 端、微信客户端、微信小程序 IDE 等都是通过 Web 技术打包成 PC 端的

续表

平 台	实 现	描 述
移动端	Cordova（旧称 PhoneGap），基于 Cordova 的 Ionic Framework	这种采用 H5 开发并打包成 IPA 或 APK 的应用，称为 Hybrid 应用（混合模式应用），通过 WebView 实现所谓的跨平台，应用广泛
物联网	Ruff.io	Ruff 是一个 JavaScript 运行时环境，专为硬件开发而设计

PC 端开发是最近两年才火起来的，通过 Web 技术开发跨平台 PC 端是一个相对不错的选型，除了节省成本，也非常迎合公司的长期发展策略，这里推荐两个 Electron 的开源项目。

Stacer 是 Linux 系统上的优化和监控软件，是基于 Electron 构建的。下面给出 Ubuntu 下的 Stacer 界面，如图 3-8 所示。

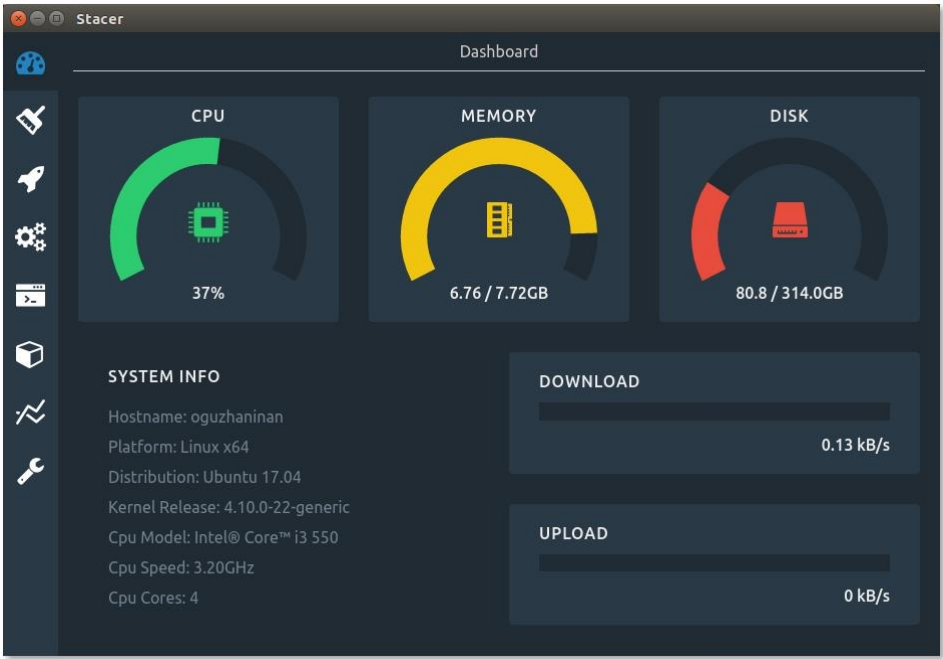


图 3-8

Medis 是 ioredis 的作者 luin（子骅）编写的 Redis 客户端，其技术选型非常时尚，采用 React 作为前端框架，采用 Electron 作为跨平台打包工具，是一个非常经典的例子，如图 3-9 所示。

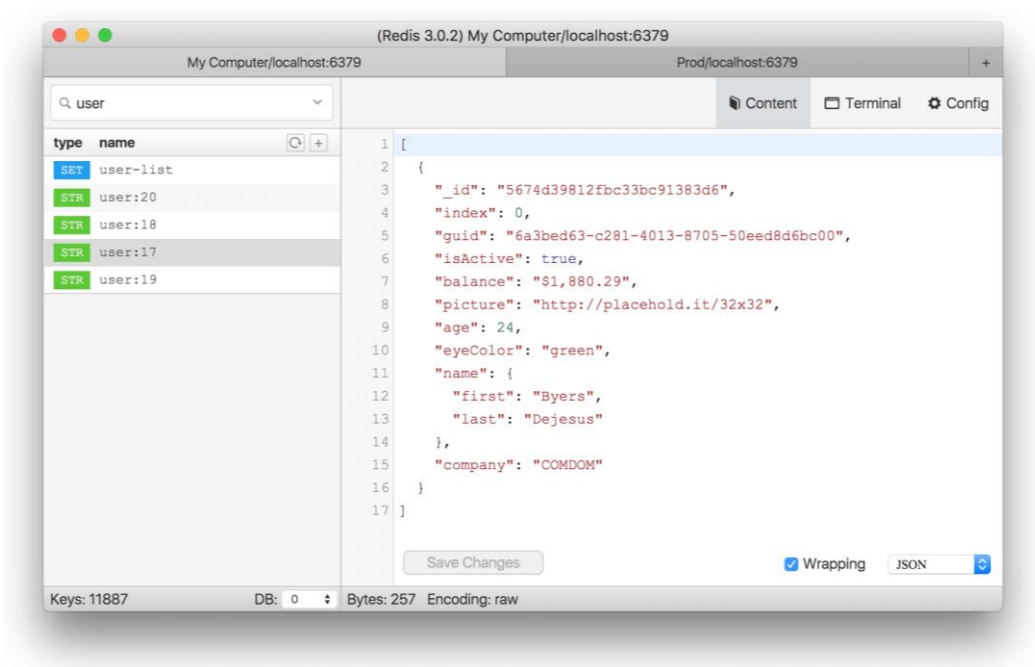


图 3-9

构建工具

说起构建工具，大概会想到 Make、Ant、Rake、Gradle 等，其实 Node.js 世界里有更多的实现，下面列举几种，如表 3-5 所示。

表 3-5

名 称	介 绍	点 评
Jake	熟悉 CoffeeScript 的大概都熟悉这个，和 Ant、Make、Rake 类似	经典
Grunt	DSL 风格的早期著名框架	配置非常麻烦
Gulp	流式构建，不会产生中间文件，利用 Stream 机制，处理大文件和内存有优势，配置简单，只要懂点 JavaScript 就能搞定	Grunt 的替代品
Webpack + npm scripts	说是构建工具有点过，但二者组合勉强算是，Loader 和 Plugin 机制还是非常强大的	流行

构建工具的源码实现都不会特别复杂，所以 Node.js 世界里有非常多的实现，还有人写过 Node.js 版本的 Make 呢，非常有趣！

3.2.3 后端开发

一般, 后端开发指的是 Web 应用开发中和视图渲染无关的部分, 主要是以数据库交互为主的重业务型逻辑处理。但架构升级后, Node.js 承担了前后端分离的重任, 于是便有了更多玩法。

从开发带视图的传统 Web 应用和面向 API 的接口应用, 到通过 RPC 调用封装对数据库进行操作, 再到提供前端 API 代理和网关、服务组装等, 这些都属于后端开发, 不是只和数据库打交道的部分才算后端开发。这样一来, 前端工程师可以更好地控制开发过程, 进行调优和性能优化。

对 Node.js 来说, 它一直没有在后端取得合理的占有率, 原因是多方面的, 其中几点如下。

- 利益分配冲突。已有的实现大多是基于 Java 或者其他语言的, 基本是没办法撼动的, 重写的成本是巨大的, 另外, 如果用 Node.js 编写, 那么那些写 Java 的人怎么办?
- Node.js 相对年轻, 大家对 Node.js 的理解不够, 回调和异步流程控制略麻烦, 很多架构师都不愿意花时间去学习。尽管对于 Web 应用的处理非常简单高效, 但在遇到问题时并不容易排查定位, 对开发者水平要求略高。
- 很多开发者是从前端开发领域转过来的, 技能单一, 数据库、架构方面的知识欠缺, 对系统设计也知之不多, 这是很危险的, 有种“麻杆打狼, 两头害怕”的感觉。
- Node.js 在科普、培训、布道等方面做得并不好, 国外使用的人非常多, 国内却很少有人知道。

尽管如此, Node.js 还是卷入了各种是非风口, 至今依然在大前端浪潮中“大红大紫”。原因在于, 它的定位非常明确, 补足了以 JavaScript 为核心的全栈体系中的服务器部分。开发人员也是人, 能够同时掌握并精通多门语言的毕竟不多, 而且能使用 JavaScript 一门语言实现所有功能, 为什么要学习更多呢? 相信很多人都有这样的想法。

👉 Web 应用

Web 应用大致分两种, 带视图的传统 Web 应用和面向 API 接口的应用, 我们先看一下 Node.js Web 应用开发框架的演进时间线, 大致如下。

- 2010 年, TJ Holowaychuk 编写了 Express。

- 2011 年, Derby.js 开始被开发, 同年 8 月, WalmartLabs 的一位成员 Eran Hammer 提交了 Hapi 的第一次 Git 记录。Hapi 原本是 Postmile 的一部分, 并且最开始是基于 Express 构建的, 后来发展成独立的框架。
- 2012 年 1 月, 专注于 RESTful API 的 Restify 发布了 1.0 版本, 同构的 Meteor 也进入开发阶段, 最像 Rails 的 Sails 也开始开发。
- 2013 年, TJ Holowaychuk 开始“玩”ES6 Generator, 编写了 co 这个 Generator 执行器, 并开始启动 Koa 项目。2013 年下半年, 李成银参考 ThinkPHP 开始开发 ThinkJS。
- 2014 年 4 月, Express 发布 4.0 版本, 进入 4.x 时代持续到今天, 随着 MEAN 架构的提出, MEAN.js 开始被开发, 意图实现大一统, 另外 Total.js 开始起步, 就像 PHP 里的 Laravel、Python 里的 Django 或 ASP.NET 的 MVC 框架, 这代表了 Node.js 的成熟, 开始向其他语言的成熟框架借鉴经验。
- 2015 年 8 月, Koa 框架发布 1.0 版本, 可以在 Node.js v0.12 以上版本中通过 co 和 Generator 实现同步逻辑模拟, 那时候 co 还是基于 thunkify 模块的。
- 2015 年 10 月, ThinkJS 发布了首个基于 ES2015+特性的 2.0 版本。
- 2016 年 9 月, 蚂蚁金服的 Eggjs 在 JSConf China 2016 上亮相并宣布开源。
- 2017 年 2 月, Koa 框架发布了 2.0 正式版。
- 2018 年 2 月, Koa 框架发布了 2.5 正式版。
- 2019 年 2 月, Koa 框架发布了 2.7 正式版。

我们可以根据框架的特性进行分类, 简单描述如表 3-6 所示。

表 3-6

框架名称	特 性	点 评
Express	简单、实用, 功能齐全	最著名的 Web 框架
Derby.js、 Meteor	支持同构	前后端都放到一起, 模糊了开发边界, 看上去更简单, 实际上对开发来说要求更高
Sails、Total	面向其他语言, 如 Ruby、PHP 等	借鉴业界优秀产品, 也是 Node.js 走向成熟的一个标志
MEAN.js	面向架构	类似于脚手架, 又期望同构, 结果只是“蹭”了热点

续表

框架名称	特 性	点 评
Hapi、Restfy	面向 API 和微服务	移动互联网时代 API 的作用被放大，故而独立分类，尤其是对于微服务开发而言更是利器
ThinkJS	面向新特性	借鉴 ThinkPHP，并慢慢走出自己的一条路，支持 async 函数等新特性，3.0 版本是基于 Koa v2.0 作为内核的
Koa	专注于异步流程改进	下一代 Web 框架
Egg	基于 Koa，在开发上有极大便利	企业级 Web 开发框架

对于框架选型，大家可以遵循以下原则。

- 考虑业务场景、特点，不必为了用而用，避免本末倒置。
- 考虑自身团队能力、喜好，有时候技术选型决定团队氛围，需要平衡激进与稳定。
- 保证在出现问题的时候有人能够做到源码级定制。Node.js 已经有 10 年历史，但模块完善程度良莠不齐，如果不慎踩到一个坑里，需要团队在无外力的情况解决问题，否则会影响进度。

个人学习求新，企业架构求稳，无非取决于喜好与场景而已。

Node.js 向后端延伸，必然会触动后端开发的利益。那么对于 Proxy 层（前后端矛盾的交界处）而言，后端不想变，前端又求变，长此以往，API 接口一定会变得越来越糟糕。后端开发人员认为 API 是前端负责的，对后端来说，只要不插手数据库和服务就可以。

但是 Node.js 能不能做这些呢？答案是能的，和 Java、PHP 类似，Node.js 一般是和数据库连接到一起处理业务逻辑的。目前国内大部分后端开发都以 Java、PHP 等为主，所以想要转变并不容易。

对于小型公司和创业公司而言，在新孵化的项目中更倾向于使用 Node.js，因为 Node.js 简单、快速、高效。对于微服务架构下的某些服务而言，使用 Node.js 开发是比较合理的。

国内在这个问题上一直没有做得很好，所以 Node.js 在大公司中还没有被很好地应用，存在安全问题、生态问题、历史遗留问题等，还有很多人对 Node.js 有一定的误解。关于这一点，我要解释一下。

- 单线程很脆弱，这是事实，但可以通过 Cluster 模块实现多核并发处理网络请求。
- 运维其实很简单，日志采集、监控也非常简单。
- 在模块稳定性上，Node.js 对 MongoDB、MySQL、Redis 等的支持还是相当不错的，但对其他的数据库的支持可能没那么好。
- 安全问题是一个伪命题，所有框架面临的安全问题都是一样的。

👉 API 代理

我猜大家能够想到的 Node.js 的应用场景，大概如下。

- 数据库 CRUD（增删改查）操作。
- 视图模板，提供静态 HTTP 资源托管。

如果只是做这些，那 Node.js 和 Java、PHP 等就没什么区别了。

有很多人认为 Node.js 不能用于后端开发，其实这不是 Node.js 的问题，而是由单一技能导致的。做后端开发也是需要具有一定沉淀的，对操作系统、数据库、运维等都有相当高的要求。

在复杂的系统里，API 也极其复杂，甚至会涉及跨部分协作，一般来讲会面临以下问题。

- 一个页面的 API 非常多。
- 跨域。
- API 返回的数据对前端不友好。
- 需求决定 API，API 反馈不及时。

产品经理需要应变，后端不易变，一变就会引起数据库、存储、数据处理等的改变，可能引发事故。而前端相对更容易变更，前端只负责组装服务，无须对数据库进行操作，所以只要服务 API 粒度合适，由前端处理是更好的。

API 即使差一些也不会影响功能，发布也相对容易，故而后端就成了应变的“利器”，俗称“背锅侠”。不同平台的交互不一样，API 也需要有所差异，因此最好同时提供多种 API，但对后端来说，能反馈一套 API 为什么还要写多套呢？后端懒得根据 UI/UE 去定制 API，最终还是

要将复杂工作交给“背锅侠”。

有了 Node.js, 对前端的支持会更友好。早在 2013 年左右, 淘宝技术曾给出一张架构图, 提出了构建模型 Proxy 层的说法, 今天看来, 依然不过时。将异构的 API (可能是 RESTful API, 可能是 RPC 服务) 组装到一起, 可以形成供前端使用的合理的新 API, 很明显这是非常好的, 当然也是前端开发者喜欢 Node.js 的一个理由。

在包含多种交互的情况下, 理想的情况是使用一样的模型, 提供基于模型的定制 API, 在 Node.js 层提供统一的模型, 然后再包装成前端需要的 API, 这样的 API 才是对前端友好的 API。

如图 3-10 所示, 图中左侧, 浏览器和 Node.js 服务通信间可以有多种协议, HTML、RESTful、BigPipe、Comet、Socket 等, 足够我们完成任何想做的事。图中右侧是 Node.js 实现的 WebServer, Node.js 服务分为如下两个部分。

- 由 Node.js 提供的 HTTP 服务。
- 模型 Proxy: 服务端的服务, 组成并转化成自身的模型层, 用于为 HTTP 服务提供更好的接口。

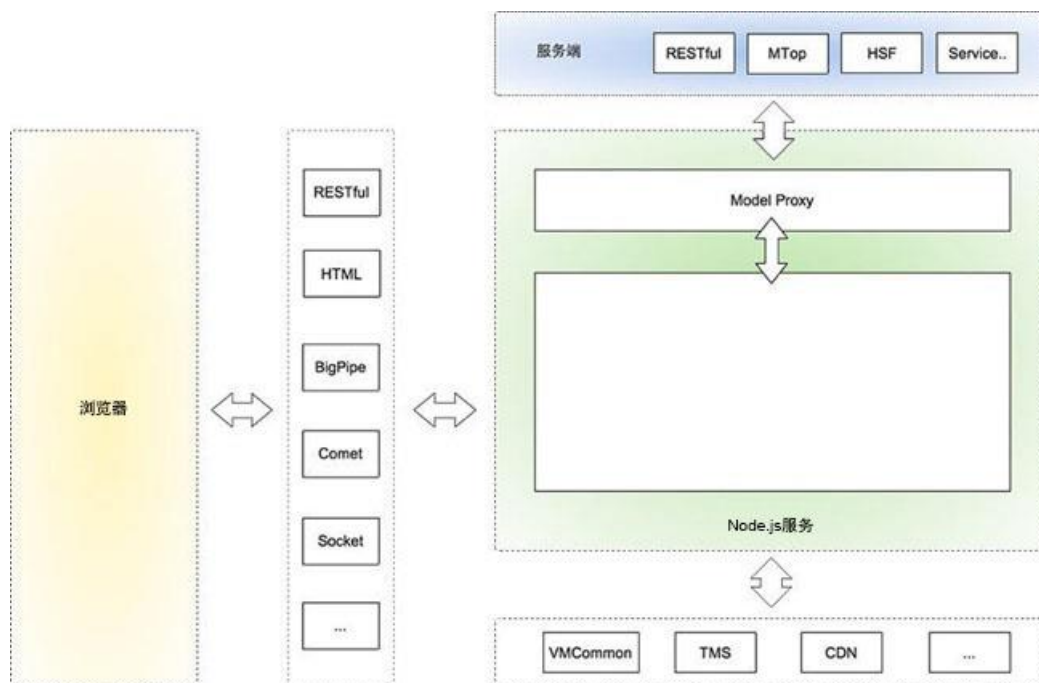


图 3-10

这里的 Model Proxy 其实就是我们所说的 API Proxy，这张图只说明了结果，即把聚合的服务转化成了模型，进而为 HTTP 服务提供 API。

在前端渲染之余，加一层 API Proxy 是非常必要的。

下面我们再深化一下 API Proxy 的概念，如图 3-11 所示。

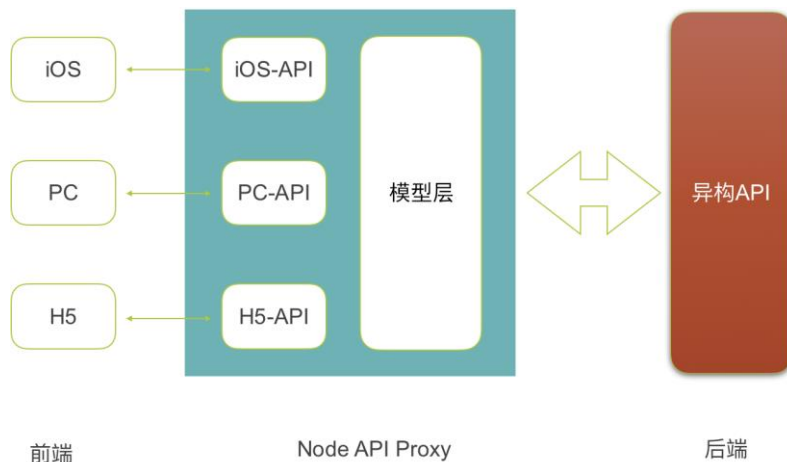


图 3-11

API Proxy 位于异构 API 之前，通过对异构 API 进行抽象，形成模型层，然后根据各种前端需求，包装成对应的 API 接口。最了解前端的只有前端，使用 API Proxy 层能够较好应变，在中间层进行各种转换处理，让 API 对前端开发更加友好。

👉 服务组装

在微服务架构十分流行的今天，服务化可谓最佳实践，应用非常多的便是基于 TCP 的 RPC 服务，相比于 HTTP 来说效率更高，再结合配置中心、服务注册等，能够很好地对系统进行解耦。微服务更是在业务边界分拆方面做到了极致，以业务维度来拆分服务，能够实现小而美，做到快速迭代。那么另外一个问题就来了，复杂的跨服务业务怎么处理呢？这时候就需要对服务进行组装。

梳理模型层和定制 API 都在 API Proxy 层实现，那么服务组装也在 API Proxy 层实现是否更好呢？答案是肯定的。无论采用哪种语言，服务组装的代价几乎一样，那么为什么不用 Node.js 来实现呢？

如图 3-12 所示, 这里的 Node Proxy 做了两件事, 分别是输出 API 和渲染辅助。

- 输出 API: 前端的异步 Ajax 请求可以直接访问 API。
- 渲染辅助: 如果是直接渲染, 或者采用 BigPipe、WebSocket 协议等, 需要在服务器端组装 API, 然后再将结果返回给浏览器。

所以 API 后面还有一个服务组装, 在微服务架构十分流行的今天, 将服务组装放到 Node Proxy 里的好处尤其明显, 既可以提高前端开发效率, 又可以让后端更加专注于服务开发。如果前端团队足够大, 还可以专门组建一个 API 小组, 从事与服务集成相关的工作。

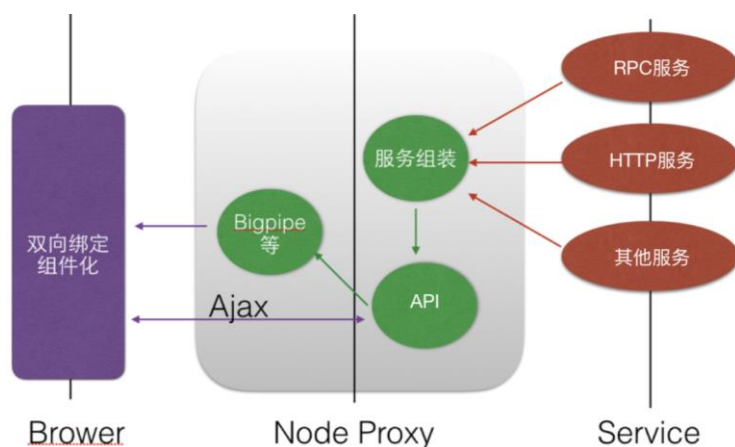


图 3-12

API 网关

Node.js 擅长 I/O 操作, 其 http 模块和 stream 模块组合使用时非常适合作为代理软件。通常, 编写很少量的代码就可以实现一个功能强大的代理。

API 网关中有两个重要功能, 请求转发和跨域 JSONP 支持。

关于请求转发, 我们来看一个简单实用的代理示例。

```

const http = require('http')
const fs = require('fs')

const app = http.createServer((req, res) => {
  if ('/remote' === req.url) {
    res.writeHead(200, {'Content-Type': 'text/plain'})
  }
})
  
```

```

    return res.end('Hello Remote Page\n')
  } else {
    proxy (req, res)
  }
})

function proxy (req, res) {
  let options = {
    host : req.host,
    port : 3000,
    headers: req.headers,
    path : '/remote',
    agent : false,
    method : 'GET'
  };

  let httpProxy = http.request(options, (response) => {
    response.pipe(res)
  })

  req.pipe(httpProxy)
}

app.listen(3000, function() {
  const PORT = app.address().port

  console.log(`Server running at http://127.0.0.1:${PORT}/`)
})

```

`http.request` 方法的返回值是 `<http.ClientRequest>`，它继承自 `OutgoingMessage`，因此可以得到如下结论。也就是说，`http.request` 方法的返回值和 `res` 是一样的。

```
http.request => http.ClientRequest => OutgoingMessage => Stream
```

关于以上的代码，其中的重点如下

- `httpProxy = http.request()` 即一个新的 `res`。
- `req.pipe(httpProxy)` 通过 `pipe` 方法，使得 `req` 有了新的代理请求，即 `httpProxy`。
- `httpProxy` 里面的是一次完整的 HTTP 请求，于是就有了 `httpProxy` 的 `response`。
- `response.pipe(res)` 将 `res` 放到 `response` 流里，完成代理功能。

说到底，`req` 和 `res` 还是原本的状态，只不过中间让 `httpProxy` 拦截了一层而已，这就是代理的真相。请求转发其实就是代理，对于 API 网关来说，在这个基础上进行扩展即可。

如图 3-13 所示, 关于跨域 JSONP 支持, 首先要理解以下几点。

- 跨域最通用的做法是采用 JSONP, 对于 API 接口而言, 将原有的 API 包装一下即可, 但对于这样一个极其简单的包装, 开发者往往会预估 0.5 人/天时间, 而且提测部署还需要时间, 所以让后端来做这件事就显得很不合适。
- 最简单的做法是在页面所在的 Node.js 服务器上使用 Node.js 作为客户端来进行请求, 最后将请求结果以 JSON 或 JSONP 的形式返回。很多公司都是这样做的, 十分简单。
- 采用上面所说的方式时, 页面 URL 如果被 Nginx 重写了几次, 是无法找到真实的 Node.js 服务器的 (不是不能, 是代价极高), 那么此时该如何去做呢? 很简单, 将页面服务和 node-proxy 转成同域的就可以了, 用 Nginx 再合适不过了。
- 上面的解决方案看起来很好, 但是多了一层 Nginx, 为什么一定要将页面服务和 node-proxy 转成同域的呢? 具体该怎么做呢? 很简单, 让 node-proxy 作为独立域名提供 JSONP 服务即可, 做到页面和接口分离。

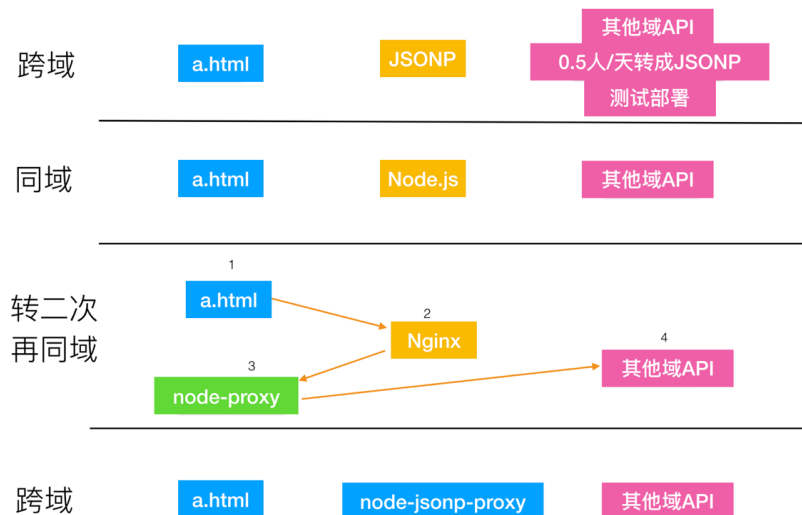


图 3-13

如果 node-proxy 返回的是普通 JSON API, 依然可以采用 Nginx 将页面服务和 node-proxy 转成同域的; 如果 node-proxy 返回的是普通 JSONP API, 且是独立域名, 则所有页面都可以直接访问 JSONP, 如图 3-14 所示。

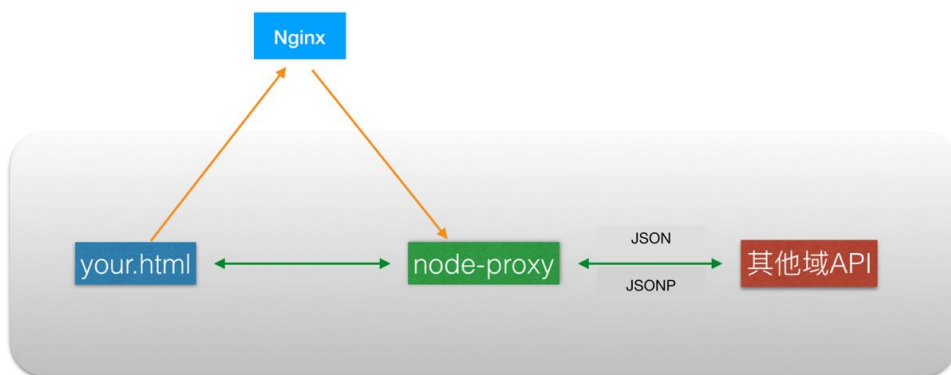


图 3-14

因此，我们特别想获得一个如下所描述的服务。

- 使用 Node.js 4.x 以上版本编写，性能高效。
- 重写解决跨域问题。
- 直接重写成 JSONP 接口。
- 支持 HTTP 和 HTTPS 转换。
- 配置化，无须部署。
- 收益 = (x 个接口个数 × 0.5 人/天) + (y 项目部署次数 × 项目部署时间)

是不是很好呢？这样一来，将 Node.js 作为中间层无疑是令前后端都感到舒服的做法。

👉 Serverless

Serverless 是一种基于互联网技术的架构理念，采用函数即服务 (Function as a Service, FaaS) 架构理念，让开发者只关注应用逻辑，而不必将全部功能都在服务端实现，通过组合多个函数的功能来实现应用程序逻辑。采用 Serverless 架构能够让开发者在构建应用的过程中无须关注服务器运维，有效节省开发成本，其优势总结如下。

- 节约使用成本。
- 简化设备运维过程。
- 提升可维护性。

在 Node.js 的世界里, 每个文件都是一个模块, 那么集成 Serverless 概念能不能让每个文件都成为一个服务呢? 通过预定的各种 API SDK, 统一自动化运维环境, 是不是可以让开发更简单呢?

对于 Node.js 应用落地, 可以先开发部分功能, 也可以花一些精力将完整的开发流程打通, 对于公司的整体架构是非常有益的。如图 3-15 所示, 从 Mock API 开始, 如果有对应的 Serverless 服务实现, 可以直接切换。但目前各个框架都没有在减少运维成本上做出更多努力, 略显可惜。如果未来都统一到 Service Mesh 上, 让前端开发人员不再担心运维难度大的问题, 将更多精力投入到业务开发上, 便可以拓宽前端的业务边界, 完全承担 RPC 服务以外的所有工作也是有可能的。

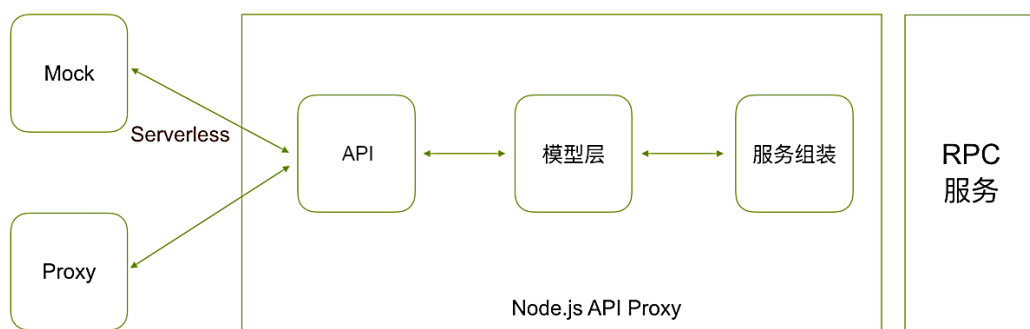


图 3-15

3.3 更多乐趣

编程是很枯燥的事, 只有“玩”出乐趣才不会觉得累。前面我们讲了很多 Node.js 能做的事, 比如架构升级、现代 Web 开发、后端开发等, 但 Node.js 能做的远不止这些, 本节会列出更多应用场景, 让 Node.js 可以做更多有趣的事!

3.3.1 更多应用场景

Node.js 的应用场景非常丰富, Node.js 可以开发操作系统, 但一般这样做的人很少。通常我习惯将 Node.js 的应用场景分为以下 7 类。

- 作为基础设施用于前端开发。
- 作为命令行辅助工具。

- 用于移动端（Cordova）和 PC 端（NW.js 和 Electron）。
- 实现前端组件化，完成组件的打包构建，增加 HTTP 代理等功能。
- 用于架构设计，实现前后端分离。
- 性能优化，反爬虫与爬虫。
- 实现全栈。

之前已经讲过很多了，这里再列出一些具体应用场景，如表 4-7 所示，以便读者能够更好地了解 Node.js。

表 3-7

编 号	场 景	描 述
1	反向代理	Node.js 可以作为反向代理，像 Nginx 一样，但我们很少在线上这样做。其实使用 Node.js 做这种请求转发是非常简单的，在后面的章节里会单独讲解
2	爬虫	有大量的爬虫模块，比如 node-crawler 等，写起来比 Python 要简单一些，搭配 jsDOM（Node.js 版本的 jQuery）类库时对前端来说尤其友好
3	命令行工具	所有辅助开发、运维、提高效率且可以用 CLI 做的，也都可以用 Node.js 来开发，是编写命令行工具最简单的方式，Java 8 以后也参考了 Node.js 的命令行实现
4	微服务与 RPC	Node.js 里有各种 RPC，比如 dnode、Seneca，也有跨语言支持的 gRPC，足够应用
5	微信公众号开发	相关 SDK、框架非常多，是快速开发的利器
6	前端流行的 SSR 和 PWA	SSR 是服务器端渲染，PWA 是渐进式 Web 应用，都是前端领域最火的技术。如果大家用过 SSR 和 PWA，一定对 Node.js 也不陌生。比如 React、Vue 都是用 Node.js 实现的 SSR，PWA 的很多模块也是用 Node.js 实现的。那么为什么不用其他语言实现呢？不是其他语言不能实现，而是使用 Node.js 简单、方便、学习成本低，能轻松获得高性能

3.3.2 C/C++扩展

Node.js 和其他语言一样，对 C/C++的支持特别好，提供了两种机制用于编写扩展。

- NAN（Native Abstractions for Node.js）：由 rvagg 牵头编写的 Node.js 扩展抽象层，使得 Node.js 扩展开发更简单，几乎不需要了解 Node.js 源码细节即可编写。
- N-API：Node.js 8 开始支持，借助 Application Binary Interface（ABI）虚拟机实现“一次编译，多个版本运行”，主要解决了 Node.js 扩展模块跨版本的编译问题。

举个 node-java 的例子, 调用 Redis 服务时, 一般公司都会为了安全而定制 Java 版本的访问库, 会对鉴权、加密解密做各种定制, 如果前端想调用, 就只有两种办法。

- 使用 Java 提供 HTTP 或 RPC 服务来调用 (前提是能提供, 有时候没法提供)。
- 参照 Java 版本的功能, 自己写一个 Node.js 客户端 (实现成本太高, 如果不懂 Java 代码和各种加密解密算法, 几乎是写不来的)。

还有另外一种解决方案, 即采用 node-java 模块, API 是 JavaScript 的, 但实际调用的是 Java 代码, 是不是听起来还挺酷的? Node.js 可以通过 NAN 或 N-API 调用 C/C++代码, 而 Java 也可以在 C++代码里调用 Java 代码, Java 代码又可以通过 Java 自身的反射机制获得类对应的属性和方法。也就是说, Node.js 通过一系列过程最终可以获得 Java 类里的属性和方法, 剩下就是实例化, 方法调用包装而已, 在后面使用 Node.js 编写扩展的章节中会详细讲解。

类似的还有很多, 本来前端不擅长 (比如多线程) 或者未曾做过的事, 都可以考虑用 Node.js 去实现, 这其实极大地解放了前端的创造力, 可以说大前端如果没有 Node.js 的助力不会有今天的繁荣。

3.3.3 团队优化

前端开发从角色独立到慢慢有部门, 到大前端涵盖各种和用户交互的部分, 再到使用 Node.js 进行各种优化, 这个过程中涉及的人数也在递增, 并且在未来还有很大的增长空间, 那么如何管理好前端团队就成了一个很大的问题, 尤其是在技术发展如此快的今天。

在前端团队引入 Node.js 的好处是非常明显的, 下面列出几条, 如表 3-8 所示。

表 3-8

	没使用 Node. js 之前	使用 Node. js 之后
工程化	各种 Hack, 没有资深工程师甚至就没法做	前端开发人员可以自行实现, 可以掌控所有流程, 固化结构, 最终适合大团队协作
模块化	基本靠约定	有了 npm、Bower 这样的包管理器, 有了 cnpm 这样的自建 npm 源, 支持私有的 Scope, 有多模块管理工具 Lerna 等, 实现模块化非常容易
工具化	各种 Hack, 没有资深工程师甚至就没法做	以极小的代价获得工具代理的高效性
工作氛围	很平常	富有创造力, 促进沟通, 对学习力有极大的提高

引入 Node.js 对于部门规划也是极好的，我喜欢用“进可攻退可守”来形容。一般前端做好 PC 端和 H5 端开发是本职工作，向前可以做组件化，主导移动端开发，PC 客户端也可以这样做。另外可以用 Node.js 的后端能力辅助开发，比如 API 模拟、检测、组装服务等，给前端更多自由。

一个好的团队才会吸引更多更好的开发者加入，大前端引入 Node.js 是团队优化的最佳手段。引入 Node.js 之后，团队成员都开始进行自主学习了，几乎不需要付出很多心力就能够营造一个健康向上的工作氛围。

3.3.4 全栈之路

讲了 Node.js 工具、前端 4 阶段、各种跨平台，就是为了介绍 Node.js 全栈的各种可能，下面讲一下如何能做到 Node.js 全栈。

要想做到全栈，核心在于以下两点。

- 掌握后端不会的 UI（界面）。
- 掌握前端不会的 DB（业务）。

只要打通这两个要点，其他就比较容易了。最怕的是哪一样都接触一点，然后就号称自己是全栈专家，建议大家不要这样，这就好比在简历里写“精通”，然后基本上都会被问到尴尬是一样的。全栈是一种信仰，不是拿来吹牛的，而是用来解决更多问题、应对更多变化的，做到全栈可以让自己的知识体系不留空白，享受自我实现的极致快乐。

👉 我的全栈之路

说起我的全栈之路，大概可以概括为以下几个阶段。

- 就业之痛：没有目标就向“钱”看，有目标就向前看。
- 迷茫之时：既然无法逃避，就热爱，最后变成兴趣。
- 而立之年：人生不只有代码，但它能让我快乐，终生受益。

在这个过程里面，我也曾懵懂，也曾迷茫，但我一直信奉“一次只做一件事，尽力做到极致”，短时间内是比较枯燥的，但一旦坚持下去，就会发现技术其实是门手艺，厚积才能薄发。

我没办法说自己最擅长什么,但我知道在什么场景下应该用什么技术。或者说,应变是我最大的本事。很多框架、新技术我都没见过、没用过,但花一点点时间浏览一下,我就能用已有的知识快速理解,这其实是长期学习带来的好处。

现在越来越忙,写代码的时间越来越少,技术发展也越来越快,我能做好的就是每日精进,用这些已有的知识储备跟年轻人比赛。我不觉得累,相反我很享受这种感觉,确认自己还没有被时代淘汰,这是一件多么幸福的事啊!

👉 从后端转

做后端开发的人对数据库是比较熟悉的,无论 MongoDB,还是 MySQL、Postgres,而对前端理解比较薄弱,可能只会基本的 HTML、CSS、模板引擎等。因此,若想从后端人员变为全栈专家,要牢记——4 阶段循序渐进,build 与工具齐飞!

前端开发的 4 阶段,我的感觉是,按照顺序循序渐进学习的效果最好。

👉 从前端转

从前端往后端转,API 接口非常容易学会,像 Koa 这类框架大部分人一周也能学会,最难的是对 DB、ER 模型的理解,即对业务需求落地的理解。

我们来想想一般的前端开发人员具备什么技能。

- ☐ HTML。
- ☐ CSS (兼容浏览器)。
- ☐ JavaScript 会一点 (可能更多的是会点 jQuery)。
- ☐ PS 切图。
- ☐ Firebug 和 Chrome Debugger 会的人不太多。
- ☐ 用过几个框架,大部分人是仅仅会用。
- ☐ 英语一般。
- ☐ 会一点 SVN 或 Git。

那么他们如果想在前端领域做得更深入，有哪些难点呢？

- 基础薄弱，如面向对象、设计模式、命令行工具、shell 编程、工程化构建等。
- 对编程思想的理解不够，MVC、IoC、约定大于配置等。
- 区分概念困难。
- 外围验收困难，如 H5 和 Hybrid 等。
- 难以追赶趋势，不知如何学习新东西。

以上皆是痛点，所以比较好的办法应该是下面这样的。

- 玩转 npm、Gulp 这样的前端工具类（此时还是前端）。
- 使用 Node.js 进行前后端分离（此时还是前端）。
- 掌握 Express、Koa 这类框架。
- 掌握 Jade、EJS 等模板引擎。
- 使用 Nginx。
- 玩转后端异步流程处理。
- 玩转后端 MongoDB、MySQL 对应的 Node.js 模块。

从我们的经验来看，这样做是比较靠谱的。先做最简单的前后端分离，里面没有任何和 DB 相关的内容，前端可以非常容易地学会。半年后，接触异步流程处理和数据库相关内容，学习后端代码，这样就可以做到全栈了。

👉 从移动端转

让我们一起来看一下移动端的发展过程，具体如下。

Native（原生开发）→ Hybrid（混搭开发）→ React Native/Weex → H5

目前 React Native 和 Weex 开发逐渐变得主流，组件化写法已经由前端主导了，国内强运营需求刺激新技术不断产生，这些新技术非常有前途。以前 iOS 和 Android 程序员占比很高，但现在只留一两个写插件的人足矣，真是差别很大。

那么面对这样的转变，该怎么办呢？要么忍！要么转！在温水里舒服了几年，也该学点东西了。Hybrid 或组件化开发，总要会一样。无论会哪种，你都离前端很近，因为 H5 或组件化都是从前端走出来的。组件化在前端领域先行，无论借鉴还是学习都不可避免。如果没时间就直接学习组件化，如果有时间就好好学学前端的完整体系，但最终也还是要学习组件化的。

所以从移动端转为全栈最好从 Cordova 开始，先学习 Hybrid 开发。

- 只要关注 `www` 目录里的 H5 即可，比较简单。
- H5 不足以应对的情况下，可以编写 Cordova 插件，即通过插件让 JavaScript 调用原生 SDK 里的功能。
- Cordova 的 CLI 可以通过 npm 安装，是学习 npm 的好方法。
- 学习 Gulp 构建工具。

只要入了 H5 的坑，其实就非常好办了。先学习 H5、Zepto、iScroll、FastClick 等，然后学习微信开发常用的 WeUI、Vux (Vue+WeUI)、JMUI (React+WeUI) 等，接着“玩”点框架，比如 jQuery Mobile、Sencha Touch，再来点高级货，比如 Ionic Framework (基于 Angular、Cordova)，然后进入前端 4 阶段，依次“打怪升级”，最后掌握 Node.js。

这个基本上是我走的路，是我从 2010 年写 iOS 和 PhoneGap (当时是 0.9.3)，一路走到现在的总结。

以前技术发展还不是那么快，写 Java 代码的时候，Apache 的开源用得比较多，那时开源的代码托管 SourceForge，Google Code 也凑合用，自从 Git 和 GitHub 出现，代码社交兴起，极大促进了开源的发展，使得大量明星项目脱颖而出。这是好事，如果没有开源，中国的软件水平可能要落后好多年。那么问题也来了，如何能在技术快速发展的今天，同时获得更好的个人成长呢？

学习有 3 种层次，跟人学最快，其次是跟书（或者博客）学，最慢的是自悟。但是牛人不容易遇到，遇到了也未必有精力教你，书本或者博客上的知识也有限，而对于自悟，如果没有深厚的积累，也是有相当大难度的。

对于开发者来说，代码是一切的基础，在掌握了一定的计算机基础后，人与人之间的差别就在于代码编写能力和眼界。编程没有捷径，能够做到每日精进就是极好的。现在开源代码非常多，能够从中获取自己所需的知识，也是一种本领！如果能够坚持每日精进，其实根本不需

要向其他人学习。

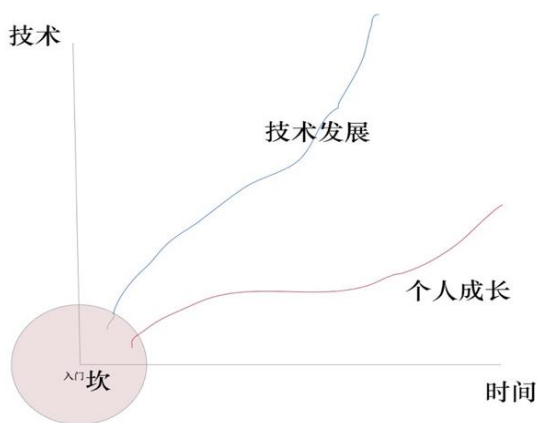


图 3-16

大家可以在 GitHub 上随便打开一个前端项目，里面有一半以上都是与 Node.js 相关的，各种包管理、测试、CI、辅助模块，如果大家对这些基础信息掌握得非常好，那么学习一个新的框架就会比别人快很多，最重要的是可以“学习一次，到处使用”。

很多人问我怎么才能成为一个 Node.js 专家，我的回答是“如果能在 CNode 论坛上坚持写文章和开源项目两年，一定能轻松进入 BAT，不用你找他们，他们自会找你”。那么，从今天起，开始重视开源项目，重视 Node.js，做到每日精进吧！

3.4 本章小结

按照 SWOT 分析法（也称道斯矩阵）来分析今时今日的 Node.js，很多人觉得它是金牛型产品，但在我看来，它现在处在从明星型产品到金牛型产品的转变过程中，距离成为金牛型产品还有一段路要走。

通过本章的分析，我们有理由相信，未来的 Node.js 会越来越好！

第 4 章

更好的 Node.js

2015 年 6 月，ES6 规范经由 TC39 标准委员会批准，成了 JavaScript 语言版本的一次最大升级，也正因如此，ES6 对 Node.js 开发也影响巨大。Node.js 自身集成的 Chrome V8 开始支持更多的 ES6 特性，另外，npm 上的大量模块和疯狂的前端开发者也在时刻提醒 Node.js 开发者：你有更好的开发方式。

下面就让我们一起来看一下更好的 Node.js 吧！

4.1 选择

Node.js（或者说 JavaScript）给我们带来了足够的选择空间，这是其他语言很难实现的，本节我们将从写法、是否适合开发大型软件，以及如何实现快速开发 3 个方面进行介绍。

4.1.1 语法可难可易

Node.js 采用 JavaScript 语法，所以在写法上非常灵活，开发者的选择非常丰富。至于采用哪种方式，要看团队的技术水平、战略规划和信仰。

进阶的 3 个阶段分别是面向过程、面向对象和函数式编程。对于从 0 开始的团队来讲，可以先面向过程，然后随着团队的成熟一点一点增加难度，甚至可以使用各种转译器（CoffeeScript、TypeScript、Babel 等）。当然，如果极客范儿十足，或者在某些特定场景下，使用函数式编程也是相当不错的选择。

JavaScript 仿佛是一门很“烂”的语言，因为上面的 3 个阶段，它做得都不好。但在不同阶段它都能陪伴你成长，在你需要的时候，它可以给你更好的选择，这样看来，还有什么比这更

好吗？！

👉 面向过程

使用 Node.js 世界著名的 Web 框架 Express 编写的代码就是典型的面向过程的代码，了解基本语法，几乎可以无障碍阅读和编写，唯一要花点心思的就是理解 Express 框架的中间件机制，示例如下。

```
const express = require('express')
const app = express()

app.get('/', function (req, res) {
  res.send('Hello World!')
})

app.listen(3000, function () {
  console.log('Example app listening on port 3000!')
})
```

这是典型的快而脏(Quick and Dirty)的代码，“简单粗暴”，能够更快地实现你想要的功能，无论你的水平如何，都能快速上手。

从这一点上来看，Node.js 无疑是对新手极其友好的平台。其实上面这样的代码用在真正的业务中也是可行的，除了看起来不太优雅，其余的也没有大碍。如果想让代码更加优雅，我们可以使用 Koa 框架。

👉 面向对象

当熟悉了 Node.js 语法之后，就会慢慢学习面向对象的写法，这是进阶的必经之路。有一本著名的红皮书《JavaScript 高级程序设计》，里面讲的一个重点内容就是 JavaScript 的面向对象用法。对于 JavaScript 这种基于原型的面向对象语言来说，它不是纯粹的面向对象语言，写法很多变，有时甚至需要自己去编写面向对象机制。对此，Node.js v5.x 之后的版本开始支持 ES6 里的 class 和 extends 关键字，可以更简单地编写面向对象程序，示例如下。

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
```



```
    console.log(this.name + ' makes a noise.');
```

```
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    console.log(this.name + ' 在叫.');  }  
}  
  
var d = new Dog('大米');  
d.speak(); // 大米 在叫.
```

ES6 里虽提供了类、继承、static 方法等，但只能说“凑合着用”，离真正的面向对象还差得很远。如果说非要使用面向对象的写法，一般都会采用一些更好的转译器来实现，比如有 Ruby 开发经验的开发者会采用 CoffeeScript，有 Java、C# 开发经验的会采用 TypeScript，这里尤其要推荐一下 TypeScript，它无缝兼容 ES6 语法，并且提供静态类型、接口、反射、泛型等特性，不需要我们自己编写更多关于类机制的实现，在代码质量、抽象程度等方面都表现得比较好，但学习成本稍高一些。

👉 函数式编程

JavaScript 作为一种典型的多范式编程语言，随着这两年 React 的火热，其函数式编程的概念也开始流行起来，RxJS、CycleJS、LodashJS、UnderscoreJS 等多种开源库都使用了函数式编程的思想。

下面定义一个名为 calcAvgCost 函数，代码如下。

```
const map = fn => array => array.map(fn);  
const prop = key => object => object[key];  
const reduce = (fn, initial) => array => array.reduce(fn, initial);  
const add = (x, y) => x + y;  
const sum = reduce(add, 0);  
const filter = fn => array => array.filter(fn);  
  
const average = items => (  
  items.length === 0  
    ? 0  
    : sum(items) / items.length  
);  
  
const flow = (...fns) => x => (  
  fns.reduce((result, fn) => fn(result), x)
```

```
);  
  
const calcAvgCost = (items, filterFn = () => true) => (  
  flow(  
    filter(filterFn),  
    map(prop('price')),  
    average  
  ) (items)  
);
```

关于以上代码，要点如下。

- 第一等公民是函数。
- 函数柯里化 (curry): 传递给函数一部分参数来调用它，让它返回一个函数去处理剩下的参数。
- 为了解决函数嵌套的问题，代码中使用了“函数组合”，calcAvgCost 和 flow 函数里的都是典型的“函数组合”。

calcAvgCost 函数的用法如下。

```
const items = [  
  { name: 'Motherboard', manufacturer: 'A', price: 65 },  
  { name: 'CPU', manufacturer: 'A', price: 240 },  
  { name: 'DRAM', manufacturer: 'B', price: 100 },  
  { name: 'CPU', manufacturer: 'B', price: 150 },  
];  
  
const avgCost = calcAvgCost(items);  
  
const avgCostCPU = calcAvgCost(items, item => (  
  item.name === 'CPU'  
));  
  
const avgCostB = calcAvgCost(items, item => (  
  item.manufacturer === 'B'  
));  
  
const avgCostCPUFromA = calcAvgCost(items, item => (  
  item.name === 'CPU' && item.manufacturer === 'A'  
));
```

用起来很简单，但写的时候是挑战传统思维习惯的(国外计算机入门时就学习函数式编程，而国内则是学习 C 语言类的编程语言)，不过也不得不承认，使用函数式编程写出的代码确实漂亮，其抽象程度也足够高。

如果是个人学习,我非常鼓励大家尝试函数式编程方式,函数式编程的思想在前面讲的两
种编程方式里一样是有借鉴意义的。如果是在团队中应用,还是要谨慎选择,持观望态度,原
因有两个:一是学习成本高;二是维护成本高。

4.1.2 开发大型软件

在学习 Node.js 的过程中,大家一定会想,Node.js 是否适合编写大型软件应用呢?我们先
来看一下开发大型软件应用的基础有哪些。

- 关于测试: TDD、BDD、测试覆盖率、持续集成是非常成熟的,相关模块和框架更是
数不胜数。以测试框架为例,有老牌的 Mocha,有对新特性支持最好的 AVA,还有 Jest
和 Jasmine 等,你可以按照自己的喜好进行选择。对于开源项目来说,可能需要和
CircleCI 或 TravisCI 进行持续集成,一般公司内部会采用 Jenkins 来实现持续集成或持
续发布。目前来看,其他语言支持的相关特性,Node.js 几乎都支持。
- 关于代码质量和规范: Standard 模块、XO 模块、Lint 工具、Hint 工具,以及编辑器,
对代码规范的支持已经非常好了,这些前端开发中的最佳实践可以直接复用到 Node.js
项目里。
- 关于构建: Gulp、Grunt、Webpack、Shipit 中都有大量插件,足以应对复杂的前端项
目,一般的 Node.js 项目远远没有前端项目复杂,因此上述这些构建工具都可以放心使
用。尤其推荐 Gulp,它是目前最主流的构建软件,基于流式构建,高效,可扩展,简
单易用。
- 生成器: 可以使用 Yeoman、Slush、Vue CLI、SAO 等,自己动手定制也非常常见,本
书后面会详细讲解。
- 包管理工具: npm、Bower 简单易用,足够用于 Node.js 中。

对于开发大型软件应用,Node.js 有足够的实现能力,它提供了良好的基础和包管理工具,
以及大量可以直接使用的成熟模块,事实证明 Node.js 非常适合用于开发大型软件应用,具体
如下。

- 具备开发大型软件应用的基础。
- 所有云计算公司都爱 Node.js。

- 各大名企，如阿里巴巴、腾讯、百度、Uber 等都大量应用 Node.js。
- 生态完善，性能调优方面有 node-clinic 和 alinode 助力。

4.1.3 特定场景下的快速开发

开发速度快慢取决于很多因素，不能一言以蔽之。以 Web 应用开发为例，在 Node.js 领域有大量 Web 开发框架，大多是小而美型的，缺少像 Rails 这样成熟的集成度高的一站式框架。

于是，很多人把 MEAN 技术栈组合起来，这样做的好处是，在熟悉这些技术和约定的前提下可以大幅提升开发速度，但缺点是实现难度大。

Metetor 是著名的同构 Web 框架，在 GitHub 上比 Node.js 的 star 数还要多，其影响力可见一斑，它模糊了服务端和客户端的界限，是同构技术的典型应用，对于实时应用场景的快速开发来说是非常高效的。但 Metetor 也有和 MEAN 组合一样的问题——集成度过高，导致性能调优非常困难。

对于同构而言，现在也要辩证地看，React 和 Vue 实现前端组件化后，客户端渲染（CSR）普遍被采用，但出于性能和 SEO 的考虑，服务端渲染（SSR）也是必需的，这种场景下，Next.js 或 Umi SSR 的这种基于组件的同构解决方案显得尤其重要。

以上这些框架都是特定场景下的提速方案，一般不敢轻易使用，调优难度非常大，如果有一天能搞定它们的源码级开发，那么，它们一定会被广泛应用。

很多时候，我们并不需要使用多么复杂的框架，对于实现特定场景下的快速开发，掌握两个要点即可：第一，保证手法熟练，简单易行；第二，约定用法。

我们可以根据业务需要来定义一个框架，约定好用法，将一些提高效率的工具、脚手架与之集成，如果能实现和自动化运维部署集成起来就更好了。这种框架几乎每个公司都有，虽然水平参差不齐，但绝大部分都是非常高效的，能够应对公司的业务变化。

使用 Node.js 实现特定场景下的快速开发是非常适合的，具体如下。

- 在写法上，可易可难，满足各类人群，是有助于团队成长的选择。
- 可以开发大型软件，开源生态已相当成熟。
- 可快可慢，可以满足特定场景下快速开发的要求。

这些优点已经足够应对绝大部分的使用场景了，我们有足够多的选择，有足够大的上升空间，这对团队的成长来说是非常有利的，也从侧面印证了上一章中我们讲过的 Node.js 对团队的优化作用。

4.2 单线程会“死”吗

单线程非常脆弱，随便一点异常都会使其“挂掉”，不幸的是 Node.js 就是单线程的，所以经常被人诟病“太脆弱，动不动就崩溃”。但是，真的是这样的吗？

在线上高可用的项目中，Node.js 是可以非常好地应对并发的，如果了解用法，就不会出现单线程脆弱的问题。

下面举一个最简单的单线程的例子，为大家演示正确的用法。

首先，在终端执行如下代码，创建 Node.js 项目标准流程。

```
$ npm init -y
$ npm install --save koa@next
$ touch app.js
```

首先进行 npm 初始化，无论项目大小，它自己必须是一个模块，必须要有 package.json。使用 npm 安装 Koa 模块，安装之后才可以使用。创建入口文件 app.js，读取文件成功后将返回“Hello Koa”字符串，代码如下。

```
const fs = require('fs');
const Koa = require('koa');
const app = new Koa();

app.use(ctx => {
  fs.readFile('somefile.txt', function (err, data) {
    if (err) throw err;
    console.log(data);

    ctx.body = 'Hello Koa';
  });
});

app.listen(3000);
```

执行 node app.js 启动服务器，然后访问 http://127.0.0.1:3000/，页面上会显示 Not Found，代码如下。

```
$ node app.js
/Users/sang/workspace/github/modern-nodejs/app.js:8
  if (err) throw err;
            ^

Error: ENOENT: no such file or directory, open 'somefile.txt'
    at Error (native)
```

如果只有一个接口，出问题也认了，可绝大部分情况下一台服务器上至少有几个、几十个，甚至更多接口，不能因为一个接口异常就连累所有的接口都无法使用！

一定要注意，这里是通过 `node app.js` 启动 `app.js` 的，很明显只启动了一个 Node.js 进程，Node.js 又是单进程的，所以实际上本次操作只使用了一个线程来处理 Node.js 代码的具体逻辑，如果代码出错，那这个线程肯定是要崩溃的。这时你一定会在心里抱怨：“多核服务，启动单一进程，太浪费了！”

4.2.1 uncaughtException

其实，捕获 `uncaughtException` 异常就可以解决上面提到的问题，改进后的 `app2.js` 代码如下。

```
const fs = require('fs');
const Koa = require('koa');
const app = new Koa();

app.use(ctx => {
  if (ctx.path === '/good') {
    return ctx.body = 'good'
  }
  fs.readFile('somefile.txt', function (err, data) {
    if (err) throw err;
    console.log(data);

    ctx.body = 'Hello Koa';
  });
});

process.on('uncaughtException', function (err) {
  console.log(err);
})

app.listen(3000);
```

使用 `process.on('uncaughtException', function (err) {})` 就不会造成接口崩溃了，可惜的是，很多应用在开发时都没有做这样的基本处理，因此都出现了问题！

4.2.2 异常捕获

异常是一种安全的分支处理技术，一旦应用程序出现状况且不及时处理，程序就会中断执行，即产生异常。产生异常的语句以及之后的语句将不再执行，默认情况下会进行异常信息的输出，然后自动结束程序的执行。

对于开发人员来说，即使程序出现了异常，也要让程序正确执行完毕。

下面来看一下改进的 app3.js 代码如何捕获并处理异常，具体如下。

```
const fs = require('fs');
const Koa = require('koa');
const app = new Koa();

app.use(ctx => {
  if (ctx.path == '/good'){
    return ctx.body = 'good'
  }

  fs.readFile('somefile.txt', function (err, data) {
    try {
      if (err) throw err;
      console.log(data);

      ctx.body = 'Hello Koa';
    } catch (e) {
      // 这里捕获不到 readCallback 函数中抛出的异常
      console.log(e)
    } finally {
      console.log('离开 try/catch');
    }
  });
});

app.listen(3000);
```

在 fs.readFile 的回调函数里加入 try/catch 异常处理后，当读取文件遇到异常时，就会直接抛出，而抛出的异常会被 try/catch 捕获，当前线程就不会因异常而意外结束了。

关于以上代码，还有几个注意要点，说明如下。

- Node.js 里约定，同步代码才能捕获异常，异步代码不能直接使用 try/catch（与你采用的异步流程控制方式有关，如果使用 Promise，就使用 Promise 的异常处理方法）。

- 代码中都是 try/catch 也有弊端，比如 Go 语言代码就有太多异常捕获。
- 使用 try/catch 成本较高，除非必要，一般不建议使用（主要针对 Node.js 8.0 以前的版本，后面的版本进行了优化）。

4.2.3 forever

进程因异常退出是很常见的事，当遇到崩溃退出的时候，重启就可以了。Node.js 中很早就有专门负责进程崩溃应用自动重启的模块，名为 **forever**。先全局安装这个模块，然后终端中就有了 **forever** 命令。

```
$ [sudo] npm install forever -g
```

以 Koa 常见入口文件 **app.js** 为例，执行 **node app.js**。

```
$ forever start app.js
```

此时访问 **http://127.0.0.1:3000/**，页面显示 **Not Found**——崩溃。然后使用 **forever** 处理 **crash** 事件，再启动一个新的 Node.js 进程，于是又可以处理请求了。也就是说，除了这种会崩溃的请求，对其他请求不会有任何影响。可以说，使用 **forever** 将服务不可用的风险降到了最低。

forever 就是“打不死的小强”，曾经非常流行，现在大部分都改用 **PM2** 了，**PM2** 支持所有 **forever** 支持的功能，并且功能更加强大，比如实现 0 秒切换等，是目前部署 Node.js 时最常用的模块。

4.2.4 小集群：单台服务器上多个实例

上面讲的都是单机单个实例。现在的单台服务器大多是多核的，所以无法充分利用多核优势。比较好的办法就是使用 **Cluster** 模块。**Cluster** 模块（集群）是 Node.js 在 v0.10 之后就有的模块，专门用于解决多核并发问题。

大家知道，**Nginx** 或 **HAProxy** 等集群都是一主多从的，主机的端口通过负载均衡算法将请求转发到 **Slave** 机器上。在一台机器上启动多个实例，原理也是类似的。

这里推荐使用 **PM2** 模块，绝大部分的产品环境部署都使用 **PM2** 模块，以前有很多项目用 **forever**，现在大部分都转为 **PM2** 了。

首先通过下面的命令全局安装这个模块，然后终端中就有 **PM2** 命令了，如图 4-1 所示。


```
$ [sudo] npm install pm2 -g
$ pm2 start app.js -i 0 --name "modern-nodejs"
```

```
+ modern-nodejs git:(master) x sudo pm2 start app.js -i 0 --name "modern-nodejs"
Password:
[PM2] Spawning PM2 daemon
[PM2] PM2 Successfully daemonized
[PM2] Starting app.js in cluster_mode (0 instance)
[PM2] Done.
```

App name	id	mode	pid	status	restart	uptime	memory	watching
modern-nodejs	0	cluster	50199	online	0	0s	29.234 MB	disabled
modern-nodejs	1	cluster	50200	online	0	0s	29.102 MB	disabled
modern-nodejs	2	cluster	50204	online	0	0s	26.074 MB	disabled
modern-nodejs	3	cluster	50208	online	0	0s	17.637 MB	disabled

```
Use `pm2 show <id|name>` to get more details about an app
+ modern-nodejs git:(master) x
```

图 4-1

是不是非常简单？当访问 <http://127.0.0.1:3000/> 时，页面显示 Not Found——崩溃，此时看一下 PM2 的状态，如图 4-2 所示。

```
+ modern-nodejs git:(master) x sudo pm2 ls
```

App name	id	mode	pid	status	restart	uptime	memory	watching
modern-nodejs	0	cluster	50199	online	0	3m	29.477 MB	disabled
modern-nodejs	1	cluster	50200	online	0	3m	29.473 MB	disabled
modern-nodejs	2	cluster	50204	online	0	3m	29.301 MB	disabled
modern-nodejs	3	cluster	50208	online	0	3m	29.309 MB	disabled

```
Use `pm2 show <id|name>` to get more details about an app
+ modern-nodejs git:(master) x sudo pm2 ls
```

App name	id	mode	pid	status	restart	uptime	memory	watching
modern-nodejs	0	cluster	51053	online	1	1s	28.969 MB	disabled
modern-nodejs	1	cluster	50200	online	0	3m	29.473 MB	disabled
modern-nodejs	2	cluster	50204	online	0	3m	29.301 MB	disabled
modern-nodejs	3	cluster	50208	online	0	3m	29.309 MB	disabled

```
Use `pm2 show <id|name>` to get more details about an app
+ modern-nodejs git:(master) x
```

图 4-2

可以看到，第一个线程的 restart 数显示为 1，也就是说它也有和 forever 一样的功能，当崩溃的时候会自动创建新的线程来继续服务。

PM2 非常强大，支持无缝重载、0 秒切换等，可实现各种监控、部署等功能。

4.2.5 大集群：多台机器

为了应对大流量，需要多台机器进行集群处理，因此可以通过负载均衡策略将流量分发到

各个机器上，通过消除单点故障提升应用系统的可用性。常见的集群处理方式是使用 Nginx 或 HAProxy 这样的软件。如果应用是部署到阿里云上的，可以直接使用阿里云提供的 SLB（Server Load Balancer）负载均衡服务对多台云服务器进行流量分发。如果深入研究，会发现 SLB 其实是基于 Tengine 的，Tengine 是阿里巴巴维护的 Nginx 的优化版本。

和使用其他语言做负载的方式一样，前端的所有请求都会落到负载层，由负载层转发到负载中可用节点的服务器上，如图 4-3 所示。

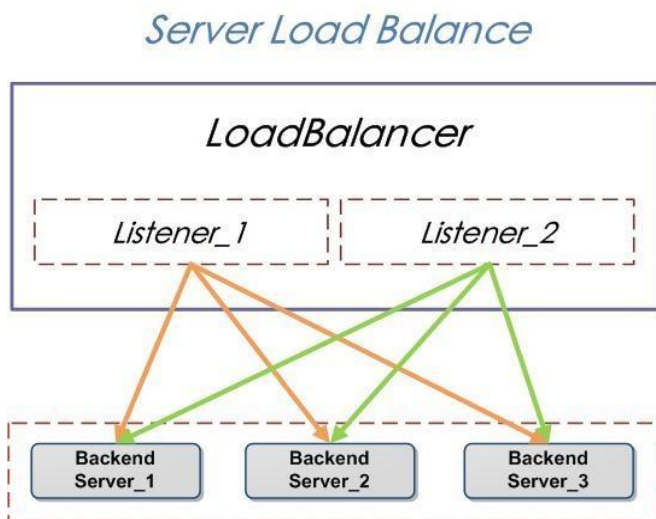


图 4-3

唯一需要注意的就是，在做负载均衡的时候一般需要提供健康检查，大致方式是在服务器里提供 `check_health.html` 或者通过 HEAD 请求来检测实际的服务器节点是否存活，以此作为判断负载节点是否可用的依据，这是极其常见的手段。

“单线程会死”是一个伪命题，大部分时候是用法不当造成的。关于单线程问题，为大家总结几个要点，具体如下。

- 单个应用实例可以适当捕获异常，减少崩溃几率。
- 单个应用实例崩溃之后，采用 `forever` 或 `PM2` 自动重启，可以继续服务。
- 利用多核集群同时在一台服务器上启动多个实例，崩溃的几率极低。
- 应用线上部署就只部署一台服务器吗？这种几率其实也很小，多台服务器也要做集群。

- 如果所有集群中的服务器都崩溃了该怎么办？这其实是运维的问题，和 Node.js 无关。

结论是，Node.js 并不脆弱，只是因为它是单线程的，所以和其他语言的处理方式稍有不同而已！

4.3 为 Node.js 正名

Node.js 自诞生以来就一直处于风口浪尖之上，是技术领域里的明星项目，所以各种误解、“八卦”自然也免不了。本节将和大家分享几个关于 Node.js 的常被议论的话题，以正视听。

4.3.1 版本帝？

Node.js 是名副其实的版本帝，版本更新确实很频繁，时间线如下。

- 2013 年，发布了 0.10 版本。
- 2015 年 1 月，发布了 1.0.0 版本 (io.js)。
- 2015 年 5 月，发布了 2.x 版本 (io.js)。
- 2015 年 8 月，发布了 3.x 版本 (io.js)。
- 2015 年 9 月，Node.js 基金会发布了 5.0 版本与 io.js 合并后的第一个版本。
- 2015 年 10 月，Node.js 5.2.0 版本成为首个 LTS（长期支持）版本。
- 2015 年年底，发布了 5.2.4 和 5.5.0 版本。
- 2016 年 3 月，发布了 5.5.0 LTS 版本和 5.9.0 Stable 稳定版本。
- 2016 年年底，6.0 版本支持 95% 以上的 ES6 特性，7.0 版本通过 flag 支持 async 函数，全面支持 99% 的 ES6 特性。
- 2017 年 2 月，发布了 7.6 版本，可以不通过 flag 使用 async 函数。
- 2017 年 5 月，发布了 8.0 版本，支持 async Hooks，N-API 等特性。
- 2018 年 4 月，发布了 10.0 版本，新增 http2 模块，将 npm 从 v5.7 更新到了 v6，并且增强了对 ESM Modules 的支持。

- 2018 年 10 月，发布了 11.0 版本，增加了多线程 Worker Threads。

整体上来说，Node.js 的发展趋于稳定。成立 Node.js 基金会能够让 Node.js 在未来获得更好的开源社区支持；发布 LTS 版本意味着 Node.js SDK API 趋于稳定；频繁发布版本虽然被很多人诟病，但换个角度来看，这也是社区活跃的一个体现，如果大家真的看了 Changelog，便会发现，新版本相比于旧版本只增加了一些小的改进，而且是边边角角的改进，也就是说，Node.js 的核心代码已经非常稳定了，可以大规模使用。

4.3.2 已无性能优势？

Node.js 在 2009 年横空出世，可以说靠异步特性获得了很大的性能优势。所有语言几乎没有能和它相比的。但是福祸相依，因为性能太出众，所以促使很多语言、编程模型都纷纷进行改进，比如产生了 Go 语言，比如 PHP 里的 Swoole 框架可以支持异步协程了，再比如鸟哥（惠新宸）对 PHP 的 VM 进行了改进，大家似乎都以不支持异步为耻。后来的故事大家都知道了，各种语言的性能都得到了提高。

那么在这种情况下，Node.js 还有优势吗？

在实现难易度上，Node.js 除了异步流程控制稍复杂外，其他的都非常简单。比如在写法上，你可以选择编写面向过程、面向对象、函数式的程序。不要因为 Node.js 变化快，就觉得自己跟不上潮流。一般后端程序员转为 Node.js 开发人员时，几乎两周就能精通，这一点相比其他语言还是很有优势的。

在调优成本上，Node.js 即使不进行优化，性能也非常好，另外，对 Node.js 进行优化也比其他语言更简单。

在学习成本上，Node.js 是有优势的。学习其他语言，前后端至少要学两种以上，如果学习 Node.js，你只需要学会 JavaScript 即可，可以少学一种语言。我想问，大前端离得开 JavaScript 吗？今日的前端还不够复杂吗？你真的有那么多精力学习更多语言吗？

其实大家可以关注一下基于 npm 的开源生态，截至 2019 年 3 月，npm 上已有超过 94.7 万个模块，“秒杀”无数竞品。npm 是所有开源包管理中最强大的，我们说“更了不起的 Node.js”，其实 npm 居功甚伟。

图 4-4 展示了来自 Module Counts 的各个包管理模块的差异。

npm 生态是 Node.js 的优势，可是说“Node.js 没有性能优势”真的对吗？这其实是对 Node.js

的误解。Node.js 的性能依然很好,不断迭代的版本其实就是在提升性能。而且 Node.js 具有 npm 极其完善的生态,可谓性能与生态双剑合璧,这是无与伦比的。

Module Counts

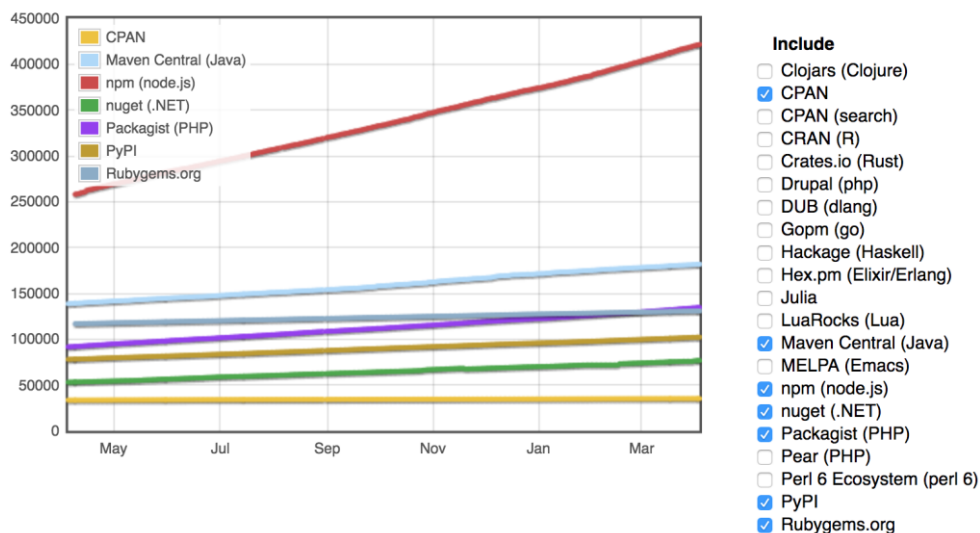


图 4-4

4.3.3 异步和回调地狱

天生异步, 成也异步, 败也异步!

正因为异步特性, Node.js API 设计只能采用错误优先 (Error-First) 风格的回调约定, 于是大家硬生生地把多层回调写成了回调地狱 (callback hell), 这时就有各种“黑粉”冒出来对 Node.js 进行攻击。

但正是因为回调地狱是最差的实践, 所以大家才不得不求变, 于是 Thunk 函数、Promise/A+ 规范等相继出现。虽然 Promise/A+ 规范不是那么完美, 但对于解决回调地狱问题来说已经足够。而且 Generator 特性和 Generator 的执行环境 co 模块也被逐渐引入新的异步解决方案, 使得异步在写法上越来越接近于同步。当 async 函数落地的时候, Node.js 已经站在了同 C#、Python 一样的高度上, 大家还有什么理由攻击它呢?

表 4-1 列举了 Node.js 支持的所有异步解决方案, 并给出了推荐建议 (5 星为最高级别)。

表 4-1

解决方案	描 述	推荐指数
callback	Node.js API 天生就是这样的，异步必须要约定错误优先风格的回调	3 星
Thunk	编译器的“传名调用”实现，通常做法是将参数放到一个临时的函数之中，再将这个临时函数传入函数体	1 星
Promise	最开始是社区定义的 Promise/A+规范，大家习惯称之为 Promise，随后成了 ES6 语言标准	4 星
Generator	ES6 中的生成器，用于计算，但 TJ 想将其用作流程控制	1 星
co	Generator 用起来非常麻烦，故而 TJ 编写了 co 这个 Generator 生成器，用法更简单	2 星
async 函数	原本计划进入 ES7 规范，结果差一点，好在 Node.js 8 实现了，所以 Node.js v7 以上的版本都可以使用，无须等 ES7 规范落地	5 星

从推荐指数可以看出，我们应首选 async 函数，但要注意版本问题，要使用最新的版本。其次就是 Promise，它都能非常好地驾驭 callback 和 async 函数，尤其是在异常捕获、扩展上，具有明显的优势。

有时，将一件事做到极致，也许能收获另一片天地。异步流程控制是 Node.js 编程的核心，掌握异步流程控制之后，Node.js 中就只剩 API 需要学习了，后面会详细讲解。

4.3.4 技术栈演进

自从 ES6 规范在 Node.js 中落地之后，整个 Node.js 开发领域都发生了翻天覆地的变化。从 v0.10 开始，Node.js 中就逐渐加入了 ES6 特性，比如 Node.js v0.12 可以使用 Generator，这也促使寻求异步流程控制的 TJ Holowaychuk 写出了 co 这个著名的模块，进而产生了 Koa 框架。但是在 v5.0 之前，必须通过 flag 才能开启 Generator 支持，因此 Koa v1.0 迟迟未发布，在 Node.js v5.0 发布后，Koa v1.0 才发布。

2015 年，传统写法终结；2016 年，变革写法开始兴起。其中核心变更是支持使用 ES6 语法编写 Node.js 代码。

- 可以使用 Node.js v5.x+里的 ES6 特性，如果想实现更高级的功能，可以使用 Babel 编译支持 ES7 特性，或者使用 TypeScript。
- 合理使用 Standard 或者 xo 模块代码风格约定。

- 适当引入 ES6 语法，只要 Node.js SDK 内置支持的，都可以使用。
- 大家要重视面向对象写法的使用，虽然 ES6 的面向对象机制不健全，但以后定会不断完善。面向对象对于大型软件开发更适合，这其实也是我推荐使用 TypeScript 的原因之一。

表 4-2 对比了变革前后的技术栈选型，希望读者能够从中感受到其中的变化。

表 4-2

分 类	2015 年	2016 年	选型原因
Web 框架	Express v5.x	Koa v1.0 和 v2.0	在流程控制上十分便利
数据库	Mongoose (MongoDB)	Mongoose (MongoDB)	对 MongoDB 和 MySQL 的支持都一样，不过 MongoDB 更简单，足以应付绝大部分场景
异步流程控制	Bluebird (通过 Promise/A+实现)	Bluebird (通过 Promise/A+实现) Koa v1.0 使用 co + Generator, Koa v2.0 使用 async 函数	从 Promise 到 async 函数，无论如何演进，Promise 都会存在，是必须要掌握的知识点
模板引擎 (视图层)	EJS 和 Jade	Jade 和 Nunjucks	这里列举了两种，一种可读性好，另一种简单高效，都是非常好的
测试	Mocha	AVA	Mocha 是 Node.js 里著名的测试框架，但对新特性的支持没有 AVA 那么好，而 AVA 基于 Babel 安装，更“高大上”
调试方式	使用内置的 node-inspector	使用 VSCode 编辑器	在 Node.js v6 和 v7 发布之后，对 node-inspector 的支持变得不是很好，相反 VSCode 编辑器使用简单，文件多时也不卡顿，体验很好

4.4 更好的实践

本节将总结一些 Node.js 开发中的更好的实践，首先要从 ES6 的新语法特性以及必备知识开始，然后讲解类型系统、模块管理器 Learn，以及 npm 的替代品 Yarn，这些应该都是当下最流行的技术，称它们为更好的实践，名副其实。

4.4.1 ES.next

Chrome V8 团队致力于让 JavaScript 演变成一门表达能力强、定义明确，更容易开发出高效、

安全、正确的 Web 应用编程语言。2015 年 6 月，ES6 规范经由 TC39 标准委员会的批准，成为 JavaScript 语言版本的一次最大升级。这次升级为 JavaScript 带来了许多新特性，包括 Class、箭头函数、Promises、Iterator/Generator、Proxies、Symbol 和一些额外的语法糖。TC39 标准委员会也加快了新规范定稿的节奏，并于 2016 年 2 月发布了 ES7 草案。由于发布周期较短，与 ES6 相比，ES7 并没有增加太多的新特性，比较引人注意的是增加了乘方运算符和 `Array.prototype.includes()`。

至此，JavaScript 引擎发展历程迎来了一个重要的里程碑：Chrome V8 支持了 ES6 和 ES7。我们知道，Node.js 使用 Chrome V8 作为 JavaScript 解释引擎，也就是说 Chrome V8 的语法在对应的 Node.js 版本里都被支持。

ES6 和 ES7 等下一代的 ECMAScript 规范通常为 ES.next，是浏览器和 Node.js 里使用的下一代 JavaScript 语言规范。无论是出于学习还是实践目的，这些 ES.next 特性都是值得开发者关注的。使用这些 ES.next 特性来编写 Node.js 代码也是开发者乐于接受的。

通过 <http://node.green> 网站，我们能够清楚知道 Node.js 各个版本对 ES6 特性的支持情况，如图 4-5 所示。

		9.0.0	8.2.1	7.10.1	7.5.0	6.11.2	6.4.0	5.12.0	4.8.4	0.12.18	0.10.48
		99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete
optimisation											
proper tail calls (tail call optimisation)											
direct recursion	❌	Error	Error	Flag	Flag	Flag	Error	Error	Error	Error	Error
mutual recursion	❌	Error	Error	Flag	Flag	Flag	Error	Error	Error	Error	Error
syntax											
default function parameters											
basic functionality	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
explicit undefined defers to the default	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
defaults can refer to previous params	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
arguments object interaction	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
temporal dead zone	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
separate scope	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
new Function() support	❌	Yes	Yes	Yes	Yes	Yes	Yes	Error	Error	Error	Error
rest parameters											
basic functionality	❌	Yes	Yes	Yes	Yes	Yes	Yes	Flag	Flag	Error	Error
function 'length' property	❌	Yes	Yes	Yes	Yes	Yes	Yes	Flag	Flag	Error	Error
arguments object interaction	❌	Yes	Yes	Yes	Yes	Yes	Yes	Flag	Flag	Error	Error
can't be used in setters	❌	Yes	Yes	Yes	Yes	Yes	Yes	Flag	Flag	Error	Error
new Function() support	❌	Yes	Yes	Yes	Yes	Yes	Yes	Flag	Flag	Error	Error

图 4-5

ES6 特性兼容情况见表 4-3。

表 4-3

Node.js 版本	支持 ES6 特性比例	说 明
v5.x 以下	v0.12 最多支持 31%，而且很多特性需要使用--harmony flag 来开启	兼容没太大意义
v5.x	对 Class 等 ES6 特性的支持占比为 57%，可以使用，但很多要通过严格模式或使用--harmony flag 来开启	v5.x 作为首个 LTS 版本，有大量用户使用，所以还是要尽量支持
v6.x~8.x	支持 90%以上的特性，几乎不需要 Babel 转译就可以用	基本不太需要考虑兼容性

在使用 Node.js 模块的时候，一定要留意 package.json 里的 engines 属性，它会强制指定兼容的 Node.js 版本信息，比如下面的配置就要求 Node.js 必须是 5.x 以上版本的。

```
"engines": {
  "node": ">= 5.0.0"
},
```

我们自己在写模块的时候，也需要考虑向下兼容的版本，一般也会这样配置，保证至少兼容到 Node.js 5.x 版本。

👉 转译工具

对于很多开发者来说，最理想的情况就是使用 ES.next 特性进行开发，又不用关注 Node.js 版本的兼容性，这并非不可行。最好的解决办法就是使用 ES.next 特性开发环境，在生成环境里使用转译工具将 ES.next 特性转译为 Node.js 版本都支持的 ES5。

将 ES6 降级到 ES5 的转译工具如下。

- Traceur: 由 Google 开发，第一个实现 ES6 到 ES5 降级的转译器。到目前为止已经不常更新，支持大约 59%左右的 ES6 特性。
- Babel: 最流行的 ES6 到 ES5 的转译器，这和它最早支持 React 的 JSX 语法有关。Babel 通过插件式架构，将 ES.next 特性加入独立插件，是目前支持 ES.next 特性最好的转译方式。它在前端开发中应用极多，用来编写 Node.js 也是非常好的，像 ThinkJS 和 Koa 这些比较新潮的框架，一般都会采用 Babel 来开发。
- TypeScript: 兼容 ES.next 的超集，而且可选的支持变量类型更适合开发大型项目，比 Babel + flow 的组合性能更好。另外，它不仅支持从 ES6 到 ES5 的转译，甚至还可以转译到 ES3。如果有 Java 开发经验，用 TypeScript 会更加得心应手。

ThinkJS 就是非常典型的结合 npm scripts 和 Babel 一起使用的框架，示例代码如下。

```
"scripts": {
  "test": "npm run eslint && npm run test-cov",
  "test-cov": "istanbul cover ./node_modules/mocha/bin/_mocha -- -t 50000 --recursive -R spec test/",
  "compile": "babel src/ --out-dir lib/",
  "watch-compile": "npm run compile -- --watch",
  "watch": "npm run watch-compile",
  "prepublish": "npm run compile",
  "eslint": "eslint src/"
},
```

关于以上代码，有两个要点，说明如下。

- 在开发环境中，执行 npm run watch。
- 在产品环境中，执行 npm run compile。

👉 语法说明及必会要点

ES.next 的特性还是相当多的，这里简单介绍，如表 4-4 所示。

表 4-4

分 类	内 容	重要程度
基础	理解 call、apply、bind 方法，理解 const	5 星
面向对象基础	掌握类、继承、static 方法、上下文绑定	4 星
流程控制	掌握 Generator（可选）、Promise、async 函数	4 星
语法糖	了解箭头函数（嵌套不深时可以使用，否则请参考函数式编程）、解构赋值、Proxies、Symbol，以及各种数据结构辅助方法	3 星

call、apply、bind 实在是基础到不能再基础的知识点了，还有 Object.defineProperty、Delegates 和 only 也是常用的必会知识点，在阅读 Node.js 源码时会经常遇到它们，下面具体介绍。

1. call 和 apply

call 和 apply 都是为了改变某个函数运行时的 context（上下文）而存在的，换句话说，就是为了改变函数体内部 this 的指向而存在的，很多自己实现的面向对象机制是借助它们来完成的，在 Node.js 模块中出现频率是极高的。

它们的区别在于，第二个参数的形式不同，如下所示。

```
obj.call(thisObj, arg1, arg2, ...);  
obj.apply(thisObj, [arg1, arg2, ...]);
```

`apply` 的第二个参数为数组形式, 函数会自动将数组展开; `call` 的第二个参数为不定参数, 需要传递几个就传递几个。 `apply` 的具体用法如下。

```
function test(arg) {  
  console.log(this.variable + arg);  
}  
var obj = {  
  variable : 1  
};  
  
test(); // NaN  
test.apply(obj, [1]); // 结果是 2  
test.call(obj, 2); // 结果是 3
```

2. bind

`bind` 方法创建了一个新的函数, 当函数被调用时, 会将 `this` 关键字设置为提供的值, 调用新函数时, 会在方法调用之前提供一个给定的参数序列。ES5 开始就引入了 `bind` 方法, 它的用法和 `call` 的用法类似, 即第二个参数是可变参数, 原型如下。

```
obj.bind(thisObj, arg1, arg2, ...);
```

把 `obj` 绑定到 `thisObj`, 这时 `thisObj` 便具备了 `obj` 的属性和方法。与 `call` 和 `apply` 不同的是, `bind` 绑定之后返回绑定完成的函数, 不会立即执行, 需要再显式执行一次此函数才能完成调用。

使用上面的函数定义, 调用如下。

```
var c = test.bind(obj, 3);  
c(); // 结果是 4
```

除了这种用法, 在 ES6 的 `Class` 里也经常需要绑定上下文, 比如下面的例子。

```
class BigView extends BigViewBase {  
  constructor(req, res, layout, data) {  
    super(req, res, layout, data);  
    ...  
  }  
  
  ...  
  
  start() {  
    debug('BigView start');  
  
    return this.before()  
      .then(this.renderLayout.bind(this))
```

```

        ...
        .catch(this.processError.bind(this))
    }

    before() {
        debug('default before');
        return PROMISE_RESOLVE;
    }

    renderLayout() {
        return PROMISE_RESOLVE;
    }

    processError() {
        return PROMISE_RESOLVE;
    }
};

```

首先，**BigView** 是一个类，在类里面，**start** 作为入口会完成一整串的流程，这串流程是基于 **Promise** 实现的，按照 **Promise** 规范，只要保证每个子流程返回的都是 **Promise** 对象即可。这种情况下，在 **then()** 方法里，子流程的上下文是丢失的，此时需要手动绑定，即使用 **this.renderLayout.bind(this)**，这在面向对象编程时几乎是不可避免的。

其实，还有更简单的做法，使用基于 ES7 的 **Decorators** 特性封装的 **autobind-decorator**，可以免去对类上下文的手动绑定，代码如下。

```

import autobind from 'autobind-decorator'

class Component {
  constructor(value) {
    this.value = value
  }

  @autobind
  method() {
    return this.value
  }
}

```

这样我们便通过 **@autobind** 实现了对 **method** 方法 **this** 上下文的注入。

```

let component = new Component(42)
let method = component.method // 不需要.bind(component)
method() // 返回 42

```

当然，你也可以对整个类里的所有方法进行上下文绑定，注意要考虑对性能的影响。

```
@autobind
class Component { }
```

ES7 中已经淘汰了 bind 的写法, 而使用两个冒号“::”来代替, 用法更简单, 例如::obj.method。

3. Object.defineProperty

Object.defineProperty 是 ES5 中新增的一个 API, 其作用是给对象的属性增加更多的控制。在我们日常的工作中, 用到这个 API 的地方不多, 然而它对于 MVVM 框架中的双向数据绑定 (two-ways data binding) 来说是至关重要的, 目前 Vue 和 Avalon 中的双向数据绑定均是通过它来实现的。

Object.defineProperty 语法如下。

```
Object.defineProperty(obj, prop, descriptor)
```

其中的参数说明如下。

- **obj**: 需要定义属性的对象 (目标对象)。
- **prop**: 需要被定义或修改的属性名 (对象上的属性或者方法)。
- **descriptor**: 需要被定义或修改的属性的描述符。

在 EventEmitter 源码 lib/event.js 里有一个经典示例, 非常实用, 具体如下。

```
// 默认 EventEmitters 只有 10 个监听者
var defaultMaxListeners = 10;

Object.defineProperty(EventEmitter, 'defaultMaxListeners', {
  enumerable: true,
  get: function() {
    return defaultMaxListeners;
  },
  set: function(arg) {
    // 强制让全局的 console 编译
    // 查看 https://github.com/nodejs/node/issues/4467
    console.log('defaultMaxListeners is set to ' + arg);
    // 检查输入参数 arg 的类型是否为 number
    if (typeof arg !== 'number' || arg < 0 || arg !== Math.floor(arg)) {
      throw new TypeError('"defaultMaxListeners" must be a positive number');
    }
    defaultMaxListeners = arg;
  }
});
```

4. delegate

在 JavaScript 里，可以将一个对象的方法、属性等委托给另一个对象。Koa 的上下文对象 ctx 可以访问一些 request 或 response 上的方法、属性，原因在于，request 或 response 上的方法、属性被委托给了 ctx 对象。

先获得 delegate 引用对象，命令如下。

```
const delegate = require('delegates');
```

具体事项如下。

```
const proto = {  
  
}  
  
/**  
 * Response delegation.  
 */  
  
delegate(proto, 'response')  
  .method('redirect')  
  ...  
  .access('body')  
  ...
```

将上面的代码委托给 response 上的属性，具体用法如下，后面会有多处用到该方法。

```
app.use(function(ctx, next) {  
  ctx.body = 'xxx'  
  //ctx.redirect('/url')  
})
```

5. only

在 Koa 源码里经常见到 toJSON() 方法的具体实现，代码如下。

```
toJSON() {  
  return only(this, [  
    'method',  
    'url',  
    'header'  
  ]);  
}
```

这里的 only 是一个 Node.js 模块，用于返回对象的白名单属性，尤其适用于“类的方法非常多但只想暴露部分属性”的情况。虽然源码只有 10 行，但这样的 API 用起来还是相当舒服

的，是遵循“小而美”原则的典范。

4.4.2 类型系统

知乎平台上有一句“名言”：动态类型一时爽，代码重构火葬场。

诚然，若没有严格的类型形态，对于团队协作来说是非常痛苦的，即使有各种代码质量检测环节、代码规范工具，仍然阻止不了一些低级错误的产生，因此大家对 JavaScript 是痛恨的。与 Java 相比，Java 虽然死板，但不会出现这么低级的错误。

首先复习一下 JavaScript 的类型系统，如表 4-5 所示。

表 4-5

对 象	类 型	实例对象	Object.prototype.toString	标 准
"abc"	String	——	String	String
new String("abc")	Object	String	String	Object
function hello(){}	Function	Function	Function	Object
123	Number	——	Number	Number
new Number(123)	Object	Number	Number	Object
new Array(1,2,3)	Object	Array	Array	Object
new MyType()	Object	MyType	Object	Object
null	Object	——	Object	Null
undefined	Undefined	——	Object	Undefined

通常，会存在一些意想不到、难于定位的错误。使用 Lint 工具可以避免一部分错误，通过对 Runtime 进行类型检查也可以做到，但是需要大量的检查代码。

针对以上问题，目前有两种解决方案，具体如下。

- 工具：使用 Facebook 开发的 flow，这是静态 JavaScript 类型检查工具，一般与 Babel 组合使用。
- 语言级别：TypeScript 支持类型系统，不过为了兼容 ES.next 语法，类型是可选的。长远来看，TypeScript 应该是真正的赢家。

4.4.3 更好的 npm 替代品——Yarn

Yarn 是开源的 JavaScript 包管理器，由于 npm 在扩展内部使用时遇到了大小、性能和安全等方面的问题，Facebook 便携手 Exponent、Google 和 Tilde，在大型 JavaScript 框架上打造并测试了 Yarn，以便令其尽可能适用于多人开发。

Yarn 承诺比各大流行 npm 包的安装更可靠，且速度更快。根据你所选的工作包的不同，Yarn 可以将安装时间从数分钟减少至几秒钟。Yarn 还兼容 npm 注册表，但包安装方法有所区别。Yarn 使用 lockfile 和一个决定性安装算法，能够为参与同一个项目的所有用户维持相同的节点模块目录结构，有助于减少难以追踪的 Bug，能在多台机器上复制。

Yarn 还致力于让安装更快速可靠，支持缓存下载的每一个包和并行操作，允许在没有互联网连接的情况下安装（如果此前安装过的话）。此外，Yarn 承诺同时兼容 npm 和 Bower 工作流，限制安装模块的授权许可。

2016 年 10 月，在 Yarn 横空出世的一周之内，GitHub 上的 star 数便已过万，可以看出社区的活跃度，以及解决问题的诚意。

npm 被 Yarn 替换的原因具体如下。

- Facebook 中的大部分 npm 工作性能都不太好。
- npm 拖慢了公司的 CI 工作流。
- 一个模块变动时，需要检查所有的模块，这是相当低效的。
- npm 被设计为具有不确定性的包管理器，而 Facebook 工程师需要为他们的 DevOps 工作流提供可依赖的系统。

Yarn 的特点如下。

- 本地缓存文件的性能更好。
- 可以并行执行一些操作，加速了对新模块的安装处理。
- 使用 lockfile，并且能够用确定的算法创建一个跨所有机器的一致文件。
- 出于安全性考虑，在安装进程里不允许编写包的开发者执行其他代码。

很多人说，Yarn 和 Ruby 的 Gem 机制类似，都生成 lockfile。这确实是一个很不错的改进，

在速度上有很大提升，配置 Cnpm 等国内源的体验非常好。

👉 安装 Yarn

使用脚本安装，代码如下。

```
$ curl -o- -L https://yarnpkg.com/install.sh | bash
```

当然也可以使用 npm 进行安装，代码如下。

```
$ npm install --global yarn
```

下面我们来看一下 Yarn 的用法。首先初始化新工程，代码如下。

```
$ yarn init
```

增加依赖模块，代码如下。

```
$ yarn add [package]
$ yarn add [package]@[version]
$ yarn add [package]@[tag]
```

更新依赖，代码如下。

```
$ yarn upgrade [package]
$ yarn upgrade [package]@[version]
$ yarn upgrade [package]@[tag]
```

移除依赖，代码如下。

```
$ yarn remove [package]
```

在项目里安装依赖，代码如下。

```
$ yarn
```

或者也可以采用如下方式安装依赖。

```
$ yarn install
```

👉 安装全局模块

通过 yarn global 命令安装全局模块，等同于使用 npm install --global 命令，这里以安装全局模块 kp 为例，kp 是一个根据端口号查杀进程的模块，安装代码如下。

```
$ yarn global add kp
yarn global v0.19.1
[1/4] □ Resolving packages...
```

```
[2/4] Fetching packages...
[3/4] Linking dependencies...
[4/4] Building fresh packages...
warning Your current version of Yarn is out of date. The latest version is "0.21.3"
while you're on "0.19.1".
info To upgrade, run the following command:
$ npm upgrade --global yarn
success Installed "kp@1.1.1" with binaries:
  - kp
Done in 3.66s.
```

对于熟悉 `npm` 的朋友来说，使用 `Yarn` 是非常简单的事，如果是初学者，推荐使用 `npm`，毕竟 `npm` 是绕不开的核心。等到哪一天 `Node.js` SDK 内置了 `Yarn`，我们也会推荐使用 `Yarn`。但作为业内最佳实践，`Yarn` 无疑是值得大家学习和使用的。

4.4.4 多模块管理器 Lerna

在设计框架的时候，经常做的事是进行模块拆分，提供插件或集成机制，这样的做法是非常好的。但问题也随之而来，当你的模块非常多时，该如何管理呢？

- 方法 1：为每个模块都建立独立的仓库（以前都是这样做的）。
- 方法 2：将所有模块都放到同一个仓库里（`Learn` 建议）。

方法 1 虽然看起来更“干净”，但模块多时，依赖安装、测试、不同版本兼容等问题会导致模块间依赖混乱，出现非常多的重复依赖，极易造成版本不兼容。这时方法 2 就显得更加有效了，对于测试、代码管理、发布等，都可以更好地支持。

`Lerna` 就是基于这种初衷而产生的专门用于管理 `Node.js` 多模块的工具。官方介绍是，`A tool for managing JavaScript projects with multiple packages.`

你可以通过 `npm` 全局模块来安装 `Lerna`，官方推荐直接使用 `Lerna 2.x` 版本。

```
$ npm install --global lerna@^2.0.0-beta.0
```

接下来，我们会创建一个 `Git` 仓库，命令如下。

```
$ git init lerna-repo && cd lerna-repo
```

初始化 `Lerna` 仓库，命令如下。

```
$ lerna init
```

你的仓库应该显示如下。

```
lerna-repo/  
  packages/  
    package.json  
  lerna.json
```

模块发布命令如下，一定要注意，npm 的源是 `npmjs.org`，否则将无法发布成功。

```
$ lerna publish
```

4.5 本章小结

通过本章的学习，你应该已经对 Node.js 有了更深的了解。Node.js 中有丰富的选择，可以开发大型软件应用，它的单线程稳定性也是非常好的，利用 `Cluster` 模块可以实现多核并行处理，执行效率非常高。最后讲的 Node.js 的更好的实践（ES.next、TypeScript、Yarn、Lerna），相信也能让你感到耳目一新。

我们有理由相信，ES.next 带来的变革会产生深远的影响，让每一个 Node.js 开发者都感受到开发的快乐！

第 5 章

Node.js 是如何执行的

2015 年是 Node.js 发展史上的一个分水岭。在 2015 年之前，因内部对 ES.next 特性的支持问题，核心开发者 fork 并维护了 io.js 项目。io.js 连续发布了 1.x、2.x 和 3.x 等大版本，势头颇为强劲。这种情况下，Joyent 公司不得不做出让步，因此共同成立了自主开源的 Node.js 基金会。2015 年 9 月，Node.js 基金会发布了 Node.js v4.0，这也是 Node.js 项目与 io.js 项目合并后推出的第一个版本，为开发者带来了大量的 ES6 语言扩展。

2015 年至今（截止到本书写作之时），Node.js 发布了 9 个大版本，4 个 LTS 版本，下一个 LTS 版本 Node.js v12 也即将完成，整体来说发展还是相当不错的。唯一的问题在于，Node.js 的核心代码变动非常大，虽然 API 向后兼容做得非常好，但对于想要学习和搞清楚原理的人来说，这可能是一场灾难。

本章我们以 Node.js 8 版本的代码为基础，剖析一下 Node.js 的核心原理，下面跟我一起来了解一下 Node.js 是如何执行的吧！

5.1 准备

首先下载源码，或者从 GitHub 上直接克隆到本地，然后按照经典的 C/C++ 程序的 3 步编译安装法，在终端里执行如下命令。

```
$ ./configure
$ make -j4
$ [sudo] make install
```

对于普通的通过源码安装来说，掌握这些已经足够了。如果想深入了解原理，甚至是给 Node.js 源码提 PR (Push Request)，就要学习更多了。比如，要想新增一个文件，要修改构建

配置哪里呢？下面我们就来探究一下。

5.1.1 编辑器

都说“工欲善其事，必先利其器”，对于 C/C++开发来说，编辑器的选择是丰富多样的，下面给出一些跨平台编辑器的对比，见表 5-1。Visual Studio 也很好，但其安装包过大，且支持跨平台时间不长，因此未提及。

表 5-1

名 称	描 述	是否付费	难 度
Vim + GDB	高手的选择，但学习曲线略陡峭	否	高
Code::Blocks	开放源码的全功能 C/C++集成开发环境，配置比较复杂	否	中
Eclipse CDT	保持 Eclipse 习惯，没有 CLion 智能，支持可视化调试，大量 Eclipse 插件可用	否	中
CLion	简单易用，功能强大，支持可视化调试，支持 CMake 配置，对代码重构、静态分析等的支持尤其好	是	低

对于非专业的 C/C++程序员来说，CLion 应该是最好的选择。如图 5-1 所示，CLion 是一款专为开发 C/C++程序开发所设计的跨平台集成开发环境，以 IntelliJ 为基础设计，包含许多智能功能，可提高开发人员的生产力。这种强大的集成开发环境可以帮助开发人员在 Linux、macOS 和 Windows 上开发 C/C++程序，同时使用智能编辑器来提高代码质量，实现自动代码重构并且深度整合 CMake 编译系统，从而提高开发人员的工作效率。

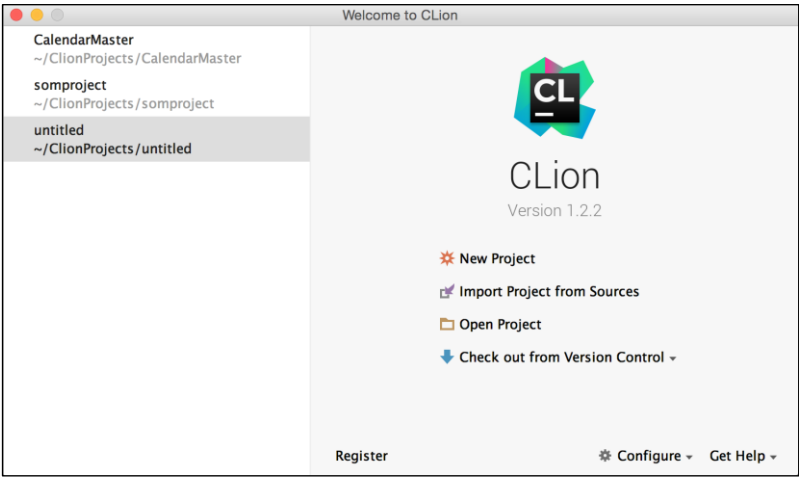


图 5-1

CLion 的特点如下。

- 简单易用，功能强大。
- 具有智能的代码辅助功能。
- 支持代码生成和重构。
- 支持静态分析。
- 支持可视化调试。
- 集成了 CMake 强大的构建功能。
- 支持单元测试。

CLion 是商业软件，功能非常强大，需要购买授权，可试用 30 天，对于掌握 Node.js 源码来说非常有用，下载地址是 <https://www.jetbrains.com/clion/download/>。

5.1.2 编译

进入 Node.js 代码文件，编译代码。

```
$ ./configure --debug
$ make -j 4
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C out BUILDTYPE=Release V=1
/Applications/Xcode.app/Contents/Developer/usr/bin/make -C out BUILDTYPE=Debug V=1
... (各种编译过程)

if [ ! -r node_g -o ! -L node_g ]; then ln -fs out/Debug/node node_g; fi
if [ ! -r node -o ! -L node ]; then ln -fs out/Release/node node; fi
```

Node.js 源码量较大，如果不并行处理，编译过程会特别慢，所以推荐使用 -j 参数。

-j[N] 和 -jobs[=N] 参数可用来提高编译效率，充分利用多核处理器的性能，并行编译，其中的 N 为并行的任务数。

如果开启 debug 参数，实际上会进行两个 CMake 任务，并且在源码根目录分别连接 Node.js 和 node_g。编译完成后，生成的所有文件将位于 out/Release 和 out/Debug 目录下。

```
$ ls out/Debug
ctest          icupkg          libicudata.a    libopenssl.a
```

```

libv8 libplatform.a libzlib.a      obj
genccode      libcares.a      libicu18n.a      libuv.a
libv8_libsampler.a mksnapshot      obj.host
genrb      libgtest.a      libicustubdata.a      libv8 base.a
libv8_nosnapshot.a node      obj.target
iculslocs      libhttp_parser.a      libicuucx.a      libv8_libbase.a
libv8_snapshot.a node.d      openssl-cli

```

编译好之后,选择 File,然后选择 import project 导入 Node.js 源代码。由于 CLion 采用 CMake 构建项目,所以需要修改 CMakeLists.txt 命令,添加自定义任务,修改后的代码如下。

```

cmake_minimum_required(VERSION 3.7)
project(node_master)

set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11")

add_custom_target(node COMMAND make -C $(node SOURCE_DIR) node g
                  CLION_EXE_DIR='./out/Debug')

```

5.1.3 调试

在 Node.js 源码根目录下创建 test.js 文件,用来读取 node.gyp 配置信息。

```

const fs = require('fs')

fs.readFile('./node.gyp', {encoding: 'utf-8'}, function (e, content) {
  console.log(content);
})

```

打开配置界面配置 Debug 选项,内容如图 5-2 所示。

- 选择 out/Debug/node 作为执行文件。
- 注意配置 Current Directory, 即要执行的 test.js 文件所在的目录。
- 添加 test.js 作为命令行参数,也可以添加 Node.js 的其他参数,比如--expose-gc 等。
- 在 Before launch 栏中把 build 去掉。

点击【OK】按钮即可完成配置,在 src/node_file.cc 的 1168 行 static void Read(const FunctionCallbackInfo<Value>& args)中增加断点,如图 5-3 所示,此时按照 Run—> Debug 'node' 顺序执行即可进入调试界面。

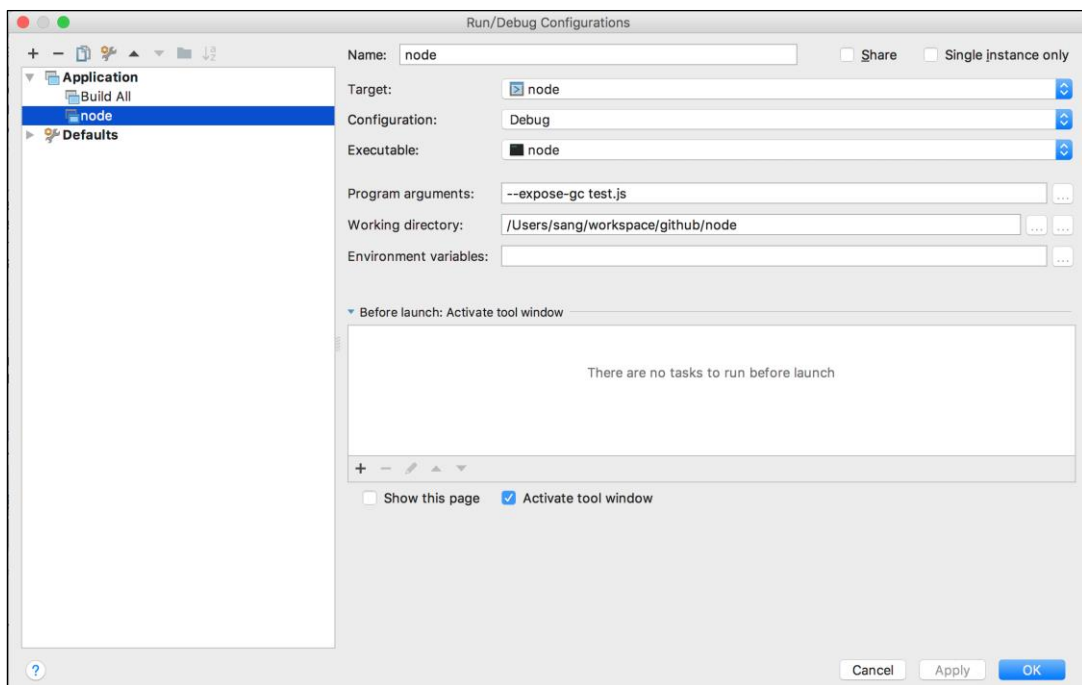


图 5-2

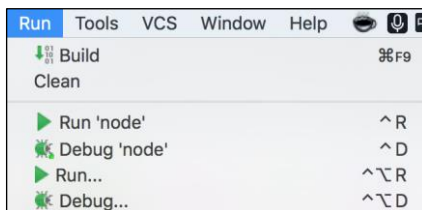


图 5-3

调试界面如图 5-4 所示，和 Chrome 的审查界面非常像。

另外一种调试方式是，使用原始的 gdb 命令进行调试，当然也可以结合 CLion 一起调试。要想做到这一点有一个要求，就是要使用调试版的 Node.js，俗称 node_g，这个需要自己编译 Node.js 才能得到，编译 Node.js 时添加--debug 参数即可。

```
$ ./configure --debug && make
$ gdb --args ./node_g test.js
```

有了 node_g 就可以用 gdb 单步调试 Node.js 及扩展程序了。gdb 命令行调试不够直观，文档众多，这里就不做详细介绍了。

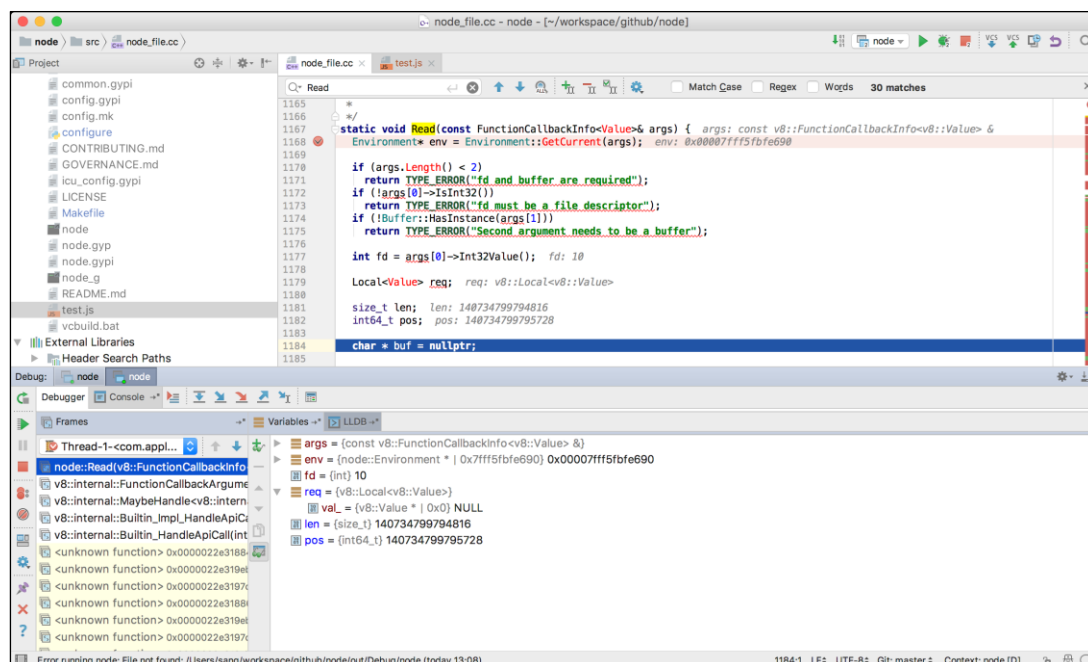


图 5-4

5.2 编译步骤

经典的 C/C++ 程序的 3 步编译安装法听起来很简单, 可对于 Node.js 源码而言却不完全适用, 原因是 Node.js 采用了 GYP 和 Python 作为构建工具。

5.2.1 configure

configure 是用来检测安装平台的目标特征的, 比如它会检测平台中是不是有 CC 或 GCC, 通常它是一个 shell 脚本, 但是 Node.js 源码里的 ./configure 是 Python 脚本, 有 1400 多行, 绝大部分都是参数处理和编译依赖的配置生成语句, 示例如下。

```
$ ./configure
creating ./icu config.gypi
* Using ICU in deps/icu-small
creating ./icu config.gypi
{ 'target defaults': { 'cflags': [],
                      'default_configuration': 'Release',
                      'defines': [],
```

```
        'include_dirs': [],
        'libraries': []},
'variables': { 'asan': 0,
               'coverage': 'false',
               'debug_devtools': 'node',
               'force_dynamic_crt': 0,
               'host_arch': 'x64',
               'icu_data_file': 'icudt59l.dat',
               'icu_data_in': '../..//deps/icu-small/source/data/in/icudt59l.dat',
               'icu_endianness': 'l',
               'icu_gyp_path': 'tools/icu/icu-generic.gyp',
               'icu_locales': 'en,root',
               'icu_path': 'deps/icu-small',
               'icu_small': 'true',
               'icu_ver_major': '59',
               'llvm_version': 0,
               'node_byteorder': 'little',
               'node_enable_d8': 'false',
               'node_enable_v8_vtunejit': 'false',
               'node_install_npm': 'true',
               'node_module_version': 56,
               'node_no_browser_globals': 'false',
               'node_prefix': '/usr/local',
               'node_release_urlbase': '',
               'node_shared': 'false',
               'node_shared_cares': 'false',
               'node_shared_http_parser': 'false',
               'node_shared_libuv': 'false',
               'node_shared_openssl': 'false',
               'node_shared_zlib': 'false',
               'node_tag': '',
               'node_use_bundled_v8': 'true',
               'node_use_dtrace': 'true',
               'node_use_etw': 'false',
               'node_use_lttng': 'false',
               'node_use_openssl': 'true',
               'node_use_perfctr': 'false',
               'node_use_v8_platform': 'true',
               'node_without_node_options': 'false',
               'openssl_fips': '',
               'openssl_no_asm': 0,
               'shlib_suffix': '56.dylib',
               'target_arch': 'x64',
               'uv_parent_path': '/deps/uv/',
               'uv_use_dtrace': 'true',
               'v8_enable_gdbjit': 0,
               'v8_enable_i18n_support': 1,
               'v8_enable_inspector': 1,
```

```

        'v8 no strict aliasing': 1,
        'v8_optimized_debug': 0,
        'v8_promise_internal_field_count': 1,
        'v8_random_seed': 0,
        'v8_use_snapshot': 'true',
        'want_separate_host_toolset': 0,
        'want_separate_host_toolset mkpeephole': 0,
        'xcode_version': '8.0'}}
creating ./config.gypi
creating ./config.mk

```

关于以上代码，概括起来就两个核心步骤。

第一步：./configure 准备 GYP 要执行的参数，其中的核心代码如下。

```

from gyp node import run_gyp
...

run_gyp(gyp_args)

```

参数如下。

```
['--no-parallel', '-f', 'make-mac']
```

这里的 `gyp_args` 是通过 Python 的 `optparse` 来获得的，还是极其简单的，`configure` 脚本配置工具是 `autoconf` 工具，当然也有很多是用 Python 编写的，用法几乎一模一样。`configure` 脚本执行非常简单，下面在终端里看一下它的具体用法。

```

$ ./configure --help
Usage: configure [options]

Options:
  -h, --help                show this help message and exit
  --prefix=PREFIX           select the install prefix [default: /usr/local]
  --coverage                Build node with code coverage enabled
  --debug                   also build debug build
  ...

```

比如，想要指定安装目录 `$ ./configure --prefix=/some/path` 是标准配置时，使用 Python 实现也很简单，代码如下。

```

# Options should be in alphabetical order but keep --prefix at the top,
# that's arguably the one people will be looking for most.
parser.add_option('--prefix',
    action='store',
    dest='prefix',
    default='/usr/local',
    help='select the install prefix [default: %default]')

```

第二步：调用 `tools/gyp_node.py` 里的 `run_gyp` 函数，生成配置文件。

```
if name == 'main':
    run_gyp(sys.argv[1:])
```

参数如下。

```
[ '--no-parallel', '-f', 'make-mac', '/Users/sang/workspace/github/node/node.gyp', '-I',
'/Users/sang/workspace/github/node/common.gypi', '-I', '/Users/sang/workspace/github/
node/config.gypi', '--depth=.', '--generator-output', '/Users/sang/workspace/github/no
de/out', '-Goutput_dir=/Users/sang/workspace/github/node/out', '-Dcomponent=static_li
brary', '-Dlibrary=static_library', '-Dlinux_use_bundled_binutils=0', '-Dlinux_use_bun
dled_gold=0', '-Dlinux_use_gold_flags=0']
```

最终生成 `icu_config.gypi`、`./config.gypi`、`./config.mk`。

GYP 可以支持更大的项目在不同的平台上编译，比如在 macOS、Windows、Linux 上，它可以生成 Xcode 工程、Visual Studio 工程、Ninja 编译文件和 Makefile。Chromium 和 V8 的编译过程中都用到了 GYP，所以 Node.js 项目也使用了 GYP，这真是一个令人又爱又恨的选择。

GYP 的输入是 `.gyp` 和 `.gypi` 文件，`.gypi` 文件在 `.gyp` 文件里是通过 `include` 来使用的，可以理解为从 `.gyp` 配置文件中抽取出来的模块。同样，`.mk` 也是用于在 Makefile 里引用的，所以这一步相当于生成了一些辅助构建 `make` 和 `gyp` 的引用文件。除此之外，还生成了 `out` 目录，将各种编译配置进行了预处理，与如下语句等同。

```
$ make out/Makefile
```

实际执行的是下面的命令。

```
$ ./tools/gyp_node.py -f make
```

生成结果如下。

```
$ tree out -L 4
out
├── Makefile
├── cctest.target.mk
├── deps
│   ├── cares
│   │   └── cares.target.mk
│   ├── gtest
│   │   └── gtest.target.mk
│   ├── http_parser
│   │   ├── http_parser.target.mk
│   │   ├── http_parser_strict.target.mk
│   │   ├── test-nonstrict.target.mk
│   │   └── test-strict.target.mk
```

```

├── openssl
│   ├── openssl-cli.target.mk
│   └── openssl.target.mk
├── uv
│   ├── libuv.target.mk
│   ├── run-benchmarks.target.mk
│   └── run-tests.target.mk
├── v8
│   └── src
│       ├── inspector
│       ├── js2c.target.mk
│       ├── mksnapshot.target.mk
│       ├── natives_blob.target.mk
│       ├── postmortem-metadata.target.mk
│       ├── v8.target.mk
│       ├── v8_base.target.mk
│       ├── v8_external_snapshot.target.mk
│       ├── v8_libbase.target.mk
│       ├── v8_libplatform.target.mk
│       ├── v8_libsampler.target.mk
│       ├── v8_maybe_snapshot.target.mk
│       ├── v8_nosnapshot.target.mk
│       └── v8_snapshot.target.mk
├── zlib
│   └── zlib.target.mk
├── gyp-mac-tool
├── mkssldef.target.mk
├── node.target.mk
├── node_dtrace_header.target.mk
├── node_dtrace_provider.target.mk
├── node_dtrace_ustack.target.mk
├── node_etw.target.mk
├── node_js2c.host.mk
├── node_perfctr.target.mk
├── specialize node d.target.mk
├── tools
│   └── icu
│       ├── genccode.host.mk
│       ├── genrb.host.mk
│       ├── icu_implementation.host.mk
│       ├── icu_implementation.target.mk
│       ├── icu_uconfig.host.mk
│       ├── icu_uconfig.target.mk
│       ├── icu_uconfig_target.target.mk
│       ├── icudata.target.mk
│       ├── icui18n.host.mk
│       ├── icui18n.target.mk
│       └── iculslocs.host.mk

```

```

├── icupkg.host.mk
├── icustubdata.target.mk
├── icutools.host.mk
├── icuuc.host.mk
├── icuuc.target.mk
├── icuucx.target.mk
└── v8 inspector compress protocol json.host.mk
12 directories, 55 files

```

毫无疑问，通过 GYP 配置生成各个平台的构建脚本是一个高度抽象的过程，带来了不可避免的复杂性，因此需要花时间去摸索和学习。

👉 node.gyp

上面说过，`gyp.main(args)`会自动读取'`/Users/sang/workspace/github/node/node.gyp`'，这个文件是.gyp 配置文件，是 Node.js 配置中最核心的。

node.gyp 分为 variable、target、condition 这 3 个部分，分别描述如下。

- **variable**（变量定义部分）：定义了键值对，可以以`<(varname)`的方式被引用，这一点和曾经流行的构建工具 Grunt 很像，都是典型的 JSON 模板应用。
- **target**（项目列表）：.gyp 文件定义的一套构建项目的规则，而且也支持构建依赖。
- **condition**（条件）：在加载.gyp 文件后，使用 Python 的 `eval()`函数对条件字符串进行处理。

关于 target 的描述见表 5-2。

表 5-2

项目名称	依赖	描述
Node.js	node_js2c	核心项目，默认编译为可执行的 Node.js 文件，也可以编译为贡献库
cctest	['node','deps/gtest/gtest.gyp:gtest','node_js2c#host','node_dtrace_header','node_dtrace_ustack','node_dtrace_provider',]	负责测试和动态跟踪
mkssldef	无	通过执行 <code>tools/mkssldef.py</code> 来处理 openssl 相关配置
node_etw	无	生成可供 Windows（ETW）使用的事件追踪的头文件和资源文件

续表

项目名称	依赖	描述
node_perfctr	无	生成 perf counter 头文件和资源文件
v8_inspector_compress_protocol_json	无	执行 tools/compress_json.py
node_js2c	无	执行 tools/js2c.py, 将 JavaScript 源代码转为 C 语言风格的字符数组。这样做是为了便于在 Chrome V8 库中嵌入 JavaScript 代码
node_dtrace_header	无	生成 Dtrace 用的文件, 输入是 src/node_provider.d, 输出是 node_provider.h
node_dtrace_provider	无	根据不同平台生成 Dtrace 需要的文件
node_dtrace_ustack	无	根据不同平台生成 Dtrace ustack 需要的文件
specialize_node_d	无	生成支持 Dtrace 的 Node.js 入口文件

核心 target 是 Node.js 和 cctest, 其他都是依赖的子项目, Node.js 项目主要负责生成可执行文件, 只依赖了 node_js2c, 这里只需要了解 node_js2c 即可。对于 cctest, 主要学习其测试写法 and 性能测试方法即可。

👉 node_js2c

Node.js 使用了 Chrome V8 附带的 js2c.py 工具把所有内置的 JavaScript 代码都转换成了 C 语言里的数组, 最终生成可供核心代码引用的 node_javascript.cc 文件, 直接包含到 Node.js 程序中, 这样做能提高内置 JavaScript 模块的编译效率, 比每次执行前动态加载要高效得多。

根据 node.gyp 定义的 node_js2c 的 target 具体如下。

```
{
  'target_name': 'node_js2c',
  'type': 'none',
  'toolsets': ['host'],
  'actions': [
    {
      'action_name': 'node_js2c',
      'process_outputs_as_sources': 1,
      'inputs': [
        '<@(library_files)',
        './config.gypi',
      ],
      'outputs': [
        '<(SHARED_INTERMEDIATE_DIR)/node_javascript.cc',
      ],
    }
  ]
}
```

```

    ],
    'conditions': [
      [ 'node_use_dtrace=="false" and node_use_etw=="false"', {
        'inputs': [ 'src/notrace_macros.py' ]
      } ],
      [ 'node_use_lttng=="false"', {
        'inputs': [ 'src/nolttng_macros.py' ]
      } ],
      [ 'node_use_perfctr=="false"', {
        'inputs': [ 'src/perfctr_macros.py' ]
      } ]
    ],
    'action': [
      'python',
      'tools/js2c.py',
      '<@(_outputs)',
      '<@(_inputs)',
    ],
  },
],
}, # end node_js2c

```

这段 target 定义的核心是 python tools/js2c.py 输入和输出，其中的输入是 '<@(library_files)' 和 './config.gypi'，输出是 node_javascript.cc 文件。如果想了解更多细节，可以看一下 GYP 生成的 out/node_js2c.host.mk，主要作用是对 node.gyp 里的变量 library_files 数组里的文件进行遍历、转换，进而生成 node_javascript.cc 文件。

library_files 变量包含 4 部分，具体如下。

- lib/*.js
- deps/v8/tools/*.js（Chrome V8 支持）
- deps/node-inspect/lib/*.js（新版本的 Chrome inspector 调试协议）
- src/*.py
 - nolttng_macros.py: 重置 LTTng 的宏，lttng 是 Linux 平台上的开源追踪框架。
 - notrace_macros.py: 重置 Dtrace 的宏，Dtrace 是动态追踪技术的鼻祖，使用 D 语言编写，提供了高级性能分析和调试功能。
 - perfctr_macros: 重置性能引用计数的宏。

下面我们来看一下 node_javascript.cc 文件和 SDK 里的模块程序 lib/*.js 之间的对应关系。

这里以 `vm.js` 为例, 在终端里执行 `python tools/js2c.py "out/Debug/obj/gen/node_javascript.cc" lib/vm.js`, 生成的代码如下。

```
#include "node.h"
#include "node_javascript.h"
#include "v8.h"
#include "env.h"
#include "env-inl.h"

namespace node {

    static const uint8_t raw_vm_key[] = { 118,109 };
    static struct : public v8::String::ExternalOneByteStringResource {
        const char* data() const override {
            return reinterpret_cast<const char*>(raw_vm_key);
        }
        size_t length() const override { return arraysize(raw_vm_key); }
        void Dispose() override { /* Default calls `delete this`. */ }
        v8::Local<v8::String> ToStringChecked(v8::Isolate* isolate) {
            return v8::String::NewExternalOneByte(isolate, this).ToLocalChecked();
        }
    } vm_key;

    static const uint8_t raw_vm_value[] = {47,47,32,67,111,112,121,114,105,103,104,116,
32,74,111,121,101,110,116,44,
32,73,110,99,46,32,97,110,100,32,111,116,104,
...
46,105,115,67,111,110,116,101,120,116,10,125,59,10 };
    static struct : public v8::String::ExternalOneByteStringResource {
        const char* data() const override {
            return reinterpret_cast<const char*>(raw_vm_value);
        }
        size_t length() const override { return arraysize(raw_vm_value); }
        void Dispose() override { /* Default calls `delete this`. */ }
        v8::Local<v8::String> ToStringChecked(v8::Isolate* isolate) {
            return v8::String::NewExternalOneByte(isolate, this).ToLocalChecked();
        }
    } vm_value;

    v8::Local<v8::String> MainSource(Environment* env) {
        return internal_bootstrap_node_value.ToStringChecked(env->isolate());
    }

    void DefineJavaScript(Environment* env, v8::Local<v8::Object> target) {
        CHECK(target->Set(env->context(),
            vm_key.ToStringChecked(env->isolate()),
            vm_value.ToStringChecked(env->isolate())).FromJust());
    }
}
```

```

    }
}

```

以上代码中的要点是转码，即将非 ASCII 码要转成 UTF-8 编码，最终将 UTF-8 编码转成 UTF-16 编码，这其实是为了适配 Chrome V8 里使用的 `uint8t` 和 `uint16t` 类型转换。另外，`vm` 对应生成两个数组：`raw_vm_key` 和 `raw_vm_value`。

最终根据模板，生成在 Node.js 的 Namespace 下使用的 C++ 代码，如下。

```

static struct : public v8::String::ExternalOneByteStringResource {
    const char* data() const override {
        return reinterpret_cast<const char*>(raw_vm_value);
    }
    size_t length() const override { return arraysize(raw_vm_value); }
    void Dispose() override { /* Default calls `delete this`. */ }
    v8::Local<v8::String> ToStringChecked(v8::Isolate* isolate) {
        return v8::String::NewExternalOneByte(isolate, this).ToLocalChecked();
    }
} vm_value;

```

代码里的结构体 `vm_value` 主要负责字符串和数据之间的转换，进而可以在 Node.js 源码里直接使用 `lib/*.js` 对应的 C++ 代码。最终生成的 `node_javascript.cc` 文件中有将近 7 万行左右的代码，都是按照上面的规则进行转换的，在性能优化上有非常好的效果。Node.js 源码编译的核心原理和外部调用关系如图 5-5 所示。

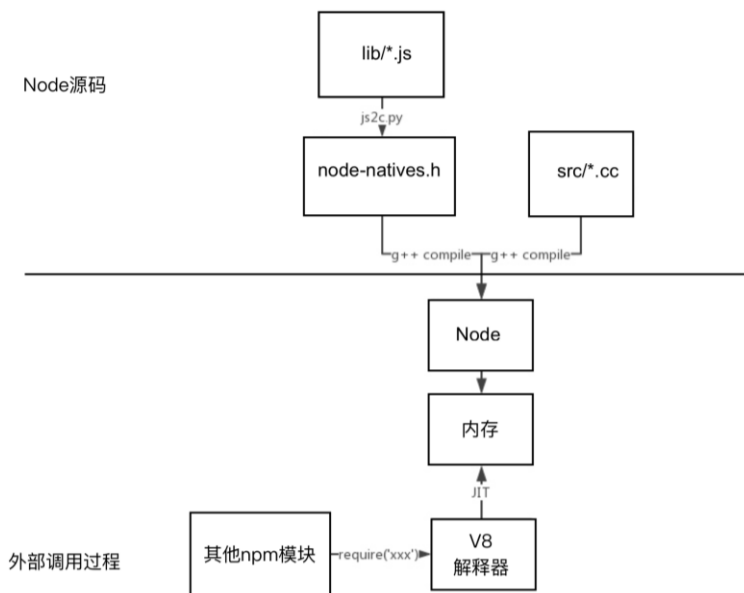


图 5-5

5.2.2 make

`make` 是一个用于解析 `Makefile` 文件中配置指令的命令工具。直接在命令中键入 `make`, 相当于执行默认的任务。`Makefile` 文件里的 `.PHONY` 目标并非实际的文件名, 它只是显式请求时执行命令的名字。有两种情况需要使用 `.PHONY` 目标: 一是避免和同名文件冲突, 二是改善性能。

`Node.js` 源码根目录的 `Makefile` 的最下面就使用了 `.PHONY` 的定义, 如下。

```
.PHONY: $(TARBALL)-headers \  
all \  
bench \  
bench \  
bench-all \  
bench-array \  
bench-buffer \  
bench-ci \  
bench-fs \  
bench-http \  
bench-http-simple \  
bench-idle \  
bench-misc \  
bench-net \  
bench-tls \  
binary \  
blog \  
blogclean \  
build-addons \  
build-addons-napi \  
build-ci \  
ctest \  
check \  
clean \  
clear-stalled \  
coverage \  
coverage-build \  
coverage-clean \  
coverage-test \  
cpplint \  
dist \  
distclean \  
doc \  
doc-only \  
docclean \  
docopen \  
dynamiclib \  
install \  

```

```
install-bin \  
install-includes \  
jslint \  
jslint-ci \  
lint \  
lint-ci \  
list-gtests \  
pkg \  
release-only \  
run-ci \  
staticlib \  
tar \  
test \  
test-addons \  
test-addons-clean \  
test-addons-napi \  
test-addons-napi-clean \  
test-all \  
test-ci \  
test-ci-js \  
test-ci-native \  
test-gc \  
test-gc-clean \  
test-v8 \  
test-v8-all \  
test-v8-benchmarks \  
test-v8-intl \  
uninstall \  
v8 \  
website-upload
```

看名字即可知道 `all` 是核心。当然，你也可以指定构建类型，如果构建类型是 `Debug`，那么生成的就是 `node_g` 命令（默认生成的是 `node` 命令），在调试场景中使用会非常方便。

```
# BUILDTYPE=Debug builds both release and debug builds. If you want to compile  
# just the debug build, run `make -C out BUILDTYPE=Debug` instead.  
ifeq ($(BUILDTYPE),Release)  
all: out/Makefile $(NODE_EXE)  
else  
all: out/Makefile $(NODE_EXE) $(NODE_G_EXE)  
endif
```

查看所有的 task，如图 5-6 所示。

<code>+ node git:(master) x make ADDONS_BINDING_GYPS=</code>		
ADDONS_BINDING_GYPS	RAWVER	build-ci
ADDONS_BINDING_SOURCES	RELEASE	cctest
ADDONS_NAPI_BINDING_GYPS	SIGN	check
ADDONS_NAPI_BINDING_SOURCES	STAGINGSERVER	clean
ARCH	TAG	clear-stalled
BINARYNAME	TARBALL	config.gypi
BINARYTAR	TARNAME	coverage
BUILDTYPE	TEST_CI_ARGS	coverage-build
BUILDTYPE_LOWER	UNAME_M	coverage-clean
BUILD_DOWNLOAD_FLAGS	USE_XCODE	coverage-test
BUILD_INTL_FLAGS	V	cpplint
BUILD_RELEASE_FLAGS	V8_ARCH	distclean
CI_JS_SUITES	V8_TEST_OPTIONS	doc
CI_NATIVE_SUITES	VERSION	doc-only
CONFLICT_RE	XZ	doc-upload
COVTESTS	XZ_COMPRESSION	docclean
CPPLINT_EXCLUDE	all	docopen
CPPLINT_FILES	apiassets	install
DESTCPU	apidoc_dirs	jslint
DESTDIR	apidoc_sources	jslint-ci
DISTTYPE	apidocs_html	lint
DISTTYPEDIR	apidocs_json	lint-ci
DOCBUILDSTAMP_PREREQS	bench	list-gtests
DOCS_ANALYTICS	bench-all	out/Makefile
EXEEXT	bench-array	out/doc/%
FLAKY_TESTS	bench-buffer	out/doc/api/%.html
FULLVERSION	bench-ci	out/doc/api/%.json
GTEST_FILTER	bench-crypto	out/doc/api/assets/%
LOGLEVEL	bench-dgram	pkg
NODE	bench-events	pkg-upload
NODE_EXE	bench-fs	release-only
NODE_G_EXE	bench-http	run-ci
NPM	bench-misc	tar
NPMVERSION	bench-net	tar-headers
OSTYPE	bench-tls	tar-headers-upload
PACKAGEMAKER	bench-url	tar-upload
PKG	bench-util	test
PKGDIR	binary	test-addons
PLATFORM	binary-upload	test-addons-clean
PREFIX	build-addons	test-addons-napi
PYTHON	build-addons-napi	test-addons-napi-clean
		test-all
		test-all-valgrind
		test-async-hooks
		test-build
		test-build-addons-napi
		test-check-deopts
		test-ci
		test-ci-js
		test-ci-native
		test-debug
		test-debugger
		test-gc
		test-gc-clean
		test-inspector
		test-internet
		test-known-issues
		test-message
		test-node-inspect
		test-npm
		test-npm-publish
		test-parallel
		test-pummel
		test-release
		test-simple
		test-tick-processor
		test-timers
		test-timers-clean
		test-v8
		test-v8-all
		test-v8-benchmarks
		test-v8-intl
		test-valgrind
		test/addons-napi/.buildstamp
		test/addons/.buildstamp
		test/addons/.docbuildstamp
		test/gc/build/Release/binding.node
		uninstall
		v8

图 5-6

5.2.3 make install

`make install` 可用于将编译出来的可执行文件和库文件复制到合适的地方（默认是 `configure` 里指定的 `prefix` 目录），对于 C/C++ 项目而言则稍微复杂一点，头文件也是要处理的。

首先查看一下 Node.js 的安装位置。

```
$ which node
/Users/sang/.nvm/versions/node/v8.0.0/bin/node
```

看一下 Node.js 安装位置的目录信息，我们就会知道安装 Node.js 时会生成哪些文件。下面就是构建需要生成的所有文件。

```
$ tree /Users/sang/.nvm/versions/node/v8.0.0 -L 3
/Users/sang/.nvm/versions/node/v8.0.0
├── CHANGELOG.md
├── LICENSE
├── README.md
├── bin
│   ├── node
│   └── npm -> ../lib/node_modules/npm/bin/npm-cli.js
└── etc
```

```
├─ include
│   └─ node
│       ├── android-ifaddrs.h
│       ├── common.gypi
│       ├── config.gypi
│       ├── libplatform
│       ├── node.h
│       ├── node_api.h
│       ├── node_api_types.h
│       ├── node_buffer.h
│       ├── node_object_wrap.h
│       ├── node_version.h
│       ├── openssl
│       ├── pthread-barrier.h
│       ├── stdint-msvc2008.h
│       ├── tree.h
│       ├── uv-aix.h
│       ├── uv-bsd.h
│       ├── uv-darwin.h
│       ├── uv-errno.h
│       ├── uv-linux.h
│       ├── uv-os390.h
│       ├── uv-sunos.h
│       ├── uv-threadpool.h
│       ├── uv-unix.h
│       ├── uv-version.h
│       ├── uv-win.h
│       ├── uv.h
│       ├── v8-debug.h
│       ├── v8-inspector-protocol.h
│       ├── v8-inspector.h
│       ├── v8-platform.h
│       ├── v8-profiler.h
│       ├── v8-testing.h
│       ├── v8-util.h
│       ├── v8-version-string.h
│       ├── v8-version.h
│       ├── v8.h
│       ├── v8config.h
│       ├── zconf.h
│       └─ zlib.h
├─ lib
│   ├── dtrace
│   │   └─ node.d
│   └─ node_modules
│       ├── npm
│       └─ yrm -> /Users/sang/workspace/github/yrm
└─ share
```

```
├─ doc
│   └─ node
├─ man
│   └─ man1
└─ systemtap
    └─ tapset
```

18 directories, 43 files

知道了 Node.js 生成的文件有哪些, 下面再来看一下 Node.js 的安装原理, 核心代码位于 Node.js 项目的 Makefile 中。

```
install: all
$(PYTHON) tools/install.py $@ '$(DESTDIR)' '$(PREFIX)'
```

很明显, 真正执行安装任务的是 tools/install.py, 具体完成了如下工作。

- 读取 config.gypi, 获得配置。
- C/C++ 安装。
 - 复制安装文件到 bin/node 下。
 - 复制 Node.js 的 headers, 对应上面的 include/node/*.h。
 - 生成文档, 比如 share/man、share/doc 下的文档。
- 处理 npm 模块, 将 deps 复制到 lib/node_modules/npm 下面, 同时将 npm 以软连接的方式链接到 bin/npm。
- 根据配置做一些特殊处理, SSL、Dtrace、SystemTap 等。

自 2009 年到现在, Node.js 的代码量已经相当大, 要想弄清楚它的构建过程也是极为不易的, 前面讲到很多, 概括一下, 如图 5-7 所示。

这一切根源都是因为 Chrome V8 选用了 GYP, 同时搭配了很多用 Python 编写的工具, 所以社区觉得 GYP 是顽疾, 只有移除了它才能真正实现 Node.js 化, 不过这样做会带来很大的工作量, 于是就有了一个完全用 Node.js 实现的兼容 GYP 的 gyp.js, 虽然目前还没有正式发布, 但号角已吹响。

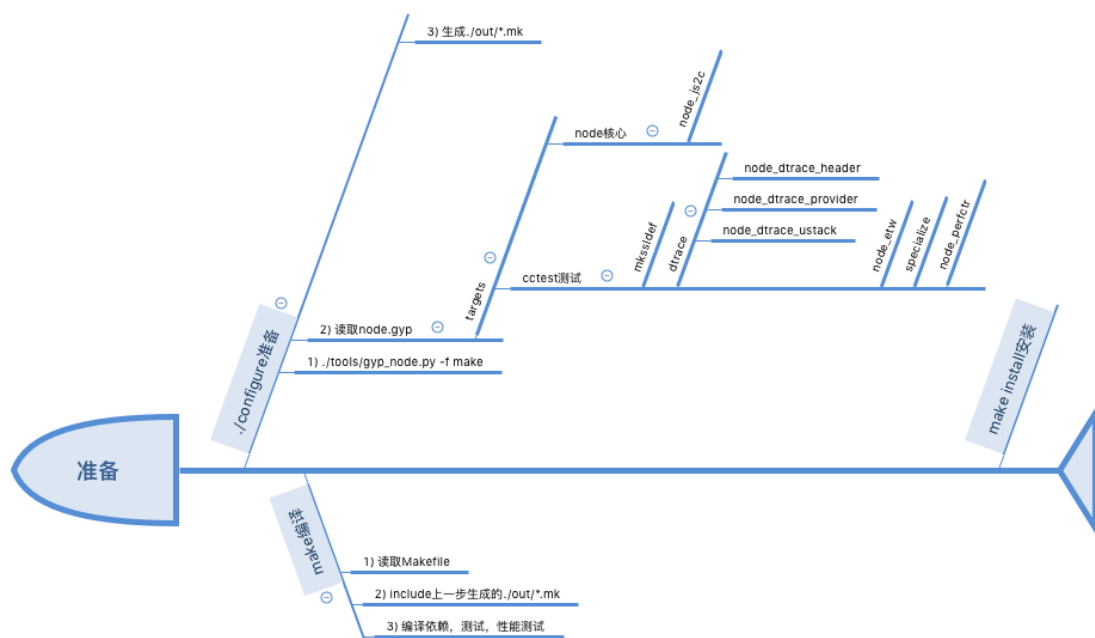


图 5-7

5.3 从入口开始

在 C/C++ 中, main 函数是整个程序的入口, 于是我们发现 src/node_main.cc 就是 Node.js 的入口文件。

```
int main(int argc, char *argv[]) {
#ifdef __linux__
    char** envp = environ;
    while (*envp++ != nullptr) {}
    Elf_auxv_t* auxv = reinterpret_cast<Elf_auxv_t*>(envp);
    for (; auxv->a_type != AT_NULL; auxv++) {
        if (auxv->a_type == AT_SECURE) {
            node::linux_at_secure = auxv->a_un.a_val;
            break;
        }
    }
}
#endif
// 禁止 stdio 缓存, 防止 Chrome V8 里的日志打印等功能执行
setvbuf(stdout, nullptr, _IONBF, 0);
setvbuf(stderr, nullptr, _IONBF, 0);
```



```
return node::Start(argc, argv);
}
```

这段代码的核心是最后一句，如下。

```
return node::Start(argc, argv);
```

翻阅代码会发现 `node::Start` 函数位于 `src/node.cc` 文件中，这便是 Node.js 的核心入口了。

```
int Start(int argc, char** argv) {
    atexit([] () { uv_tty_reset_mode(); });
    PlatformInit();

    CHECK_GT(argc, 0);

    // 透传 argv 指针, 比如 process.title = "blah".
    argv = uv_setup_args(argc, argv);

    // 初始化
    int exec_argc;
    const char** exec_argv;
    Init(&argc, const_cast<const char**>(argv), &exec_argc, &exec_argv);

#ifdef HAVE_OPENSSL
    {
        std::string extra_ca_certs;
        if (SafeGetenv("NODE_EXTRA_CA_CERTS", &extra_ca_certs))
            crypto::UseExtraCaCerts(extra_ca_certs);
    }
#endif
#ifdef NODE_FIPS_MODE
    OPENSSL_init();
#endif // NODE_FIPS_MODE
    V8::SetEntropySource(crypto::EntropySource);
#endif // HAVE_OPENSSL

    v8_platform.Initialize(v8_thread_pool_size);
    // 当 argv 的值是--trace-events-enabled的时候, 开启跟踪功能
    if (trace_enabled) {
        fprintf(stderr, "Warning: Trace event is an experimental feature "
            "and could change at any time.\n");
        v8_platform.StartTracingAgent();
    }
    V8::Initialize();
    v8_initialized = true;
    const int exit_code =
        Start(uv_default_loop(), argc, argv, exec_argc, exec_argv);
    if (trace_enabled) {
        v8_platform.StopTracingAgent();
    }
}
```

```

v8 initialized = false;
V8::Dispose();

v8 platform.Dispose();

delete[] exec_argv;
exec_argv = nullptr;

return exit_code;
}

```

5.3.1 核心流程

对 Start() 函数进行简单梳理，其中有 5 步是非常重要的，具体如下。

- PlatformInit();
- argv = uv_setup_args(argc, argv);
- Init(&argc, const_cast(argv), &exec_argc, &exec_argv);
- V8::Initialize();
- Start(uv_default_loop(), argc, argv, exec_argc, exec_argv);

下面，我们会对这几个步骤进行一一讲解。

👉 PlatformInit

为了解释 PlatformInit 的功能，我们需要了解有关信号（Signals）的基本知识。

信号是类 UNIX 操作系统中进行进程间通信的一种异步通知机制。在操作系统中，当信号被发送到一个进程中并产生中断时，任何非原子操作都会中断。如果进程中使用了信号处理函数，那么它将被执行，否则就执行默认的处理函数。在一个运行中的程序的控制终端键入特定的组合，可以发送某些特定信号，比如键入 Ctrl+C 组合会发送 INT 信号，键入 Ctrl+Z 组合会发送 TSTP 信号（SIGTSTP），它们将分别导致进程终止和挂起。

PlatformInit 主要用于对文件进行描述，以及注册两个信号处理函数，代码如下。

```

RegisterSignalHandler(SIGINT, SignalExit, true);
RegisterSignalHandler(SIGTERM, SignalExit, true)

```

在 Node.js 里通过 on 来捕获 SIGINT 事件的代码如下。

```
process.on('SIGINT', () => {
  console.log('Received SIGINT. Press Control-D to exit.');
```

👉 uv_setup_args

uv_setup_args 是定义在 libuv 的 deps/uv/include/uv.h 中的方法，用于进行 process.title 的设置和读取。

👉 Init

关于 Init 的作用，有以下几点说明。

- 初始化 Uptime 值。
- 对 node 命令行接收的参数和 Chrome V8 的 flag 参数进行映射处理。
- 将 node_is_initialized 标记为 true。

👉 V8::Initialize

首先，我们要明确 Chrome V8 的作用，如图 5-8 所示。

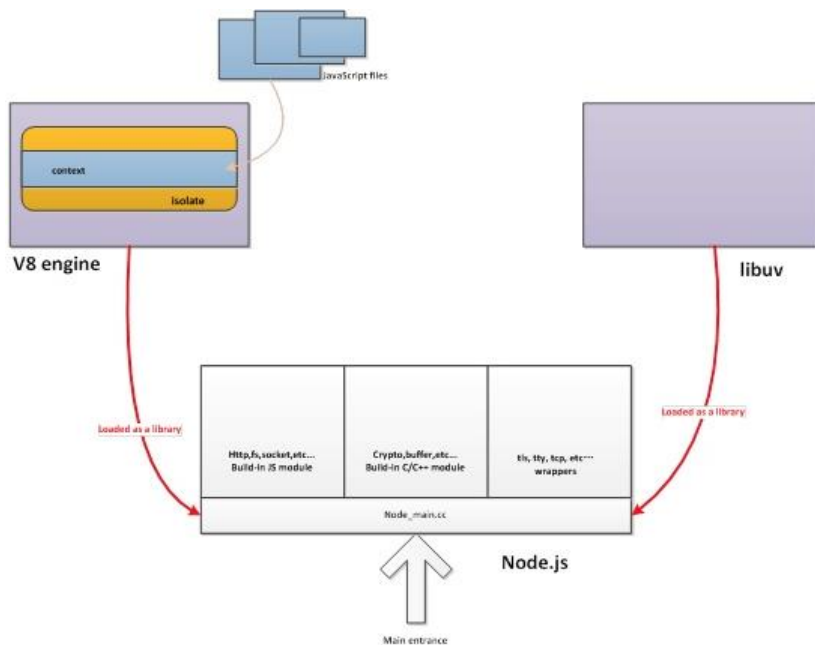


图 5-8

从图中，我们可以知道，所有的 Node.js 源码（JavaScript 文件）都会先由 Chrome V8 引擎来解释并执行。在 Chrome V8 的示例里，调用方式如下。

```
int main(int argc, char* argv[]) {  
    // 初始化 Chrome V8 引擎  
    V8::InitializeICUDefaultLocation(argv[0]);  
    V8::InitializeExternalStartupData(argv[0]);  
    Platform* platform = platform::CreateDefaultPlatform();  
    V8::InitializePlatform(platform);  
    V8::Initialize();  
}
```

👉 内联的 Start 方法

直接看下面的代码。

```
inline int Start(Isolate* isolate, IsolateData* isolate_data,  
                int argc, const char* const* argv,  
                int exec_argc, const char* const* exec_argv)
```

对于 Node.js 的两大核心库 Chrome V8 和 libuv 来说，前面我们已经完成了 Chrome V8 的初始化，内联的 Start 方法主要是针对 libuv 的，具体完成了下面 4 项工作。

- 准备工作。
- 执行 LoadEnvironment(&env);。
- 开启 Event Loop，无限循环。
- 收尾，内存回收，断开 debug 连接。

5.3.2 构造 process 对象

process 是 Node.js 内置的全局对象，无须引用，在 Node.js 项目的任意位置都可以使用。Node.js 是单进程、单线程的，所以把所有的进程信息都放到 process 对象中是非常合理的，在 Java 8 里，也有对 process 的借鉴。

如果了解 Node.js 的原理，自然不能错过 process 对象。

👉 调用关系

在 CLion 里，为 Start 函数增加断点，通过调试可知，调用栈如图 5-9 所示。



图 5-9

下面对图 5-9 中 5 个核心调用过程进行讲解。

- main 是程序入口。
- node::Start 位于 node.cc 里，负责启动主流程，进行 Chrome V8 初始化。
- 内联的 Start 函数负责初始化 env 上下文环境，开启 Event Loop。
- void Environment::Start 调用 node.cc 里的 SetupProcessObject 实现。
- LoadEnvironment 负责加载运行环境。

看一下 env.cc 里的 Environment::Start 函数，设置 process 对象的方法如下。

```
auto process_object =
    process_template->GetFunction()->NewInstance(context()).ToLocalChecked();
set_process_object(process_object);

SetupProcessObject(this, argc, argv, exec_argc, exec_argv);
```

SetupProcessObject 最终调用的是 env 上的 process_object() 函数，如下。

```
Local<Value> arg = env->process_object();
```

👉 SetupProcessObject

Node.js 是单线程的(单一进程内只有一个线程)，所以当前进程的所有信息都会放到 process

对象中，这样是非常合理的，其中包含的信息也非常丰富，我们在终端里执行以下命令。

```
$ node -e 'console.log(process)'
```

执行后将打印出来的 process 对象信息整理成表格，见表 5-3。

表 5-3

名 称	示例数据	说 明
title	默认是'node'	终端上显示的标题
version	'v8.1.1'	Node.js 的版本号
pid	13176	Node.js 进程的版本号
moduleLoadList	['Binding contextify',... 'Binding signal_wrap'],	加载的模块列表
versions	{ http_parser: '2.7.0',node: '7.2.1',v8: '5.4.500.44',uv: '1.10.1'... }	Node.js 依赖模块的版本信息
env	{ TERM_PROGRAM: 'Apple_Terminal', SHELL: '/bin/zsh', ..._: '/Users/sang/.nvm/versions/node/v7.2.1/bin/node' },	当前进程使用的环境变量
config	{ target_defaults: { cflags: [], variables: {} }	当前 Node.js 构建时使用的配置信息， 可以辅助定位
features	{ debug: false, uv: true, ipv6: true, tls_npn: true,... }	当前 Node.js 版本开启和关闭的新功能

如果是指定文件，process 对象中还包含 mainModule 属性，如下。

```
mainModule:
Module {
  id: '.',
  exports: {},
  parent: null,
  filename: '/Users/sang/workspace/github/node/test.js',
  loaded: false,
  children: [ [Object] ],
  paths:
    [ '/Users/sang/workspace/github/node/node_modules',
      '/Users/sang/workspace/github/node_modules',
      '/Users/sang/workspace/node_modules',
      '/Users/sang/node_modules',
      '/Users/node_modules',
      '/node_modules' ] } }
```

下面举例说明一下 process 的用法。

(1) 统计信息：CPU、内存等

将 process 用于统计信息时，代码如下。

```
const startUsage = process.cpuUsage();  
// { user: 38579, system: 6986 }  
  
// 等待 500 毫秒  
const now = Date.now();  
while (Date.now() - now < 500);  
  
console.log(process.cpuUsage(startUsage));  
// { user: 514883, system: 11226 }
```

(2) 事件循环机制: process.nextTick

要想了解 process.nextTick, 先要了解一下 nextTick 和_tickCallback 两个方法, 示例代码如下。

```
function laterCallback() {  
  console.log('print me later');  
}  
  
process.nextTick(laterCallback);  
console.log('print me first');
```

对于上面的代码, 相信编写过 Node.js 的人都很熟悉, nextTick 的作用就是把 laterCallback 放到下一个事件循环中去执行。而_tickCallback 方法则是一个非公开的方法, 是在当前事件循环结束之后调用, 以进行下一个事件循环的入口函数。换言之, Node.js 为事件循环维持了一个队列, nextTick 入队列, _tickCallback 出队列。

(3) uncaughtException 事件

uncaughtException 是一个非常古老的事件。当 Node.js 发现一个未被捕获的异常时, 便会触发这个事件。如果这个事件中存在回调函数, Node.js 就不会强制结束进程, 用法如下。

```
process.on('uncaughtException', (err) => {  
  ...  
});
```

(4) 其他

- 进程管理: exit、kill。
- I/O 相关: stdout、stderr、stdin。
- 路径处理: cwd、chdir 等。

👉 process.binding 内部的 C++ 模块绑定

process 里有 moduleLoadList 属性, 是用来描述当前进程已经加载的模块的, 示例代码如下。

```
moduleLoadList:
[ 'Binding contextify',
  'Binding natives',
  'NativeModule events',
  'Binding config',
  'Binding icu',
  'NativeModule util',
  'Binding uv',
  ...
  'Binding stream_wrap',
  'Binding signal_wrap' ]
```

其中, 当前进程加载的模块又可以分为两种, 具体如下。

- Binding 模块: 通过 process.binding 绑定的内部 C++ 模块。
- NativeModule 模块: 内部 JavaScript 模块。

在 src/node.cc 里的 SetupProcessObject() 函数中, Binding 是用 C++ 代码实现的。

```
env->SetMethod(process, "binding", Binding);
```

具体的 Binding 函数的实现如下。

```
static void Binding(const FunctionCallbackInfo<Value>& args) {
    Environment* env = Environment::GetCurrent(args);

    Local<String> module = args[0]->ToString(env->isolate());
    node::Utf8Value module_v(env->isolate(), module);

    Local<Object> cache = env->binding_cache_object();
    Local<Object> exports;

    if (cache->Has(env->context(), module).FromJust()) {
        exports = cache->Get(module)->ToObject(env->isolate());
        args.GetReturnValue().Set(exports);
        return;
    }

    // 给 process.moduleLoadList 赋值
    char buf[1024];
    snprintf(buf, sizeof(buf), "Binding %s", *module_v);
```



```

Local<Array> modules = env->module load list array();
uint32_t l = modules->Length();
modules->Set(l, OneByteString(env->isolate(), buf));

node_module* mod = get_builtin_module(*module_v);
if (mod != nullptr) {
    exports = Object::New(env->isolate());
    // 内部绑定没有"module" 对象, 仅仅是 exports
    CHECK_EQ(mod->nm_register_func, nullptr);
    CHECK_NE(mod->nm_context_register_func, nullptr);
    Local<Value> unused = Undefined(env->isolate());
    mod->nm_context_register_func(exports, unused,
        env->context(), mod->nm_priv);
    cache->Set(module, exports);
} else if (!strcmp(*module_v, "constants")) {
    exports = Object::New(env->isolate());
    CHECK(exports->SetPrototype(env->context(),
        Null(env->isolate())) .FromJust());
    DefineConstants(env->isolate(), exports);
    cache->Set(module, exports);
} else if (!strcmp(*module_v, "natives")) {
    exports = Object::New(env->isolate());
    DefineJavaScript(env, exports);
    cache->Set(module, exports);
} else {
    char errmsg[1024];
    snprintf(errmsg,
        sizeof(errmsg),
        "No such module: %s",
        *module_v);
    return env->ThrowError(errmsg);
}

args.GetReturnValue().Set(exports);
}

```

在加载内建模块时, 我们要按照如下步骤进行操作。

- 先创建一个 exports 空对象。
- 调用 get_builtin_module() 方法获得 node_module 对象 mod。
- 调用 mod 的 nm_context_register_func() 方法来构建 exports 对象。
- 将 exports 对象按模块名进行缓存, 即通过 cache->Set(module, exports) 方法。
- 返回调用方, 完成导出。

之前的模块列表中还有两个特殊的模块，即 Binding constants 和 Binding natives，在 Binding 函数里也对它们进行了特殊处理。将前面提到的 JavaScript 转为 C/C++ 文件 node_javascript.cc 里的内容，然后通过 process.binding('native') 取出，放到 lib/internal/bootstrap_node.js 文件里的 NativeModule._source 中，代码如下。

```
NativeModule._source = process.binding('natives');
```

然后看一下 NativeModule 的 compile 方法，代码如下。

```
NativeModule.getSource = function(id) {
  return NativeModule._source[id];
};

NativeModule.prototype.compile = function() {
  var source = NativeModule.getSource(this.id);
  ...
};
```

你会发现，模块里的 getSource 方法的具体实现其实就是在上面提到的 process.binding('natives')。

```
NativeModule.getSource == NativeModule._source == process.binding('natives')
```

✎ process.binding('contextify') 示例

大家常见的.js 文件最后都是通过 process.binding('contextify') 进行编译的，因此可以说，这是一个相当重要的模块。

lib/vm.js 代码如下。

```
'use strict';

const binding = process.binding('contextify');
const Script = binding.ContextifyScript;
...
const realRunInThisContext = Script.prototype.runInThisContext;
const realRunInContext = Script.prototype.runInContext;

Script.prototype.runInThisContext = function(options) {
  if (options && options.breakOnSigint && process._events.SIGINT) {
    return sigintHandlersWrap(realRunInThisContext, this, [options]);
  } else {
    return realRunInThisContext.call(this, options);
  }
}
```

```
};
...
```

lib/internal/bootstrap_node.js 代码如下。

```
const ContextifyScript = process.binding('contextify').ContextifyScript;

function runInThisContext(code, options) {
  const script = new ContextifyScript(code, options);
  return script.runInThisContext();
}
```

以上两段代码的功能是一样的, 差别在于, vm.js 在开发者自己创建线程的时候进行调用, 而 bootstrap_node.js 是在对 NativeModule 进行编译时调用。按照前面的讲解, 凡是 process.binding 里的项目都是 C++ 代码, 由此推断, process.binding('contextify') 对应的是 src/node_contextify.cc, 事实上也的确如此。

src/node_contextify.cc 文件里定义了两个类, 即 ContextifyContext 和 ContextifyScript, 很明显, ContextifyScript 是核心, 可操作 ContextifyContext 上下文, Chrome V8 上下文在 ContextifyContext 里初始化。通过下面的核心 JavaScript 代码可以更好地理解其原理。

```
function runInThisContext(code, options) {
  const script = new ContextifyScript(code, options);
  return script.runInThisContext();
}
```

通过以上代码可以看到, new ContextifyScript() 调用 Script, 获得 Script 对象, 然后执行 script.runInThisContext() 并返回结果。

下面查看 C++ 源码里关于 runInThisContext 方法的定义。

```
public:
static void Init(Environment* env, Local<Object> target) {
  HandleScope scope(env->isolate());
  Local<String> class_name =
    FIXED ONE BYTE STRING(env->isolate(), "ContextifyScript");

  Local<FunctionTemplate> script_tmpl = env->NewFunctionTemplate(New);
  script_tmpl->InstanceTemplate()->SetInternalFieldCount(1);
  script_tmpl->SetClassName(class_name);
  env->SetProtoMethod(script_tmpl, "runInContext", RunInContext);
  env->SetProtoMethod(script_tmpl, "runInThisContext", RunInThisContext);

  target->Set(class_name, script_tmpl->GetFunction());
  env->set_script_context_constructor_template(script_tmpl);
}
```

对应的 `RunInThisContext` 代码如下。

```
static void RunInThisContext(const FunctionCallbackInfo<Value>& args) {
    ...
    EvalMachine(env, timeout, display_errors, break_on_sigint, args,
                &try_catch);
}
```

`EvalMachine` 代码如下。

```
static bool EvalMachine(Environment* env,
                        const int64_t timeout,
                        const bool display_errors,
                        const bool break_on_sigint,
                        const FunctionCallbackInfo<Value>& args,
                        TryCatch* try_catch) {
    ContextifyScript* wrapped_script;
    ASSIGN_OR_RETURN_UNWRAP(&wrapped_script, args.Holder(), false);
    Local<UnboundScript> unbound_script =
        PersistentToLocal(env->isolate(), wrapped_script->script_);
    Local<Script> script = unbound_script->BindToCurrentContext();
    ...
    result = script->Run();
    ...
    args.GetReturnValue().Set(result);
    return true;
}
```

以上整个过程实现了 Chrome V8 上下文的初始化，处理 `Script` 对象，对处理后的 `Script` 调用 `run` 函数获得结果并返回。整段代码设计得可圈可点，结构清晰，有很多可取之处。

5.3.3 LoadEnvironment

所有的 JavaScript 输入文件都处在 `LoadEnvironment` 阶段，由 Chrome V8 引擎负责加载并执行，可以说这一步是非常重要的入口。

首先将 `node_js2c` 任务里生成的 `node_javascript.cc` 的 `bootstrap_node.js` 代码加载出来，这一步非常重要。先来看一下 `bootstrap_node.js`，它的参数是 `process` 的匿名函数。

```
(function(process) {

    function startup() {
        ...
    }

    ...

})
```

```

    startup();
});

```

调用方式应该为 `f(process)`，掌握这个对于理解后面的代码至关重要。加载脚本的代码如下。

```

Local<String> script_name = FIXED_ONE_BYTE_STRING(env->isolate(),
                                                "bootstrap node.js");
Local<Value> f_value = ExecuteString(env, MainSource(env), script_name);

```

初始化全局对象并对外暴露，这样便可以实现随处访问。

```

Local<Object> global = env->context()->Global();

global->Set(FIXED_ONE_BYTE_STRING(env->isolate()), "global", global);

```

从 `env` 里获取 `process` 对象，作为参数传递给最终要执行的 `bootstrap_node.js`，即最终调用 `f(process)`，翻译为 C++ 代码如下。

```

Local<Value> arg = env->process object();
f->Call(Null(env->isolate()), 1, &arg);

```

总结一下，整个过程分为 3 步，具体如下。

- 从 `node_javascript.cc` 的 `bootstrap_node.js` 代码中加载出 `f` 变量。
- 准备全局对象以便全局使用，准备 `f` 的参数 `process` 对象。
- 执行 `f->Call` 来调用 `f(process)`。

5.3.4 bootstrap_node.js

`bootstrap_node.js` 的代码结构如下，参数是 `process` 对象，一开始就执行了 `startup` 函数。

```

(function(process) {

    function startup() {
        ...
    }

    ...

    startup();
});

```

下面带大家解读一下源码。

👉 process 继承 EventEmitter

如下面的代码所示，这里的 `EventEmitter` 是通过 `NativeModule.require` 函数来加载的，这和 `CommonJS` 规范里的 `require` 是一样的。或者更准确点来说，`NativeModule.require` 是 `CommonJS` 规范里的 `require` 的具体实现。

```
const EventEmitter = NativeModule.require('events');
process.eventsCount = 0;

const origProcProto = Object.getPrototypeOf(process);
Object.setPrototypeOf(process, Object.create(EventEmitter.prototype, {
  constructor: Object.getOwnPropertyDescriptor(origProcProto, 'constructor')
}));

EventEmitter.call(process);
```

👉 设置 process 对象

设置 `process` 对象的具体代码如下。

```
setupProcessObject();

setupProcessFatal();

setupProcessICUVersions();

setupGlobalVariables();

...

const _process = NativeModule.require('internal/process');

process.setup hrtime();

...

Object.defineProperty(process, 'argv0', {
  enumerable: true,
  configurable: false,
  value: process.argv[0]
});

process.argv[0] = process.execPath;
```

其中的 `setupGlobalVariables` 函数代码如下。

```
function setupGlobalVariables() {
  Object.defineProperty(global, Symbol.toStringTag, {
    value: 'global',
    writable: false,
    enumerable: false,
    configurable: true
  });
  global.process = process;
  const util = NativeModule.require('util');

  ...

  global.Buffer = NativeModule.require('buffer').Buffer;
  process.domain = null;
  process.exiting = false;
}
```

这段代码值得解读一下。

- `global.process = process;`: 声明全局 `process` 对象。
- `global.Buffer = NativeModule.require('buffer').Buffer;`: 声明类型 `Buffer`。

另外还有一个大家可能都想要了解的细节, 即 `console.log` 到底是怎么来的, 具体代码如下。

```
function setupGlobalConsole() {
  const originalConsole = global.console;
  let console;
  Object.defineProperty(global, 'console', {
    configurable: true,
    enumerable: true,
    get: function() {
      if (!console) {
        console = originalConsole === undefined ?
          NativeModule.require('console') :
            installInspectorConsole(originalConsole);
      }
      return console;
    }
  });
}
```

根据 `NativeModule.require('console')` 可知, `Console` 是 `NativeModule` 模块, 即 `lib/console.js`。同样的还有我们常用的 `console.log` 函数, 代码如下。

```
Console.prototype.log = function log(...args) {
  write(this._ignoreErrors,
    this._stdout,
```

```

    `${util.format.apply(null, args)}\n`,
    this._stdoutErrorHandler);
};

```

由于 Node.js 8 源码支持 99% 以上的 ES6 语法，所以这里使用了 rest 参数和模板字符串，应该很容易看明白。

🔪 调试处理

调试的代码如下。

```

if (process. invalidDebug) {
  process.emitWarning(
    '`node --debug` and `node --debug-brk` are invalid. ' +
    'Please use `node --inspect` or `node --inspect-brk` instead.',
    'DeprecationWarning', 'DEP0062', startup, true);
  process.exit(9);
} else if (process. deprecatedDebugBrk) {
  process.emitWarning(
    '`node --inspect --debug-brk` is deprecated. ' +
    'Please use `node --inspect-brk` instead.',
    'DeprecationWarning', 'DEP0062', startup, true);
}

```

Node.js 支持两种调试协议（基于 TCP 实现的，默认端口为 5858），即老版本的 debug 和新版本的 inspect，另外 VSCode 的通用调试协议也进行了适配，如图 5-10 所示。

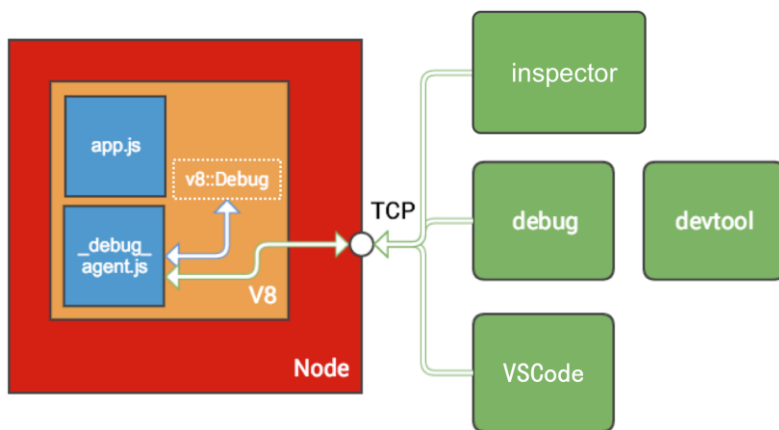


图 5-10

对比一下两种调试协议，如表 5-4 所示。

表 5-4

	debug (旧协议)	inspect (新协议)
支持版本	目前支持所有版本, 以后会被移除	Node.js v6.3 以上、Chrome 55 以上
用法	<code>node --debug app.js</code>	<code>node --inspect app.js</code>

说明: `node --inspect` 如果没有设置断点, 则不会自动停在代码第一行的地方。而 `node --inspect-brk` 会在第一行代码处停住。

👉 模块加载过程

先来介绍一下模块加载过程的几项原则, 具体如下。

- 如果存在 `_third_party_main` 模块, 则直接执行。
- 如果参数中有 `inspect` 或 `debug` 选项, 则直接启动调试。
- 如果有 `--remote_debugging_server` 选项, 则启动远程调试。
- 如果有 `-e` 选项, 则直接执行。
- 如果是 `repl` 模式, 则按 CLI 命令行方式执行。
- 如果是文件 `eval`, 则通过 `eval` 的方式直接执行。

在 Node.js 源码里, 模块加载过程如下。

```
preloadModules();

const internalModule = NativeModule.require('internal/module');
internalModule.addBuiltinLibsToObject(global);
evalScript(['eval']);
```

其中的核心是 `evalScript` 代码, 具体如下。

```
function evalScript(name) {
  const Module = NativeModule.require('module');
  const path = NativeModule.require('path');
  const cwd = tryGetCwd(path);

  const module = new Module(name);
  module.filename = path.join(cwd, name);
  module.paths = Module._nodeModulePaths(cwd);
  const body = process._eval;
```

```

const script = `global.  filename = ${JSON.stringify(name)};\n` +
  'global.exports = exports;\n' +
  'global.module = module;\n' +
  'global.  dirname =  dirname;\n' +
  'global.require = require;\n' +
  'return require("vm").runInThisContext(' +
  `${JSON.stringify(body)}, { filename: ` +
  `${JSON.stringify(name)}, displayErrors: true });\n`;
const result = module._compile(script, `${name}-wrapper`);
if (process._print_eval) console.log(result);

process._tickCallback();
}

```

如果模块加载的是文件，则加载代码如下。

```

// 保证 process.argv[1] 是一个完整路径
const path = NativeModule.require('path');
process.argv[1] = path.resolve(process.argv[1]);

const Module = NativeModule.require('module');

if (process._syntax_check_only != null) {
  const fs = NativeModule.require('fs');
  // 读取文件中的源代码
  const filename = Module._resolveFilename(process.argv[1]);
  var source = fs.readFileSync(filename, 'utf-8');
  checkScriptSyntax(source, filename);
  process.exit(0);
}

preloadModules();
Module.runMain();

```

对于上面这段代码，有几个要点要着重讲解一下，具体如下。

1. `const Module = NativeModule.require('module');`: 根据约定，Module 其实就是 `lib/module.js`。

2. `checkScriptSyntax(source, filename)`: 主要是通过 `vm` 模块对应源码进行编译的，然后加载源码到上下文对象中，代码如下。

```

function checkScriptSyntax(source, filename) {
  const Module = NativeModule.require('module');
  const vm = NativeModule.require('vm');
  const internalModule = NativeModule.require('internal/module');

  // 移除 Shebang (#! 开头) 的语句
  source = internalModule.stripShebang(source);

```

```
// 移除 BOM (编辑器使用的当前文件采用编码信息)
source = internalModule.stripBOM(source);
// 将代码模块包装
source = Module.wrap(source);
// vm 执行对应代码, 如果失败会显示错误信息
new vm.Script(source, {displayErrors: true, filename});
}
```

3.preloadModules();: preloadModules 是负责进行模块预加载的, 加载的模块位于 process.preload_modules 中, 而 process.preload_modules 是通过 argv 参数解析获得的, 所以它其实是对 Node.js 里的 -r 选项的实现。preloadModules 函数的代码如下。

```
function preloadModules() {
  if (process._preload_modules) {
    NativeModule.require('module')._preloadModules(process._preload_modules);
  }
}
```

Node.js 命令在执行的时候, 可以通过 -r 来指定预加载的模块, 具体方法如下。

```
$ node --help|grep require
-r, --require          module to preload (option can be repeated)
```

4.Module.runMain();: 在 lib/module.js 里, Module.runMain();代码如下。

```
// 启动 main 模块。
Module.runMain = function() {
  // 加载 main 模块
  Module._load(process.argv[1], null, true);
  // 将 nextTickQueue 中的回调函数先执行完
  process._tickCallback();
};
```

其中的核心方法代码如下。

```
Module._load = function(request, parent, isMain) {
  if (parent) {
    debug('Module._load REQUEST %s parent: %s', request, parent.id);
  }

  var filename = Module._resolveFilename(request, parent, isMain);

  var cachedModule = Module._cache[filename];
  if (cachedModule) {
    return cachedModule.exports;
  }

  if (NativeModule.nonInternalExists(filename)) {
    debug('load native module %s', request);
```

```
    return NativeModule.require(filename);
  }

  var module = new Module(filename, parent);

  if (isMain) {
    process.mainModule = module;
    module.id = '.';
  }

  Module._cache[filename] = module;

  tryModuleLoad(module, filename);

  return module.exports;
};
```

在以上代码中，需要注意一些特殊情况。

- 如果缓存里有加载过的文件，就返回缓存里的内容。
- 如果要加载的文件是 `NativeModule`，就使用 `NativeModule.require()` 来加载。

对于其他的常规情况而言，代码就显得简单多了，具体流程及代码实现如下。

- 实例化 `Module` 获得 `module` 对象。
- 将 `module` 对象添加到缓存 `Module._cache` 里。
- 通过 `tryModuleLoad` 来加载对应模块。
- 将 `module.exports` 返回。

```
var module = new Module(filename, parent);

if (isMain) {
  process.mainModule = module;
  module.id = '.';
}

Module._cache[filename] = module;

tryModuleLoad(module, filename);

return module.exports;
```

下面来看一下 `tryModuleLoad` 里调用的 `module.load` 函数，代码如下。

```
// 根据文件名进行加载, 此处由文件后缀判断
Module.prototype.load = function(filename) {
  debug('load %j for module %j', filename, this.id);

  assert(!this.loaded);
  this.filename = filename;
  this.paths = Module._nodeModulePaths(path.dirname(filename));

  var extension = path.extname(filename) || '.js';
  if (!Module._extensions[extension]) extension = '.js';
  Module._extensions[extension](this, filename);
  this.loaded = true;
};
```

paths 即 `Module._nodeModulePaths`, 此函数在不同平台中有不一样的实现方式, 这里就不一一列举了。

👉 Native 模块和 CommonJS 实现

Native 模块的定义如下。

```
function NativeModule(id) {
  this.filename = `${id}.js`;
  this.id = id;
  this.exports = {};
  this.loaded = false;
  this.loading = false;
}
```

看一下 `require` 的用法, 代码如下。

```
NativeModule.require = function(id) {
  if (id === 'native module') {
    return NativeModule;
  }

  const cached = NativeModule.getCached(id);
  if (cached && (cached.loaded || cached.loading)) {
    return cached.exports;
  }

  if (!NativeModule.exists(id)) {
    const err = new Error(`No such built-in module: ${id}`);
    err.code = 'ERR_UNKNOWN_BUILTIN_MODULE';
    err.name = 'Error [ERR_UNKNOWN_BUILTIN_MODULE]';
    throw err;
  }
}
```

```
process.moduleLoadList.push(`NativeModule ${id}`);

const nativeModule = new NativeModule(id);

nativeModule.cache();
nativeModule.compile();

return nativeModule.exports;
};
```

关于以上代码，说明如下。

- 实例化 `nativeModule`。
- 通过 `NativeModule._cache[this.id] = this`;缓存 `this` 对象。
- 编译的步骤很多，是核心部分。
- 对外暴露 `exports`。

编译过程的代码如下。

```
NativeModule.getSource = function(id) {
  return NativeModule._source[id];
};

NativeModule.wrap = function(script) {
  return NativeModule.wrapper[0] + script + NativeModule.wrapper[1];
};

NativeModule.wrapper = [
  '(function (exports, require, module, __filename, __dirname) { ',
  '\n});'
];

NativeModule.prototype.compile = function() {
  var source = NativeModule.getSource(this.id);
  source = NativeModule.wrap(source);

  this.loading = true;

  try {
    const fn = runInThisContext(source, {
      filename: this.filename,
      lineOffset: 0,
      displayErrors: true
    });
  }
```

```
fn(this.exports, NativeModule.require, this, this.filename);

this.loaded = true;
} finally {
  this.loading = false;
}
};
```

关于以上代码，说明如下。

- 通过 `NativeModule.getSource` 获得 `source`。
- 使用 `NativeModule.wrap` 对获得的 `source` 进行包裹，注入 `CommonJS` 相关参数，变成新的函数。
- 通过 `runInThisContext` 在 `Chrome V8` 里执行 `script`，获得 `fn` 函数。
- 执行 `fn` 函数，即执行模块内的代码，并注入相关参数。

在 `lib/module.js` 里，代码如下。

```
const NativeModule = require('native_module');

...
Module.wrapper = NativeModule.wrapper;
Module.wrap = NativeModule.wrap;
```

`require` 实际是 `NativeModule.require`，所以这里进行了特殊处理，直接返回了 `NativeModule`。

```
NativeModule.require = function(id) {
  if (id === 'native_module') {
    return NativeModule;
  }
}
```

`lib/module.js` 和 `lib/internal/bootstrap_node.js` 里的 `NativeModule` 互相调用，这部分是比较难理解的，各位读者要注意思考。

👉 内置模块加载过程

内置的模块在 `Node.js` 启动的时候就加载了，它的加载速度比其他模块要快很多。为了探寻内置模块的加载过程，我们试着进行如下分析。

第一步：在调用 `require('fs')` 时查看 `require` 方法的实现。

第二步：探寻 `require` 方法的来源。

在 Node.js 里，CommonJS 是通过包裹来实现的，核心是 `NativeModule.wrapper` 方法，其中的第二个参数就是 `require`。也就是说，`require` 就是在调用 `require('fs')` 时被注入的，示例如下。

```
NativeModule.wrapper = [
  '(function (exports, require, module, __filename, __dirname) { ',
  '\n});'
];
```

调用 `NativeModule.wrapper` 方法时，注入的第二个参数其实是 `NativeModule.require`。示例如下。

```
var source = NativeModule.getSource(this.id);
source = NativeModule.wrap(source);
const fn = runInThisContext(source, {
  filename: this.filename,
  lineOffset: 0,
  displayErrors: true
});
fn(this.exports, NativeModule.require, this, this.filename);
```

至此，相信大家已经看明白了，`require('fs')` 等价于 `NativeModule.require('fs')`。如果想探究内置模块的加载原理，需要从 `NativeModule.require` 入手。

第三步：从 `process.binding('natives')` 中加载 `node_javascript.cc` 里的 JavaScript 源文件信息。

之前的模块列表中还有两个特殊的模块，即 `Binding constants` 和 `Binding natives`，在 `Binding` 函数里也对它们进行了特殊处理。将前面提到的 JavaScript 转为 C/C++ 文件 `node_javascript.cc` 里的内容，然后通过 `process.binding('native')` 取出，放到 `lib/internal/bootstrap_node.js` 文件里的 `NativeModule._source` 中。前面讲过，`NativeModule.getSource`、`NativeModule._source`、`process.binding('natives')` 是等价的。

第四步：获得内置模块。

- 调用 `get_builtin_module()` 方法获得 `node_module` 对象 `mod`。
- 调用 `mod` 的 `nm_context_register_func()` 方法构建 `exports` 对象。

第五步：宏定义。

- 第一个宏：`NODE_MODULE(modname, regfunc)` 定义普通模块。
- 第二个宏：`NODE_MODULE_CONTEXT_AWARE_BUILTIN(modname, regfunc)` 定义

内置模块。

上面两个宏定义的实现方法大同小异，都通过 `NODE_MODULE_X` 方法进行加载，代码如下。差别在于，`regfunc` 的类型不同

```
#define NODE_MODULE X(modname, regfunc, priv, flags) \
extern "C" { \
    static node::node_module _module = \
    { \
        NODE_MODULE_VERSION, \
        flags, \
        NULL, \
        __FILE__, \
        (node::addon_register_func) (regfunc), \
        NULL, \
        NODE_STRINGIFY(modname), \
        priv, \
        NULL \
    }; \
    NODE_C_CTOR(_register_ ## modname) { \
        node module_register(& module); \
    } \
}
```

5.3.5 EventLoop 启动方法

本节我们一起来看一下最核心的 EventLoop 启动方法，其定义如下。

```
inline int Start(Isolate* isolate, IsolateData* isolate_data,
                int argc, const char* const* argv,
                int exec_argc, const char* const* exec_argv)
```

调用方式如下。参数为 `isolate`，值为 `uv_default_loop()`。

```
Start(uv_default_loop(),
```

核心代码如下。

```
{
    SealHandleScope seal(isolate);
    bool more;
    do {
        v8_platform.PumpMessageLoop(isolate);
        more = uv_run(env.event_loop(), UV_RUN_ONCE);

        if (more == false) {
            v8_platform.PumpMessageLoop(isolate);
```

```

    EmitBeforeExit(&env);

    more = uv_loop_alive(env.event_loop());
    if (uv_run(env.event_loop(), UV_RUN_NOWAIT) != 0)
        more = true;
}
} while (more == true);
}

```

调用 libuv 的 `uv_run()` 函数，此函数是 libuv 里最重要的 API，每次调用此函数都会进行一次事件循环处理。下面展示一个 UNIX 版本的 `uv_run()` 实现（位于 `deps/uv/src/unix/core.c` 文件里）的示例，代码如下。

```

int uv_run(uv_loop_t* loop, uv_run_mode mode) {
    int timeout;
    int r;
    int ran_pending;

    r = uv__loop_alive(loop);
    if (!r)
        uv__update_time(loop);

    while (r != 0 && loop->stop_flag == 0) {
        uv__update_time(loop);
        uv__run_timers(loop);
        ran_pending = uv__run_pending(loop);
        uv__run_idle(loop);
        uv__run_prepare(loop);

        timeout = 0;
        if ((mode == UV_RUN_ONCE && !ran_pending) || mode == UV_RUN_DEFAULT)
            timeout = uv__backend_timeout(loop);

        uv__io_poll(loop, timeout);
        uv__run_check(loop);
        uv__run_closing_handles(loop);

        if (mode == UV_RUN_ONCE) {
            /* 只处理一次时间，UV_RUN_ONCE 会在没有任务的时候阻塞，而 UV_RUN_NOWAIT 会立刻返回 */
            uv__update_time(loop);
            uv__run_timers(loop);
        }

        r = uv__loop_alive(loop);
        if (mode == UV_RUN_ONCE || mode == UV_RUN_NOWAIT)
            break;
    }
}

```

```
    if (loop->stop_flag != 0)
        loop->stop_flag = 0;

    return r;
}
```

从 Chrome V8、process 对象、处理 process.binding、加载环境变量、bootstrap_node.js 对 Native Nodule 进行封装，到最后启动 libuv 的 EventLoop。整个过程还是有些复杂的，但对于我们理解 Node.js 原理来说，是必要的过程。之所以用这么长的篇幅来讲解这个过程，也是希望大家在用的时候能够真真切切地明白其中的原理，当然，如果能促使大家对 Node.js 源码有所贡献，也是极好的。

5.4 API 调用过程

文件 fs 模块是最常见的 Node.js 内置模块之一，其中最常用的当然是读文件和写文件方法，这里以读文件（readFile）为例进行讲解，常用代码如下。

```
const fs = require('fs')

fs.readFile('/etc/passwd', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

下面看一下 lib/fs.js 里的 fs.readFile 函数的定义，具体如下。

```
fs.readFile = function(path, options, callback) {
  callback = maybeCallback(callback || options);
  options = getOptions(options, { flag: 'r' });

  if (handleError((path = getPathFromURL(path)), callback))
    return;
  if (!nullCheck(path, callback))
    return;

  var context = new ReadFileContext(callback, options.encoding);
  context.isUserFd = isFd(path); // file descriptor ownership
  var req = new FSReqWrap();
  req.context = context;
  req.oncomplete = readFileAfterOpen;

  if (context.isUserFd) {
    process.nextTick(function() {
      req.oncomplete(null, path);
    });
  }
}
```

```

    });
    return;
  }

  binding.open(pathModule._makeLong(path),
    stringToFlags(options.flag || 'r'),
    0o666,
    req);
};

```

5.4.1 相关的引用

我们先来看一下 lib/fs.js 里的模块引用方法，核心如下。

```

const binding = process.binding('fs');
const FSReqWrap = binding.FSReqWrap;

```

ReadFileContext 是一个对象，负责维护当前文件的上下文信息（比如 fd、pos、encoding 等），该对象只有 read 和 close 方法。

5.4.2 FSReqWrap

在上面的相关引用里，你会很容易发现 FSReqWrap 是核心实现，其实底层的 Open() 函数会在一系列辅助操作之后进入 uv_fs_open() 函数，其参数就是 FSReqWrap 对象。

FSReqWrap() 是一个与 fs 相关的请求包装。也就是说，它基于某一个现成的机制来实现与 fs 相关的请求操作，这个现成的机制就是 ReqWrap。图 5-11 完整展示了 FSReqWrap 类的继承关系。

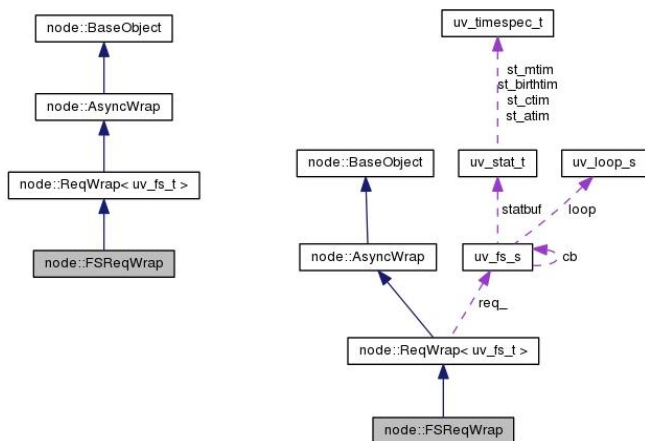


图 5-11

5.4.3 核心 open 方法

核心 open 方法的代码如下。

```
binding.open(pathModule._makeLong(path),
             stringToFlags(options.flag || 'r'),
             0o666,
             req);
```

那么 process.binding('fs')里的 open 方法到底是怎么定义的呢？请大家思考。

5.4.4 src/node_file.cc

直接看 src/node_file.cc 的代码，具体如下。

```
static void Open(const FunctionCallbackInfo<Value>& args) {
    Environment* env = Environment::GetCurrent(args);

    int len = args.Length();
    if (len < 1)
        return TYPE_ERROR("path required");
    if (len < 2)
        return TYPE_ERROR("flags required");
    if (len < 3)
        return TYPE_ERROR("mode required");
    if (!args[1]->IsInt32())
        return TYPE_ERROR("flags must be an int");
    if (!args[2]->IsInt32())
        return TYPE_ERROR("mode must be an int");

    BufferValue path(env->isolate(), args[0]);
    ASSERT_PATH(path)

    int flags = args[1]->Int32Value();
    int mode = static_cast<int>(args[2]->Int32Value());

    if (args[3]->IsObject()) {
        ASYNC_CALL(open, args[3], UTF8, *path, flags, mode)
    } else {
        SYNC_CALL(open, *path, *path, flags, mode)
        args.GetReturnValue().Set(SYNC_RESULT);
    }
}
```

其中的核心在于 ASYNC_CALL 和 SYNC_CALL 这两个具体实现的宏，如下。

```

#define ASYNC_DEST_CALL(func, request, dest, encoding, ...) \
    Environment* env = Environment::GetCurrent(args); \
    CHECK(request->IsObject()); \
    FSReqWrap* req_wrap = FSReqWrap::New(env, request.As<Object>(), \
                                           #func, dest, encoding); \
    int err = uv_fs_ ## func(env->event_loop(), \
                             req_wrap->req(), \
                             __VA_ARGS__, \
                             After); \
    req_wrap->Dispatched(); \
    if (err < 0) { \
        uv_fs_t* uv_req = req_wrap->req(); \
        uv_req->result = err; \
        uv_req->path = nullptr; \
        After(uv_req); \
        req_wrap = nullptr; \
    } else { \
        args.GetReturnValue().Set(req_wrap->persistent()); \
    } \

#define ASYNC_CALL(func, req, encoding, ...) \
    ASYNC_DEST_CALL(func, req, nullptr, encoding, __VA_ARGS__) \

#define SYNC_DEST_CALL(func, path, dest, ...) \
    fs_req_wrap req_wrap; \
    env->PrintSyncTrace(); \
    int err = uv_fs_ ## func(env->event_loop(), \
                             &req_wrap.req, \
                             VA_ARGS, \
                             nullptr); \
    if (err < 0) { \
        return env->ThrowUVException(err, #func, nullptr, path, dest); \
    } \

#define SYNC_CALL(func, path, ...) \
    SYNC_DEST_CALL(func, path, nullptr, __VA_ARGS__) \

```

以 ASYNC_CALL 为例,调用的方法为 ASYNC_CALL(open, args[3],UTF8,*path,flags, mode), 定义的方法为 ASYNC_DEST_CALL(func, request, dest, encoding, ...), 即 func=open。“翻译”一下, 用代码表示如下。

```

function ASYNC_DEST_CALL('open', request, dest, encoding, ...)
    Environment* env = Environment::GetCurrent(args);
    CHECK(request->IsObject());
    FSReqWrap* req_wrap = FSReqWrap::New(env, request.As<Object>(),
                                           'open', dest, encoding);
    int err = uv fs open(env->event loop(),
                        req_wrap->req(),

```

```

        VA_ARGS ,
        After);
req_wrap->Dispatched();
if (err < 0) {
    uv_fs_t* uv_req = req_wrap->req();
    uv_req->result = err;
    uv_req->path = nullptr;
    After(uv_req);
    req_wrap = nullptr;
} else {
    args.GetReturnValue().Set(req_wrap->persistent());
}
}
}

```

实际上，最终调用的是 libuv 里提供的 uv_fs_open 函数，代码如下。

```

UV_EXTERN int uv_fs_open(
    uv_loop_t* loop, uv_fs_t* req, const char* path, int flags, int mode, uv_fs_cb cb);

```

图 5-12 描述了 libuv 处理 File I/O 请求时涉及的各种操作，具体如下。

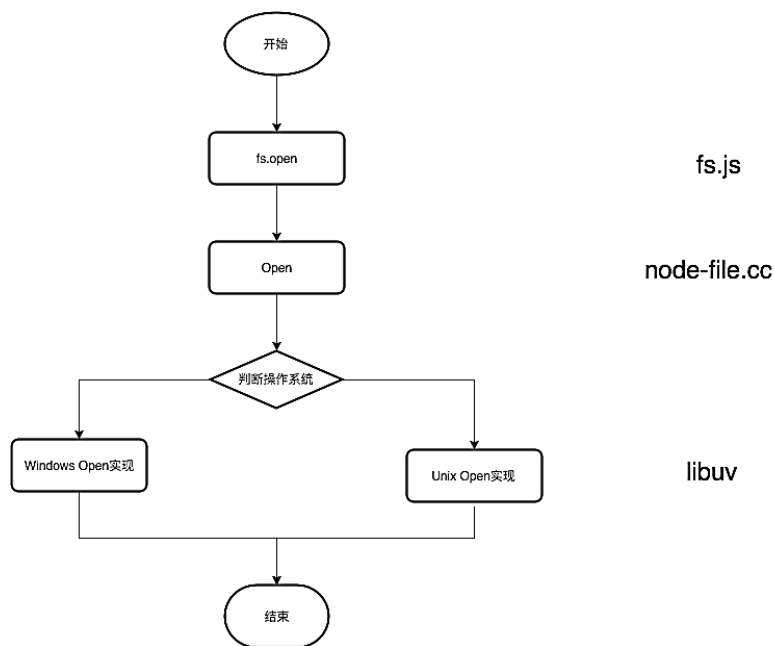


图 5-12

libuv 所做的工作当然远不止这些，限于篇幅，很多细节没有讨论到。

5.5 事件循环机制

事件循环（EventLoop）是 libuv 的核心，也称为 I/O Loop，它建立在所有的 I/O 内容操作的基础上，如图 5-13 所示。



图 5-13

5.5.1 概览

Ryan Dahl 在开发 Node.js 的时候，便希望能够解决一些快速开发异步任务的问题，为此他比较了 Lua 语言和 JavaScript 语言的差异，最终非常明智地选择了 JavaScript。当时可供选择的异步事件处理库有 libevent 和 Marc Lehmann 的 libev，但是 libev 只能在 UNIX 环境下运行，不能支持 Windows 系统，无法满足日益流行的 Node.js 的包容性，因此没有被应用。

Windows 平台上与 kqueue(FreeBSD)或者(e)poll(Linux)等内核事件通知响应相关的机制是 IOCP，libuv 诞生后，它提供了一个跨平台的抽象，可以决定使用 libev 还是 IOCP。在 Node.js v0.9.0 中，libuv 彻底移除了 libev 的内容。

其实 Node.js 对于 JavaScript 语言层面的优化，Chrome V8 引擎都已经做得相当不错了，所以就有了 Node.js 跟随 Chrome V8 引擎发布版本提升性能的说法。但是 Chrome V8 并不具有如 I/O 操作等能力，而 libuv 却可以补齐这个能力。Node.js 在 Chrome V8 引擎层发起有关文件、网络等 I/O 操作，并在事件循环中加入事件及对应的回调，当 libuv 里的任务执行完成之后，会调用注册的回调函数并注入处理结果。图 5-14 是 libuv 官方文档中给出的设计图，很清晰地展

示了 libuv 的架构。

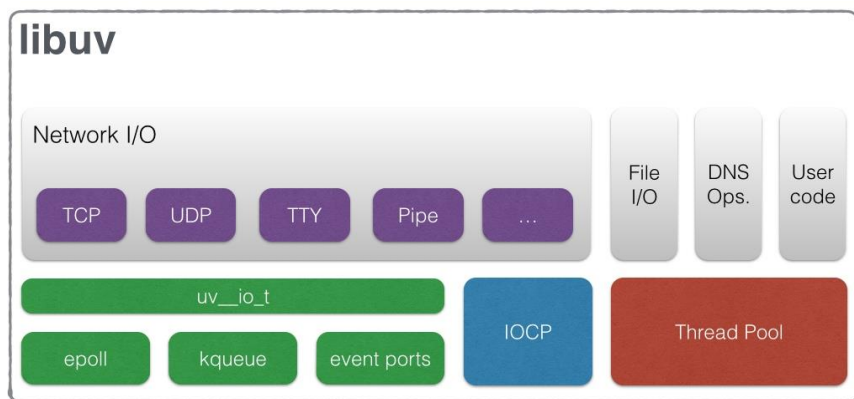


图 5-14

需要注意的是，Node.js 是单进程、单线程的，但 libuv 并不是单线程的，它依赖一个伴随 Node.js 启动而初始化的线程池来实现。

在 libuv 事件编程模型中，应用程序只负责监视特定的事件，并在事件发生后进行响应，下面是事件驱动编程模型的伪代码。

```
while there are still events to process:
    e = get the next event
    if there is a callback associated with e:
        call the callback
```

为了便于理解原理，来看一个最简单的 libuv 的例子，核心是 uv_run 方法，代码如下。

```
#include <stdio.h>
#include <stdlib.h>
#include <uv.h>

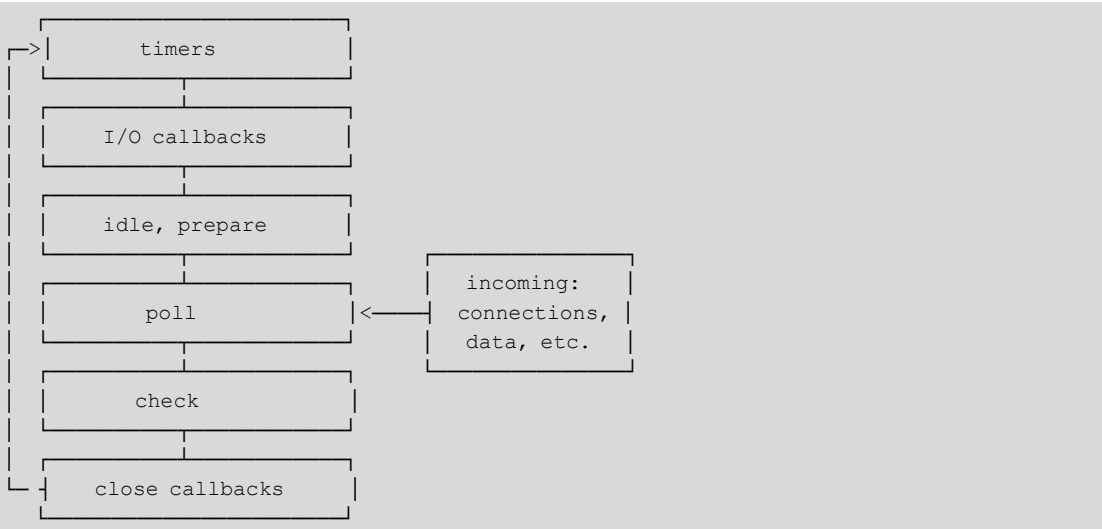
int main() {
    uv_loop_t *loop = malloc(sizeof(uv_loop_t));
    uv_loop_init(loop);

    printf("Now quitting.\n");
    uv_run(loop, UV_RUN_DEFAULT);

    uv_loop_close(loop);
    free(loop);
    return 0;
}
```

5.5.2 生命周期

Node.js 启动时会初始化事件循环机制，每次事件循环都会包含如下 6 个阶段。



关于上面的几个阶段，说明如下，见表 5-5。

表 5-5

编 号	阶段名称	说 明
1	timers	执行 setTimeout() 和 setInterval() 设定的回调
2	I/O callbacks	执行绝大部分回调，除了 close、timers 和 setImmediate() 设定的回调
3	idle, prepare	仅内部使用
4	poll	获取新的 I/O 事件，在适当的条件下，Node.js 会在这里阻塞
5	check	setImmediate() 设定的回调会在这一阶段执行
6	close callbacks	socket.on('close', callback) 的回调会在这个阶段执行

每个阶段都有一个先入先出（FIFO）的用于执行回调的队列，通常当事件循环运行到某个阶段时，Node.js 会先执行该阶段的操作，然后再去执行该阶段队列里的回调，直到队列里的内容耗尽，或者执行的回调数量达到最大（maximum number）。当队列已耗尽或者对应的回调数达到最大时，事件循环就会进入下一个阶段，如此循环往复，直至进程结束。

在 poll 阶段，任何一步操作都可能会调度更多操作和新的被处理事件，这些都是由操作系统内核来进行调度的，当选中的事件等待处理时，poll 事件便要进行排队。因此，正在运行的

回调过长会导致 poll 阶段比 timers 阶段运行的时间更长。

注意：虽然 Windows 和 UNIX\Linux 在系统层面的实现是不同的，但这不影响事件循环生命周期的几个阶段，最重要的部分都是一样的。事实上还有其他阶段，但我们关注的仅仅是 Node.js 实际用到的部分，即上面所述的部分。上面讲的 6 个阶段是一次事件循环的过程，不包括 `process.nextTick()` 的内容。

为了更好地理解事件循环，需要掌握以下要点。

- 队列管理
 - `process.nextTick()` 入列
 - `process._tickCallback` 出列
- 如何在合适的时候加入操作
 - 通过 timers（定时器、`setTimeout`、`setInterval` 等）
 - 通过 `setImmediate()`

5.5.3 microtask 和 macrotask

在 JavaScript 的事件循环机制中，还有两个必须了解的概念，即 microtask（微任务）和 macrotask（宏任务）。当前调用栈执行完毕时，会分两种情况进行处理。首先处理 microtask 队列里的事件，然后再从 macrotask 队列中取出一个事件并执行。在同一次事件循环中，microtask 永远在 macrotask 之前执行。

- microtask 举例
 - `process.nextTick`
 - `promise`
- macrotask 举例
 - `setTimeout`
 - `setInterval`

- setImmediate
- I/O

如果理解了 Node.js 的事件循环机制，就会比较容易理解这部分内容。先看一段代码，具体如下。

```
console.log('start')

const interval = setInterval(() => {
  console.log('setInterval')
}, 0)

setTimeout(() => {
  console.log('setTimeout 1')
  Promise.resolve()
    .then(() => {
      console.log('promise 3')
    })
    .then(() => {
      console.log('promise 4')
    })
    .then(() => {
      setTimeout(() => {
        console.log('setTimeout 2')
        Promise.resolve()
          .then(() => {
            console.log('promise 5')
          })
          .then(() => {
            console.log('promise 6')
          })
          .then(() => {
            clearInterval(interval)
          })
        }, 0)
    })
  }, 0)

Promise.resolve()
  .then(() => {
    console.log('promise 1')
  })
  .then(() => {
    console.log('promise 2')
  })
```

在一个事件循环的周期中, 一个任务应该从 `macrotask` 队列开始执行, 当这个 `macrotask` 结束后, 所有的 `microtask` 将在同一个周期中执行。在 `microtask` 执行时还可以加入更多的 `microtask`, 然后一个一个执行, 直到 `microtask` 队列被清空。

这个规范理解起来有点晦涩, 可以参考下面的执行结果来辅助理解具体原理。

```
start
promise 1
promise 2
setInterval
setTimeout 1
promise 3
promise 4
setInterval
setTimeout 2
promise 5
promise 6
```

下面解释一下在事件循环过程中, 每个循环的具体执行过程。

- 循环 1: `setInterval` 被列为任务。`setTimeout 1` 被列为任务。`Promise.resolve 1` 中的两个 `then` 方法被列为 `microtask`。

任务队列里有两个任务: `setInterval` 和 `setTimeout 1`。

- 循环 2: `microtask` 队列清空, `setInterval` 的回调可以执行, 另一个 `setInterval` 被列为任务, 位于 `setTimeout 1` 后面。

任务队列里有两个任务: `setTimeout 1` 和 `setInterval`。

- 循环 3: `microtask` 队列清空, `setTimeout 1` 的回调可以执行, `promise 3` 和 `promise 4` 被列为 `microtask`。`promise 3` 和 `promise 4` 执行, `setTimeout 2` 被列为任务。

任务队列里有两个任务: `setInterval` 和 `setTimeout 2`。

- 循环 4: `microtask` 队列清空, `setInterval` 的回调可以执行, 然后另一个 `setInterval` 被列为任务, 位于 `setTimeout 2` 后面。`setTimeout 2` 的回调执行, `promise 5` 和 `promise 6` 被列为 `microtask`。

任务队列里有两个任务: `setTimeout 2` 和 `setInterval`。

现在 `promise 5` 和 `promise 6` 的回调应该被执行, 并且清空了 `Interval`, 但有的时候不知道为

什么 `setInterval` 还会再执行一遍，结果变成下面这样。

```
...
setTimeout 2
setInterval
promise 5
promise 6
```

将上面的代码放入 Chrome Console 中执行是没有问题的，因此要注意一下不同的 Node.js 版本产生的不同影响。

5.5.4 process.nextTick(callback)

`process.nextTick(callback)` 是用于在事件循环的下一循环中调用回调函数的，与 `setTimeout(fn, 0)` 函数的功能类似，但效率更高。`process.nextTick(callback)` 将一个函数推迟到代码执行的下一个同步方法执行完毕，或异步事件回调函数开始执行时再执行。原理很简单，可以基于 Node.js 的事件循环进行分析，每次循环就是一次 tick，每次 tick 时，Chrome V8 引擎都会从事件队列中取出所有事件依次进行处理，如果遇到 `nextTick` 事件，则将其加入事件队尾，等待下一次 tick 到来时执行。这样产生的结果是，`nextTick` 事件被延迟执行。以下是 `nextTick` 的源码 `lib/internal/process/next_tick.js`。

```
function _tickCallback() {
  do {
    // Node.js 在执行任务时，会一次性把 Eventloop 队列中的所有任务都取出来，依次执行
    while (tickInfo[kIndex] < tickInfo[kLength]) {
      ++tickInfo[kIndex];
      const tock = nextTickQueue.shift();
      const callback = tock.callback;
      const args = tock.args;

      // CHECK (Number.isSafeInteger(tock[async_id_symbol]))
      // CHECK (tock[async_id_symbol] > 0)
      // CHECK (Number.isSafeInteger(tock[trigger_id_symbol]))
      // CHECK (tock[trigger_id_symbol] > 0)

      emitBefore(tock[async_id_symbol], tock[trigger_id_symbol]);

      // 如果全部任务都顺利执行，则删除刚才取出的所有任务，等待下一次执行
      if (async_hook_fields[kDestroy] > 0)
        emitDestroy(tock[async_id_symbol]);

      combinedTickCallback(args, callback);
    }
  } while (tickInfo[kIndex] < tickInfo[kLength]);
}
```

```

    emitAfter(tock[async id symbol]);

    if (kMaxCallbacksPerLoop < tickInfo[kIndex])
        tickDone();
    }
    tickDone();
    _runMicrotasks();
    // 如果中途出错, 则删除已经完成的任务和出错的任务, 等待下一次循环
    emitPendingUnhandledRejections();
} while (tickInfo[kLength] !== 0);
}

function nextTick(callback) {
    if (typeof callback !== 'function')
        throw new errors.TypeError('ERR_INVALID_CALLBACK');

    if (process._exiting)
        return;

    var args;
    switch (arguments.length) {
        case 1: break;
        case 2: args = [arguments[1]]; break;
        case 3: args = [arguments[1], arguments[2]]; break;
        case 4: args = [arguments[1], arguments[2], arguments[3]]; break;
        default:
            args = new Array(arguments.length - 1);
            for (var i = 1; i < arguments.length; i++)
                args[i - 1] = arguments[i];
    }

    const asyncId = ++async_uid_fields[kAsyncUidCntr];
    const triggerAsyncId = initTriggerId();
    const obj = new TickObject(callback, args, asyncId, triggerAsyncId);
    // nextTick 把参数里的任务放到了队列的最后
    nextTickQueue.push(obj);

    ++tickInfo[kLength];

    if (async_hook_fields[kInit] > 0)
        emitInit(asyncId, 'TickObject', triggerAsyncId, obj);
}

```

5.6 本章小结

通过本章的学习，你应该已经对 Node.js 源码有了一定的了解，阅读源码是一个长期过程，只要你准备好编译环境，理解编译步骤和 Node.js 核心流程，剩下的就是不断实践了。

从另一个角度来看，理解 Node.js 源码关键就是明白 Chrome V8 和 libuv 的作用、了解 JavaScript 如何与 C/C++ 通信、掌握模块机制。

本章的知识点比较零碎，涉及的知识面也比较广，大家可以根据自己的实际情况进行学习，看懂多少是多少，即使没有完全掌握，对使用 Node.js 也没有多大的影响。等你对 Node.js 的理解深入到一定程度时，记得返回来重新看一下，相信你一定会会心一笑。

第6章

模块与核心

模块是语言、框架、平台的必要组成部分，Node.js 作为新生平台，采用 CommonJS 规范作为模块标准，即将 CommonJS 作为 Node.js 模块编写的基石。有了模块规范，便需要包管理器 npm，有了更好的 ES6 语法，才需要集成面向未来的 ES 模块。

以上提到的就是本章的主要内容，具体要点如下。

- 介绍 CommonJS 规范，以及它和 Node.js 之间的关系。
- Node.js 模块详解：从 SDK、内置模块、全局对象、C/C++ 扩展等方面展开。
- 未来展望：详细介绍 ES 模块。

6.1 CommonJS 规范

本节主要介绍 CommonJS 规范，把模块化加载历史与 Node.js 实现之间的关系说清楚，通过实例来进一步阐述规范的核心内容，最后通过源码解析来阐述其实现原理。

CommonJS 是一个规范，通过简单的 API 声明服务器的模块，目标是让 JavaScript 可以运行在浏览器之外的所有地方，例如在服务器或本地桌面应用程序上。

6.1.1 简介

2009 年，Node.js 横空出世，人们可以用 JavaScript 来编写服务端的代码。如果说浏览器端的 JavaScript 即便没有模块化也可以勉强接受的话，那么，服务端没有模块化是万万不能接受的。

当时，牛人云集的 CommonJS 社区猛然发力，制定了 Modules/1.0 规范，首次定义了 JavaScript 世界里的模块应该是什么样的。这个项目由摩斯拉工程师 Kevin Dangoor 发起，起初被命名为 ServerJS。2009 年 8 月，这个项目被重新命名为 CommonJS，可以展示更广泛的 API。

CommonJS 规范的创建和审核过程是公开的，该规范是经过深思熟虑后得到的结果，CommonJS 并不属于 TC39 制定的 ECMAScript 标准，但是 TC39 部分成员也参与了该项目。

📁 Modules/1.0 规范

具体来说，Modules/1.0 规范包含以下内容。

- 规定模块的标识应遵循的规则（书写规范）。
- 定义全局函数 `require`，通过传入模块标识来引入其他模块，执行的结果即其他模块暴露出来的 API。
- 如果被 `require` 函数引入的模块中也包含依赖，则依次加载这些依赖。
- 如果引入模块失败，那么 `require` 函数应该抛出异常。
- 模块通过变量 `exports` 来向外暴露 API，`exports` 只能是一个对象，暴露的 API 须作为此对象的属性。

此规范一出，立刻获得了良好的反响，由于其简单直接，因此在 Node.js 中首次成功实现后便立刻被推广开来。Modules/1.0 规范源于服务端，无法直接用于浏览器端，所以社区意识到，要想同时在浏览器环境中实现模块化，需要对规范进行升级。而就在社区讨论下一版规范的时候，CommonJS 社区内部发生了比较大的分歧，分裂出两个不同的派别，具体如下。

- Modules/1.x 派：在现有基础上进行改进，制定了 Modules/Transport 规范，提出了“先通过工具把现有模块转化为复合浏览器上使用的模块，然后再使用”的方案。Browserify 就是这样一个工具，可以把 Node.js 的模块编译成浏览器可用的模块，Webpack、node-libs-browser 是常见的浏览器端使用的 Browserify 模块集合。

目前 Modules/1.x 中的最新版本是 Modules/1.1.1，增加了一些 `require` 属性，以及 `module` 变量来描述模块信息，变动不大。

- Modules/async 派：不属于 CommonJS 的范畴，独立制定了浏览器端的 JS 模块化规范

AMD (Asynchronous Module Definition), 包括后来的 CMD、UMD 等都和 AMD 有着密不可分的关系。代表项目有 RequireJS 和 SeaJS。

其实还应该有个 Modules/2.0 派, 即支持 Modules/Wrappings 规范的群体。此规范指出了模块应该如何“包装”, 由于未见成功案例, 因此不详细讲解了。

👉 CommonJS 规范与 Node.js 的关系

Node.js 借鉴 CommonJS 模块规范实现了一套非常易用的模块系统, npm 对模块规范的完美支持, 也使得 Node.js 应用开发事半功倍。Node.js 能以一种较成熟的姿态展现在开发者面前, 离不开 CommonJS 规范的贡献。在服务器端, CommonJS 能以一种寻常的姿态进入各个公司的项目代码, 也离不开 Node.js 的优异表现。

尽管它们有些不同, 但大家还是习惯说, Node.js 是基于 CommonJS 规范的。先有规范, 后有实现, 这种在实现过程中做过改进的规范与原规范的关系, 用“基于”来概括并不为过, 当然, 这样也更加便于初学者理解。

CommonJS 项目定义了一系列的规范, 可以辅助 JavaScript 应用程序在服务器端进行开发。Node.js 开发人员起初打算完全遵循 CommonJS 规范, 但后来又推翻了起初的设想, 因为设计模块时, CommonJS 非常影响 Node.js 的实现。

Node.js 和 CommonJS 在模块系统中主要通过两个关键字进行交互, 即 `require` 和 `exports`。`require` 是一个用于引入模块的函数, 参数是所需模块的标识。在 Node.js 的实现中, 模块的名字在 `node_modules` 目录下, 如果不在, 就会去查找指定路径。`exports` 是一个特殊的对象, 它的任何输出都将作为一个对外暴露的公共 API。以下为 `require` 和 `exports` 的用法示例

```
// 在 circle.js 中
const PI = Math.PI;

exports.area = (r) => PI * r * r;

exports.circumference = (r) => 2 * PI * r;

// 在某些文件中
const circle = require('./circle.js');
console.log(`The area of a circle of radius 4 is ${circle.area(4)}`);
```

Node.js 和 CommonJS 的区别主要体现在 `module.exports` 对象的具体实现上。

- 在 Node.js 中, `module.exports` 是真正的特殊对象, 是真正的对外暴露出口, 而 `exports`

只是一个变量，是被默认的 `module.exports` 绑定的。

- CommonJS 规范里没有 `module.exports` 对象。在 Node.js 中，它的实际含义是一个完全预先构建的对象，不经过 `module.exports` 是不可能对外暴露的。

当二者同时存在时，以 `module.exports` 为准，代码如下。

```
// 这种写法不生效，exports 里的内容会被 module.exports 替换
exports = (width) => {
  return {
    area: () => width * width
  };
}

// 最终导出的是如下函数
module.exports = (width) => {
  return {
    area: () => width * width
  };
}
```

2013 年 5 月，Node.js 包管理工具 npm 的作者 Isaac Z. Schlueter 发表了一个简短的声明，内容如下，其中表达了 CommonJS 不能很好地符合 Node.js 的应用场景，需要对 CommonJS 进行增强修改等意思。

"Standardizing" before you have implementation is naive, in the most literal sense of showing a lack of experience, wisdom, or judgement. There is no reliable way to know the problems with an API before using it in a variety of different situations, and standardizing increases the commitment to this untested API, when what you ought to be doing is decreasing the commitment (and thus, the cost of fixing it later).

尽管如此，开发者还是习惯称 Node.js 是基于 CommonJS 规范的。个中缘由，读者自己体会吧。

👉 从 Node.js 到浏览器

前面讲过，CommonJS 原来是叫 ServerJS 的，从名字可以看出，它是专攻服务端的，为了统一前后端而改名为 CommonJS。由此可见，其野心是很大的，当然，能够统一前后端写法，当然是最理想不过的。

图 6-1 展示了 Node.js 和 Browser 之间的关系，既有重合也有差异。

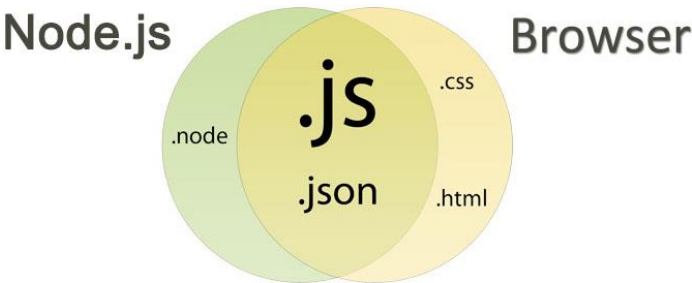


图 6-1

下面看一下前端模块化的演进过程，如表 6-1 所示。

表 6-1

模块规范	加载器	打包器	寿 命
CommonJS	Webmake	Browerify、Webpack、Ykit	Webmake 用得不多，但打包器很常用
AMD	require.js	r.js	require.js 算是第一个比较著名的实现，像著名的 ACE 编辑器就是基于 require.js 编写的，后来衍生出了 Sea.js 等
ES6	system.js	Webpack	在 React、Vue 流行之后慢慢变为主流，是未来的趋势

当前浏览器客户端有两个流行的选择：Browserify 和 Webpack。

Browserify 是一个使用 Node.js 风格 require() 来管理浏览器端代码的构建工具，它可以加载 npm 模块，使前端开发和 Node.js 开发同时使用 CommonJS 规范，降低学习成本，提高开发效率。

Browserify 其实也不是纯粹的 CommonJS 规范，而是基于 CommonJS 规范的 Node.js 实现方式，只不过从服务器端被移植到了浏览器端使用而已。这是一个微创新，但很有意义，在写法上降低了学习成本，统一了服务器端和浏览器端的代码编写方式。在 ES6 模块变成通用解决方案之前，它可能是唯一一个前后端通用的规范。

以下代码为使用 CommonJS 规范编写的前端脚本。

```
var foo = require('./foo.js');
var bar = require('../lib/bar.js');
var gamma = require('gamma');

var elem = document.getElementById('result');
```

```
var x = foo(100) + bar('baz');  
elem.textContent = gamma(x);
```

我们可以通过 `npm i -g browserify` 以全局的方式安装 Browserify。安装成功后，通过 `browserify` 命令进行打包，比如将下面的 `main.js` 及其依赖一起打包，生成最终的 `bundle.js` 文件，将 `bundle.js` 放到浏览器里就可以直接运行了。

```
$ browserify main.js > bundle.js
```

本节主要讲的是编写 Node.js 代码最基础的模块规范：Node.js 是基于 CommonJS 规范的具体实现，完整实现了 Modules/1.0 规范，并在实现过程中扩展了部分功能。我们不仅可以按照 CommonJS 写法编写 Node.js 代码，也可以通过 Browerfy 进行转译，让同样写法的代码在浏览器中运行。

6.1.2 核心技术

Node.js 对模块的定义十分简单，主要分为模块引用、模块定义和模块标识 3 个部分，其中常用的模块处理命令如下。

- `require`：用来引用模块。
- `export`：用来导出模块，包括标识符（identifier）和模块内容（contents）。
 - `module.exports`：对外导出的对象只能有 1 个。
 - `exports.xxx`：对外导出的值可以有多个。

`require` 其实还有按需加载的含义，就像前端常见的 AMD、CMD、UMD 规范等，当多次引用一个模块的时候，该模块只会被加载一次，其他情况下都在缓存中加载，不需要重新加载，这其实就是 Node.js 的模块缓存机制。

👉 什么是模块

模块可以将关联代码封装到一个代码单元中。创建一个模块可以理解为把全部有关联的函数放到一个文件中。下面我们举例说明，首先创建文件 `greetings.js`，它包含如下两个函数。

```
sayHelloInEnglish = function() {  
  return "Hello";  
};  
  
sayHelloInChinese = function() {
```

```
    return "你好";  
};
```

将两个不同功能的 `sayHello` 方法封装到一起, 就可以称为一个模块, 不过此时还不能算 Node.js 模块, 除了定义函数, 还需要对外导出模块。

👉 模块导出

代码封装后被用到其他文件中可以提高实用性, 下面让我们来重构上面的 `greetings.js`, 以达成这个目的。为了更好地理解, 我们将分三步进行讲解。

第一步: 把 `exports` 放在 `greetings.js` 的第一行代码中。

```
var exports = module.exports = {};
```

第二步: 在 `greetings.js` 中指派任何我们想在其他文件中使用的内容, 这里以 `exports` 为例。

```
exports.sayHelloInEnglish = function() {  
    return "HELLO";  
};  
  
exports.sayHelloInChinese = function() {  
    return "你好";  
};
```

第三步: 使用 `module.exports` 实现和步骤二一样的功能。

```
module.exports = {  
    sayHelloInEnglish: function() {  
        return "HELLO";  
    },  
  
    sayHelloInChinese: function() {  
        return "你好";  
    }  
};
```

在上述代码中, 使用 `module.exports` 替换 `exports` 可以得到相同的结果。如果感觉有些混乱, 那么只需要记住 `module.exports` 和 `exports` 引用的是相同的对象即可。

- `exports` 多用于编写对外暴露多个 API 的工具类代码。
- `module.exports` 用于编写对外暴露同一对象 API 的代码。

注意, 核心的是 `module.exports`, `exports` 只是 `module.exports` 的引用而已。

👉 引用模块

`require` 是 CommonJS 规范中 Modules/1.0 里用来引用模块的关键词，在 Node.js 中当然也是一样的用法，具体如下。

```
var require = function(id) {  
  
    // ...  
  
    return module.exports;  
};
```

关于上述代码，要点说明如下。

- `require` 是一个函数，参数是模块标识。
- `require` 函数的返回值是 `module.exports` 对象。
- `require` 函数是模块标识，模块标识有两种。
 - 如果是 Node.js 模块，要发布在 npm 上，安装到 `node_modules` 目录下面。
 - 如果是自定义的本地模块，则是带有相对路径的。

引用 `greetings.js` 公用模块到一个新的文件 `main.js` 中，也分为三步，具体如下。

第一步：定义 `greetings.js` 对外暴露的 API。

```
// greetings.js  
module.exports = {  
    sayHelloInEnglish: function() {  
        return "HELLO";  
    },  
  
    sayHelloInChinese: function() {  
        return "你好";  
    }  
};
```

第二步：在 `main.js` 中引入 `greetings.js` 模块。

```
// main.js  
var greetings = require("./greetings.js");
```

这一步相当于在当前文件里定义了 `greetings` 对象，等价代码如下。


```
var greetings = {  
  sayHelloInEnglish: function() {  
    return "HELLO";  
  },  
  
  sayHelloInChinese: function() {  
    return "你好";  
  }  
};
```

第三步：将 `greetings.js` 中对外暴露的 API 作为 `greetings` 对象的方法。

```
// main.js  
var greetings = require("./greetings.js");  
  
// "Hello"  
greetings.sayHelloInEnglish();  
  
// "你好"  
greetings.sayHelloInChinese();
```

如果上面第一步中的代码不使用 `module.exports`，而使用 `exports`，会有什么区别呢？各位读者可以自行体会一下。

👉 模块间的循环引用

在 iOS 早期使用 ARC 管理内存时，经常会出现由于循环引用导致内存无法释放的问题。当两个不同的对象各有一个强引用指向对方时，循环引用便产生了。在 Node.js 里也一样，模块之间互相引用，也会出现这样的问题，官方文档里称之为 `modules cycles`，译为模块间循环引用。模块间循环引用的示例如图 6-2 所示。

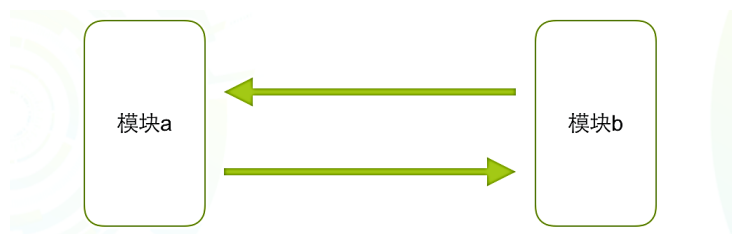


图 6-2

下面看一下 `a` 和 `b` 两个模块的循环引用关系。

`a.js` 的内容如下。

```
console.log('a starting');
exports.done = false;
var b = require('./b.js');
console.log('in a, b.done = %j', b.done);
exports.done = true;
console.log('a done');
```

b.js 的内容如下。

```
console.log('b starting');
exports.done = false;
var a = require('./a.js');
console.log('in b, a.done = %j', a.done);
exports.done = true;
console.log('b done');
```

main.js 的内容如下。

```
console.log('main starting');
var a = require('./a.js');
var b = require('./b.js');
console.log('in main, a.done=%j, b.done=%j', a.done, b.done);
```

当 main.js 引用 a.js 的时候，a.js 引用 b.js，同时，b.js 也想要引用 a.js，这时候就产生了依赖闭环的问题。为了避免无限循环，需要打破这个闭环。

CommonJS Modules/1.0 规范中有说明——在这种情况下，require 返回的对象必须至少包含此外部模块在调用 require 函数之前就已经准备完毕的输出。这句话有些绕，从依赖闭环产生的地方说起，b.js 需要引用 a.js，这里 b.js 作为当前模块，a.js 相对于 b.js 来说就是外部模块，那么 a.js 应该在引用 b.js（即进入当前模块执行环境）之前就返回输出，执行过程如下。

先执行 a.js，代码如下。

```
console.log('a starting');
exports.done = false;
// 只执行到这里，将输出返回调用模块(b.js)，以下被丢弃
var b = require('./b.js');
console.log('in a, b.done = %j', b.done);
exports.done = true;
console.log('a done');
```

然后 b.js 继续执行，以下是执行结果。

```
$ node main.js
main starting
a starting
b starting
in b, a.done = false
```

```
b done
in a, b.done = true
a done
in main, a.done=true, b.done=true
```

注意, 虽然 `main.js` 同时通过 `require` 引用了 `a.js` 和 `b.js`, 但是根据 `Node.js` 的模块缓存策略, 模块只执行一次, 不用担心内存问题。另外, 当模块间出现循环引用时, 一定要注意代码的位置。

👉 入门实例

`helloworld2` 中会包含 `helloworld2.js` 和 `main.js` 两个文件, 用来演示多文件的直接引用关系。

创建 `helloworld2.js`, 代码如下。

```
module.exports = function () {
  console.log('hello world');
}
```

这里使用 `module.exports` 导出一个函数, 里面只打印了 `hello world`, 下面我们来看一下如何在 `main.js` 里调用上面的代码。

创建 `main.js`, 代码如下。

```
var hello = require('./helloworld2');

hello()
```

需要说明的是, 通过 `require` 来引用 `helloworld2` 这个模块时, 一旦引用成功, 那么这个模块对外暴露的方法或变量就可以调用了。

执行 `main.js`, 命令如下。

```
$ node demo/main.js
hello world
```

`helloworld2` 这个模块导出的是一个函数, 既然是函数, 那么就一定可以直接调用。

在 `main.js` 里进行如下修改。

```
require('./helloworld2')();
```

执行下面的代码, 效果是一样的。

```
$ node demo/main2.js
hello world
```

👉 对比模块导出的两种方式

`module.exports` 只暴露了一个函数，如果想暴露更多方法或变量该怎么做呢？

我们来假设一下，如果希望在 `helloworld5` 里同时实现吃饭和打招呼，该怎么做呢？也就是说，模块要提供两个方法：吃饭和打招呼。

创建 `helloworld5.js`，代码如下。

```
function say(person) {
  console.log('i am say hello world to ' + person);
}

function eat(food) {
  console.log('i am eat ' + food);
}

exports.eat = eat;
exports.say = say;
```

创建 `main5.js` 文件，代码如下。

```
var h5 = require('./helloworld5')

h5.eat('兰州拉面');

h5.say('海角');
```

执行 `main5.js`。

```
$ node demo/main5.js
i am eat 兰州拉面
i am say hello world to 海角
```

如果你还记得模块是如何导出的，就会发现，之前用的是 `module.exports`，而现在用的是 `exports.xxx`。那么它们有什么差别呢？

其实 `module.exports` 才是真正的接口，`exports` 只不过是它的一个辅助工具。最终将结果返回给调用方的是 `module.exports`，而不是 `exports`。所有的 `exports` 收集到的属性和方法，最终都赋值给了 `module.exports`。当然，这里有一个前提，即 `module.exports` 本身不具备任何属性和方法。如果 `module.exports` 已经具备了一些属性和方法，那么 `exports` 收集来的信息将被忽略。当二者共存时，会产生问题，我们来看一下。

修改 `exports.js`，具体如下。

```
module.exports = 'Exports IT!';
exports.name = function () {
  console.log('hello world');
}
```

再次引用执行 `e_main.js`。

```
var hello = require('./exports');

console.log(hello.name())
```

发现报错：对象 `hello` 没有 `name` 方法。这是因为 `exports` 模块忽略了 `exports` 收集的 `name` 方法，返回了一个字符串“Exports IT!”。由此可知，模块并不一定非得返回实例化对象。

模块可以是任何合法的 JavaScript 对象：布尔值、日期、JSON、字符串、函数、数组、对象等。如果没有显式地给 `module.exports` 设置任何属性和方法，那么模块返回值就是 `exports` 设置给 `module.exports` 的属性。

下面看一下常见的 `module.exports` 的赋值写法。

```
module.exports = 1
module.exports = NaN
module.exports = 'foo'
module.exports = { foo: 'bar' }
module.exports = ['foo', 'bar']
module.exports = function foo () {}
```

现在你应该明白了，如果你希望模块是一个特定的类型，就用 `module.exports`。如果你希望模块是一个典型的实例化对象，就用 `exports`。给 `module.exports` 添加属性类似于给 `exports` 添加属性，代码如下所示。

```
module.exports.name = function() {
  console.log('My name is Lemmy Kilmister');
};
exports.name = function() {
  console.log('My name is Lemmy Kilmister');
};
```

请注意，这两种结果并不相同。前面已经提到，`module.exports` 是真正的接口，`exports` 是它的辅助工具，所以推荐使用 `exports` 进行导出，除非你希望实例化对象改变成另一个类型。

我们知道，默认情况下 `module.exports` 的返回值是空对象。如果只是添加方法或属性，只要操作 `exports` 就可以了。只有需要覆盖模块返回值的时候才需要用到 `module.exports`。因为 `exports` 是形参，形参只能在作用域内改变，出了作用域就将被还原。所以想覆盖对外导出的返回值时，需要使用 `module.exports`。

除了工具类用 `exports.xxx`，绝大多数情况下我们都用 `module.exports`，只要遵循这个简单的约定即可，最佳写法如下。

```
exports = module.exports = function (opts) {}
```

6.2 Node.js 模块

上一节主要介绍了 CommonJS 规范，需要掌握在 Node.js 里如何使用模块，核心是记住 `require`、`module.exports`、`exports` 这 3 个关键字的用法。

如果能够理解 `exports.xxx` 和 `module.exports` 的区别，就会理解为什么说 CommonJS 在 Node.js 里实现是至关重要的。可以说，掌握了 CommonJS 规范，就相当于掌握了 Node.js 最核心、最基础的模块。

在代码执行前，Node.js 执行了什么操作？这可能是每个 Node.js 初学者都有的疑问。其实，每个模块在被引用的时候都被头尾包装，代码执行后将 `exports` 对象返回给引用方。你可以看到，`exports` 是通过参数传递进来的。如果 `exports` 的引用被修改，那么返回的将是原来的对象，所以这个时候就要用 `module.exports` 来代替，比如最简单的模块如下。

```
module.exports = function () {  
  console.log('hello world');  
}
```

在模块代码被执行之前，Node.js 会把它包裹到函数里，类似下面这样。

```
(function (exports, require, module, __filename, __dirname) {  
  
  console.log('hello world');  
});
```

6.2.1 从源码分析实现原理

在模块代码被执行之前，Node.js 会把我们写的代码包裹到函数里，那么我们就顺着这个思路来探究一下 Node.js 代码的执行原理吧。

首先，在 Node.js 源码里搜索与模块相关的代码，如下。

```
$ ack "(exports, require, module,  filename,  dirname)"  
doc/api/modules.md  
437:(function (exports, require, module, __filename, __dirname) {
```

```
lib/internal/bootstrap node.js
420:  '(function (exports, require, module, __filename, __dirname) { ',
```

在 lib/internal/bootstrap_node.js 里，我们发现了如下实现。

```
NativeModule.wrapper = [
  '(function (exports, require, module, __filename, __dirname) { ',
  '\n});'
];
```

然后看一下如何调用 wrapper 方法的 wrap 方法，具体如下。

```
NativeModule.wrap = function(script) {
  return NativeModule.wrapper[0] + script + NativeModule.wrapper[1];
};
```

这样一来，我们就非常容易理解 Node.js 是如何将代码包裹到函数里的了，方法如下。

```
(function (exports, require, module, __filename, __dirname) {
//script 读取文件里的代码。
})
```

下面看一下具体的编译代码。

```
NativeModule.prototype.compile = function() {
  var source = NativeModule.getSource(this.id);
  source = NativeModule.wrap(source);

  this.loading = true;

  try {
    var fn = runInThisContext(source, {
      filename: this.filename,
      lineOffset: 0,
      displayErrors: true
    });
    fn(this.exports, NativeModule.require, this, this.filename);

    this.loaded = true;
  } finally {
    this.loading = false;
  }
};
```

针对以上代码，说明如下。

- `source = NativeModule.wrap(source);`: 调用的就是上面的 wrap 方法。
- 具体的执行代码是 `runInThisContext` 方法。

以上是我们对 Node.js 代码在执行阶段如何被包裹进行的源码分析。执行源码，Node.js 将完成以下操作。

- 保证顶层的变量（通过 `var`、`const` 或 `let` 定义的变量）只在模块内起作用。
- 帮助提供一些全局查找的变量，实际上是指定给模块的变量，具体如下。
 - `module.exports` 对象可用于从模块上对外暴露值。
 - 为了方便，Node.js 加载源码时注入了 `filename` 和 `dirname`，即模块的文件名和路径，这样一来我们便可以在任意文件里使用它们。

6.2.2 从 Node.js 代码执行开始

一般我们执行 Node.js 代码时，命令通常是这样的：`node xxx.js`。

在 `lib/internal/bootstrap_node.js` 里可以找到模块执行的核心代码逻辑，具体如下。

```
// 保证 process.argv[1] 是完整路径
const path = NativeModule.require('path');
process.argv[1] = path.resolve(process.argv[1]);

const Module = NativeModule.require('module');

// 检查用户输入的 -c 或 --check 参数
if (process. syntax check only != null) {
  const fs = NativeModule.require('fs');
  // 读取源码
  const filename = Module.resolveFilename(process.argv[1]);
  var source = fs.readFileSync(filename, 'utf-8');
  checkScriptSyntax(source, filename);
  process.exit(0);
}

preloadModules();
Module.runMain();
```

关于以上代码，要点如下。

- 通过 `NativeModule.require` 引入 `lib/module.js`，然后赋值给 `Module`。
- 预加载模块 `preloadModules()`。
- 执行模块 `Module.runMain()`。

以上代码中的核心是 `Module.runMain()`，在 `lib/module.js` 里实现如下。

```
// 启动主模块
Module.runMain = function() {
  // 加载主模块
  Module.load(process.argv[1], null, true);
  // 先执行 nextTickQueue 中的回调函数
  process._tickCallback();
};
```

上面代码的核心是 `Module._load` 方法，具体如下。

```
Module.load = function(request, parent, isMain) {
  if (parent) {
    debug('Module._load REQUEST %s parent: %s', request, parent.id);
  }

  var filename = Module._resolveFilename(request, parent, isMain);

  var cachedModule = Module._cache[filename];
  if (cachedModule) {
    updateChildren(parent, cachedModule, true);
    return cachedModule.exports;
  }

  if (NativeModule.nonInternalExists(filename)) {
    debug('load native module %s', request);
    return NativeModule.require(filename);
  }

  var module = new Module(filename, parent);

  if (isMain) {
    process.mainModule = module;
    module.id = '.';
  }

  Module._cache[filename] = module;

  tryModuleLoad(module, filename);

  return module.exports;
};
```

以上代码的要点在于对缓存和模块类型的处理，说明如下。

- 如果模块已经存在于缓存中，则返回 `cachedModule.exports` 对象。

- 如果模块是 `native` 模块，则调用 `NativeModule.require()`，参数是文件名，最后返回结果。
- 其他情况下会为该文件创建一个新的模块，并将其保存到缓存中。随后，加载文件内容，然后返回给 `module.exports` 对象。

对于我们自己编写的代码，一般就是最后这种情况，核心是 `tryModuleLoad` 部分，示例如下。

```
function tryModuleLoad(module, filename) {
  var threw = true;
  try {
    module.load(filename);
    threw = false;
  } finally {
    if (threw) {
      delete Module.cache[filename];
    }
  }
}
```

以上示例中的核心是 `module.load` 部分，举例如下。

```
Module.prototype.load = function(filename) {
  debug('load %j for module %j', filename, this.id);

  assert(!this.loaded);
  this.filename = filename;
  this.paths = Module.nodeModulePaths(path.dirname(filename));

  var extension = path.extname(filename) || '.js';
  if (!Module.extensions[extension]) extension = '.js';
  Module.extensions[extension](this, filename);
  this.loaded = true;
};
```

这里的文件后缀有 3 种：`.js`、`.json`、`.node`，默认后缀是 `.js`，下面分别对 3 种文件后缀进行简单介绍并给出示例代码。

- `.js` 是通过 `module._compile` 来进行编译处理的，通过 `fs` 模块同步读取文件后编译执行。
- `.json` 通过 `JSON.parse` 将结果直接返回 `module.exports`，通过 `fs` 模块同步读取文件后编译执行。
- `.node` 是用 C/C++ 编写的扩展文件，通过 `process.dlopen` 方法加载最后编译生成的文件

并返回结果。

```
// .js 原生扩展
Module._extensions['.js'] = function(module, filename) {
  var content = fs.readFileSync(filename, 'utf8');
  module._compile(internalModule.stripBOM(content), filename);
};

// .json 原生扩展
Module._extensions['.json'] = function(module, filename) {
  var content = fs.readFileSync(filename, 'utf8');
  try {
    module.exports = JSON.parse(internalModule.stripBOM(content));
  } catch (err) {
    err.message = filename + ': ' + err.message;
    throw err;
  }
};

// .node 原生扩展
Module._extensions['.node'] = function(module, filename) {
  return process.dlopen(module, path._makeLong(filename));
};
```

其他文件后缀会默认以.js 的方式进行处理。为了验证上面的结论，我们创建 c.js 文件，直接打印 require，代码如下。

```
console.log(require)
```

在终端里执行 c.js 文件，结果如下。

```
$ node c
{ [Function: require]
  resolve: [Function: resolve],
  main:
    Module {
      id: '.',
      exports: {},
      parent: null,
      filename: '/Users/sang/workspace/github/node/c.js',
      loaded: false,
      children: [],
      paths:
        [ '/Users/sang/workspace/github/node/node_modules',
          '/Users/sang/workspace/github/node_modules',
          '/Users/sang/workspace/node_modules',
          '/Users/sang/node_modules',
```

```

    '/Users/node modules',
    '/node_modules' ] },
extensions: { '.js': [Function], '.json': [Function], '.node': [Function] },
cache:
{ '/Users/sang/workspace/github/node/c.js':
  Module {
    id: '.',
    exports: {},
    parent: null,
    filename: '/Users/sang/workspace/github/node/c.js',
    loaded: false,
    children: [],
    paths: [Object] } } }

```

以上代码中的 `extensions` 对应就是上面的加载方式。如果想自定义后缀处理方式，可以自己扩展 `require.extensions` 进行扩展，不过官方不推荐这样做，Node.js 的启动速度本就不快，这样做会让 Node.js 的启动过程变得更慢。

6.2.3 深入理解模块

在 `lib/module.js` 里，我们可以通过以下代码查看模块的定义。

```

function Module(id, parent) {
  this.id = id;
  this.exports = {};
  this.parent = parent;
  updateChildren(parent, this, false);
  this.filename = null;
  this.loaded = false;
  this.children = [];
}
module.exports = Module;

```

其经典用法 `lib/internal/bootstrap_node.js` 里的 `eval` 实现如下。

```

function evalScript(name) {
  const Module = NativeModule.require('module');
  const path = NativeModule.require('path');
  const cwd = tryGetCwd(path);

  const module = new Module(name);
  module.filename = path.join(cwd, name);
  module.paths = Module.prototype.paths(cwd);
  const body = process._eval;
  const script = `global.__filename = ${JSON.stringify(name)};\n` +
    'global.exports = exports;\n' +
    'global.module = module;\n' +

```

```

        'global. dirname =  dirname;\n' +
        'global.require = require;\n' +
        'return require("vm").runInThisContext(' +
        `${JSON.stringify(body)}, { filename: ` +
        `${JSON.stringify(name)}, displayErrors: true });\n`;
const result = module._compile(script, `${name}-wrapper`);
if (process. print eval) console.log(result);

process._tickCallback();
}

```

evalScript 没有依赖模块，因此只传了一个 name 参数。下面来看一个实例，首先创建 a.js。

```
console.log(module)
```

然后执行 a.js。

```

$ node a
Module {
  id: '.',
  exports: {},
  parent: null,
  filename: '/Users/sang/workspace/github/book-source/a.js',
  loaded: false,
  children: [],
  paths:
    [ '/Users/sang/workspace/github/book-source/node_modules',
      '/Users/sang/workspace/github/node_modules',
      '/Users/sang/workspace/node_modules',
      '/Users/sang/node_modules',
      '/Users/node_modules',
      '/node_modules' ] }

```

创建 b.js，在 b.js 里通过 require 来引用模块 a，这样模块 b 就变成了模块 a 的父（parent）模块。

```
require('./a')
```

执行 b.js，结果如下。

```

$ node b
Module {
  id: '/Users/sang/workspace/github/book-source/a.js',
  exports: {},
  parent:
    Module {
      id: '.',
      exports: {},
      parent: null,
      filename: '/Users/sang/workspace/github/book-source/b.js',

```

```

loaded: false,
children: [ [Circular] ],
paths:
  [ '/Users/sang/workspace/github/book-source/node_modules',
    '/Users/sang/workspace/github/node_modules',
    '/Users/sang/workspace/node_modules',
    '/Users/sang/node_modules',
    '/Users/node_modules',
    '/node_modules' ] },
filename: '/Users/sang/workspace/github/book-source/a.js',
loaded: false,
children: [],
paths:
  [ '/Users/sang/workspace/github/book-source/node_modules',
    '/Users/sang/workspace/github/node_modules',
    '/Users/sang/workspace/node_modules',
    '/Users/sang/node_modules',
    '/Users/node_modules',
    '/node_modules' ] }

```

通过以上实例，相信大家应该一看就明白模块里具体有什么了。

👉 理解模块引用

Node.js 中的模块有两种类型：核心模块和文件模块，核心模块直接使用名称获取，比如最常用的 `http` 模块，获取方法如下。

```
const http = require('http');
```

`require` 方法接受以下几种类型的参数传递。

- `http`、`fs`、`path` 等 Node.js 内置模块。
- `./mod` 或 `../mod`：相对路径的文件模块。
- `/path/module/mod`：绝对路径的文件模块。
- `mod`：非原生模块的文件模块。

👉 require 实现

`require` 实现代码位于 `lib/internal/module.js` 里，具体如下。

```
function makeRequireFunction(mod) {
  const Module = mod.constructor;
```

```
function require(path) {
  try {
    exports.requireDepth += 1;
    return mod.require(path);
  } finally {
    exports.requireDepth -= 1;
  }
}

function resolve(request) {
  return Module._resolveFilename(request, mod);
}

require.resolve = resolve;

require.main = process.mainModule;

//支持外部扩展类型
require.extensions = Module.extensions;

require.cache = Module._cache;

return require;
}
```

makeRequireFunction 是在 lib/module.js 里的 compile 方法中调用的, 代码如下。

```
Module.prototype._compile = function(content, filename) {
  ...
  var require = internalModule.makeRequireFunction(this);
  ...
};
```

以上代码中的的核心部分如下。

```
function require(path) {
  try {
    exports.requireDepth += 1;
    return mod.require(path);
  } finally {
    exports.requireDepth -= 1;
  }
}
```

require 其实就是 mod.require 的别名, 而 mod 其实是 NativeModule 的实例。所以, require 是 NativeModule.require 的别名。

👉 搜索模块加载的方式

搜索模块加载的方式有如下几种。

1. 直接使用名字加载

核心模块优先级最高，在有命名冲突的时候要首先加载核心模块。

文件模块只能按照路径加载，可以省略默认的.js 拓展名，不是.js 后缀的情况下需要显式声明。加载路径分为绝对路径和相对路径。

2. 查找 node_modules 目录来加载

我们知道在调用 `npm install <name>` 命令的时候，会在当前目录下创建 `node_module` 目录（如果不存在）来安装模块，当 `require` 遇到一个既不是核心模块又不是以路径形式表示的模块名称时，会试图在当前目录下的 `node_modules` 目录中来查找是否包含这样一个模块。如果没有找到，则会在当前目录的上一层 `node_modules` 目录中继续查找，反复执行这一过程，直到找到根目录为止。

3. 使用全局安装的模块来加载

如果模块依赖 `npm`，你肯定安装过不少全局模块，那为什么不直接使用这些全局模块呢？使用全局安装模块的途径有 3 种，具体如下。

第 1 种：设置 `NODE_PATH` 环境变量。

在 Linux 中是 `export NODE_PATH = /usr/lib/node_modules`，具体取决于全局模块实际的安装位置。

第 2 种：通过命令行 `node--global`，代码如下

```
npm install --global express
```

第 3 种：引用绝对路径，代码如下。

```
var pg = require("/usr/local/lib/node_modules/pg");
```

当然也有 `requireg` 这样的模块。理论上讲，我们使用全局安装模块的几率非常小，这里把它列出来，主要是为了让大家更好地理解 `npm` 及 `node_modules` 加载机制，其实 `require` 加载规则在 `Node.js` 文档里有更详细的说明，如下所示。


```

require(X) from module at path Y
1. If X is a core module,
  a. return the core module
  b. STOP
2. If X begins with './' or '/' or '../'
  a. LOAD_AS_FILE(Y + X)
  b. LOAD_AS_DIRECTORY(Y + X)
3. LOAD_NODE_MODULES(X, dirname(Y))
4. THROW "not found"

LOAD_AS_FILE(X)
1. If X is a file, load X as JavaScript text. STOP
2. If X.js is a file, load X.js as JavaScript text. STOP
3. If X.json is a file, parse X.json to a JavaScript Object. STOP
4. If X.node is a file, load X.node as binary addon. STOP

LOAD_AS_DIRECTORY(X)
1. If X/package.json is a file,
  a. Parse X/package.json, and look for "main" field.
  b. let M = X + (json main field)
  c. LOAD AS FILE(M)
2. If X/index.js is a file, load X/index.js as JavaScript text. STOP
3. If X/index.json is a file, parse X/index.json to a JavaScript object. STOP
4. If X/index.node is a file, load X/index.node as binary addon. STOP

LOAD NODE MODULES(X, START)
1. let DIRS=NODE_MODULES_PATHS(START)
2. for each DIR in DIRS:
  a. LOAD AS FILE(DIR/X)
  b. LOAD_AS_DIRECTORY(DIR/X)

NODE MODULES PATHS(START)
1. let PARTS = path split(START)
2. let I = count of PARTS - 1
3. let DIRS = []
4. while I >= 0,
  a. if PARTS[I] = "node_modules" CONTINUE
  c. DIR = path join(PARTS[0 .. I] + "node modules")
  b. DIRS = DIRS + DIR
  c. let I = I - 1
5. return DIRS

```

👉 内建库 (builtinLibs)

通过 `ack` 命令搜索内建库关键字 `addBuiltinLibsToObject`, 结果如下。

```
$ ack addBuiltinLibsToObject
lib/internal/bootstrap_node.js
138:     internalModule.addBuiltinLibsToObject(global);

lib/internal/module.js
90:function addBuiltinLibsToObject(object) {
130:  addBuiltinLibsToObject,

lib/repl.js
652:  internalModule.addBuiltinLibsToObject(context);
```

结果显示有如下 3 处相关代码。

第 1 处：位于 lib/internal/bootstrap_node.js 里，具体如下。

```
if (process. eval != null && !process. forceRepl) {
  preloadModules();

  const internalModule = NativeModule.require('internal/module');
  internalModule.addBuiltinLibsToObject(global);
  evalScript(['eval']);
}
```

这个其实就是终端里的 node -e 部分的代码实现，上面的代码实现了 3 个功能，分别如下。

- 预加载模块 preloadModules()。
- 通过 NativeModule.require() 引入 'internal/module' 代码。
- 将内建的库绑定到全局对象上。

下面以打印 module 对象为例来看一下 node -e 的用法

```
$ node -e "console.log(module)"
Module {
  id: '[eval]',
  exports: {},
  parent: undefined,
  filename: '/Users/sang/workspace/github/node/[eval]',
  loaded: false,
  children: [],
  paths:
    [ '/Users/sang/workspace/github/node/node_modules',
      '/Users/sang/workspace/github/node_modules',
      '/Users/sang/workspace/node_modules',
      '/Users/sang/node_modules',
      '/Users/node_modules',
      '/node_modules' ] }
```

如果各位有兴趣, 也可以自己在终端里键入 `node -e "console.log(global)"` 查看全局对象里到底有什么内容。

第 2 处: 位于 `lib/internal/module.js` 文件里, 是最核心的部分, 代码如下。

```
const builtinLibs = [
  'assert', 'async_hooks', 'buffer', 'child_process', 'cluster', 'crypto',
  'dgram', 'dns', 'domain', 'events', 'fs', 'http', 'https', 'net',
  'os', 'path', 'punycode', 'querystring', 'readline', 'repl', 'stream',
  'string_decoder', 'tls', 'tty', 'url', 'util', 'v8', 'vm', 'zlib'
];

const { exposeHTTP2 } = process.binding('config');
if (exposeHTTP2)
  builtinLibs.push('http2');

function addBuiltinLibsToObject(object) {
  // Make built-in modules available directly (loaded lazily).
  builtinLibs.forEach((name) => {

    const setReal = (val) => {
      delete object[name];
      object[name] = val;
    };

    Object.defineProperty(object, name, {
      get: () => {
        const lib = require(name);

        delete object[name];
        Object.defineProperty(object, name, {
          get: () => lib,
          set: setReal,
          configurable: true,
          enumerable: false
        });

        return lib;
      },
      set: setReal,
      configurable: true,
      enumerable: false
    });
  });
}
```

以上代码中有几个要点, 说明如下。

- 声明了内置库 `builtinLibs` 变量，里面都是 Node.js SDK 里的模块。
- `addBuiltinLibsToObject` 函数其实就是遍历 `builtinLibs` 的，通过 `Object.defineProperty()` 进行绑定。
- 如果没对外暴露 `http2`，它是不出现在内置库里的。

第 3 处：位于 `lib/repl.js` 文件里，主要和 CLI 部分相关。

REPL (Read-Eval-Print Loop) 一般翻译成“交互式解释器”或“交互式编程环境”，不过我觉得不用翻译，直接称为 REPL 即可。下面是对 REPL 的英文解释。

A Read-Eval-Print-Loop (REPL) is available both as a standalone program and easily includable in other programs. REPL provides a way to interactively run JavaScript and see the results. It can be used for debugging, testing, or just trying things out.

基本上，脚本语言都支持 REPL，在 Node.js 里，REPL 为运行 JavaScript 脚本与查看运行结果提供了一种交互方式，通常用于调试、测试以及验证某种想法。下面来看一段简单的示例代码。

```
// repl_test.js
var repl = require("repl"),
    msg = "message";

repl.start("> ").context.m = msg;
```

然后，在终端里执行 `repl_test.js`，如下。

```
$ node repl test.js
> m
'message'
>
```

其实我们之前调试用的 API 在命令行里都是有效的，比如 `global`、`module`、`require` 等。这些全局的对象和 SDK 都是通过 `addBuiltinLibsToObject` 注入的。

`repl` 和 `node -e` 在功能上是类似的，唯一不同的是，`node -e` 只能在一行语句里执行。利用命令行和 `repl` 开发一些交互式的生成器是非常好的主意。如果用过 Rails，一定很喜欢 Rails Console，它可以直接调用 AR (ActiveRecord) 里的各种方法，非常简单地测试对 DB 进行的各种操作。那么作为一个 Node.js 开发者，是否也可以实现如下功能呢？请各位读者认真思考。

- 用 `.list` 命令打印出所有的可用实体。
- 执行 `User.find({},function(err,doc){console.log(doc)})` 可以查出具体内容。

👉 模块缓存机制

要解决模块代码更新的问题，我们就需要阅读 Node.js 模块管理器的实现机制。通过简单阅读代码，我们可以发现核心代码是 `Module._load` 部分，下面我们给出稍微精简过的代码，具体如下。

```
Module._load = function(request, parent, isMain) {
  var filename = Module._resolveFilename(request, parent);

  var cachedModule = Module._cache[filename];
  if (cachedModule) {
    return cachedModule.exports;
  }

  var module = new Module(filename, parent);
  Module._cache[filename] = module;
  module.load(filename);

  return module.exports;
};

require.cache = Module._cache;
```

通过以上代码可以发现，其中的核心部分是 `Module._cache`，只要清除了这个模块缓存，下一次执行 `require` 的时候，模块管理器就会重新加载最新的代码了，如下。

```
console.log('模块 modA 开始加载...')
exports = function() {
  console.log('Hi')
}
console.log('模块 modA 加载完毕')
```

创建测试文件 `init.js`，并编写测试代码，如下。

```
var mod1 = require('./modA')
var mod2 = require('./modA')
console.log(mod1 === mod2)
```

在命令行执行 `init.js` 代码，结果如下。

```
$ node init.js
模块 modA 开始加载...
```

```
模块 modA 加载完毕  
true
```

从上面的执行结果可以看到，虽然执行 `require` 两次，但 `modA.js` 仍然只执行了一次。`mod1` 和 `mod2` 是相同的，即两个引用都指向同一个模块对象。

👉 热更新：动态 `require`

理解了 `require` 的缓存机制，如果想热更新代码，就必须要在文件变动的时候清理或更新已有缓存。下面是 `clean_cache.js` 的内容。

```
function cleanCache (module) {  
  var path = require.resolve(module);  
  require.cache[path] = null;  
}
```

执行以上代码可以清除缓存，但也非常容易带来更多的内存问题，比如文件描述符没有手动关闭、数据库连接没有关闭等，类似的问题几乎是很难解的。

```
cleanCache('./clean_cache.js');  
var code = require('./clean_cache.js');  
console.log(code);
```

上面代码第二行中的 `require` 确实会重新加载代码，但这和 Java 里的 OSGi（Java 动态化模块化规范）差距较大，已实现的开源模块有 GitHub 上的 `yyrdl`、`dload` 等，只能说这是火中取栗的做法，非常危险。

和热更新相比，我更认可配置中心的做法，配置根据配置中心内容来自动同步，当代码变动时，配合集群重载当前系统，这比热更新更“靠谱”。

6.2.4 全局对象

模块有两种写法：第一种写法是基于 `CommonJS` 规范编写的，前面已经讲过；第二种写法是全局对象写法，全局对象很容易理解，即无须引用就可以直接使用的对象。本节将主要介绍第二种全局对象写法。另外，需要注意全局对象和 `global` 关键字之间的区别。

👉 内置对象

内置全局对象指的是在所有 `Node.js` 模块中无须安装就可以使用的模块，具体如表 6-2 所示。

表 6-2

编 号	对 象	描 述
1	Buffer	数据类型
2	__dirname	当前文件目录
3	__filename	当前文件名称
4	console	控制台输出模块
5	exports	CommonJS 关键字
6	global	共享的全局对象
7	module	CommonJS 关键字
8	process	当前 Node.js 进程对象
9	require()、require.cache、require.extensions、require.resolve()	CommonJS 关键字
10	setImmediate(callback[, ...args])	Event Loop 相关 API
11	setInterval(callback, delay[, ...args])	Event Loop 相关 API
12	setTimeout(callback, delay[, ...args])	Event Loop 相关 API
13	clearImmediate(immediateObject)	Event Loop 相关 API
14	clearInterval(intervalObject)	Event Loop 相关 API
15	clearTimeout(timeoutObject)	Event Loop 相关 API

内置对象可以分为以下几类。

第一类：为模块包装而提供的全局对象。

Node.js 模块源码是包装在下面的结构里的，其中 exports、require、module、__filename、__dirname 都是为模块包装而提供的全局对象，示例代码如下。

```
(function (exports, require, module, __filename, __dirname) {  
  // 开发者编写的业务代码  
})
```

第二类：内置 process 对象。

在前面的章节中，我们针对 process 模块进行了详细介绍。作为核心模块，它可以对当前 Node.js 的各种信息进行绑定，作为全局模块，使用它是一个极其明智的选择，Java 8 也借鉴了这种实现思想。

第三类：控制台 Console 模块。

比如我们常用的 console.log()，在 JavaScript 语言里有实现，在 Node.js 中又单独实现了一

次，原因在于，Node.js 需要在终端输出，这和在浏览器里的行为是不一样的。在 `lib/console.js` 文件里有 `console.log` 的具体实现，如下。

```
Console.prototype.log = function log(...args) {
  write(this._ignoreErrors,
    this._stdout,
    `${util.format.apply(null, args)}\n`,
    this._stdoutErrorHandler);
};
```

Console 模块在 `lib/internal/bootstrap_node.js` 里被绑定为全局对象，如下。

```
if (!process._noBrowserGlobals) {
  setupGlobalTimeouts();
  setupGlobalConsole();
}
```

第四类：Event Loop 相关 API 的实现。

这一类主要是指 `setImmediate`、`setInterval`、`setTimeout` 和对应的 `clear` 方法的实现。其实大家也可以想想，如果想对 Event Loop 的队列进行操作，用全局模块来实现是非常方便的。

除了上述几种类型，还有特定类型 `Buffer` 和全局对象，下面分别进行讲解。

👉 Buffer

Java NIO 中的 `Buffer` 用于和 NIO 通道进行交互，其核心是利用缓存机制将内存块包装成 NIO `Buffer` 对象进行网络传输，并提供访问该内存块的一组方法。

Node.js 世界也面临着一样的问题。虽然 JavaScript 能很好地处理 Unicode 编码的字符串，但对于二进制或非 Unicode 编码的数据，处理起来就显得无能为力了，所以 Node.js 在 SDK 里内置了 `Buffer` 类，可以像其他程序语言一样，处理各种类型的数据。

`Buffer` 代表一个缓冲区，用于存储二进制数据，俗称字节流，是 I/O 传输时常用的处理方式。相比于字符串，`Buffer` 可以免去编码和解码的过程，节省 CPU 成本，因此在使用 Node.js 进行服务端开发时，HTTP、TCP、UDP、IO、数据库、处理图片、表文件上传等操作，都会用到 `Buffer`。另外，`Buffer` 其实也是 `stream` 的基础，可见其重要程度。

`Buffer` 的应用场景有以下几种。

- 在使用 `net` 或 `http` 模块来接收网络数据时，可用 `Buffer` 作为数据结构进行传输，即 `data` 事件的参数。

- 用于大文件的读取和写入。以前 fs 读取的内容是 string，后来都改用 Buffer，在大文件读取上，性能和内存有明显优势。
- 用于字符转码、进制转换。Unicode 编码虽然能满足绝大部分场景，但有时候还是不够的，由于 Node.js 内置的转换编码并不支持 GBK，因此如果要处理编码为 GBK 的文档，就需要 iconv 和 iconv-lite 来补充一部分，string_decoder 模块提供了一个 API，用于把 Buffer 对象解码成字符串，但会保留编码过的多字节 UTF-8 与 UTF-16 字符。
- 用作数据结构，处理二进制数据，也可以处理字符编码。

初始化 Buffer 有很多方法，具体如下。

- new Buffer()（遗留 API，不推荐）
- Buffer.from()
- Buffer.alloc()
- Buffer.allocUnsafe()
- Buffer.allocUnsafeSlow()

从命名上来看，Buffer.allocUnsafe()和 Buffer.allocUnsafeSlow()都是不安全的，有泄漏内存中敏感信息的危险，所以用得比较多的是 from 和 alloc 方法。

```
let buffer = Buffer.from('Hello World!');
console.log(buffer);
```

执行以上代码，输出结果如下。

```
<Buffer 48 65 6c 6c 6f 20 57 6f 72 6c 64 21>
```

我们可以看到，这些是十六进制的字节编码，每个元素都是 0~255 之间的整数。

- H 对应的 ASCII 码是 72，十进制转为十六进制后的结果是 48。
- e 对应的 ASCII 码是 101，十进制转为十六进制后的结果是 65。
- l 对应的 ASCII 码是 108，十进制转为十六进制后的结果是 6C。

一个 Buffer 类似于一个整数数组，但它对应于 Chrome V8 堆内存之外的一块原始内存。Buffer.from(string[,encoding])方法接收两个参数，第二个参数是编码方法，如果不传递这个参数，将默认使用 UTF-8 编码。Buffer 和 JavaScript 字符串对象之间的转换需要显式地调用编码方法

来完成，目前支持 ASCII、UTF-8、UTF-16LE (UCS2)、BASE64、Latin1 (Binary)、Hex 共 6 种编码方式。

下面给出典型的文件读取示例。

```
const fs = require('fs')

var a = fs.readFileSync('./a.js')
console.log(a)
console.log(typeof a)
console.log(a instanceof Buffer)
```

结果返回的 **a** 是 **Buffer** 类型，如果是文本类型，则需要手动调用 **toString()** 方法。

```
$ node readbuffer.js
<Buffer 0a 65 78 70 6f 72 74 73 2e 6e 61 6d 65 20 3d 20 27 69 35 74 69 6e 67 27 0a 63 6f 6
e 73 6f 6c 65 2e 6c 6f 67 28 6d 6f 64 75 6c 65 29 0a>
object
true
```

读取大文件，示例如下。

```
var fs = require('fs');
var rs = fs.createReadStream('chinese.md', { highWaterMark: 5 });
var dataArr = [], len = 0, data;
rs.on("data", function (chunk) {
    dataArr.push(chunk);
    len += chunk.length;
});
rs.on("end", function () {
    data = Buffer.concat( dataArr, len ).toString();
    console.log(data);
});
```

Buffer 与字符串间的转换会产生极小的性能损耗，在文件读取时候，**highWaterMark** 设置也会对性能产生重要影响。**highWaterMark** 是缓存区能够容纳的最大字节数，默认为 16KB。同时，**highWaterMark** 的大小设置对 **Buffer** 内存的分配和使用有一定影响，如果 **highWaterMark** 设置过小，可能导致系统调用次数过多。如果此处不设置 **{ highWaterMark: 5 }**，则默认的 **stream** 中切割 **data** 块的单位是 8KB。

看一下内存分配方法的示例代码。

```
function allocate(size) {
    if (size <= 0) {
        return new FastBuffer();
    }
    if (size < (Buffer.poolSize >>> 1)) {
```

```
if (size > (poolSize - poolOffset))
  createPool();
var b = new FastBuffer(allocPool, poolOffset, size);
poolOffset += size;
alignPool();
return b;
} else {
  return createUnsafeBuffer(size);
}
```

总体来说, 内存的分配策略遵循以下原则: 大块内存直接分配, 小块内存通过内存池来分配。定义内存池大小的方法为 `Buffer.poolSize`, 内存池大小通常为 8KB, 在 `lib/buffer.js` 里命令如下。

```
Buffer.poolSize = 8 * 1024;
```

可以看到, 初次加载该模块时就执行了一次 `createPool()`, 即创建了一个 8KB 大小的内存池。之后每次在创建新的 `Buffer` 时, 如果小于 4KB (`size < (Buffer.poolSize >>> 1)`), 则需要通过内存池来分配内存, 否则直接通过 `createBuffer(size)` 方法来分配。

在通过内存池来分配内存的时候, 会先检查内存池的可用容量是否满足所需大小, 如果不满足, 则会创建一个新的内存池, 相关代码片段如下。

```
if (size > (poolSize - poolOffset))
  createPool();
```

一个特例是, 当 `size` 为 0 的时候, 会直接通过 `createPool(size)` 来分配内存, 这样可以避免后面对 `size` 以及内存池可用容量进行判断, 从而提高分配效率。

另外需要注意 `SlowBuffer` 和 `FastBuffer` 的区别, `SlowBuffer` 并不是慢速的 `Buffer`, 只是当内存池大小大于 8KB 或者不够用的情况下可用于申请空间, 大于 8KB 的以及直接用 `SlowBuffer` 方法新获得的空间将是独享的, 内存占用小于 8KB 的 `Buffer` 将会共享内存空间。目前 `SlowBuffer` 基本作为内部 API 使用, 对外建议使用 `Buffer.allocUnsafeSlow(size)` 方法。当然, `Buffer` 的 API 与 Node.js 版本的兼容问题也是一件极其令人头疼的事。

Node.js 中有一个字符串解码类, 即 `string_decoder`, 可以通过 `require('string_decoder')` 来引用和使用。字符串解码器 `StringDecoder` 可以将缓存 `Buffer` 解码为字符串, `StringDecoder` 是 `buffer.toString()` 方法的简单实现, Node.js 的 `child_process`、`crypto`、`readline` 等核心模块中都引用了这个模块。它的特殊之处在于, 当传入的 `Buffer` 不完整 (比如 3 个字节的字符只传入了两个) 时, Node.js 内部会维护一个 `internal buffer` 方法, 将不完整的字节缓存, 等到使用者再次调用 `stringDecoder.write(buffer)` 传入剩余的字节时, 拼成完整的字符。这样可以有效避免因 `Buffer`

不完整而产生的错误，对于很多场景，比如网络请求中的包体解析等，都非常有用，尤其是对较大的中文 GBK 编码的文件，你可以通过 stream 读取，然后传递给 iconv，使用 GBK 编码解码流将其转换成 UTF-8 编码的流，最后通过 string_decoder 输出。

还有一种特殊的用法，即将 Buffer 作为数据结构，和 C 语言里一样，毕竟通过位操作会更节省内存。知名 Node.js 程序员老雷（雷宗民）曾给出这样一个例子，假设有一个学生考试成绩数据库，每条记录的结构如表 6-3 所示。

表 6-3

学 号	课程代码	分 数
XXXXXX	YYYY	ZZ

其中学号是一个 6 位的数字，课程代码是一个 4 位数字，分数最高分为 100 分。在使用文本格式来存储这些数据时，比如使用 CSV 格式存储时，可能如下。

```
100001,1001,99
100002,1001,67
100003,1001,88
```

其中每条记录占用 15 字节的空间，而使用二进制存储时其结构如表 6-4 所示。

表 6-4

学 号	课程代码	分 数
3 字节	2 字节	1 字节

每条记录仅需要 6 字节的空间，仅是使用文本格式存储的 40%！下面是用来操作这些记录的程序代码。

```
// 读取一条记录
// 参数 buf: Buffer 对象
// 参数 offset: 本条记录在 Buffer 对象中的开始位置
// 参数 data: 包括 number、lesson、score
function writeRecord (buf, offset, data) {
  buf.writeUIntBE(data.number, offset, 3);
  buf.writeUInt16BE(data.lesson, offset + 3);
  buf.writeInt8(data.score, offset + 5);
}

// 写入一条记录
// 参数 buf: Buffer 对象
// 参数 offset: 本条记录在 Buffer 对象中的开始位置
function readRecord (buf, offset) {
```

```
    return {
      number: buf.readUIntBE(offset, 3),
      lesson: buf.readUInt16BE(offset + 3),
      score: buf.readInt8(offset + 5)
    };
  }
}

// 写入记录列表
// 参数 list: 记录列表, 每一条中包含 number、lesson、score
function writeList (list) {
  var buf = new Buffer(list.length * 6);
  var offset = 0;
  for (var i = 0; i < list.length; i++) {
    writeRecord(buf, offset, list[i]);
    offset += 6;
  }
  return buf;
}

// 读取记录列表
// 参数 buf: Buffer 对象
function readList (buf) {
  var offset = 0;
  var list = [];
  while (offset < buf.length) {
    list.push(readRecord(buf, offset));
    offset += 6;
  }
  return list;
}
```

我们可以再编写一段程序来查看执行效果, 具体如下。

```
var list = [
  {number: 100001, lesson: 1001, score: 99},
  {number: 100002, lesson: 1001, score: 88},
  {number: 100003, lesson: 1001, score: 77},
  {number: 100004, lesson: 1001, score: 66},
  {number: 100005, lesson: 1001, score: 55},
];
console.log(list);

var buf = writeList(list);
console.log(buf);
// 输出 <Buffer 01 86 a1 03 e9 63 01 86 a2 03 e9 58 01 86 a3 03 e9 4d 01 86 a4 03 e9 42 01
// 86 a5 03 e9 37>

var ret = readList(buf);
console.log(ret);
```

```

/* 输出
[ { number: 100001, lesson: 1001, score: 99 },
  { number: 100002, lesson: 1001, score: 88 },
  { number: 100003, lesson: 1001, score: 77 },
  { number: 100004, lesson: 1001, score: 66 },
  { number: 100005, lesson: 1001, score: 55 } ]
*/

```

在上面的例子中，当每一条记录的结构发生变化时，我们需要修改 `readRecord()` 和 `writeRecord()`，重新计算每一个字段在 `Buffer` 中的偏移量，当记录的字段比较复杂时，就很容易出错。为此老雷编写了 `lei-proto` 模块，允许通过简单定义每条记录的结构生成对应的 `readRecord()` 和 `writeRecord()` 函数。

首先执行以下命令安装 `lei-proto` 模块。

```
$ npm install lei-proto --save
```

使用 `lei-proto` 模块后，上面提到的例子可以改为下面这样。

```

var parsePorto = require('lei-proto');

// 生成指定记录结构的数据编码器和解码器
var record = parsePorto([
  ['number', 'uint', 3],
  ['lesson', 'uint', 2],
  ['score', 'uint', 1]
]);

function readList (buf) {
  var list = [];
  var offset = 0;
  while (offset < buf.length) {
    list.push(record.decode(buf.slice(offset, offset + 6)));
    offset += 6;
  }
  return list;
}

function writeList (list) {
  return Buffer.concat(list.map(record.encodeEx));
}

```

运行与前面相同的测试程序，可以看到，结果是一样的。关于 `lei-proto` 模块的详细使用方法，感兴趣的读者可以研究一下其实现原理。

如果想学习更多的 `Buffer` 用法，最好的方式是深入了解 `msgpack5` 的源码，`msgpack` 是号称比 JSON 快 10 倍的序列化包，`msgpack5` 就是 `msgpack` 在 Node.js 端的最好实现，一般在网络传

输过程中, 为了提高访问速度, 通常会采用 msgpack 或 protobuf 协议。msgpack5 的核心就是对各种基础类型进行编码解码, 所以测试程序中会包含各种对 Buffer 进行转换的处理, 是极好的学习示例。

msgpack5 里使用了 BL 模块, BL 即 BufferList, 是用于拼接 Buffer 对象的, 尤其是对 stream API 有极好的支持。大家都知道, 一个中文字占了 3 个 Buffer 单位, 但是在数据流分块的时候将一个中文的 3 个标识分开再进行 toString 式转义, 会出现乱码。对此, 解决方式就是在拿到分块数据时千万不能直接进行 toString 式转义, 而是要将每段 Buffer 保存, 最后合并成一个大的 Buffer 再进行转义。类似的模块还有朴灵写的 bufferhelper, 如果大家细心解读源码会发现, Node.js 源码里其实也有一个“丑陋”的实现, 即 lib/internal/streams/BufferList.js, 对初学者而言还是有借鉴价值的。

👉 全局对象

全局对象和 JavaScript 里的普通对象是一样的, 主要用于扩展变量和方法。

扩展变量时的写法如下。

```
global.debug = true;

if (debug === true){
}
```

扩展方法时的写法如下。

```
global.log = console.log;

log('something')
```

下面我们创建两个文件用来演示全局对象的用法。假设 a.js 代码如下。

```
global.debug = true;

global.log = debug? console.log : function(x){};
```

b.js 代码如下。

```
global.debug = false;

global.log = debug? console.log : function(x){};
```

测试代码 c.js 的内容如下。

```
require('./a');
require('./b');
```

测试代码 d.js 的内容如下，与上面示例的差别在于，两个 `require` 方法的调用位置不同。

```
require('./b');
require('./a');
```

以上 4 段示例代码会不会让人感觉特别乱？这就是 `global` 关键字的位置不同造成的。多个文件共享变量时不容易控制，所以一般我们要尽量慎用 `global`，否则会出现莫名其妙的 `Bug`。

当然，只在某个文件里适当利用 `global` 关键字的特性还是非常不错的，比如可以使用本例中的 `log` 方法来减少代码量，也可以使用本例中的 `globod.debug` 方法设置是否开启日志，如果开启，开发者就可以在当前项目的任意文件中引用 `log` 方法来打印日志。

6.2.5 Node.js 模块详解

Node.js 自身提供了基本的模块，但在开发实际应用的过程中仅使用这些基本模块是远远不够的。幸运的是，Node.js 内置 `npm` 包管理器，开发者可以很快找到要使用的包进行下载、安装并管理已经安装的包。

Node.js 以包的形式组织程序模块，包的定义十分简单——包含 `package.json` 文件的目录或归档文件，并通过 `@` 来唯一标识。

Node.js 模块分为两部分，分别是 Node.js 内置的 SDK 和 C/C++ 扩展模块，如图 6-3 所示。

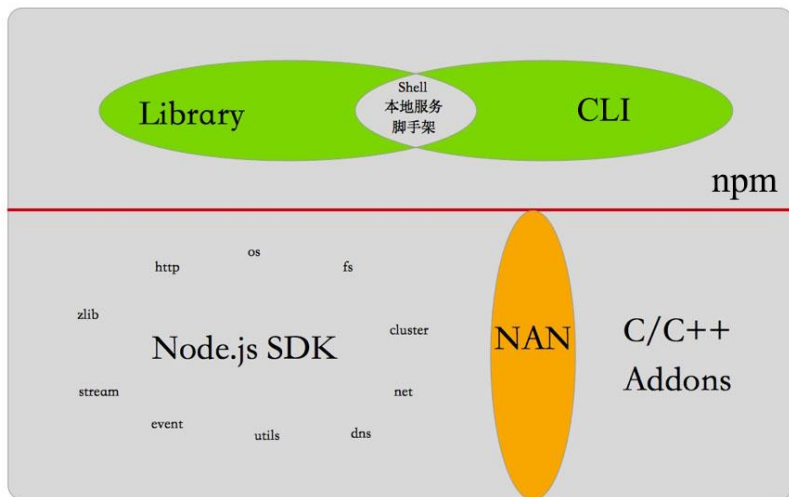


图 6-3

👉 Node.js SDK

SDK 翻译成中文就是“软件开发工具集”，是用来帮一个产品或平台开发应用程式的工具组，由产品的厂商提供，供开发者使用。Node.js SDK 就是 Node.js 平台提供的各种内置模块的统称。安装 Node.js 后，可以直接使用的 Node.js 模块都是 Node.js SDK 里的模块。

Node.js SDK 内的代码量不大，都存在于 Node.js 源码的 lib 目录下。表 6-5 给出了部分 SDK 模块及描述，读者只要了解即可。

表 6-5

模块名称	功 能	描 述
assert.js	断言，一般用在必要判断或测试框架中	API 举例：assert.ok 或 assert.equal
async_hooks.js	以前称为 AsyncWrap，用于追踪 Node.js 中异步资源的生命周期	API 发出的消息会将 Node.js 中所有句柄对象的生命周期告知给 Consumer
buffer.js	用于实现数据缓冲	Buffer 一般用于处理二进制数据，也可以处理字符编码
child_process.js	实现多进程任务	创建子进程，实现子进程和主进程之间的通信
cluster.js	利用多核 CPU 实现并行	可以帮助我们简化多进程并行化程序的开发难度，轻松构建一个用于负载均衡的集群。著名的 pm2 模块就使用了 Cluster 实现
console.js	主要将浏览器里的 console 与终端进行适配	和浏览里的用法一样，比如 console.log、console.dir 等
constants.js	对外暴露一些常量	废弃，不推荐使用
crypto.js	加密模块	对 OpenSSL 里的 HMAC、Cipher、Decipher 等算法进行加解密封装，一般用户在进行密码处理时都会用到该模块
dgram.js	实现 UDP 报文 Socket	UDP 不属于连接型协议，因而具有资源消耗小、处理速度快等优点，通常在音频、视频和普通数据传送时使用较多
dns.js	域名解析	主要 API 是 lookup 和 resolve
domain.js	Node.js 0.8 以上的版本中才有，用来捕捉异步回调中出现的异常	即将废弃，不推荐使用
events.js	事件处理，即 EventEmitter 的实现	EventEmitter 的核心功能就是对事件触发和事件监听器功能进行封装
fs.js	文件系统模块，提供一组类似 UNIX（POSIX）标准的文件操作 API	主要针对目录、文件进行操作，开发中使用极其广泛

续表

模块名称	功 能	描 述
http.js	搭建 HTTP 服务端和客户端	是 Node.js 里使用多的模块,可以非常简单地构建 Web 应用,是 Web 框架的底层核心库, lib/_http_*是它的私有子模块
http2.js	下一代 HTTP 协议	在 Node.js 8 里是需要通过 flag 开启的体验功能
https.js	HTTPS 实现,即以安全为目标的 HTTP 通道,是 HTTP 的安全加强版	HTTPS 的安全基础是 SSL,在架构选择上,可以通过 Nginx 实现,也可以在 Node.js 应用层上实现
inspector.js	新版本调试协议	主要对 inspector 进行封装,核心源码主要是 Agent、Socket 和 Session 这三类

👉 模块化和模块依赖

我们从结构化编程时代就开始提倡模块化（Modularization），希望像拼乐高积木一样进行开发，将系统拆分为不同的组件，各自实现，然后再组装到一起。各个语言中都有第三方模块，比如 Java 里的 jar、Ruby 里的 gem、Perl 里的 pm 等。在 Node.js 中，这样的第三方模块称为 node module。npm 是官方推荐的包管理器，我们可以简单地认为 node module 和 npm 上的 package 是一样的。

- 每个模块的根目录都包含 package.json，用来定义模块和该模块的依赖模块。
- 每个模块都可以作为其他模块的依赖独立存在，即可以将多个模块组成一个模块。
- 模块按照功能可以分为基础模块和集成模块，集成模块是由多个基础模块组成的。

Node.js 提供了良好的模块化机制和简单易用的 npm 包管理工具，因此社区能够将优秀的模块生态作为宣传点，目前 npm 是开源社区里最大的包管理工具。

👉 包结构和小而美的哲学

前面提到，在很长一段时间内，JavaScript 中是缺少包管理工具的，最核心的原因是没有统一模块规范。CommonJS 的出现改变了这种情况，通过定义包的结构规范，模块化变成了可能，采用定制的 CommonJS 规范作为 Node.js 内置模块规范，解决了 npm 包的安装、卸载、依赖管理、版本管理等问题。

一个符合 CommonJS 规范的包应该具有如下结构。

- 一个 `package.json` 文件，存在于包的顶级目录下。
- 二进制文件应该包含在 `bin` 目录下（可选）。
- JavaScript 代码入口默认是 `index.js`，其他代码包含在 `lib` 目录下。
- 文档应该包含在 `doc` 目录下（可选）。
- 单元测试应该包含在 `test` 目录下（可选）。

我们要遵循 UNIX 哲学 9 条里的第 1 条——Small is beautiful，保持模块足够小（内聚），对于 Node.js 来说，也实在是再合适不过了。

对于 Node.js 这种新生平台来说，采用小而美哲学发展社区生态是非常明智的选择。至于 npm 包的质量问题，并非 Node.js 独有的问题，相信读者自可甄别。

👉 Node.js 模块扩展

Node.js 和其他语言一样，对 C/C++ 的支持特别好，提供了两种机制用于编写扩展。

- NAN (Native Abstractions for Node.js): Rod Vagg 牵头编写了 Node.js 扩展抽象层，使得 Node.js 扩展开发更加简单，几乎不需要了解 Node.js 源码细节即可编写。
- N-API: Node.js 8 之后开始支持的新 API，借助 ABI 虚拟机可以实现“一次编译，多个版本运行”，主要解决了 Node.js 扩展模块跨版本的编译问题。

自从 Node.js 8 发布之后，N-API 的“一次编译，多个版本运行”特性就成为一个亮点。与跨平台类似，它是跨版本的，这样解决了以前一直困扰 Node.js 的每次都需要编译 addon 的问题。尤其是像 Sass 这种用 C 语言编写的模块，通常编译报错如下。

```
Error: The module 'node-sass'
was compiled against a different Node.js version using
NODE_MODULE_VERSION 51. This version of Node.js requires
NODE_MODULE_VERSION 55. Please try re-compiling or re-installing
the module (for instance, using 'npm rebuild' or 'npm install').
```

NODE_MODULE_VERSION 是每一个 Node.js 版本内人为设定的数值，意思为 ABI 的版

本号。一旦这个号码与已经编译好的二进制模块号码不符，便判断为 ABI 不兼容，需要用户重新编译。

这其实是一个工程难题，也让我们必须思考：如何降低 Node.js 上游的代码变化对 C++ 模块的影响从而维持一个良好的向下兼容的模块生态系统。最坏的情况下，每次发布 Node.js 新版本时，因为 API 的变化，C++ 模块的作者都要修改源代码，而那些不再有人维护或作者失联的老模块就会无法继续使用，在作者修改代码之前社区就失去了这些模块的可用性。其次比较坏的情况是，每次发布 Node.js 新版本时，虽然 API 保持兼容使得 C++ 模块的作者不需要修改源代码，但 ABI 发生变化导致这些模块必须重新编译。而最好的情况就是，Node.js 新版本发布后，所有已编译的 C++ 模块都可以继续正常工作，完全不需要任何人工干预。

Node.js 8 中的 N-API 就是为了解决这个问题而产生的，N-API 可以实现上述最好的情况，为 Node.js 模块生态系统的长期发展铺平道路。N-API 追求以下目标。

- 有稳定的 ABI。
- 抽象消除 Node.js 版本之间的接口差异。
- 抽象消除 Chrome V8 不同版本之间的接口差异。
- 抽象消除 Chrome V8 与其他 JavaScript 引擎（如 ChakraCore）之间的接口差异。

N-API 采取以下手段达到上述目标。

- 采用 C 语言头文件而不是 C++ 头文件，消除 Name Mangling 以便最小化一个稳定的 ABI 接口。
- 不使用 Chrome V8 的任何数据类型，所有 JavaScript 数据类型都变成了不透明的 `napi_value`。
- 重新设计异常管理 API，所有 N-API 都返回 `napi_status`，通过统一的手段处理异常。
- 重新设计对象的生命周期 API，通过 `napi_open_handle_scope` 等 API 替代 Chrome V8 的 Scope 设计。

N-API 目前在 Node.js 8 中仍是实验阶段的功能，需要配合命令行参数 `--napi-modules` 来使用。

在前面的章节中，我们曾列举过 `node-java` 的例子。Rust 语言对 N-API 进行封装也是极为方便的。Rust 的作者说，使用 Rust 绑定 N-API 有 3 点好处。

- N-API 是纯的 C 语言扩展，故而进行 Rust 扩展会更简单。
- N-API 是稳定的，ABI 可以跨版本兼容，对于模块的维护非常友好。
- API 是独立的，可以跨 Chrome V8、ChakraCore、SpiderMonkey，以及未来更多的平台使用。

Node.js 虽好，但同步和 CPU 密集运算一直是痛点，如今这些问题可以在 Native 模块中解决，这些恰恰是 Rust 这种系统级的语言所擅长的，它比 Go 语言的性能还要出色。

6.3 未来展望：ES 模块

现在 ES6 自带了模块标准，这是 JavaScript 第一次在语言级别支持模块(之前的 CommonJS、AMD、CMD、UMD 都不算)，但 Node.js v8.9 之前的所有版本都无法直接使用 ES6 模块，只能通过 Traceur、BabelJS 或 TypeScript 等转译器将 ES6 代码转化为兼容 ES5 版本的 JavaScript 代码才能使用。尽管如此，ES6 模块的新特性依然非常吸引人。

ES 模块的目标是创建一个同时兼容 CommonJS 和 AMD 的格式，使语法更加紧凑，通过编译时加载，在编译时就能确定模块的依赖关系，比 CommonJS 模块的加载效率更高。而在异步加载和配置模块加载方面，则借鉴 AMD 规范，执行效率、灵活度都远远好于 CommonJS 写法。总结来说，ES 模块的优势如下。

- 语法更紧凑。
- 结构更适用于静态编译（比如静态类型检查、优化等）。
- 对循环引用的支持更好。
- 用法简单，不需要关注实现细节。
- 采用声明式语法：模块导入用 import 关键字，导出用 export 关键字，没有 require 关键字。
- 程式化加载 API：可以设置模块如何加载，并按需加载。

简单总结一下，Chrome 61 开始便支持了 ES 模块，Node.js v8.9 之后也开始支持 ES6 模块。除了 Node.js 内置的实现，也有参照草案的实现，@std/esm 是第一个让 Node.js 同时支持两种模块标准的实现。

6.3.1 ES 模块入门

模块可以通过 `export` 关键字对外导出很多内容，这些导出的内容可以通过名字进行访问。下面的代码是 ES 模块的简单写法。

```
//----- lib.js -----
export const sqrt = Math.sqrt;
export function square(x) {
  return x * x;
}
export function diag(x, y) {
  return sqrt(square(x) + square(y));
}

//----- main.js -----
import { square, diag } from 'lib';
console.log(square(11)); // 121
console.log(diag(4, 3)); // 5
```

当然，也可以将所有内容导出，并以别名的形式来访问，如下所示。

```
//----- main.js -----
import * as lib from 'lib';
console.log(lib.square(11)); // 121
console.log(lib.diag(4, 3)); // 5
```

作为对比，下面给出使用 CommonJS 写法实现同样功能的示例，如下所示。

```
//----- lib.js -----
var sqrt = Math.sqrt;
function square(x) {
  return x * x;
}
function diag(x, y) {
  return sqrt(square(x) + square(y));
}
module.exports = {
  sqrt: sqrt,
  square: square,
  diag: diag,
};

//----- main.js -----
var square = require('lib').square;
var diag = require('lib').diag;
console.log(square(11)); // 121
console.log(diag(4, 3)); // 5
```

6.3.2 模块导入

模块导入有两种方式，直接导入和按需导入，下面具体来看一下。

👉 直接导入

直接导入模块对象的写法如下。

```
import * as fs from 'fs';
```

这种写法和 CommonJS 中的写法一样。

```
const fs = require('fs');
```

👉 按需导入

按需导入写法如下。

```
import { readFile } from 'fs';
```

以上代码的实质是从 fs 模块中加载 readFile 方法，不加载其他方法。这种加载称为编译时加载或静态加载，即 ES6 可以在编译时完成模块加载，效率更高。

6.3.3 模块导出

ES6 提供了很多种对外导出的方式。

对外导出所有内容（除了默认导出部分），方法如下。

```
export * from 'src/other_module';
```

通过别名导出，方法如下。

```
export { foo as myFoo, bar } from 'src/other_module';

export { default } from 'src/other module';
export { default as foo } from 'src/other_module';
export { foo as default } from 'src/other_module';
```

👉 具名导出

导出对象的指定别名，即所谓的具名导出，方法如下。

```
export { MY CONST as FOO, myFunc };  
export { foo as default };
```

👉 内联具名导出

变量声明的方式有多种，在内联具名导出时不做限制，方法如下。

```
export var foo;  
export let foo;  
export const foo;
```

函数声明的方式如下。

```
export function myFunc() {}  
export function* myGenFunc() {}
```

类声明的方式如下。

```
export class MyClass {}
```

👉 默认导出

默认导出是指使用 **default** 关键字，在引用时默认返回导出对象。对于默认导出，函数声明（可以是匿名函数）的方式如下。

```
export default function myFunc() {}  
export default function () {}  
  
export default function* myGenFunc() {}  
export default function* () {}
```

类声明（可以是匿名类）的方式如下。

```
export default class MyClass {}  
export default class {}
```

使用表达式声明的方式如下，注意最后的“分号”。

```
export default foo;  
export default 'Hello world!';  
export default 3 * 7;  
export default (function () {});
```

6.3.4 ES 模块示例

Vue 是最早一批对 ES 模块支持的框架，我们看一下 Vue v2.5.13 的 package.json 文件里的具

体配置项，如下。

```
{
  "name": "vue",
  "version": "2.5.13",
  "description": "Reactive, component-oriented view layer for modern web interfaces.",
  "main": "dist/vue.runtime.common.js",
  "module": "dist/vue.runtime.esm.js",
  ...
}
```

这里的 `module` 对应的是以 `esm.js` 为后缀的文件，就是 ES 模块的具体实现。

6.3.5 兼容性更好的@std/esm

上面的用法只适合 Node.js v8.9 之后的版本，那么如果想兼容 Node.js v4 该如何实现呢？答案是，我们需要用到 `@std/esm` 模块。`@std/esm` 模块其实就是一个 `esm` 解释器，属于纯 Node.js 的实现，所以能够完美兼容更多 Node.js 版本。而 Node.js 的 `esm` 是基于 Chrome 61 的 feature 实现的，所以原生的 `esm` 实现是依赖 Node.js 版本的。

首先通过 `npm i --save @std/esm` 命令安装 `@std/esm` 模块，然后编写如下代码引入 `@std/esm` 模块。

```
require = require("@std/esm")(module, options)
module.exports = require("./main.mjs").default
```

剩下的代码可以按照 `esm` 模块的写法来编写。开心地使用你喜欢的 `import` 和 `from` 关键字吧，不用担心不兼容的问题！

6.4 本章小结

通过本章的学习，你应该已经对 Node.js 与 CommonJS 规范有了一定的了解，它们可以在服务器端和前端达成用法上的统一。当然，模块写法除了遵守 CommonJS 规范，还要更好地面向 ES6 模块写法，虽然目前使用 CommonJS 规范编写的模块和 ES 模块还没法完全“打通”，但社区一直在努力，比如创建了试图解决这一问题的新项目，见 <https://github.com/nodejs/ecmascript-modules>，相信不久会有更好的实现。在此之前，使用各种转译器来编写更好的代码也是一个不错的选择！

第 7 章

异步写法与流程控制

大家都知道，流程控制是程序中的逻辑控制的统称，那么，为什么在 Node.js 的世界里就变成异步流程控制了呢？这和 Node.js 本身的异步机制有关，若每个函数都是异步的，性能虽会更好，但却容易造成 callback hell（回调地狱）问题。在 Node.js 里，为了解决 API 级别的回调地狱问题，便引入了用于流程控制的部分——异步流程控制。

Node.js 从一开始就破旧立新，因此经常出现回调地狱问题。但也正是因为存在回调地狱问题，Node.js 才能在异步流程控制方面发展得很快，比如实现了 Thunk、将 Promise/A+ 规范落地、在 ES6 中支持了 Generator 及 co 模块，以及实现了目前最被看好的 async 函数。各种关于异步流程控制的 Node.js 辅助模块更是多到数不清，比如著名 async.js 模块。

可以这样说，掌握了 Node.js 里的异步流程控制，你就掌握了 Node.js 一半以上的内容。Node.js 中的异步流程控制由于发展过快而被人诟病，事实上，这样的行为并不客观。本章试图去繁就简，希望能够让大家了解 Node.js 流程控制简单易用的一面。

本章主要分为 3 个部分：了解异步流程的调用和模式；掌握流程改进的 5 个阶段及重要特性；总结学习重点。

如图 7-1 所示，Node.js SDK 自带的 API 多采用 Promise 规范进行处理，Node.js v0.12 之后的版本开始大规模支持 Generator，在 Node.js v7.6 之后又支持 async 函数，这几乎是 3 个里程碑式的事件，囊括了 Node.js 中目前所有主流的异步流程控制方案，也是学习 Node.js 过程中的重点内容。本章我们要在这些内容中推导出学习重点，有针对性地进行学习。

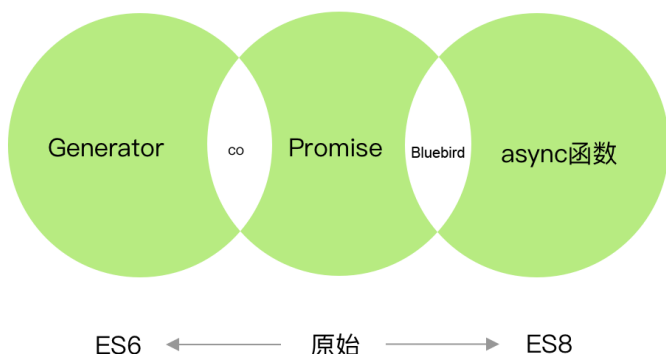


图 7-1

7.1 异步调用

异步调用是 Node.js 的精髓，理解了异步写法，就能够在编程时游刃有余。很多人认为掌握了源码才算精通，但在深入源码之前，最好能对 API 用法非常熟悉，避免理解不全面。这部分非常基础，也非常重要，希望各位读者用心理解。

7.1.1 异步与同步

首先要了解什么是异步，什么是同步。

举个例子来说，比如你去 CNode 论坛发帖提问，多半是不会立即收到回复的，这时可以采取两种做法。

- 一直等待（不停按 F5 键进行刷新），直到有人回复，解决问题。
- 该忙什么忙什么，当有人回复的时候，系统会通知你。chrome-cnode-notifier 这个插件就是专门做这件事的。

第一种做法就是同步的——必须等到别人回复（服务器返回信息）才进行其他事情，“至死方休”。而第二种做法就是异步的——不耽误时间，合理利用时间高效做事，即充分利用服务器并行执行操作。

遇到类似情况，你会采用哪种办法呢？显然异步并行操作更能提升效率。

7.1.2 浏览器中的异步

Web 2.0 时代的特色主要体现在对交互的创新上，其核心技术是 Ajax，即异步 JavaScript 与 XML。不再需要重新加载浏览器即可更新 Web 页面的内容，再也不用不断地打开信息页面去查看新内容了。

在不同的浏览器中实现 Ajax 代码可能很乏味，因此大量的 jQuery 库应运而生。这些库使得我们能够以很容易并且很优雅的方式实现客户端与服务器端的通信。Ajax 中的“A”表示 Asynchronous，即“异步”的意思。下面我们便通过理解 Ajax 的原理，再次深入理解所谓的异步是什么。

以常用的 jQuery 为例，代码如下。

```
$.getJSON("/api/1",function(result){
    // 执行某些操作
    $("div").append("x");
});
console.log(1 + '\n')
$.getJSON("/api/2",function(result){
    // 执行某些操作
    $("div").append("y");
});
console.log(2 + '\n')
```

这里一共发送了两个请求，两个 \$.getJSON 命令是顺序执行的，所以会出现如下结果。

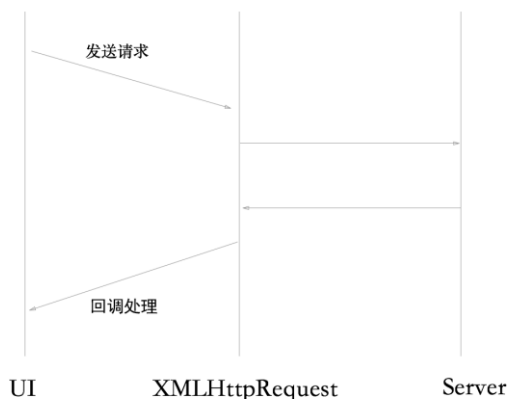
```
1
2
```

至于 x 和 y 哪个先在页面中输出，则要看具体服务器的响应，这就是所谓的异步操作。

7.1.3 Node.js 异步原理

为了让大家更好地理解异步原理，这里将 Ajax 和 Node.js 对比来讲，以便更好地突出各自的特点。Ajax 的核心是 XHR（XMLHttpRequest），其原理如图 7-2 所示，而 Node.js 的核心是 EventLoop，二者有异曲同工之妙。

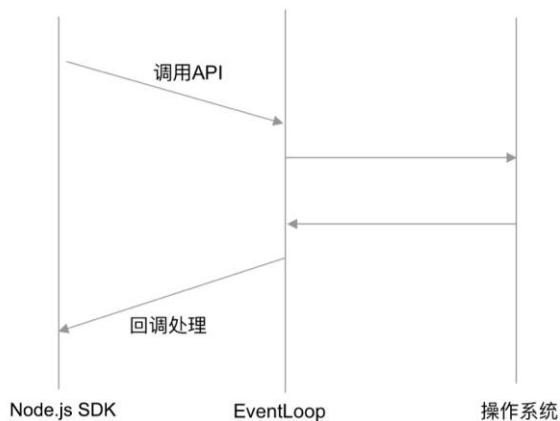
Ajax 定义好请求和回调函数后，剩下的事情就交由 XHR 去处理。XHR 与服务器交互会产生时间差异，所以异步操作能够很好地解决这个问题，不需要刷新页面就可以获取数据。



Ajax原理

图 7-2

如图 7-3 所示, Node.js 异步原理的核心是 EventLoop, 我们调用 Node.js API 方法的时候, 它会把具体操作和回调函数交给 EventLoop 去执行, EventLoop 维护了一个回调函数队列, 当异步函数执行时, 回调函数会被放入这个队列。JavaScript 引擎直到异步函数执行完成后, 才会开始处理 EventLoop, 这意味着 JavaScript 代码不是多线程的, 即使表现出来的行为相似。EventLoop 维护的是一个先进先出 (FIFO) 的队列, 这说明回调函数在队列中是顺序执行的。JavaScript 被 Node.js 选为开发语言, 就是因为编写这样的代码非常简单。



Node.js异步原理

图 7-3

EventLoop 是用 C/C++ 编写的 libuv 库。libuv 采用了异步和事件驱动的编程风格，主要功能是为开发人员提供一套基于 I/O（或其他活动）通知的回调函数。所以，应该由 libuv 处理的 I/O 任务，实际是由操作系统处理的。在这种情况下，执行快慢可能会有差异，因此异步是最好的选择。

综上所述，无论是 Ajax 还是 Node.js，它们都是借助“中间层”进行实际操作的。在使用时，我们无须过多关注中间层之后的操作就可以非常简单地完成功能开发，这其实也是 Node.js 最大的好处——能够以最小的成本获取高性能，只需要专注于编写代码即可，更复杂的执行过程由 libuv 来处理。

7.1.4 API 和示例

API 是应用程序接口 Application Programming Interface 的简称，通过 Node.js 异步原理，我们可以知道，异步的核心在于 Node.js SDK 中的 API 调用，然后交由 EventLoop（libuv）去执行，所以我们一定要熟悉 Node.js 的 API 操作。

Node.js 的 API 都是异步的，同步的函数在 Node.js 中是一种奢求，因此使用前要先查询 API 文档，在高并发场景下慎用。

笔者推荐使用 Dash 或 Zeal 来查看离线文档，经常查看离线文档对深入理解 API 很有帮助，比 IDE 辅助效果要好，可以有效避免面临“离开 IDE 就不会写代码”的窘境。

👉 获取目录下所有文件的 API

为了说明 Node.js 的 API 用法，这里我们以获取目录下所有文件的接口 fs.readdir 为例进行介绍，代码如下。

```
fs.readdir(path, callback) #
Asynchronous readdir(3). Reads the contents of a directory.
The callback gets two arguments (err, files)
where files is an array of the names of the files
in the directory excluding '.' and '..'.
```

这里解释一下 Node.js SDK 中使用的错误优先回调（error-first callback）的写法，具体如下。

- 回调函数通常都是 API 的最后一个参数。
- 回调函数中会约定回调内容，比如上例中的(err, files)。
 - err 在前，没有 err 时第一个参数为空，此为 API 的约定写法。

- 具体返回结果在后，可以有多种结果。

👉 异步写法

使用回调函数的写法都是异步写法，示例如下。

```
const fs = require('fs')

const path = '.'

fs.readdir(path, function(err, files) {
  console.log(files)
})
```

👉 同步写法

同步写法即直接返回结果的写法，不需要在回调函数中处理，它是顺序执行的，会发生阻塞，示例如下。

```
var fs = require('fs')

console.log(fs.readdirSync('.'))
```

执行以上代码并查看结果。从结果中我们可以看出，返回值是数组类型的。

```
$ node nodejs/async/readdir-sync.js
[ '.DS_Store',
  '.git',
  '.gitignore',
  'README.md',
  'SUMMARY.md',
  '_book',
  'async',
  ...
  'other',
  'practice',
  'reference.md',
  'wechat' ]
```

👉 找出以 koa 开头的文件

若想找出以 koa 开头的文件，该怎么做呢？关键在于使用正则表达式，下面是使用 `test` 方法的简单示例。

```
files.forEach(function (file) {  
  if (/^koa/.test(file)) { //查找以 koa 开头的文件  
    console.log(file)  
  }  
})
```

首先看一下异步写法，代码如下。

```
const fs = require('fs')  
  
const path = '.'  
  
fs.readdir(path, function(err, files) {  
  files.forEach(function (file) {  
    if (/^koa/.test(file)) {  
      console.log(file)  
    }  
  })  
})
```

在上例中，获取文件的结果是在回调函数中实现的，很明显这是异步写法。下面我们再来看一下同步写法，代码如下。

```
var fs = require('fs')  
  
var files = fs.readdirSync('.')  
  
files.forEach(function (file) {  
  if (/^koa/.test(file)) {  
    console.log(file)  
  }  
})
```

在同步写法中，首先获取文件结果，然后遍历文件并打印结果，这里没有用回调函数，顺序执行的代码逻辑是非常容易读懂的，所以这样写更容易让人理解。

要注意的是，在高并发场景下要慎用同步写法，不然可能会成为性能瓶颈，毕竟同步写法是模拟实现的，和 Node.js 本身异步的设计初衷是相悖的。

7.1.5 代码优化

7.1.4 节中的示例代码还是比较冗余的，本节我们就来介绍一下代码优化的方法。

把过滤处理的代码抽象成一个独立的函数，这样做能让代码具有更好的可读性。下面我们通过一个例子来比较一下同步和异步的差别，先来看一个异步示例。


```
var fs = require('fs')

var path = '.';

function filter(files){
  files.forEach(function (file) {
    if (/^koa/.test(file)) {
      console.log(file)
    }
  })
}

// 入口
fs.readdir(path, function(err, files) {
  // 过滤
  filter(files)
})
```

其中的核心代码如下。

```
// 入口
fs.readdir(path, function(err, files) {
  // 过滤
  filter(files)
})
```

这是一个比较容易理解的异步示例，代码看起来还比较清晰。不过我们换个角度想想，如果嵌套 5 层会怎么样呢？是不是很可怕？很明显，在多层嵌套的情况下，这种写法并不好，因此我们就要想办法将代码进行优化。

来看看同步优化后的代码，具体如下。

```
var fs = require('fs')

function filter(files){
  files.forEach(function (file) {
    if (/^koa/.test(file)) {
      console.log(file)
    }
  })
}

var files = fs.readdirSync('.')

filter(files)
```

通过以上代码明显可以看出，顺序执行更容易理解。

👉 极端情况

以 `readdir-sync3.js` 为例，我们来看看完全同步的代码是什么样的，具体如下。

```
var files = fs.readdirSync('.')

// fs.readdirSync 执行完才会执行 filter
filter(files)
```

这样的代码非常容易理解，那么我们再极 endpoint。

在文件数量较少的情况下，代码执行效率非常不错，可是如果文件数量很多呢？如果有上亿个文件，程序执行就需要一定的时间，这种同步代码会阻塞后面方法的执行，那是不是无法提高并行效率呢？这种情况下，如果采用异步方法，会是什么状况呢？代码如下。

```
// 入口
fs.readdir(path, function(err, files1) {
  // 过滤
  filter(files1)
})

fs.readdir(path, function(err, files2) {
  // 过滤
  filter(files2)
})
```

以上代码其实执行了两个 `fs.readdir` 方法，这样能够更好地利用系统资源完成更多的任务。通过 `EventLoop` 执行 I/O 任务可以更好地利用系统资源，只要系统资源还有剩余，就会不断执行。这是 Node.js 的好处，但在这种方式下处理回调函数有些麻烦，因此受到很多人诟病。但异步依然是最佳实现，是解决性能问题最直接的方式，连 Java 9 中的异步 API 也开始支持异步写法了。

👉 模拟阻塞

以下是一段模拟阻塞的代码。

```
var fs = require('fs')

function filter(files){
  files.forEach(function (file) {
    if (/^koa/.test(file)) {
      console.log(file)
    }
  })
}
```

```
    })
  }

  function sleep(milliSeconds) {
    var startTime = new Date().getTime();
    while (new Date().getTime() < startTime + milliSeconds);
  };

  var files = fs.readdirSync('.');

  sleep(10000);

  filter(files)
```

在以上示例中，`sleep` 函数执行了 10 秒，然后才执行 `filter` 函数。写这个例子是为了告诉各位读者：同步虽好，但要慎用；在 `Node.js` 里，异步才是常态，最好以异步的方式来思考问题。当然，如果你开发的是一些工具类的非高并发的软件，同步阻塞方式也是推荐的，会使开发效率更高。

👉 并行任务

相比于上面的阻塞的例子，我们更希望程序能够快速执行完成，因此可以使用并行方式。下面是一个简单的并行任务的示例。

```
const fs = require('fs')

const path = '.'

// 任务 1
fs.readdir(path, function(err, files) {
  // 过滤
  files.forEach(function (file) {
    if (/^koa/.test(file)) {
      console.log('1= ' + file)
    }
  })
})

// 任务 2
fs.readdir(path, function(err, files) {
  // 过滤
  files.forEach(function (file) {
    if (/^koa/.test(file)) {
      console.log('2= ' + file)
    }
  })
})
```

```
  })  
})
```

多次执行以上代码，返回结果是不一样的。有的时候任务 1 先执行完，有的时候任务 2 先执行完，这其实是因为两个任务是并行的，哪个任务耗时短，哪个任务先执行完。

第一次的执行结果如下。

```
$ node nodejs/async/async.js  
2= koa-core  
2= koa-http  
2= koa-in-action  
2= koa.sketch  
1= koa-core  
1= koa-http  
1= koa-in-action  
1= koa.sketch
```

第二次的执行结果如下。

```
$ node nodejs/async/async.js  
1= koa-core  
1= koa-http  
1= koa-in-action  
1= koa.sketch  
2= koa-core  
2= koa-http  
2= koa-in-action  
2= koa.sketch
```

明显异步执行的效率更高，但结果却不一定是我们想要的。异步执行的结果具有一定的不确定性。

本节以 `fs.readdir` 为例，讲解了异步和同步的区别，总结如下。

- 同步方式更容易理解，但会造成线程阻塞，无法最大限度地利用系统资源。
- 异步方式需要嵌套回调，即使代码编写得非常规范也不容易理解和维护，但它能够并行执行，同时处理更多任务，效率更高。

对于编程而言，我们一定希望得到更高的执行效率，以及最大程度的可控性。`Node.js` 实现了高的执行效率，至于如何提升可控性则是开发人员要解决的问题，也是 `Node.js` 中最难的点。

7.2 Node.js 自带的异步写法

Node.js 中有两种事件处理方式，分别是 `callback`（回调）和 `EventEmitter`（事件发射器）。本节将围绕这两种异步模式进行讲解，希望大家能够玩转异步世界里的 Node.js。

简单来说，`callback` 采用错误优先（`error-first`）的回调方式，`EventEmitter` 则是事件驱动里的事件发射器。下面我们具体来看一下。

7.2.1 错误优先的回调方式

跨模块和应用之间的异步非阻塞流程控制是可以平衡的。Node.js 在设计之初就已经确立了异步机制和回调方式。绝大多数回调处理确实需要一个通用的可依赖协议，因此 `error-first` 回调（可以称为 `errorback`、`errback` 或 `node-style callback`）被引进，并且成了 Node.js 回调方式的标准。

Node.js 重度使用回调，主要是受到比 JavaScript 更古老的一种编程风格的影响。CPS（Continuation-Passing Style）是目前 Node.js 使用的回调方式曾经用过的名称。在 CPS 里，`continuation` 函数中会被传入一个参数，当其他代码执行完成后再调用这个参数，这使得不同的函数可以跨应用进行异步处理，然后再执行后续操作。

Node.js 依赖异步代码来保证执行效率，所以依赖回调模式是极其重要的。没有一个程序员愿意在不同模块间维护不同的程序写法和编程风格，于是错误优先的回调方式便被引入 Node.js 核心模块，用于解决这个问题，并最终成为标准。无论一个 API 拥有如何不一样的需求和响应，错误优先的回调模式都可以非常好地满足需求。

✎ 错误优先回调方式的写法

定义错误优先的回调方式只需要注意两条规则即可，具体如下。

- 回调函数的第一个参数返回的是 `error` 对象，如果发生错误，该对象会作为第一个参数返回。如果执行正常，一般做法是返回 `null`。
- 回调函数的第二个参数返回的是所有成功响应的结果数据。如果结果正常，即没有发生错误，参数 `err` 就会被设置为 `null`，并在第二个参数处返回成功响应的结果。

下面我们来看一下调用函数示例，Node.js 文档里最常采用下面这样的回调方式。

```
function(err, res) {  
  // 处理错误和结果  
}
```

这里的回调指的是带有两个参数（`err` 和 `res`）的函数。从语义上讲，非空的 `err` 相当于程序异常，而空的 `err` 相当于可以正常返回结果 `res`，无任何异常。

👉 API 写法约定

模块应该暴露错误优先的回调接口，示例如下。

```
module.exports = function (dragonName, callback) {  
  // 逻辑处理  
  var dragon = createDragon(dragonName);  
  
  // 注意，第一个参数是 error，如果没有错误，则它的默认值是 null  
  return callback(null, dragon);  
}
```

应该经常在回调中检查错误。为了理解这一点，我们从一个违反该规则的例子看起，代码如下。

```
const fs = require('fs')  
  
function readJSON(filePath, callback) {  
  fs.readFile(filePath, function(err, data) {  
    callback(JSON.parse(data));  
  });  
}  
  
readJSON('./package.json', function (err, pkg) { ... })
```

`readJSON` 函数最主要的问题是，如果在执行过程中发生错误，它并不会检查这个错误。改进后的代码如下。

```
function readJSON(filePath, callback) {  
  fs.readFile(filePath, function(err, data) {  
    // 如果出现错误  
    if (err) {  
      // 注意此处，必须调用回调，且参数是具体的错误信息  
      return callback(err);  
    }  
  
    // 如果没有错误，则第一个参数值为 null，第二个参数是具体结果  
    callback(null, JSON.parse(data));  
  })  
}
```

```
});  
} **在回调中返回! **
```

上面的代码仍然存在问题, 当发生错误的时候, 程序并不会在 if 语句中停止执行, 而是会继续执行。这会导致很多意料之外的问题发生, 例如上面代码提示的那样, 总是在回调中返回, 所以正确的写法应该如下。

```
function readJSON(filePath, callback) {  
  fs.readFile(filePath, function(err, data) {  
    if (err) {  
      return callback(err);  
    }  
  
    callback(null, JSON.parse(data));  
  });  
}
```

需要注意的是, `JSON.parse` 方法在无法将指定的字符串格式转化为 JSON 格式的时候会抛出一个异常。因为 `JSON.parse` 是一个同步函数, 所以我们可以将其放在 `try-catch` 语句块中。只有同步代码块才能使用 `try-catch` (在 Node.js 8 之后的优化版本里, `try-catch` 几乎没有性能损坏, 可以放心使用), 在回调函数中不能随意使用! 正确增加异常捕获的写法如下。

```
function readJSON(filePath, callback) {  
  fs.readFile(filePath, function(err, data) {  
    var parsedJson;  
  
    // 处理错误  
    if (err) {  
      return callback(err);  
    }  
  
    // 解析 JSON  
    try {  
      parsedJson = JSON.parse(data);  
    } catch (exception) {  
      return callback(exception);  
    }  
  
    // 一切正常, 直接调用回调函数  
    return callback(null, parsedJson);  
  });  
}
```

除了要掌握约定写法, 我们还需要了解异常处理方法。

异常处理是异步流程控制里最难的部分。异常主要分为两种, 系统错误和程序员错误。系

统错误包括请求超时、系统内存不足、连接远程服务失败等，一般需要搭配系统监控等辅助软件解决。程序员错误即程序的 Bug，产生原因有很多，比如，在调用异步方法时没有使用回调、无法读取 `undefined` 的属性、在高并发场景下使用了同步阻塞代码、没有及时处理异常、内存泄漏。这类错误是可以避免的，应对办法是启用日志服务，通过日志记录一切。

👉 函数定义

通常情况下，函数定义的方式如下。

```
function(arg1, arg2, ..., callback) {  
  // 进行参数校验，然后调用回调函数  
}
```

调用程序会进行约定，将回调函数作为最后一个参数进行调用。从语义上讲，遵守约定是指程序根据输入参数完成某些工作，在特定场景下调用回调对异步处理进行注册，在异步处理完成后通过回调函数通知程序调用方。

在只有回调函数没有其他参数的极端情况下，程序代码如下。

```
function(callback) {  
  // 进行参数校验，然后调用回调函数  
}
```

让我们来看一个 Node.js 文件读写的例子，代码如下。

```
const fs = require('fs')  
  
const path = '.'  
  
fs.readdir(path, function(err, files) {  
  // 总是需要在回调函数中检查错误  
  if (err) {  
    return console.log(err)  
  }  
  console.log(files)  
})
```

关于以上代码，说明如下。

- `fs.readdir` 是具体程序。
- 第一个参数是 `path`，即具体的文件地址。

- 第二个参数是 `function(err, files) {}`，即文件读取完成后的具体回调函数。
- 在回调里一定要先检查错误，命令是 `if (err) {...}`。

这是 Node.js 世界里最常见的写法，简单易懂。我们在编写对外模块的时候，也要尽量采用这种风格。大家都遵守一样的约定能够有效降低沟通成本。写法越底层，使用者越有可发挥的空间。

7.2.2 EventEmitter

事件模块是 Node.js 内置的对观察者模式的实现，通过 `EventEmitter` 属性提供一个构造函数。该构造函数的实例中具有两个常用方法，其中 `on` 方法可以用来监听指定事件，并触发回调函数，另一个 `emit` 方法可以用来发布事件。我们可以通过发布订阅模型来理解，一般是按照“订阅(`on`)—>发布(`emit`)”的顺序进行的，这和消息队列有异曲同工之处。

👉 EventEmitter 入门

`EventEmitter` 是 Node.js 的基础模块，Node.js SDK 里有很多模块是继承自 `EventEmitter` 的。最简单的测试代码如下。

```
const EventEmitter = require('events')
const observer = new EventEmitter();

observer.on('topic', function () {
  console.log('topic has occurred');
});

function main() {
  console.log('start');
  observer.emit('publish topic');
  console.log('end');
}

main()
start
topic has occurred
end
```

上面的代码在加载 `events` 模块后，通过 `EventEmitter` 属性建立了一个 `EventEmitter` 对象实例，这个实例就是消息中心，然后通过 `on` 方法为 `topic` 事件指定回调函数，最后通过 `emit` 方法触发了 `topic` 事件。

EventEmitter 可以简单理解为“发布/订阅”模式，topic 是主题，observer 对象首先通过 on 方法进行注册，对 topic 事件进行订阅，当 observer 调用 emit 方法时，所有通过 on 注册该 topic 事件的回调函数都会被调用。

上面的代码也表明，EventEmitter 对象的事件触发和监听是同步的，即只有在事件的回调函数是异步的情况下，函数 emit 才会被触发执行。

EventEmitter 和前端的事件机制很类似，其实 Vue 里的 \$emit 和 \$on 也是一模一样的机制。这里以 jQuery 官方文档里的 on 和 trigger 为例进行说明，代码如下。

```
$( "#foo" ).on( "click", function() {  
    alert( $( this ).text() );  
});  
  
$( "#foo" ).trigger( "click" );
```

on 方法都是一样的，用于定义事件的监听器。emit 方法和 trigger 也是一样的，是事件的触发器。

👉 继承 EventEmitter 类

events 模块只提供了一个对象：events.EventEmitter。EventEmitter 的核心部分就是对事件触发与事件监听器功能的封装。

可以通过 require("events"); 命令来访问 EventEmitter 模块，代码如下。

```
// 引入 events 模块  
const events = require('events');  
// 创建 EventEmitter 对象  
const EventEmitter = new events.EventEmitter();
```

当 EventEmitter 示例遇到错误时，它会触发 error 事件。当增加一个监听者的时候，会触发 newListener 事件。当移除监听者时，会触发 removeListener 事件。EventEmitter 提供了多个方法，比如 on 和 emit，on 用于绑定事件的监听函数，而 emit 用于触发事件。

对于 Node.js 的不同版本而言，EventEmitter 在用法上略有差异，Node.js v6 之后的写法可以更简单，并且 API 向前兼容，源代码里有如下兼容写法。

```
function EventEmitter() {  
    EventEmitter.init.call(this);  
}  
  
module.exports = EventEmitter;
```

```
// 向后兼容 Node.js v0.10.x
EventEmitter.EventEmitter = EventEmitter;
```

在 Node.js v6 之后, 可以直接使用 `require('events')` 类, 代码如下。

```
var EventEmitter = require('events')
var util = require('util')

var MyEmitter = function () {

}

util.inherits(MyEmitter, EventEmitter)

const myEmitter = new MyEmitter();

myEmitter.on('event', (a, b) => {
  console.log(a, b, this);
});

myEmitter.emit('event', 'a', 'b');
```

而在 Node.js v6 之前, 需要使用 `require('events').EventEmitter` 方法, 代码如下。

```
var EventEmitter = require('events').EventEmitter
var util = require('util')

var MyEmitter = function () {

}

util.inherits(MyEmitter, EventEmitter)

const myEmitter = new MyEmitter();

myEmitter.on('event', (a, b) => {
  console.log(a, b, this);
});

myEmitter.emit('event', 'a', 'b');
```

👉 设置最大监听数

`emit` 方法用于触发事件, `on` 方法用于注册事件, 其参数是因事件被触发而调用的回调函数。下面我们分别来看一下这两种方法。

对于 `on` 方法而言, 默认情况下, Node.js 允许同一个事件最多指定 10 个回调函数, 指定回

调函数的语句如下。

```
ee.on("someEvent", function () { console.log("event 1"); });
ee.on("someEvent", function () { console.log("event 2"); });
ee.on("someEvent", function () { console.log("event 3"); });
```

超过 10 个回调函数时会发出一个警告。这个阈值可以通过 `setMaxListeners` 方法改变，例如，将阈值改为 20 的方法如下。

```
ee.setMaxListeners(20);
```

特殊情况下，也可以使用 `Infinity` 来设置最大阈值，方法如下。

```
ee.setMaxListeners(Infinity);
```

`EventEmitter` 实例的 `emit` 方法是用来触发事件的，它的第一个参数表示事件名称，其余的参数都会依次被传入回调函数。传参示例如下。

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

function connection (id) {
  console.log('client id: ' + id);
};

myEmitter.on('connection', connection);
myEmitter.emit('connection', 6);
```

👉 事件

Node.js SDK 里关于事件处理的 API 是非常完善的，涵盖生命周期、事件管理等方方面面的内容，极大地减轻了开发者的负担。本节会对事件操作接口、`once` 方法、获取监听器信息、事件错误处理等 API 进行介绍。

1. 事件操作接口

`events` 模块默认支持如下两个事件：`newListener` 事件和 `removeListener` 事件。`newListener` 事件在添加新的回调函数时被触发，`removeListener` 事件在移除回调时被触发。与事件添加和移除相关的 API 非常丰富，这里就不一一列举了。

2. `once` 方法

`once` 方法类似于 `on` 方法，但是在该方法中，回调函数只被触发一次，而在 `on` 方法中会被触发多次。

以下代码触发了三次 `message` 事件，但是回调函数只会在第一次调用时运行。

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter ();

myEmitter.once('message', function(msg) {
  console.log('message: ' + msg);
});

myEmitter.emit('message', 'this is the first message');
myEmitter.emit('message', 'this is the second message');
myEmitter.emit('message', 'welcome to nodejs');
```

以下代码指定，一旦服务器连接成功，只会触发一次回调函数。执行该方法会返回一个 `EventEmitter` 对象，因此可以链式加载监听函数。

```
server.once('connection', function (stream) {
  console.log('Ah, we have our first user!');
});
```

3. 获取监听器信息

获取监听器信息是通过 `listeners` 方法实现的，该方法接收一个事件名称作为参数，返回由该事件所有回调函数组成的数组，示例代码如下。

```
const EventEmitter = require('events');
const emitter = new EventEmitter ();

function onlyOnce () {
  console.log(ee.listeners("firstConnection"));
  ee.removeListener("firstConnection", onlyOnce);
  console.log(ee.listeners("firstConnection"));
}

ee.on("firstConnection", onlyOnce)
ee.emit("firstConnection");
ee.emit("firstConnection");

// [ [Function: onlyOnce] ]
// []
```

以上代码显示两次由回调函数组成的数组，第一次只有一个回调函数 `onlyOnce`，第二次是一个空数组，因为 `removeListener` 方法取消了回调函数。

4. 事件错误处理

事件错误处理的方法和常规回调方式处理错误的方法一样，即在回调函数里判断 `error` 参数，

同时结合 Event 里最常用的 on 方法来监听 error 事件。

通过访问 <http://nodejs.org/dist/index.json>，可以获得各种 Node.js 发布版本的详细信息。下面我们以 http.get 方法为例来测试一下，代码如下。

```
http.get('http://nodejs.org/dist/index.json', (res) => {
  const statusCode = res.statusCode;
  const contentType = res.headers['content-type'];

  let error;
  if (statusCode !== 200) {
    error = new Error('Request Failed.\n' +
      'Status Code: ${statusCode}');
  } else if (!/^application\/json/.test(contentType)) {
    error = new Error('Invalid content-type.\n' +
      'Expected application/json but received ${contentType}');
  }

  // 常规回调的处理方式
  if (error) {
    console.log(error.message);
    // 释放 response 数据占用的内存
    res.resume();
    return;
  }

  res.setEncoding('utf8');
  let rawData = '';
  res.on('data', (chunk) => rawData += chunk);
  res.on('end', () => {
    try {
      let parsedData = JSON.parse(rawData);
      console.log(parsedData);
    } catch (e) {
      console.log(e.message);
    }
  });

}).on('error', (e) => {
  // 通过 on 监听 error 事件
  console.log(`Got error: ${e.message}`);
});
```

我们来想一想，为什么 http.get 方法可以通过 on 来进行错误处理呢？原因很简单，我们来看一下 lib/http.js 里对外暴露的 get 源码，如下。

```
const client = require('http client');
const ClientRequest = client.ClientRequest;
```

```
...

function get(options, cb) {
  var req = request(options, cb);
  req.end();
  return req;
}

function request(options, cb) {
  return new ClientRequest(options, cb);
}
```

以上代码是通过 `request` 方法来进行请求处理的。`request` 方法是通过 `ClientRequest` 来实现的，而 `ClientRequest` 只是客户端的一个属性而已，所以核心代码实现其实是在客户端所在的 `lib/_http_client.js` 里。

```
const OutgoingMessage = require('_http_outgoing').OutgoingMessage;

function ClientRequest(options, cb) {
  OutgoingMessage.call(this);
  ...
}
```

继续查找 `OutgoingMessage` 所在的 `lib/_http_outgoing` 文件，代码如下。

```
const Stream = require('stream');

function OutgoingMessage() {
  Stream.call(this);
  ...
}
```

其实，所有的 `http` 模块对外发送的消息都是基于 `Stream` 的，而 `Stream` 是继承自 `events` 的，看一下 `lib/stream.js` 里 `Stream` 的定义，代码如下。

```
const Stream = module.exports = require('internal/streams/legacy');
```

在 `lib/internal/streams/legacy.js` 里，我们终于见到 `events` 模块了。

```
const EE = require('events');

function Stream() {
  EE.call(this);
}

util.inherits(Stream, EE);
```

在 `Node.js` 的世界里，`http` 是最常用的模块，它简单易用且效率非常高，是被应用最广泛的模块。如果说 `http` 是 `Node.js` 的核心模块，那么 `Stream` 就是核心中的核心，是保证 `http` 高效的

秘密武器。相比之下，events 显得极为“底层”，是为核心模块服务的。

7.2.3 该选择哪种风格的写法

大家可能会疑惑，对于事件和回调写法，该选哪种？我曾经参与过 Kenneth Auchenberg 的一个小模块 expirable-hash-table 的构建，它和 Redis 或 MongoDB 里的 expire 类似。

expirable-hash-table 最初的功能非常简单，示例代码如下。

```
var ExpirableHashTable = require('expirable-hash-table')

var table = new ExpirableHashTable(1000)

table.set('key', 'value', 3000)
```

这样维护一个数据结构，不是特别有效果。比如 3000ms 后要把 key 移除，同时在回调中完成某些操作，这才是意义最大的业务切入点。

最开始我给出的 API 如下。

```
table.set('key2', 'value', 500, function callback(key, value, timeout){
  console.log(key)
});
```

从表达上看，这个 API 是很清晰的，非常容易理解。

当 key 超时且被移除的时候，会触发 callback(key, value, timeout)命令，这一点怎么看都是非常合理的。那么我们为什么不采用 EventEmitter 机制呢？先看一下如下所示的 EventEmitter 机制的示例代码。

```
table.set('key2', 'value', 500)
table.once('key2:expired', function() {
  console.log('key2:expired in callback function')
})
```

对比一下，可以发现：采用回调函数写法，代码的可读性很强，是参数同步语义化的传统思路；使用 EventEmitter 写法则语义更为清晰，可以帮助学习者非常容易地理解异步原理。它们都能实现一样的功能，只是风格不一样而已。

API 推荐使用错误优先的回调方式，统一使用 Node.js SDK 的回调风格。在同一对象内，使用 EventEmitter 解耦可以合理利用集成，使代码可读性更强。

7.3 更好的异步流程控制

流程控制改进一直是 Node.js 社区在努力做的事。本节将按照改进的先后顺序进行讲解, 先介绍什么是回调地狱, 然后分别介绍 4 种异步流程控制方式: Thunk、Promise、Generator 和 async 函数。了解每种方式的优缺点, 可以帮助开发者在 Node.js 开发及前端开发过程中更加游刃有余地进行操作。

7.3.1 回调地狱

Node.js 因为采用了错误优先的回调写法, 导致 SDK 里导出的都是回调函数。如果组合调用这些函数, 经常会出现回调里嵌套回调的问题。大家都非常厌烦这种写法, 称之为回调地狱。来看一个经典的例子, 来自著名的 Promise 模块的 q 文档, 代码如下。

```
step1(function (value1) {
  step2(value1, function(value2) {
    step3(value2, function(value3) {
      step4(value3, function(value4) {
        // 对 value4 进行一些逻辑处理
      });
    });
  });
});
```

这里只有 4 步操作, 却已经嵌套了 4 层回调。如果还有更多步骤呢? 很多新手浅尝辄止, 到这里就望而却步了。这种心态明显不够成熟, 起码要看看这个问题的应对方案吧!

👉 迷失的回调地狱

异步流程控制随处可见, 可以说只要有异步回调的地方就有异步流程控制。这是程序的通病, 并非只有 Node.js 代码中才有, 只不过 Node.js 极度追求异步性能, 故而更明显地采用了这种写法。可以在 Node.js SDK 的 API 写法上进行约定, 这是其他语言和模块里没有的。关于这一点, 这里主要介绍两个场景: 数据库和爬虫。

1. 数据库

数据库是 Web 应用中最重要的一部分。比如最常见的 CNode 论坛操作, 发帖完成后, 系统会更新用户的发帖数。一个简单的发帖系统至少要完成以下两步操作。

- step 1: 发帖, 在帖子列表里创建一条记录。
- step 2: 更新用户的发帖数。

更复杂的业务操作可能会涉及几十个甚至上百个表, 所以, 类似这种有先后顺序且涉及多个步骤的流程都需要使用异步流程控制机制。

2. 爬虫

想写好爬虫代码, 需要具备 3 项技能, 具体如下。

- 使用 `node-crawler` 进行爬取, 发送 `http` 请求。`node-crawler` 是基于 `request` 模块实现的。
- 结合 `jsdom`, 使用类似于 `jQuery` 的 `DOM` 操作来解析 `HTML` 结果。
- 将这些结果持久化, 使结果既可以被写入文件又可以被存入数据库。

爬虫示例代码如下。

```
var Crawler = require("crawler");
var jsdom = require('jsdom');

var c = new Crawler({
  jQuery: jsdom,
  maxConnections : 100,
  forceUTF8:true,
  callback : function (error, result, $) {
    var urls = $('#list a');
    console.log(urls)
  }
});

c.queue('http://www.biquku.com/0/330/');
```

在 `node-crawler` 中, `c.queue` 方法的用法如下。

- `c.queue` 方法中如果无回调, 则调用全局回调。
- `c.queue` 方法中如果有回调, 则调用传入的回调。

在这个过程中, 我们可以很好地学习 `node-crawler` 和 `jQuery` 的 `DOM` 操作, 知识点整体来说比较少, 更加容易掌握。如果不使用递归遍历, 直接通过函数调用来解决, 该怎么做呢? 各位读者可以思考一下。

👉 async.js 模块

async.js 是使用 Node.js 编写的用于简化异步流程控制的模块。它可以简化 Node.js 异步操作的写法, 使代码更容易被读懂, 是远离回调地狱的早期解决方案之一。这里需要注意的是, async.js 和 async 函数容易混淆, 所以在讲 async.js 时要加上后缀, 避免大家误解。async.js 对各种场景、API 和数据结构都有扩展, 其中下面 3 个函数是最常用到的。async.js 是一个简化异步流程控制的非常实用的模块, 但现在已经很少有人使用它了。

1. 对集合进行操作的 each 函数

each 函数可以传多个参数, 并且约定最后一个参数为执行完成的回调函数, 具体写法如下。

```
async.each(openFiles, saveFile, function(err) {  
  });
```

在上例中, 我们对一个传入的 openFiles 文件执行 saveFile 操作, 当所有文件都保存完成后, 则调用最后一个回调函数。

2. 顺序执行的 series 函数

顺序执行是最基本的流程控制方式, 但在 Node.js 中却不太容易实现, 这里重点介绍一下 series 函数。

当前面的函数执行完成后, 就立即执行下一个函数。如果函数调用过程中触发了错误, 则可以通过回调函数将错误交给最后面的回调函数来处理。示例代码如下。

```
async.series([  
  function(callback) {  
    callback(null, 'one');  
  },  
  function(callback) {  
    callback(null, 'two');  
  }  
],  
function(err, results) {  
});
```

在以上代码中, 第一个参数是数组, 每个元素都是一个异步处理函数, 约定参数是 callback,

遵守错误优先写法，将错误和结果一起返回。第二个参数是结果处理函数，如果发生错误就直接调用此函数。如果没有发生错误，则当数组内的异步处理函数都执行完成后，会调用此函数，通过 `results` 参数来获得最终结果。

3. 瀑布式的 waterfall 函数

`waterfall` 和上面介绍的 `series` 基本一样，不同的是，它可以在流程执行的过程中传递参数，示例代码如下。

```
async.waterfall([
  function(callback) {
    callback(null, 'one', 'two');
  },
  function(arg1, arg2, callback) {
    // arg1 的值是'one', arg2 的值是'two'
    callback(null, 'three');
  },
  function(arg1, callback) {
    // 此时 arg1 的值是'three'
    callback(null, 'done');
  }
], function (err, result) {
  // 此时 result 的值是'done'
});

// 或者使用具名函数的方式
async.waterfall([
  myFirstFunction,
  mySecondFunction,
  myLastFunction,
], function (err, result) {
  // 此时 result 的值是'done'
});

function myFirstFunction(callback) {
  callback(null, 'one', 'two');
}

function mySecondFunction(arg1, arg2, callback) {
  // arg1 的值是'one', arg2 的值是'two'
  callback(null, 'three');
}

function myLastFunction(arg1, callback) {
  // 此时 arg1 的值是'three'
  callback(null, 'done');
}
```

总结一下，`async.js` 是一个不错的工具库，但过于独立，和 `Promise/A+` 规范实现的模块不一

样, 也就是说, 我们需要额外学习 `async.js` 的 API, 但又不能和 `Promise/A+` 规范一起使用。与其同时学习两个, 不如直接使用 `Promise/A+` 规范。另外, `async.js` 也没有 `Bluebird` 性能好。当然, 如果是在中小项目中, 用 `async.js` 还是非常不错的。

7.3.2 Thunk

Thunk 曾在某段时间内非常受到关注, 著名的 `co` 模块在 `v4` 以前的版本中曾大量使用 Thunk 函数, `Redux` 中也有 `redux-thunk` 这样的中间件。整体来说, 掌握 Thunk 用法并不会对我们学习 `Node.js` 产生很大帮助, 因此了解一下即可。

👉 Thunk 函数

JavaScript 采用的是“传名调用”的参数求值策略, 即将参数放到一个临时函数之中, 再将这个临时函数传入函数体。这个临时函数就叫作 Thunk 函数。在 `co` 应用中, 为了能像编写同步代码那样编写异步代码, 比较常见的方式就是使用 Thunk 函数。但这不是唯一的方式, 还可以用 `Promise` 和 `Generator`。

举例来说, 比如读取文件内容的 `fs.readFile()` 函数, 可将其转化为 Thunk 函数。

Thunk 函数具备两个要素: 有且只有一个参数是 `callback` (回调) 函数; `callback` 的第一个参数是 `error`。Thunk 函数的作用是将多参数替换成单参数, 示例代码如下。

```
function Thunk (fileName) {
  return function (callback){
    return fs.readFile(fileName, callback);
  };
};

// 正常版本的 readFile (多参数版本)
fs.readFile(fileName, callback);

// Thunk 版本的 readFile (单参数版本)
var readFileThunk = Thunk(fileName);
readFileThunk(callback);
```

任何函数只要参数中有回调函数, 就能写成 Thunk 函数的形式。下面是一个简单的 Thunk 函数转换器的示例。

```
var Thunk = function(fn) {
  return function (...args) {
    return function (callback) {
```

```

    return fn.call(this, ...args, callback);
  }
};
};

```

使用上面的转换器，我们可以生成 `fs.readFile` 的 `Thunk` 函数。

```

var readFileThunk = Thunk(fs.readFile);
readFileThunk(fileA)(callback);

```

使用 `Thunk` 函数，同时结合 `co` 模块，我们就可以像写同步代码那样来写异步代码了。先举个例子，大家来感受一下。

```

var co = require('co'),
    fs = require('fs'),
    Promise = require('es6-promise').Promise;

function readFile(path, encoding){
  return function(cb){
    fs.readFile(path, encoding, cb);
  };
}

co(function* (){
  // 外面不可见，但在 co 内部其实已经转化成了 promise.then().then()..链式调用的形式
  var a = yield readFile('a.txt', {encoding: 'utf8'});
  console.log(a); // a
  var b = yield readFile('b.txt', {encoding: 'utf8'});
  console.log(b); // b
  var c = yield readFile('c.txt', {encoding: 'utf8'});
  console.log(c); // c
  return yield Promise.resolve(a+b+c);
}).then(function(val){
  console.log(val); // abc
}).catch(function(error){
  console.log(error);
});

```

这里 `a`、`b`、`c` 是顺序执行的，这要得益于 `Generator` 机制，在 `yield` 关键字后面紧接着执行 `Thunk` 函数是一种不错的方式。

👉 Thunkify

每次都自己编写一个 `Thunk` 函数还是比较麻烦的。如果使用 `Thunkify` 模块，我们便能轻松实现。`Thunkify` 和 `Promise` 里的 `Promisify` 实现有着异曲同工之处，将前面的代码改为使用 `Thunkify` 实现的形式，修改后的代码如下。

```
const co = require('co'),
      thunkify = require('thunkify'),
      fs = require('fs');

const readFile = thunkify(fs.readFile);

co(function* () {
  // 外面不可见, 但在 co 内部已经转化成了 promise.then()..链式调用的形式
  var a = yield readFile('a.txt', {encoding: 'utf8'});
  console.log(a); // a
  var b = yield readFile('b.txt', {encoding: 'utf8'});
  console.log(b); // b
  var c = yield readFile('c.txt', {encoding: 'utf8'});
  console.log(c); // c
  return yield Promise.resolve(a+b+c);
}).then(function(val) {
  console.log(val); // abc
}).catch(function(error) {
  console.log(error);
});
```

在 co v4 以前是大量使用 Thunk 函数的, 由于 Node.js 社区更趋于 Promise 化, 所以在 co v4 发布之后, Thunk 基本就完成了它的历史使命, 慢慢淡出了大家的视野。

7.3.3 Promise

Node.js 约定所有 API 都采用错误优先的回调方式, 而 Promise 是对回调地狱的思考, 或者说是改良方案。Promise 目前使用非常普遍, 可以说是在 async 函数普及之前唯一的通用性规范, 甚至 Node.js 社区都在考虑进行 Promise 化, 可见其影响力之大。

Promise 最早也是在 CommonJS 社区被提出来的, 当时提出了很多规范, 比较被接受的是 Promise/A 规范。后来人们在这个基础上提出了 Promise/A+ 规范, 也就是现在业内推行的规范, ES6 也采用这种规范。

👉 Promise/A+ 规范

为了方便读者阅读, 本书中所有的 Promise 都默认指 Promise/A+ 规范。

promise 这个词意味着“承诺”一个暂时还没有完成但将来会完成的事情。与 Promise 进行交互的最主要的方法是, 通过将函数传入它的 then 函数从而获得 Promise 的最终结果, 告诉下一个 then 函数如何操作。

Promise 的要点如下。

- 递归：每个异步操作返回的都是 Promise 对象。
- 状态机：三种状态转换，只在 Promise 对象内部可以控制，不能在外部分改变状态。
- 全局异常处理。

首先来看一下 Promise 的定义。每个 Promise 的定义都是一样的，在构造函数里传入一个匿名函数，参数是 resolve 和 reject，分别代表成功和失败时候的处理方法。示例代码如下。

```
var promise = new Promise(function(resolve, reject) {  
  
  if (/* everything turned out fine */) {  
    resolve("Stuff worked!");  
  }  
  else {  
    reject(Error("It broke"));  
  }  
});
```

再来看一下 Promise 的调用，Promise 的交互主要是通过 then 函数实现的。如果 Promise 成功执行了 resolve，那么它就会将 resolve 的值传给最近的 then 函数，作为 then 函数的参数。如果出错（reject），那就交给 catch 来捕获异常。示例代码如下。

```
promise.then(function(text) {  
  console.log(text) // Stuff 功能正常  
  return Promise.reject(new Error('我是故意的'))  
}).catch(function(err) {  
  console.log(err)  
})
```

Promise 的最大优势是标准化，各类异步工具库都按照统一规范实现，即使是 async 函数也可以无缝集成。在 async 函数普及之前，绝大部分应用都是采用 Promise 来做异步流程控制的，所以掌握 Promise 是 Node.js 学习过程中的重中之重。下面我们来看一下 Promise 中的要点。

1. 术语

为了便于学习，下面这些术语是需要了解的。

- Promise 对象：指的是 Promise 实例对象。
- ES6 Promise：如果想明确表示使用 ECMAScript 6 标准的话，可以将 ES6 作为前缀。

- Promises/A+: 这是 ES6 Promise 的前身，是一个社区规范，和 ES6 Promise 有很多共同的内容。
- Thenable 类 Promise 对象：拥有 then 方法的对象。

2. 你好，Promise

下面给出一个最简单的读写文件的 API 示例，它是典型的回调优先风格的 API，代码如下。

```
// 回调
var fs = require("fs");

fs.readFile('./package.json', (err, data) => {
  if (err) throw err;
  console.log(data.toString());
});
```

下面，我们把以上代码变成简单的 Promise 示例，如下。

```
// callback 转 Promise 的写法
var fs = require("fs");

function hello (file) {
  return new Promise(function(resolve, reject) {
    fs.readFile(file, (err, data) => {
      if (err) {
        reject(err);
      } else {
        resolve(data.toString())
      }
    });
  });
}

hello('./package.json').then(function(data) {
  console.log('promise result = ' + data)
}).catch(function(err) {
  console.log(err)
})
```

这两段代码的执行效果是一模一样的。它们唯一的差别在于，前一种写法是 Node.js 默认的 API 写法，以回调为主，而后一种写法则通过返回 Promise 对象在 fs.readFile 的回调函数里将结果延后处理了，是最简单的 Promise 实现。

Promise 函数的定义如下。

```
new Promise(function(resolve, reject){
})
```

其中包括两个参数，分别对不同结果进行处理，具体说明如下。

- **resolve**: 解决，进入下一个流程。
- **reject**: 拒绝，跳转到捕获异常流程。

当对异步函数进行 **Promise** 包装之后，调用方法也会随之改变，具体如下。

```
hello('./package.json').then(function(data) {
})
```

当读取完 **package.json** 文件之后，**hello** 函数会调用 **resolve** 方法，将最终的文件读取结果交给 **then** 方法的参数进行处理。该参数是一个函数，它的参数是上一个方法里的 **resolve** 返回的结果。

下面介绍一下在 **Promise** 链式写法中如何处理异常，常见方法如下。

```
hello('./package.json').then(function(data) {
}).catch(function(err) {
})
```

通过上面的讲解，相信大家可以了解 **Promise** 的核心：将回调函数中的结果延后到 **then** 函数里处理或交给全局异常处理。只要遵守这个约定，异步流程控制就简单多了。

3. 封装 API 的过程

还是以上面的 **fs.readFile** 为例，我们来看一下封装 **API** 的过程，一般调用 **Node.js SDK** 的代码如下。

```
fs.readFile('./package.json', (err, data) => {
  if (err) throw err;
  console.log(data.toString());
});
```

对于参数处理，除了回调，其他内容都被放到新的函数参数中了，形式如下。

```
function hello (file) {
  ...
}
```

关于返回值，这里返回 **Promise** 实例对象，代码如下。

```
function hello (file) {  
  return new Promise(function(resolve, reject){  
    ...  
  });  
}
```

对于结果处理，这里通过 **resolve** 和 **reject** 重塑了流程，代码如下。

```
function hello (file) {  
  return new Promise(function(resolve, reject){  
    fs.readFile(file, (err, data) => {  
      if (err) {  
        reject(err);  
      } else {  
        resolve(data.toString())  
      }  
    });  
  });  
}
```

我们知道所有的 Node.js API 都采用错误优先的回调方式，所有的 Node.js API 都可以像以上示例一样采用 **Promise** 来处理。只要是异步回调函数，并且写法遵守 **Promise** 规范即可。

4. 约定函数的返回值

说到返回值，这里要强调，为了简化编程的复杂性，我们约定每个函数的返回值都得是 **Promise** 对象。这可以理解为递归的变种思想应用，只要是 **Promise** 对象，就可以控制状态并支持 **then** 方法，将无限个 **Promise** 对象链接在一起，示例代码如下。

```
// callback 转 Promise  
var fs = require("fs");  
  
function hello (file) {  
  return new Promise(function(resolve, reject){  
    fs.readFile(file, (err, data) => {  
      if (err) {  
        reject(err);  
      } else {  
        resolve(data.toString())  
      }  
    });  
  });  
}  
  
function world (file) {
```

```
return new Promise(function(resolve, reject){
  fs.readFile(file, (err, data) => {
    if (err) {
      reject(err);
    } else {
      resolve(data.toString())
    }
  });
});
}

function log(data){
  return new Promise(function(resolve, reject){
    console.log('promise result = ' + data)
    resolve(data)
  });
}

hello('./package.json').then(log).then(function(){
  return world('./each.js').then(log)
}).catch(function(err) {
  console.log(err)
})
```

通过以上代码可以看出，`hello`、`world`、`log` 返回单个 `Promise` 对象，`hello('./each.js').then(log)` 返回流程链。

无论返回单个对象，还是返回流程链，它们的返回值都是 `Promise` 对象，因此都可以继续使用 `then` 方法进行处理。

5. 链式写法

每个 `Promise` 对象都有 `then` 方法，也就是说，`then` 方法是定义在原型对象 `Promise.prototype` 上的，它的作用是为 `Promise` 实例添加状态改变时的回调函数，示例代码如下。

```
Promise.prototype.then = function(sucess, fail) {
  this.done(sucess);
  this.fail(fail);
  return this;
};
```

以上方法的返回值是 `this` 对象，这就是 `then` 可以链式操作的原因(`jQuery` 也是这样实现的)。每个方法返回的都是 `this` 对象，那么就可以继续调用 `this` 对象上的函数并形成链式写法了。

`then` 函数的两个参数如下。

- success: fulfilled 状态的回调函数。
- fail: rejected 状态的回调函数。

一般情况下, 只传 success 回调函数即可, fail 函数可选, 使用 catch 来捕获异常比通过 fail 函数进行处理更加可控。

6. 状态转换

一个 Promise 对象必须处于 pending、fulfilled、rejected 其中之一的状态。

- pending: 初始状态, 非 fulfilled 或 rejected。
- fulfilled: 完成 (成功) 状态。
- rejected: 拒绝 (失败) 状态。

可以从 pending 状态切换到 fulfilled 状态, 也可以从 pending 状态切换到 rejected 状态, 状态切换不可逆, 且 fulfilled 和 rejected 两个状态之间是不能互相切换的。

我们可以把 Promise 对象理解为一个乐高积木, 它会向下一个流程传送状态和具体结果, 如图 7-4 所示。

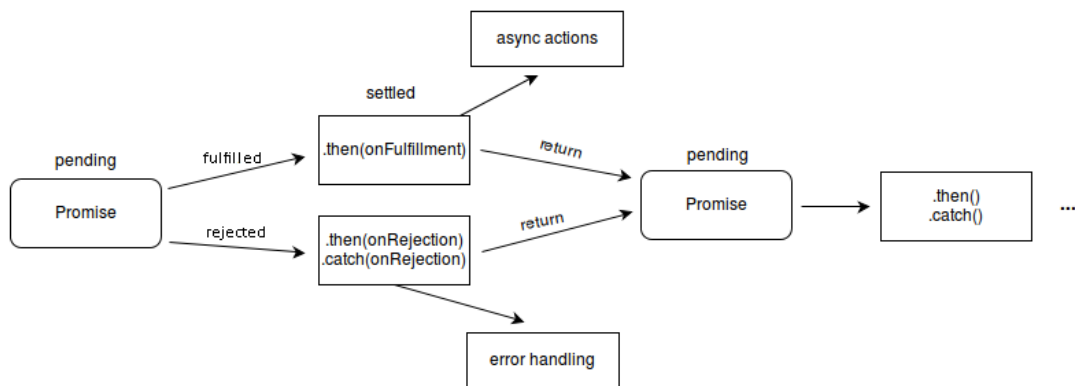


图 7-4

如果处于 pending 状态, 则 Promise 可以转换到 fulfilled 或 rejected 状态。

如果处于 fulfilled 状态, 则 Promise 不能转换到任何其他状态, 必须返回结果, 且这个结果不能被改变。

如果处于 `rejected` 状态，则 `Promise` 不能转换到任何其他状态，必须返回出错原因，且不能被改变。

一定要注意，只有异步操作有结果的时候，才可以决定当前 `Promise` 处于哪种状态，任何其他操作都无法改变这个状态。这些状态是由 `Promise` 内部维护的，我们能做的只是在写法上对其进行控制。

7. 使用 `reject` 和 `resolve` 重塑流程

前面讲到，每个函数的返回值都是 `Promise` 对象，每个 `Promise` 对象都有 `then` 方法，这是 `Promise` 得以实现递归写法的原因。

那么问题来了，如何在连续的操作步骤里完成流程重塑呢？这其实才是异步流程控制中最核心的问题。

下面仍然以 `reflow.js` 为例，来看一下简单模式和嵌套模式的写法。

(1) 简单模式

简单模式是很常规的写法，即在 `then` 里面的 `Promise` 对象里通过 `resolve` 使流程顺利进行，再在执行 `reject` 时抛出异常，代码如下。

```
hello('./package.json').then(function(data) {
  console.log('way 1:\n')
  return new Promise(function(resolve, reject) {
    console.log('promise result = ' + data)
    resolve(data)
  });
}).then(function(data) {
  return new Promise(function(resolve, reject) {
    resolve('1')
  });
}).then(function(data) {
  console.log(data)

  return new Promise(function(resolve, reject) {
    reject(new Error('reject with custom err'))
  });
}).catch(function(err) {
  console.log(err)
})
```

在每一个 `Promise` 对象里，我们都可以这样做，那么是不是就表示这个操作流程是可控的

呢？简单情况下还好，但复杂情况下并不可控。

(2) 嵌套模式

嵌套模式的代码如下。

```
hello('./package.json').then(function(data) {
  return new Promise(function(resolve, reject) {
    console.log('promise result = ' + data)
    resolve(data)
  }).then(function(data) {
    return new Promise(function(resolve, reject) {
      resolve('1')
    });
  }).catch(function(err) {
    console.log(err)
  })
}).then(function(data) {
  console.log(data)

  return new Promise(function(resolve, reject) {
    reject(new Error('reject with custom err'))
  });
}).catch(function(err) {
  console.log(err)
})
})
```

这里的做法是，把第一个 `then` 和第二个 `then` 合并到一个流程里。为了进行更细粒度的异步处理，这样做是非常有好处的。

(3) 重构的清晰版

重构的清晰版代码如下。

```
var step1 = function(data) {
  console.log('\n\nway 3:\n')
  return new Promise(function(resolve, reject) {
    console.log('promise result = ' + data)
    resolve(data)
  }).then(function(data) {
    return new Promise(function(resolve, reject) {
      resolve('1')
    });
  }).catch(function(err) {
    console.log(err)
  })
}
```

```
var step2 = function(data) {
  console.log(data)

  return new Promise(function(resolve, reject) {
    reject(new Error('reject with custom err'))
  });
}

hello('./package.json').then(step1).then(step2).catch(function(err) {
  console.log(err)
})
```

这里把每个独立的操作抽象成独立函数，然后函数的返回值是 **Promise** 对象，这样就可以在真正的流程链里随意进行组织了。

这些包含异步操作的独立函数就好比积木，让代码更具可读性和可维护性，从而使代码逻辑更清晰。再极端一点，如果把每个操作都放到独立文件里，使其变成模块，是不是体验更好呢？

(4) 最终版

最终版就是上面提到的“把每个操作都放到独立文件里变成模块”的版本，核心是使用 **require-directory** 模块。根据 **CommonJS** 规范，**require** 只能引用某一个文件，当一个文件夹里有很多文件时，处理起来就会很麻烦。而 **require-directory** 就是一个便捷模块，可以把某个文件夹内的多个文件挂载到同一个对象上。

require-directory 模块的原理是递归遍历文件，读取具体文件，如果模块遵循 **CommonJS** 规范，就将文件挂载在它的返回值对象上。

最终版本的代码如下。

```
var requireDirectory = require('require-directory');
module.exports = requireDirectory(module);
```

这样一来，**reflow/tasks** 目录下所有遵循 **CommonJS** 规范的模块便都可以挂载了。

再来看一下 **reflow/tasks/hello.js** 的内容，具体如下。

```
var fs = require("fs");

module.exports = function hello (file) {
  return new Promise(function(resolve, reject) {
    fs.readFile(file, (err, data) => {
      if (err) {
```



```
    reject(err);
  } else {
    resolve(data.toString())
  }
});
});
}
```

这其实和上面的定义是一模一样的，唯一的差别就是变成了独立的 Node.js 模块并使用了 `module.exports` 方法进行导出。下面我们来看一下具体调用的代码，如下。

```
var tasks = require('./tasks')

tasks.hello('./package.json')
  .then(tasks.step1)
  .then(tasks.step2)
  .catch(function(err) {
    console.log(err)
  })
```

以上代码的具体执行流程如图 7-5 所示。

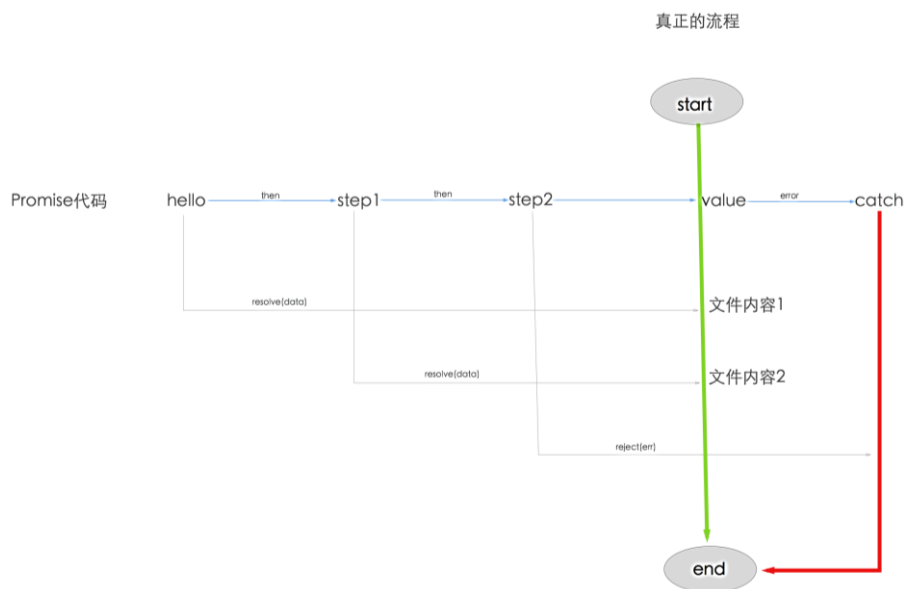


图 7-5

首先通过 `require('./tasks')` 获得 `tasks` 目录下的所有操作任务定义，然后在如下的 Promise 流程中进行处理。从上面的代码中可以看出，定义和实现是分离的，代码的可读性更好。如果这

时我们恰好需要调整 `step1` 和 `step2` 的顺序，是不是就非常简单了呢？代码如下。

```
var tasks = require('./tasks')

tasks.hello('./package.json')
  .then(tasks.step2)
  .then(tasks.step1)

.catch(function(err) {
  console.log(err)
})
```

8. 流程控制中的错误处理

在常用的处理流程控制的方法中，错误的是全局处理方式，即所有的异步操作都由一个 `catch` 来处理，示例代码如下。

```
promise.then(function(result) {
  console.log('Got data!', result);
}).catch(function(error) {
  console.log('Error occurred!', error);
});
```

这里的 `then` 方法中只有一个参数。其实，`then` 方法中可以有二个参数，第二个参数是可选的，示例代码如下。

```
promise.then(function(result) {
  console.log('Got data!', result);
}).then(undefined, function(error) {
  console.log('Error occurred!', error);
});
```

在以上示例中，`undefined` 是第一个参数，而 `function(error){}` 是第二个参数，即 `reject` 函数，用于处理上一个 `Promise` 的异常。

那么，如果有多个与 `then` 方法配对的 `reject` 函数，错误处理是不是就更加灵活了呢？其实不一定，这要取决于业务的复杂程度。

最简单的错误处理办法是在 `Promise` 里使用 `try/catch` 语句。`try/catch` 可以捕获异常，并显式处理异常，代码如下。

```
try {
  readFile()
  .then(function (data) {
```

```
        throw new Error('never will know this happened')
      })
    } catch (e) {
      console.error(e)
    }
  }
```

为了打印异常，我们可以使用`.then(null, onRejected)`语句，代码如下。

```
readFile()
  .then(function (data) {
    throw new Error('now I know this happened')
  })
  .then(null, console.error)
```

一些 Node.js 模块中会包括一些暴露异常的其他选项。比如 `q` 模块就提供了 `done` 方法，可以再次运行出异常。链式写法很方便，例如，`api/catch.js` 的代码如下。

```
var p1 = new Promise(function(resolve, reject) {
  resolve('Success');
});

p1.then(function(value) {
  console.log(value); //此时是成功的，下一句会进入异常捕获流程
  return Promise.reject('oh, no!');
}).catch(function(e) {
  console.log(e); // 此时会打印"oh, no!"值
}).then(function() {
  console.log('after a catch the chain is restored');
}, function () {
  console.log('Not fired due to the catch');
});
```

在终端中，执行以下 `api/catch.js` 脚本，结果如下。

```
$ node api/catch.js
Success
oh, no!
after a catch the chain is restored
```

以上代码在最近处的 `catch` 语句中捕获了异常，如果后面还是报错，则依然按照就近处理原则进行处理。

以下是 `api/catch2.js` 的代码，可以看到，最近处的 `catch` 语句捕获了异常，然后再次抛出异常，在后面的 `then` 语句或 `catch` 语句里还是可以继续处理的。

```
var p1 = new Promise(function(resolve, reject) {
  resolve('Success');
```

```
});

p1.then(function(value) {
  console.log(value); //此时是成功的，下一句会进入异常捕获流程
  return Promise.reject('oh, no!');
}).catch(function(e) {
  console.log(e); //此时会打印"oh, no!"值，下一句继续进入异常捕获流程
  return Promise.reject('oh, no! 2');
}).then(function() {
  console.log('after a catch the chain is restored');
}, function () {

  //第二个异常捕获流程会在这里进行处理。
  console.log('Not fired due to the catch');
});
```

在终端中，执行 `api/catch2.js` 脚本，结果如下。

```
$ node api/catch2.js
Success
oh, no!
Not fired due to the catch
```

👉 Promise 的 API 方法

Promise 规范非常简单，只包含一个构造函数和六个方法，接下来会一一介绍。

1. 构造方法

构造方法的语法如下。

```
new Promise( /* executor */ function(resolve, reject) { ... } );
```

所有 Promise 都要通过这种方式来创建，函数的两个参数 `resolve` 和 `reject` 是唯一可以改变 Promise 对象状态的接口。

- `resolve` 会让状态从 `pending` 切换到 `fulfilled`。
- `reject` 会让状态从 `pending` 切换到 `rejected`（可选）。
 - `Promise.prototype.then()` 可以捕获当前操作的 `reject` 异常。
 - `Promise.prototype.catch()` 可以捕获全局操作的 `reject` 异常。

注意，这里的 `resolve` 相当于 `Promise.resolve` 的别名，`reject` 相当于 `Promise.reject` 的别名。

下面我们通过几个例子来进行详细讲解。先来看一下 `Promise.resolve` 的用法，代码如下。

```
new Promise(function(resolve) {
  resolve(1);
}).then(function(value) {
  console.log('new Promise ' + value);
});

Promise.resolve(1).then(function(value) {
  console.log('Promise.resolve ' + value);
});
```

在以上示例中，两种 `resolve` 用法的效果是一样的，可以看出 `new Promise` 更为强大，而 `Promise.resolve` 更为便捷。

再来看一下 `Promise.reject` 的用法，代码如下。

```
var error = new Error('this is a error')

new Promise(function(resolve, reject) {
  reject(error);
}).catch(function(err) {
  console.log('new Promise ' + err);
});

Promise.reject(error).catch(function(err) {
  console.log('Promise.resolve ' + err);
});
```

在以上示例中，两种 `reject` 的效果是一样的，可以看出 `Promise.reject` 是便捷用法。

既然 `resolve` 和 `reject` 都有别名，那么我们能不能不使用构造函数，直接使用便捷用法呢？答案是不可行的。请看以下代码。

```
// 以下做法是错误的
new Promise(function() {
  return Promise.resolve(1)
}).then(function(value) {
  console.log('Promise.resolve 1 ' + value);
});
```

`Node.js` 的原生 `Promise` 是不支持以上写法的。要想更加便捷，一般采用下面这样的写法。

```
// 以下做法是正确的
function hello(i) {
  return Promise.resolve(i)
}
```

```
hello(1).then(function(value){
  console.log('Promise.resolve 1 =' + value);
});
```

这种写法可行原因是，`Promise.resolve` 返回的是 `Promise` 对象，相当于 `new Promise(resolve, reject)` 实例，`Promise.resolve` 静态方法只是对工厂方法的简单包装。

但是一定要注意，一旦函数确定要返回 `Promise` 对象，就要同时确保全部可能的分支都返回 `Promise` 对象，不然出了问题会非常难以定位。

举个简单的例子，根据 `hello` 函数的参数 `i` 是奇数或偶数进行不同结果的处理，逻辑上一定要严谨。先来看一下分支逻辑演示，代码如下。

```
// 奇数和偶数
function hello(i){
  if (i % 2 == 0) {
    return Promise.resolve(i)
  } else {
    return Promise.reject(i)
  }
}

hello(1).then(function(value){
  console.log('Promise.reject 1 ' + value);
});

hello(2).then(function(value){
  console.log('Promise.resolve 1 ' + value);
});
```

其实 `Promise.resolve` 和 `Promise.reject` 还有更多用法，对于其他用法，我们给出如下的语法定义，大家了解即可。

```
Promise.resolve(value);
Promise.resolve(promise);
Promise.resolve(thenable);
Promise.reject(reason);
```

2. 核心方法 `Promise.prototype.then()`

`Promise.prototype.then()` 方法的语法如下。

```
p.then(onFulfilled, onRejected);
p.then(function(value) { // fulfillment }, function(reason) { // rejection });
```

3. 次核心方法 Promise.prototype.catch()

Promise.prototype.catch()方法的语法如下。

```
p.catch(onRejected);  
p.catch(function(reason) { // rejection });
```

4. 工具方法

工具方法有两个，分别是 Promise.all(iterable)和 Promise.race(iterable)，两者都非常实用。

Promise.all 在所有接收到的 Promise 对象都变为 fulfilled 或者 rejected 状态之后才会继续进行后面的处理，与之相对的是 Promise.race，只要有一个 Promise 对象进入 fulfilled 或者 rejected 状态，就会继续进行后面的处理。

一定要注意，上述两个方法中的参数是多个 if 函数，这些 if 函数是并行执行的，只是处理结果的时间点不一样而已。在 async 函数里是无法并行处理的，所以需要适当地依赖 Promise 里的 all 和 race 方法，这是目前还无法避开的问题。

Promise.all 和 Promise.race 的使用方法是一样的，都是接收一个 Promise 对象数组为参数，代码分别如下。

all.js 的内容如下。

```
'use strict'  
  
let sleep = (time, info) => {  
  return new Promise(function (resolve) {  
    setTimeout(function () {  
      console.log(info)  
      resolve('this is ' + info)  
    }, time)  
  }, time)  
})  
  
let loser = sleep(1000, 'loser')  
let winner = sleep(4, 'winner')  
  
// main  
Promise.all([winner, loser]).then(value => {  
  console.log("所有都完成后会执行 then，它们是并行的哦：" + value)    // => 'this is winner'  
})
```

执行以上代码，结果如下。

```
$ node api/all.js
winner
loser
```

当所有任务都完成后会执行 `then` 函数，注意 `Promise.all()` 是并行执行的。

`race.js` 的内容如下。

```
'use strict'

let sleep = (time, info) => {
  return new Promise(function (resolve) {
    setTimeout(function () {
      console.log(info)
      resolve('this is ' + info)
    }, time)
  })
}

let loser = sleep(1000, 'loser')
let winner = sleep(4, 'winner')

// main
Promise.race([winner, loser]).then(value => {

})
```

执行以上代码，结果如下。

```
$ node api/race.js
winner
```

`winner` 和 `loser` 这两个任务中只要有一个成功执行，接下来就会执行 `then`，因此执行 `then` 与执行 `winner` 和 `loser` 的顺序无关，只看执行速度。

👉 Node.js 里的 Promise

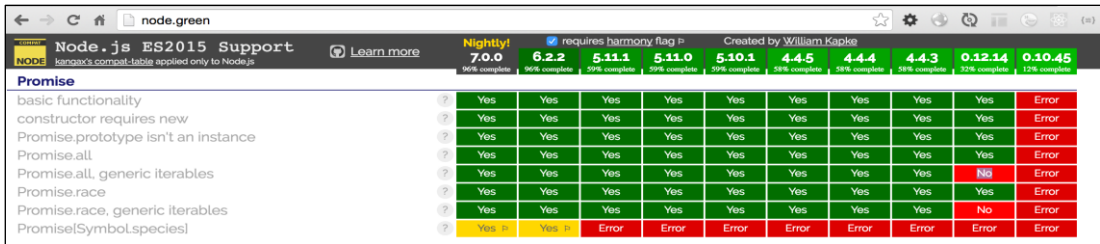
通常我们讲的 `Promise` 都是指 `Promise/A+` 规范，而 `Node.js` 因为异步而无法避免回调地狱，故而 `Promise` 在 `Node.js` 里被更加放大，变成了大家都默认的约定，各种模块也非常丰富，下面做一个简单总结。

1. 内置的 Promise

在 `Node.js v0.12` 里，这一功能实现了 9/11，而在 `Node.js v6.2` 和 `Node.js v7` 中已经 100% 实

现。所以 Node.js 对 Promise 支持得非常好，v0.12 之后的绝大部分版本都支持得不错。

要了解不同版本的 Node.js 对新特性的支持情况，可以访问网站 node.green 进行查看，如图 7-6 所示。



	Nightly!	7.0.0	6.2.2	5.11.1	5.11.0	5.10.1	4.4.5	4.4.4	4.4.3	0.12.14	0.10.45
	95% complete	95% complete	95% complete	95% complete	95% complete	95% complete	95% complete	95% complete	95% complete	95% complete	12% complete
Promise											
basic functionality	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Error
constructor requires new	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Error
Promise.prototype isn't an instance	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Error
Promise.all	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Error
Promise.all, generic iterables	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Error
Promise.race	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Error
Promise.race, generic iterables	?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Error
Promise(Symbol.species)	?	Yes	Yes	Error	Error	Error	Error	Error	Error	Error	Error

图 7-6

Node.js 源码里实际上使用的是 Chrome V8 内置的 Promise，当然这是特意为了 ES6 实现的，C++代码如下。

```
using v8::Promise;
```

著名 Node.js 程序员 tj 曾说过：“既然 Bluebird 被大量使用，那么为什么不使用 Chrome V8 自带的 Promise 呢？”这句话在当时是没什么问题的，不过还是要辩证地看。

Node.js 内置的 Promise 除了对版本有要求，对 Node.js 的性能也有一定要求。之前版本的性能可能不见得比 Bluebird 好，但未来前景是极好的。目前，还是推荐使用 Bluebird 这样的开源实现方案，原因在于 Bluebird 是通用模块，能够运行在所有的 Node.js 版本中，性能也足够好。另外，Node.js 之前版本中内置的 Promise 是有内存泄漏的，而使用 Bluebird 不会遇到这样的问题。

2. Promise 开源实现

Promise 扩展模块除了实现了 Promise/A+规范，还增加了自定义的功能。要想知道 Node.js 中有哪些比较好的 Promise 实现，最好的办法就是看一下 Bluebird 基准测试里用到了哪些模块，常见模块如下。

- asyn
- babel
- davy
- deferred

- ☐ kew
- ☐ lie
- ☐ neo-async
- ☐ optimist
- ☐ promise
- ☐ q
- ☐ rsvp
- ☐ streamline
- ☐ text-table
- ☐ vow
- ☐ when

Promise 扩展类库的数量非常多，我们只介绍其中两个比较有名的类库，具体如下。

(1) q

在 Node.js 里，q 模块实现了 Promise/A+ 和 Deferred 等规范。Deferred 本质上是对 Promise 的增强，核心是将 resolve 和 reject 方法暴露在构造函数外面，改变 Promise 对象的状态。它自 2009 年开始开发，还提供了面向 Node.js 的文件操作 API 的扩展模块 q-io，是一个在很多场景下都能用到的类库。比如 Angular v1.x 中就内置了精简版的 \$q。

(2) Bluebird

这个类库除了兼容 Promise/A+ 规范，还扩展了很多方法，比如取消了 Promise 对象的执行，提供了错误处理的扩展检测等功能，此外在性能上，该类库在实现时也下了很大的功夫，是目前性能最好的 Promise 模块。

q 和 Bluebird 这两个类库除了都能在浏览器里运行，还都拥有充实的 API 扩展。由于 Bluebird 的性能比较好，所以我们用 Bluebird 的时候会比较多。q 的文档里详细介绍了其中的 Deferred 和 jQuery 里的 Deferred 有哪些异同，以及要怎么进行迁移。Bluebird 的文档中除了提供了丰富

的 Promise 实现方式，还介绍了在出现错误时的应对方法以及 Promise 中的反模式等内容。

这两个类库的文档都很好，是非常棒的学习材料，即使我们不使用这两个类库，阅读一下文档也具有一定的参考价值。

3. Bluebird

Bluebird 是 Node.js 世界里性能最好的 Promise/A+规范的实现，API 非常齐全，功能强大，是除原生 Promise 外的不二选择。具体来说，Bluebird 的优势如下。

- Node.js 原生 Promise 存在版本问题，Bluebird 可以兼容所有版本。
- 避免 Node.js v4 中曾经出现过的内存泄漏问题。
- 内置更多扩展，如 timeout、promisifyAll 等，对 Promise/A+规范提供了强有力的补充。

我们先来看一下 Bluebird 的基本用法，安装 Bluebird 的语句如下。

```
$ npm i -S bluebird
```

下面编写一段 Bluebird 的示例代码。

```
const fs = require("fs");
const Promise = require("bluebird");

function hello (file) {
  return new Promise(function(resolve, reject){
    fs.readFile(file, (err, data) => {
      if (err) {
        reject(err);
      } else {
        resolve(data.toString())
      }
    });
  });
}

hello('./package.json').then(function(data){
  console.log('promise result = ' + data)
}).catch(function(err) {
  console.log(err)
})
```

上面的代码和之前示例的执行结果是一模一样的，差别只有一行，如下。

```
const Promise = require("bluebird");
```

由此可以看出,Node.js 原生的 Promise 和 Bluebird 的实现是兼容的。只要掌握了其中一种,几乎可以零成本学会另一种。

这里用 `const` 来声明 Promise,主要是为了在当前文件中使用 Promise 对象,在 Koa 或 Express 这样的 Web 项目里声明时则可以使用全局声明的方式。在应用的入口文件 `app.js` 里使用 `global` 关键字进行全局替换即可,代码如下。

```
global.Promise = require("bluebird");
```

就执行这样简单的一步,性能就会有明显的提升。

下面我们再来介绍一下 Bluebird 里的超时接口,当任务的操作时间超过设置值时,便会自动触发异常,示例如下。

```
const Promise = require("bluebird");

function sleep(time) {
  return new Promise((resolve)=> setTimeout(resolve, time))
}

sleep(200).then(function() {
  return sleep(2000)
})
.timeout(2100)
.catch(Promise.TimeoutError, function(e) {
  console.log("could not read file within 100ms");

  return Promise.resolve(1)
})
.then(function() {
  console.log('xxxx')
})
```

下面我们再来介绍一下 Bluebird 中的 `promisification`。在 JavaScript 中,并不是所有的异步函数和库都支持开箱即用的 Promise。大多数情况下,我们必须将一个典型的基于回调的函数转换成一个返回 Promise 的函数,这个过程也被称为 `promisification`,这里主要介绍两个核心 API: `promisify` 和 `promisifyAll`。

`promisifyAll` 可以对类方法或者对象方法进行 `promisify` 处理,用法如下。

```
const Promise = require("bluebird");
const fs = Promise.promisifyAll(require("fs"));

fs.readFileAsync("./package.json", "utf8").then(function(contents) {
  console.log(contents);
})
```

```
}).catch(function(e) {  
    console.error(e.stack);  
});
```

再来看一个稍微复杂一些的例子，下面这个例子中有 a、b、c 三个方法，每个方法都是一个普通函数，通过 Bluebird 的 `promisifyAll`，函数可以变成 `Promise` 对象，进而完成流程控制。

```
const Promise = require("bluebird");  
  
const obj = {  
  a: function(){  
    console.log('a')  
  },  
  
  b: function(){  
    console.log('b')  
  },  
  
  c: function(){  
    console.log('c')  
  }  
}  
  
Promise.promisifyAll(obj);  
  
obj.aAsync().then(obj.bAsync()).then(obj.cAsync()).catch(function(err) {  
  console.log(err)  
})
```

这些写法更加简单方便，但是便利常常伴随着危险，滥用 `promisifyAll` 可能会有性能问题，这一点要特别注意。

7.3.4 Generator

`Generator Function`（生成器函数）和 `Generator`（生成器）是 ES6 中引入的新特性，该特性早就出现在了 Python、C# 等语言中。生成器本质上是一种特殊的迭代器。

`Generator` 本意是 `Iterator` 生成器，函数运行到 `yield` 时退出，并保留上下文，在下次进入时可以继续运行。生成器函数也是一种函数，语法上仅比普通函数多了一个星号 “*”，其函数体内部可以使用 `yield` 和 `yield*` 关键字。

👉 入门

简单来说，`Generator` 是 ES6 的新特性，函数后面带 “*” 的便叫作 `Generator`，示例如下。

```
function* doSomething() {
  ...
}
```

先来看一下 Generator 如何执行，Generator 是生成器，它要先生成一个对象，然后才能继续执行，示例代码如下。

```
function* doSomething() {
  console.log('1');
  yield; // Line (A)
  console.log('2');
}

var gen1 = doSomething();

gen1.next(); // 打印出 1，然后悬停在 Line (A) 处
gen1.next(); // 恢复 Line (A) 点的执行，然后打印出 2
```

以上代码中有几处需要注意的地方，具体如下。

- gen1 是生成的 Generator 对象。
- 第一个 next 会打印出 1，之后悬停在 yield 的所在行，即 Line (A)。
- 第二个 next 恢复 Line (A) 处的执行，然后打印出 2。

理解 Generator 的执行原理对于理解 co 模块源码是极其重要的，使用 Generator 来处理复杂运算也是一种非常高效的手段。

👉 next 的返回结果说明

同上例中的代码，这里对两次 next 返回的结果进行说明。

- 第一个 gen1.next() 返回的结果是 {value: "1", done: false} 对象。
- 第二个 gen1.next() 返回的结果是 {value: "2", done: true} 对象。

两次 next 结果最大的差别在于 done 值不同，如果 done 为 true，则代表 Generator 里的异步操作都执行完了，这一点一定要搞清楚，这直接关系到对下面要介绍的 co 模块的理解。

👉 神奇的 co 模块

如果想在 Generator 里使用多个 yield 该怎么办呢？按照上面讲的执行原理，会有多个 next，

这样明显是没办法应对流程控制的。于是 TJ Holowaychuk 就编写了 co 这个著名的 ES6 模块，co 目前已经到了 4.6 版本，彻底面向 Promise 了。

co 是 ES6 Generator 的执行器，它可以使用 Generator 里的 yieldable 支持的所有形式，支持 Thunks、Promises，但不能和 async 函数混用，它的返回结果是 Promise 对象，方便与现有的模块集成。

co 适用于组合调用 Generator 的场景。在 async 函数里，await 关键字后面可以接 Promise，co 的返回值是 Promise，所以 await 和 co 搭配是一个非常好的选择。

co 的典型用法是作为 Generator 执行器使用，示例代码如下。

```
co(function* () {
  var result = yield Promise.resolve(true);
  return result;
})
.then(function (value) {
  console.log(value);
}, function (err) {
  console.error(err.stack);
});
```

以上代码中有如下两个要点。

- co 是 Generator 执行器，因此只要管好 yield 即可，不用关心 Generator 是怎么执行的。
- co 的返回值是 Promise，后面可以直接接 Promise 的 then 函数。

co 模块的 4.6 版本中只有不到 240 行代码，整体来说还算比较简单，但并不容易阅读，来看一下下面的示例。

```
// 核心代码
function co(gen) {
  // 缓存 this
  var ctx = this;
  var args = slice.call(arguments, 1)

  // 重点: co 的返回值是 Promise 对象。这就是 co 可以接 then 函数的原因
  return new Promise(function(resolve, reject) {
    // 如果懂 Promise 规范，就知道这是解决状态回调的，这是首次调用
    onFulfilled();

    /**
     * @param {Mixed} res
     * @return {Promise}
     */
  });
}
```

```

* @api private
*/

function onFulfilled(res) {
  var ret;
  try {
    ret = gen.next(res);
  } catch (e) {
    return reject(e);
  }
  next(ret);
}

/**
 * @param {Error} err
 * @return {Promise}
 * @api private
 */
// 如果懂 Promise 规范，就知道这是拒绝状态回调的
function onRejected(err) {
  var ret;
  try {
    ret = gen.throw(err);
  } catch (e) {
    return reject(e);
  }
  next(ret);
}

// Generator 执行器
// 如果 ret.done 的值等于 true，返回 ret.value
// 否则
function next(ret) {
  // 如果执行完成，直接调用 resolve 把 Promise 设置为成功状态即可
  if (ret.done) return resolve(ret.value);
  // 把 yield 的值转换成 Promise
  // 支持 Promise、Generator、generatorFunction、array、object
  // toPromise 的实现可以先不管，只要知道是转换成 Promise 了即可
  var value = toPromise.call(ctx, ret.value);
  // 成功转换就可以直接给新的 Promise 添加 onFulfilled、onRejected，
  // 当新的 Promise 状态变成结束态（成功或失败），就会调用对应的回调，
  // 整个 next 链路就可以执行下去了，解决了 Generator 不能自动执行 next 的问题
  // return value.then(onFulfilled, onRejected); 意味着又要执行 onFulfilled 了
  // 在 onFulfilled 里调用 next(ret);
  if (value && isPromise(value)) return value.then(onFulfilled, onRejected);

  // 如果以上情况都没发生，则报错
  return onRejected(new TypeError('You may only yield a function, promise,

```



```
generator, array, or object, '
    + 'but the following object was passed: "' + String(ret.value) + '"');
  }
});
}
```

阅读此源码的要点如下。

- 必须深刻理解 Promise 实现原理, 知道构造函数里的 onFulfilled 和 onRejected 是什么意思。
- 必须了解 Generator 的执行机制, 理解迭代器里的 next 以及 next 的返回对象 {value:",done: true}。

核心代码入口是 onFulfilled, 无论如何, 初次出现的 next(ret) 是一定要执行的, 因为 Generator 必须要执行 next(), 所以 next(ret) 是重点内容, 而且我们可以看到, onFulfilled 和 onRejected 里都会调用它, 即所有的逻辑都会出现在 next(ret) 方法里, 它实际上是一个状态机的简单实现。

下面看一下 co 中的 next 函数的实现。next 函数维护了一个简单的状态机, 有如下几种使用情景。

情景 1: 如果 Generator 里所有任务都执行完成, 即 ret.done 等于 true, 代表 Generator 处于“完成”状态, 如下。

```
// 如果执行完成, 直接调用 resolve 把 promise 置为成功状态
if (ret.done) return resolve(ret.value);
```

情景 2: 如果还有 yield 语句, 则需要执行 next, 跳回 onFulfilled, 进而实现调用下一个 next 的效果, 如下。

```
// 成功转换就可以直接给新的 Promise 添加 onFulfilled、onRejected
// 当新的 Promise 状态变成结束态 (成功或失败) 后, 就会调用对应的回调, 整个 next 链路就可以执行下去了
// 为什么 Generator 可以无限执行 next 呢?
// return value.then(onFulfilled, onRejected); 意味着又要执行 onFulfilled 了
// 在 onFulfilled 里调用 next;
if (value && isPromise(value)) return value.then(onFulfilled, onRejected);
```

情景 3: 捕获异常, 代码如下。

```
// 如果以上情况都没发生, 则报错
return onRejected(new TypeError('You may only yield a function, promise, generator, a
rray, or object, '
    + 'but the following object was passed: "' + String(ret.value) + '"'));
```

以上是对核心代码的说明。之前我们讲过, co 模块实际有两种 API, 有参数的和无参数的,

很明显以上代码是无参数的 Generator 执行器，那么有参数的 wrap 是什么样呢？示例如下。

```
// 为有参数的 Generator 调用提供简单包装
co.wrap = function (fn) {
  createPromise.__generatorFunction__ = fn;
  return createPromise;
  function createPromise() {
    // 重点：将 arguments 作为 fn 的参数
    // call 和 apply 是常规 JavaScript API
    return co.call(this, fn.apply(this, arguments));
  }
};
```

可以看到，将 call 和 apply 组合使用是比较巧妙的。

ES6 中的 Generator 是为了计算而设计的迭代器，但 TJ Holowaychuk 觉得它可以用于流程控制，于是就有了 co。co 和 Generator 在提供强大便利的同时，也增加了非常多新的概念，有些可能是过渡性的，有些也可能是过时的。可是，我们真的需要了解这么多概念吗？从实用的角度来看，可能并不需要。

👉 5 种 yieldable

本来是没有 yieldable 这个词的，因为在 Generator 里有 yield 关键字，而 yield 后面可以接 5 种可能的形式，因此将这种可以与 yield 连接的形式称为 yieldable，具体如下。

- Promises
- Thunk 函数
- Array 并行执行
- Object 并行执行
- Generators 和 GeneratorFunctions

这里将 co 和 Promise 做了简单的关联，同时区分 yieldable 里的并行和顺序执行处理方式，如图 7-7 所示，以便大家能够更好地理解 co 和 yieldable。

顺序执行主要有 Promise 和 Thunk，并行执行主要有 Array 和 Object。

无论是哪种，使用 co 包裹执行，内部可以是各种 yieldable 方式，co 返回的结果是 Promise，Promise 可以实现 Thenable，形成闭环。有了 co，所有异步解决方式都能混用。

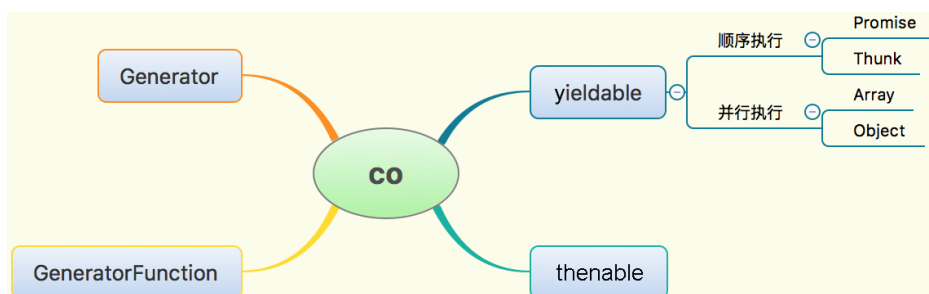


图 7-7

最关键的是，ES6 Generator 是用来进行计算的迭代器，它是过渡性的产物。yieldable 足够强大，只是学习成本稍高，理解起来有些难度。

7.3.5 async 函数

Generator 的弊病是没有执行器，它本身也不是为流程控制而生的，所以出现了 co，较好地解决了这个问题。可是，为什么非要借助 co 呢？为什么不能直接执行，并且获得和 yieldable 一样的同步执行代码的效果呢？

为了解决上述问题，async/await 被创造出来了，很多人认为它是异步操作的终极解决方案。不过很可惜，它最终没能进入 ES7 规范，但在 Chrome V8 引擎里得以实现，所以 Node.js v7.6 中就集成了 async 函数。下面我们具体来看一下。

👉 async 函数的执行过程

下面给出一段 Koa v2 应用里的代码。

```

exports.list = async (ctx, next) => {
  try {
    let students = await Student.getAllAsync();

    await ctx.render('students/index', {
      students : students
    })
  } catch (err) {
    return ctx.api_error(err);
  }
};

```

以上代码完成了 3 项操作，具体如下。

- 通过 `await Student.getAllAsync();` 获取所有的 student 信息。
- 通过 `await ctx.render` 渲染页面。
- 由于是同步代码，因此使用 `try/catch` 进行异常处理。

👉 await 的用法

`await` 的用法大概可以分为以下 3 种。

- `await + async` 函数
- `await + Promise`
- `await + co + Generator`

前两种用法是比较常见的，在第 3 种用法中，`co` 作为 `Promise` 生成器，是一种可以实现但有点麻烦的办法。下面给出以上用法的示例代码。

`await+async` 函数的示例代码如下。

```
//常规写法
const pkgConf = require('pkg-conf');

async function main(){
  const config = await pkgConf('unicorn');

  console.log(config.rainbow);
  //=> true
}

main();
// “变态” 写法
const pkgConf = require('pkg-conf');

(async () => {
  const config = await pkgConf('unicorn');

  console.log(config.rainbow);
  //=> true
})();
```

`await + Promise` 用法的示例代码如下。

```
const Promise = require('bluebird');
const fs = Promise.promisifyAll(require("fs"));

async function main(){
    const contents = await fs.readFileAsync("myfile.js", "utf8")
    console.log(contents);
}

main();
```

await + co + Generator 用法的示例代码如下。

```
const co = require('co');
const Promise = require('bluebird');
const fs = Promise.promisifyAll(require("fs"));

async function main(){
    const contents = co(function* () {
        var result = yield fs.readFileAsync("myfile.js", "utf8")
        return result;
    })

    console.log(contents);
}

main();
```

对于上面的第 3 种用法，要点如下。

- co 的返回值是 Promise，所以 await 可以直接与 co 连接起来使用。
- co 的参数是 Generator。
- 在 Generator 里可以使用 yield。

由上面 3 种基本用法，我们可以知道，async 函数的要点如下。

- async 函数从语义层面来说非常好。
- async 不需要执行器，本身具有执行能力，不像 Generator 一样需要借助 co 模块。
- async 函数进行异常处理时采用 try/catch 和 Promise 的错误处理方式，非常强大。
- await 后面接 Promise，Promise 自身就足够应对所有流程，async 函数中没有纯粹的并行处理机制，因此可以采用 Promise 里的 all 和 race 来补齐。
- await 结合 Promise 的用法是非常强大的，尤其是对并行处理方法的支持，外加 co 和

Promise 中的 then，几乎可以覆盖目前所有应用场景。

在终端里执行上面的脚本，要使用 Babel 或者其他支持 async 函数的编译工具，这里使用 Node.js v8.9 以上的版本，执行结果如下。

```
async.js
3babel presets path = /Users/sang/.npm/versions/node/v4.4.5/lib/node_modules/runkoa/no
de_modules/
hello a0 and start a1
hello a1 and start a2
hello end a2
hello end a1
```

✎ 异常处理

Node.js 里关于异常处理有一个约定，即同步代码采用 try/catch 进行处理，异步代码采用错误优先的回调方式进行处理。对于 async 函数来说，它的 await 语句是同步执行的，所以最正常的流程处理需要采用 try/catch 语句捕获，这一点和 Generator 函数是一样的。

下面的代码是通用的异常处理方法。

```
try {
  console.log(await asyncFn());
} catch (err) {
  console.error(err);
}
```

很多时候，我们需要将异常粒度设置得更细一些，这时只要把 Promise 的异常处理好就可以了。在 Promise 里有两种处理异常的方法，具体如下。

- then(onFulfilled, onRejected)里的 onRejected，处理当前 Promise 里的异常。
- 通过 try/catch 捕获代码中的异常。

async/await 是异步操作的终极解决方案，在 Node.js v7.6 发布之后，Koa 立马发布了 Koa 2 正式版本，并且推荐使用 async 函数来编写 Koa 中间件。

综上所述，可以得出如下结论。

- 使用 async 函数是趋势，从 Chrome v52 和 Node.js v8 5.1 开始就已经支持 async 函数了，目前已经广泛应用。

- `async` 函数和 `Generator` 都支持 `Promise`，所以 `Promise` 是必须会的。
- `Generator` 和 `yield` 异常强大，不过不会成为主流，所以学会基本用法和 `Promise` 即可。
- `co` 作为 `Generator` 执行器是不错的，但更好的是作为 `Promise` 包装器，通过 `Generator` 支持 `yieldable`，最后返回 `Promise`。

从 2009 年到现在，为解决异步流程控制问题，整个 `Node.js` 社区进行了大量尝试，如图 7-8 所示，大家可以先了解一下概况。

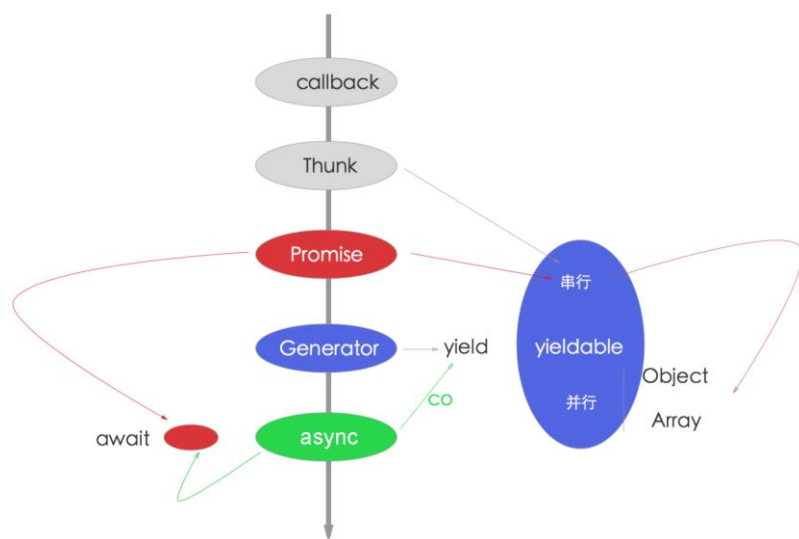


图 7-8

其中，`Promise` 是使用最多的，无论 `async` 还是 `Generator` 都可用。`Generator` 是过渡性的产品，`async` 函数则是未来的趋势。因此，`Promise` 是必须会的。推荐使用“`async` 函数+`Promise`”的用法。

一般情况下，我们并不需要掌握图 7-8 中的所有技术便可以应对一般的场景，对于初学者来说，够用即可。所以，精简一下，只了解图 7-9 中的 3 个方法就足够了。

合理地结合 `Promise` 和 `async` 函数是非常高效的，但需要注意具体的使用场景。例如，批量给函数增加 `Promise` 支持，使用 `Bluebird` 模块的 `promisifyAll` API 是非常容易的，但使用 `async` 函数是无法进行批量操作的。

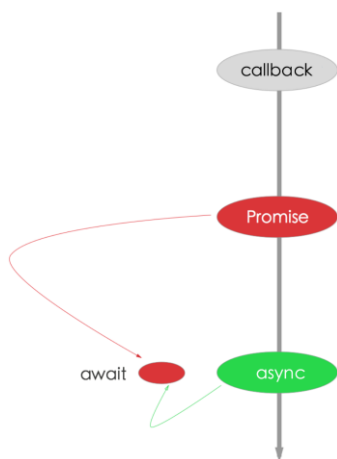


图 7-9

因此，在常见的 Web 应用里，我们总结的实践经验是，DAO 层使用 Promise 比较好，而 Service 层使用 async 函数更好。DAO 层使用 Promise 基本上只需几行代码，一般单一模型的操作不会特别复杂，应变的需求基本不大。而 Service 层一般针对多模型进行操作，可以拆分成多个简单的操作，然后使用 await 来组合，这样看起来会更加清晰，需求应变也非常容易。

7.4 本章小结

通过本章的学习，相信你已经掌握了 Node.js 异步流程控制的 5 个演进过程。除了掌握 Node.js SDK 里的两种 API 风格外，现在你应该也学会了 Node.js 里流程控制的两个必会技能：Promise 和 async 函数，掌握这两个技能便足以应对 Node.js 所有的应用场景了。

异步流程控制是 Node.js 学习过程中的核心内容，如果能掌握本章的内容，恭喜你，你已经掌握了 Node.js 中一大半的知识点了！